Spack Documentation

Release 0.9

Todd Gamblin

October 05, 2016

Tutorials

1 Feature overview **3**
- 1.1 Simple package installation . . . 3
- 1.2 Custom versions & configurations . . . 3
- 1.3 Customize dependencies . . . 4
- 1.4 Non-destructive installs . . . 4
- 1.5 Packages can peacefully coexist . . . 4
- 1.6 Creating packages is easy . . . 4

2 Getting Started **7**
- 2.1 Prerequisites . . . 7
- 2.2 Installation . . . 7
- 2.3 Compiler configuration . . . 8
- 2.4 Vendor-Specific Compiler Configuration . . . 13
- 2.5 System Packages . . . 15
- 2.6 Utilities Configuration . . . 17
- 2.7 Spack on Cray . . . 19

3 Basic usage **23**
- 3.1 Listing available packages . . . 23
- 3.2 Installing and uninstalling . . . 27
- 3.3 Seeing installed packages . . . 29
- 3.4 Specs & dependencies . . . 31
- 3.5 Virtual dependencies . . . 35
- 3.6 Integration with module systems . . . 37
- 3.7 Extensions & Python support . . . 45
- 3.8 Filesystem requirements . . . 48
- 3.9 Getting Help . . . 48

4 Workflows **51**
- 4.1 Definitions . . . 51
- 4.2 Building Packages . . . 52
- 4.3 Running Binaries from Packages . . . 54

5 Developing Software with Spack **61**
- 5.1 Write the CMake Build . . . 61
- 5.2 Write the Spack Package . . . 62
- 5.3 Build with Spack . . . 63
- 5.4 Build Other Software . . . 64

5.5 Release Your Software . . . 64
5.6 Distribute Your Software . . . 65
5.7 Other Build Systems . . . 65
5.8 Conclusion . . . 66

6 Upstream Bug Fixes **67**
6.1 Buggy New Version . . . 67
6.2 No Version Works . . . 67
6.3 Patches . . . 68

7 Configuration **71**
7.1 Temporary space . . . 71
7.2 External Packages . . . 72
7.3 Concretization Preferences . . . 73

8 Mirrors **75**
8.1 `spack mirror` . . . 76
8.2 `spack mirror create` . . . 76
8.3 `spack mirror add` . . . 77
8.4 `spack mirror list` . . . 78
8.5 `spack mirror remove` . . . 78
8.6 Mirror precedence . . . 78
8.7 Local Default Cache . . . 78

9 Command index **79**

10 Packaging Guide **81**
10.1 Creating & editing packages . . . 81
10.2 Naming & directory structure . . . 84
10.3 Trusted Downloads . . . 86
10.4 Package Version Numbers . . . 86
10.5 Adding new versions . . . 87
10.6 Fetching from VCS repositories . . . 89
10.7 Expanding additional resources in the source tree . . . 91
10.8 Automatic caching of files fetched during installation . . . 92
10.9 Licensed software . . . 92
10.10 Patches . . . 93
10.11 Handling RPATHs . . . 95
10.12 Finding new versions . . . 96
10.13 Parallel builds . . . 97
10.14 Dependencies . . . 98
10.15 Extensions . . . 100
10.16 Virtual dependencies . . . 102
10.17 Abstract & concrete specs . . . 104
10.18 Inconsistent Specs . . . 105
10.19 Implementing the `install` method . . . 106
10.20 The install environment . . . 107
10.21 Failing the build . . . 110
10.22 Prefix objects . . . 110
10.23 Spec objects . . . 110
10.24 Shell command functions . . . 113
10.25 Sanity checking an installation . . . 114
10.26 File manipulation functions . . . 115
10.27 Coding Style Guidelines . . . 116
10.28 Packaging workflow commands . . . 117

10.29 Graphing dependencies . . . 119
10.30 Interactive shell support . . . 122

11 Developer Guide **127**
11.1 Overview . . . 127
11.2 Directory Structure . . . 128
11.3 Code Structure . . . 128
11.4 Spec objects . . . 130
11.5 Package objects . . . 130
11.6 Stage objects . . . 131
11.7 Writing commands . . . 131
11.8 Unit tests . . . 131
11.9 Unit testing . . . 131
11.10 Developer commands . . . 131
11.11 Profiling . . . 131

12 spack package **133**
12.1 Subpackages . . . 133
12.2 Submodules . . . 183
12.3 spack.abi module . . . 183
12.4 spack.architecture module . . . 183
12.5 spack.build_environment module . . . 185
12.6 spack.compiler module . . . 187
12.7 spack.concretize module . . . 188
12.8 spack.config module . . . 189
12.9 spack.database module . . . 192
12.10 spack.directives module . . . 193
12.11 spack.directory_layout module . . . 194
12.12 spack.environment module . . . 197
12.13 spack.error module . . . 199
12.14 spack.fetch_strategy module . . . 199
12.15 spack.file_cache module . . . 203
12.16 spack.graph module . . . 204
12.17 spack.mirror module . . . 205
12.18 spack.modules module . . . 206
12.19 spack.multimethod module . . . 207
12.20 spack.package module . . . 209
12.21 spack.package_test module . . . 218
12.22 spack.parse module . . . 219
12.23 spack.patch module . . . 219
12.24 spack.preferred_packages module . . . 220
12.25 spack.provider_index module . . . 220
12.26 spack.repository module . . . 221
12.27 spack.resource module . . . 224
12.28 spack.spec module . . . 225
12.29 spack.stage module . . . 231
12.30 spack.url module . . . 234
12.31 spack.variant module . . . 236
12.32 spack.version module . . . 236
12.33 spack.yaml_version_check module . . . 238
12.34 Module contents . . . 238

13 Indices and tables **253**

Python Module Index **255**

Spack is a package management tool designed to support multiple versions and configurations of software on a wide variety of platforms and environments. It was designed for large supercomputing centers, where many users and application teams share common installations of software on clusters with exotic architectures, using libraries that do not have a standard ABI. Spack is non-destructive: installing a new version does not break existing installations, so many configurations can coexist on the same system.

Most importantly, Spack is *simple*. It offers a simple *spec* syntax so that users can specify versions and configuration options concisely. Spack is also simple for package authors: package files are written in pure Python, and specs allow package authors to maintain a single file for many different builds of the same package.

See the Feature overview for examples and highlights.

Get spack from the github repository and install your first package:

```
$ git clone https://github.com/llnl/spack.git
$ cd spack/bin
$ ./spack install libelf
```

If you're new to spack and want to start using it, see Getting Started, or refer to the full manual below.

CHAPTER 1

Feature overview

This is a high-level overview of features that make Spack different from other package managers and port systems.

1.1 Simple package installation

Installing the default version of a package is simple. This will install the latest version of the `mpileaks` package and all of its dependencies:

```
$ spack install mpileaks
```

1.2 Custom versions & configurations

Spack allows installation to be customized. Users can specify the version, build compiler, compile-time options, and cross-compile platform, all on the command line.

```
# Install a particular version by appending @
$ spack install mpileaks@1.1.2

# Specify a compiler (and its version), with %
$ spack install mpileaks@1.1.2 %gcc@4.7.3

# Add special compile-time options by name
$ spack install mpileaks@1.1.2 %gcc@4.7.3 debug=True

# Add special boolean compile-time options with +
$ spack install mpileaks@1.1.2 %gcc@4.7.3 +debug

# Add compiler flags using the conventional names
$ spack install mpileaks@1.1.2 %gcc@4.7.3 cppflags=\"-O3 -floop-block\"

# Cross-compile for a different architecture with arch=
$ spack install mpileaks@1.1.2 arch=bgqos_0
```

Users can specify as many or few options as they care about. Spack will fill in the unspecified values with sensible defaults. The two listed syntaxes for variants are identical when the value is boolean.

1.3 Customize dependencies

Spack allows *dependencies* of a particular installation to be customized extensively. Suppose that `mpileaks` depends indirectly on `libelf` and `libdwarf`. Using `^`, users can add custom configurations for the dependencies:

```
# Install mpileaks and link it with specific versions of libelf and libdwarf
$ spack install mpileaks@1.1.2 %gcc@4.7.3 +debug ^libelf@0.8.12 ^libdwarf@20130729+debug
```

1.4 Non-destructive installs

Spack installs every unique package/dependency configuration into its own prefix, so new installs will not break existing ones.

1.5 Packages can peacefully coexist

Spack avoids library misconfiguration by using `RPATH` to link dependencies. When a user links a library or runs a program, it is tied to the dependencies it was built with, so there is no need to manipulate `LD_LIBRARY_PATH` at runtime.

1.6 Creating packages is easy

To create a new packages, all Spack needs is a URL for the source archive. The `spack create` command will create a boilerplate package file, and the package authors can fill in specific build steps in pure Python.

For example, this command:

```
$ spack create http://www.mr511.de/software/libelf-0.8.13.tar.gz
```

creates a simple python file:

```
from spack import *


class Libelf(Package):
    """FIXME: Put a proper description of your package here."""

    # FIXME: Add a proper url for your package's homepage here.
    homepage = "http://www.example.com"
    url      = "http://www.mr511.de/software/libelf-0.8.13.tar.gz"

    version('0.8.13', '4136d7b4c04df68b686570afa26988ac')

    # FIXME: Add dependencies if required.
    # depends_on('foo')

    def install(self, spec, prefix):
        # FIXME: Modify the configure line to suit your build system here.
        configure('--prefix={0}'.format(prefix))

        # FIXME: Add logic to build and install here.
        make()
        make('install')
```

It doesn't take much python coding to get from there to a working package:

```
from spack import *


class Libelf(Package):
    """libelf lets you read, modify or create ELF object files in an
       architecture-independent way. The library takes care of size
       and endian issues, e.g. you can process a file for SPARC
       processors on an Intel-based system."""

    homepage = "http://www.mr511.de/software/english.html"
    url      = "http://www.mr511.de/software/libelf-0.8.13.tar.gz"

    version('0.8.13', '4136d7b4c04df68b686570afa26988ac')
    version('0.8.12', 'e21f8273d9f5f6d43a59878dc274fec7')

    provides('elf')

    def install(self, spec, prefix):
        configure("--prefix=" + prefix,
                  "--enable-shared",
                  "--disable-dependency-tracking",
                  "--disable-debug")
        make()

        # The mkdir commands in libelf's install can fail in parallel
        make("install", parallel=False)
```

Spack also provides wrapper functions around common commands like `configure`, `make`, and `cmake` to make writing packages simple.

CHAPTER 2

Getting Started

2.1 Prerequisites

Spack has the following minimum requirements, which must be installed before Spack is run:

1. Python 2.6 or 2.7
2. A C/C++ compiler
3. The `git` and `curl` commands.

These requirements can be easily installed on most modern Linux systems; on Macintosh, XCode is required. Spack is designed to run on HPC platforms like Cray and BlueGene/Q. Not all packages should be expected to work on all platforms. A build matrix showing which packages are working on which systems is planned but not yet available.

2.2 Installation

Getting Spack is easy. You can clone it from the github repository using this command:

```
$ git clone https://github.com/llnl/spack.git
```

This will create a directory called `spack`.

2.2.1 Add Spack to the Shell

We'll assume that the full path to your downloaded Spack directory is in the `SPACK_ROOT` environment variable. Add `$SPACK_ROOT/bin` to your path and you're ready to go:

```
$ export PATH=$SPACK_ROOT/bin:$PATH
$ spack install libelf
```

For a richer experience, use Spack's shell support:

```
# For bash users
$ export SPACK_ROOT=/path/to/spack
$ . $SPACK_ROOT/share/spack/setup-env.sh

# For tcsh or csh users (note you must set SPACK_ROOT)
$ setenv SPACK_ROOT /path/to/spack
$ source $SPACK_ROOT/share/spack/setup-env.csh
```

This automatically adds Spack to your `PATH`.

2.2.2 Clean Environment

Many packages' installs can be broken by changing environment variables. For example, a package might pick up the wrong build-time dependencies (most of them not specified) depending on the setting of `PATH`. `GCC` seems to be particularly vulnerable to these issues.

Therefore, it is recommended that Spack users run with a *clean environment*, especially for `PATH`. Only software that comes with the system, or that you know you wish to use with Spack, should be included. This procedure will avoid many strange build errors.

2.2.3 Check Installation

With Spack installed, you should be able to run some basic Spack commands. For example:

```
$ spack spec netcdf
  ...
  netcdf@4.4.1%gcc@5.3.0~hdf4+mpi arch=linux-SuSE11-x86_64
      ^curl@7.50.1%gcc@5.3.0 arch=linux-SuSE11-x86_64
          ^openssl@system%gcc@5.3.0 arch=linux-SuSE11-x86_64
          ^zlib@1.2.8%gcc@5.3.0 arch=linux-SuSE11-x86_64
      ^hdf5@1.10.0-patch1%gcc@5.3.0+cxx~debug+fortran+mpi+shared~szip~threadsafe arch=linux-SuSE11-x8
          ^openmpi@1.10.1%gcc@5.3.0~mxm~pmi~psm~psm2~slurm~sqlite3~thread_multiple~tm+verbs+vt arch=l
      ^m4@1.4.17%gcc@5.3.0+sigsegv arch=linux-SuSE11-x86_64
          ^libsigsegv@2.10%gcc@5.3.0 arch=linux-SuSE11-x86_64
```

2.2.4 Optional: Alternate Prefix

You may want to run Spack out of a prefix other than the git repository you cloned. The `spack bootstrap` command provides this functionality. To install spack in a new directory, simply type:

```
$ spack bootstrap /my/favorite/prefix
```

This will install a new spack script in `/my/favorite/prefix/bin`, which you can use just like you would the regular spack script. Each copy of spack installs packages into its own `$PREFIX/opt` directory.

2.2.5 Next Steps

In theory, Spack doesn't need any additional installation; just download and run! But in real life, additional steps are usually required before Spack can work in a practical sense. Read on...

2.3 Compiler configuration

Spack has the ability to build packages with multiple compilers and compiler versions. Spack searches for compilers on your machine automatically the first time it is run. It does this by inspecting your `PATH`.

2.3.1 spack compilers

You can see which compilers spack has found by running `spack compilers` or `spack compiler list`:

```
$ spack compilers
==> Available compilers
-- gcc ----------------------------------------------------------
    gcc@4.9.0  gcc@4.8.0  gcc@4.7.0  gcc@4.6.2  gcc@4.4.7
    gcc@4.8.2  gcc@4.7.1  gcc@4.6.3  gcc@4.6.1  gcc@4.1.2
-- intel --------------------------------------------------------
    intel@15.0.0  intel@14.0.0  intel@13.0.0  intel@12.1.0  intel@10.0
    intel@14.0.3  intel@13.1.1  intel@12.1.5  intel@12.0.4  intel@9.1
    intel@14.0.2  intel@13.1.0  intel@12.1.3  intel@11.1
    intel@14.0.1  intel@13.0.1  intel@12.1.2  intel@10.1
-- clang --------------------------------------------------------
    clang@3.4  clang@3.3  clang@3.2  clang@3.1
-- pgi ----------------------------------------------------------
    pgi@14.3-0   pgi@13.2-0  pgi@12.1-0   pgi@10.9-0  pgi@8.0-1
    pgi@13.10-0  pgi@13.1-1  pgi@11.10-0  pgi@10.2-0  pgi@7.1-3
    pgi@13.6-0   pgi@12.8-0  pgi@11.1-0   pgi@9.0-4   pgi@7.0-6
```

Any of these compilers can be used to build Spack packages. More on how this is done is in *Specs & dependencies*.

2.3.2 spack compiler add

An alias for `spack compiler find`.

2.3.3 spack compiler find

If you do not see a compiler in this list, but you want to use it with Spack, you can simply run `spack compiler find` with the path to where the compiler is installed. For example:

```
$ spack compiler find /usr/local/tools/ic-13.0.079
==> Added 1 new compiler to /Users/gamblin2/.spack/compilers.yaml
    intel@13.0.079
```

Or you can run `spack compiler find` with no arguments to force auto-detection. This is useful if you do not know where compilers are installed, but you know that new compilers have been added to your `PATH`. For example, you might load a module, like this:

```
$ module load gcc-4.9.0
$ spack compiler find
==> Added 1 new compiler to /Users/gamblin2/.spack/compilers.yaml
    gcc@4.9.0
```

This loads the environment module for gcc-4.9.0 to add it to `PATH`, and then it adds the compiler to Spack.

2.3.4 spack compiler info

If you want to see specifics on a particular compiler, you can run `spack compiler info` on it:

```
$ spack compiler info intel@15
intel@15.0.0:
        cc  = /usr/local/bin/icc-15.0.090
        cxx = /usr/local/bin/icpc-15.0.090
```

```
        f77 = /usr/local/bin/ifort-15.0.090
        fc  = /usr/local/bin/ifort-15.0.090
        modules  = []
        operating system  = centos6
```

This shows which C, C++, and Fortran compilers were detected by Spack. Notice also that we didn't have to be too specific about the version. We just said `intel@15`, and information about the only matching Intel compiler was displayed.

2.3.5 Manual compiler configuration

If auto-detection fails, you can manually configure a compiler by editing your `~/.spack/compilers.yaml` file. You can do this by running `spack config edit compilers`, which will open the file in your `$EDITOR`.

Each compiler configuration in the file looks like this:

```
compilers:
- compiler:
    modules = []
    operating_system: centos6
    paths:
      cc: /usr/local/bin/icc-15.0.024-beta
      cxx: /usr/local/bin/icpc-15.0.024-beta
      f77: /usr/local/bin/ifort-15.0.024-beta
      fc: /usr/local/bin/ifort-15.0.024-beta
    spec: intel@15.0.0:
```

For compilers, like `clang`, that do not support Fortran, put `None` for `f77` and `fc`:

```
paths:
  cc: /usr/bin/clang
  cxx: /usr/bin/clang++
  f77: None
  fc: None
spec: clang@3.3svn:
```

Once you save the file, the configured compilers will show up in the list displayed by `spack compilers`.

You can also add compiler flags to manually configured compilers. The valid flags are `cflags`, `cxxflags`, `fflags`, `cppflags`, `ldflags`, and `ldlibs`. For example:

```
compilers:
- compiler:
    modules = []
    operating_system: OS
    paths:
      cc: /usr/local/bin/icc-15.0.024-beta
      cxx: /usr/local/bin/icpc-15.0.024-beta
      f77: /usr/local/bin/ifort-15.0.024-beta
      fc: /usr/local/bin/ifort-15.0.024-beta
    parameters:
      cppflags: -O3 -fPIC
    spec: intel@15.0.0:
```

These flags will be treated by spack as if they were entered from the command line each time this compiler is used. The compiler wrappers then inject those flags into the compiler command. Compiler flags entered from the command line will be discussed in more detail in the following section.

2.3.6 Build Your Own Compiler

If you are particular about which compiler/version you use, you might wish to have Spack build it for you. For example:

```
$ spack install gcc@4.9.3
```

Once that has finished, you will need to add it to your `compilers.yaml` file. You can then set Spack to use it by default by adding the following to your `packages.yaml` file:

```
packages:
  all:
    compiler: [gcc@4.9.3]
```

Note: If you are building your own compiler, some users prefer to have a Spack instance just for that. For example, create a new Spack in `~/spack-tools` and then run `~/spack-tools/bin/spack install gcc@4.9.3`. Once the compiler is built, don't build anything more in that Spack instance; instead, create a new "real" Spack instance, configure Spack to use the compiler you've just built, and then build your application software in the new Spack instance. Following this tip makes it easy to delete all your Spack packages *except* the compiler.

2.3.7 Compilers Requiring Modules

Many installed compilers will work regardless of the environment they are called with. However, some installed compilers require `$LD_LIBRARY_PATH` or other environment variables to be set in order to run; this is typical for Intel and other proprietary compilers.

In such a case, you should tell Spack which module(s) to load in order to run the chosen compiler (If the compiler does not come with a module file, you might consider making one by hand). Spack will load this module into the environment ONLY when the compiler is run, and NOT in general for a package's `install()` method. See, for example, this `compilers.yaml` file:

```
compilers:
- compiler:
    modules: [other/comp/gcc-5.3-sp3]
    operating_system: SuSE11
    paths:
      cc: /usr/local/other/SLES11.3/gcc/5.3.0/bin/gcc
      cxx: /usr/local/other/SLES11.3/gcc/5.3.0/bin/g++
      f77: /usr/local/other/SLES11.3/gcc/5.3.0/bin/gfortran
      fc: /usr/local/other/SLES11.3/gcc/5.3.0/bin/gfortran
    spec: gcc@5.3.0
```

Some compilers require special environment settings to be loaded not just to run, but also to execute the code they build, breaking packages that need to execute code they just compiled. If it's not possible or practical to use a better compiler, you'll need to ensure that environment settings are preserved for compilers like this (i.e., you'll need to load the module or source the compiler's shell script).

By default, Spack tries to ensure that builds are reproducible by cleaning the environment before building. If this interferes with your compiler settings, you CAN use `spack install --dirty` as a workaround. Note that this MAY interfere with package builds.

2.3.8 Licensed Compilers

Some proprietary compilers require licensing to use. If you need to use a licensed compiler (eg, PGI), the process is similar to a mix of build your own, plus modules:

1. Create a Spack package (if it doesn't exist already) to install your compiler. Follow instructions on installing *Licensed software*.
2. Once the compiler is installed, you should be able to test it by using Spack to load the module it just created, and running simple builds (eg: `cc helloWorld.c; ./a.out`)
3. Add the newly-installed compiler to `compilers.yaml` as shown above.

2.3.9 Mixed Toolchains

Modern compilers typically come with related compilers for C, C++ and Fortran bundled together. When possible, results are best if the same compiler is used for all languages.

In some cases, this is not possible. For example, starting with macOS El Capitan (10.11), many packages no longer build with GCC, but XCode provides no Fortran compilers. The user is therefore forced to use a mixed toolchain: XCode-provided Clang for C/C++ and GNU `gfortran` for Fortran.

In the simplest case, you can just edit `compilers.yaml`:

```
compilers:
  darwin-x86_64:
    clang@7.3.0-apple:
      cc: /usr/bin/clang
      cxx: /usr/bin/clang++
      f77: /path/to/bin/gfortran
      fc: /path/to/bin/gfortran
```

Note: If you are building packages that are sensitive to the compiler's name, you may also need to slightly modify a few more files so that Spack uses compiler names the build system will recognize.

Following are instructions on how to hack together `clang` and `gfortran` on Macintosh OS X. A similar approach should work for other mixed toolchain needs.

Better support for mixed compiler toolchains is planned in forthcoming Spack versions.

1. Create a symlink inside `clang` environment:

```
$ cd $SPACK_ROOT/lib/spack/env/clang
$ ln -s ../cc gfortran
```

2. Patch `clang` compiler file:

```
$ diff --git a/lib/spack/spack/compilers/clang.py b/lib/spack/spack/compilers/clang.py
index e406d86..cf8fd01 100644
--- a/lib/spack/spack/compilers/clang.py
+++ b/lib/spack/spack/compilers/clang.py
@@ -35,17 +35,17 @@ class Clang(Compiler):
     cxx_names = ['clang++']

     # Subclasses use possible names of Fortran 77 compiler
-    f77_names = []
+    f77_names = ['gfortran']

     # Subclasses use possible names of Fortran 90 compiler
```

```
-    fc_names = []
+    fc_names = ['gfortran']

     # Named wrapper links within spack.build_env_path
     link_paths = { 'cc'  : 'clang/clang',
                    'cxx' : 'clang/clang++',
                    # Use default wrappers for fortran, in case provided in compilers.yaml
-                   'f77' : 'f77',
-                   'fc'  : 'f90' }
+                   'f77' : 'clang/gfortran',
+                   'fc'  : 'clang/gfortran' }

     @classmethod
     def default_version(self, comp):
```

2.3.10 Compiler Verification

You can verify that your compilers are configured properly by installing a simple package. For example:

```
$ spack install zlib%gcc@5.3.0
```

2.4 Vendor-Specific Compiler Configuration

With Spack, things usually "just work" with GCC. Not so for other compilers. This section provides details on how to get specific compilers working.

2.4.1 Intel Compilers

Intel compilers are unusual because a single Intel compiler version can emulate multiple GCC versions. In order to provide this functionality, the Intel compiler needs GCC to be installed. Therefore, the following steps are necessary to successfully use Intel compilers:

1. Install a version of GCC that implements the desired language features (`spack install gcc`).
2. Tell the Intel compiler how to find that desired GCC. This may be done in one of two ways: (text taken from Intel Reference Guide):

   > By default, the compiler determines which version of `gcc` or `g++` > you have installed from the `PATH` environment variable. > > If you want use a version of `gcc` or `g++` other than the default > version on your system, you need to use either the `-gcc-name` > or `-gxx-name` compiler option to specify the path to the version of > `gcc` or `g++` that you want to use.

Intel compilers may therefore be configured in one of two ways with Spack: using modules, or using compiler flags.

Configuration with Modules

One can control which GCC is seen by the Intel compiler with modules. A module must be loaded both for the Intel Compiler (so it will run) and GCC (so the compiler can find the intended GCC). The following configuration in `compilers.yaml` illustrates this technique:

```
compilers:
- compiler:
    modules = [gcc-4.9.3, intel-15.0.24]
    operating_system: centos7
    paths:
      cc: /opt/intel-15.0.24/bin/icc-15.0.24-beta
      cxx: /opt/intel-15.0.24/bin/icpc-15.0.24-beta
      f77: /opt/intel-15.0.24/bin/ifort-15.0.24-beta
      fc: /opt/intel-15.0.24/bin/ifort-15.0.24-beta
    spec: intel@15.0.24.4.9.3
```

Note: The version number on the Intel compiler is a combination of the "native" Intel version number and the GNU compiler it is targeting.

Command Line Configuration

. warning:

```
As of the writing of this manual, added compilers flags are broken;
see `GitHub Issue <https://github.com/LLNL/spack/pull/1532>`_.
```

One can also control which GCC is seen by the Intel compiler by adding flags to the `icc` command:

1. Identify the location of the compiler you just installed:

```
$ spack location -i gcc
/home2/rpfische/spack2/opt/spack/linux-centos7-x86_64/gcc-4.9.3-iy4rw...
```

2. Set up `compilers.yaml`, for example:

```
compilers:
- compiler:
    modules = [intel-15.0.24]
    operating_system: centos7
    paths:
      cc: /opt/intel-15.0.24/bin/icc-15.0.24-beta
      cflags: -gcc-name /home2/rpfische/spack2/opt/spack/linux-centos7-x86_64/gcc-4.9.3-iy4rw...
      cxx: /opt/intel-15.0.24/bin/icpc-15.0.24-beta
      cxxflags: -gxx-name /home2/rpfische/spack2/opt/spack/linux-centos7-x86_64/gcc-4.9.3-iy4rw.
      f77: /opt/intel-15.0.24/bin/ifort-15.0.24-beta
      fc: /opt/intel-15.0.24/bin/ifort-15.0.24-beta
      fflags: -gcc-name /home2/rpfische/spack2/opt/spack/linux-centos7-x86_64/gcc-4.9.3-iy4rw...
    spec: intel@15.0.24.4.9.3
```

2.4.2 PGI

PGI comes with two sets of compilers for C++ and Fortran, distinguishable by their names. "Old" compilers:

```
cc:  /soft/pgi/15.10/linux86-64/15.10/bin/pgcc
cxx: /soft/pgi/15.10/linux86-64/15.10/bin/pgCC
f77: /soft/pgi/15.10/linux86-64/15.10/bin/pgf77
fc:  /soft/pgi/15.10/linux86-64/15.10/bin/pgf90
```

"New" compilers:

```
cc:  /soft/pgi/15.10/linux86-64/15.10/bin/pgcc
cxx: /soft/pgi/15.10/linux86-64/15.10/bin/pgc++
f77: /soft/pgi/15.10/linux86-64/15.10/bin/pgfortran
fc:  /soft/pgi/15.10/linux86-64/15.10/bin/pgfortran
```

Older installations of PGI contains just the old compilers; whereas newer installations contain the old and the new. The new compiler is considered preferable, as some packages (`hdf4`) will not build with the old compiler.

When auto-detecting a PGI compiler, there are cases where Spack will find the old compilers, when you really want it to find the new compilers. It is best to check this `compilers.yaml`; and if the old compilers are being used, change `pgf77` and `pgf90` to `pgfortran`.

Other issues:

- There are reports that some packages will not build with PGI, including `libpciaccess` and `openssl`. A workaround is to build these packages with another compiler and then use them as dependencies for PGI-build packages. For example:

```
$ spack install openmpi%pgi ^libpciaccess%gcc
```

- PGI requires a license to use; see *Licensed Compilers* for more information on installation.

Note: It is believed the problem with `hdf4` is that everything is compiled with the `F77` compiler, but at some point some Fortran 90 code slipped in there. So compilers that can handle both FORTRAN 77 and Fortran 90 (`gfortran`, `pgfortran`, etc) are fine. But compilers specific to one or the other (`pgf77`, `pgf90`) won't work.

2.4.3 NAG

At this point, the NAG compiler is *known to not work<https://github.com/LLNL/spack/issues/590>*.

2.5 System Packages

Once compilers are configured, one needs to determine which pre-installed system packages, if any, to use in builds. This is configured in the file *~/.spack/packages.yaml*. For example, to use an OpenMPI installed in /opt/local, one would use:

```
packages:
    openmpi:
        paths:
            openmpi@1.10.1: /opt/local
        buildable: False
```

In general, Spack is easier to use and more reliable if it builds all of its own dependencies. However, there are two packages for which one commonly needs to use system versions:

2.5.1 MPI

On supercomputers, sysadmins have already built MPI versions that take into account the specifics of that computer's hardware. Unless you know how they were built and can choose the correct Spack variants, you are unlikely to get a working MPI from Spack. Instead, use an appropriate pre-installed MPI.

If you choose a pre-installed MPI, you should consider using the pre-installed compiler used to build that MPI; see above on `compilers.yaml`.

2.5.2 OpenSSL

The `openssl` package underlies much of modern security in a modern OS; an attacker can easily "pwn" any computer on which they can modify SSL. Therefore, any `openssl` used on a system should be created in a "trusted environment" — for example, that of the OS vendor.

OpenSSL is also updated by the OS vendor from time to time, in response to security problems discovered in the wider community. It is in everyone's best interest to use any newly updated versions as soon as they come out. Modern Linux installations have standard procedures for security updates without user involvement.

Spack running at user-level is not a trusted environment, nor do Spack users generally keep up-to-date on the latest security holes in SSL. For these reasons, a Spack-installed OpenSSL should likely not be trusted.

As long as the system-provided SSL works, you can use it instead. One can check if it works by trying to download an `https://`. For example:

```
$ curl -O https://github.com/ImageMagick/ImageMagick/archive/7.0.2-7.tar.gz
```

The recommended way to tell Spack to use the system-supplied OpenSSL is to add the following to `packages.yaml`. Note that the `@system` "version" means "I don't care what version it is, just use what is there." This is reasonable for OpenSSL, which has a stable API.

```
packages:
    openssl:
        paths:
            openssl@system: /false/path
        version: [system]
        buildable: False
```

Note: Even though OpenSSL is located in `/usr`, We have told Spack to look for it in `/false/path`. This prevents `/usr` from being added to compilation paths and RPATHs, where it could cause unrelated system libraries to be used instead of their Spack equivalents.

The adding of `/usr` to `RPATH` in this sitution is a known issue and will be fixed in a future release.

2.5.3 Git

Some Spack packages use `git` to download, which might not work on some computers. For example, the following error was encountered on a Macintosn during `spack install julia-master`:

```
==> Trying to clone git repository:
  https://github.com/JuliaLang/julia.git
  on branch master
Cloning into 'julia'...
fatal: unable to access 'https://github.com/JuliaLang/julia.git/':
    SSL certificate problem: unable to get local issuer certificate
```

This problem is related to OpenSSL, and in some cases might be solved by installing a new version of `git` and `openssl`:

1. Run `spack install git`
2. Add the output of `spack module loads git` to your `.bahsrc`.

If this doesn't work, it is also possible to disable checking of SSL certificates by using either of:

```
$ spack -k install
$ spack --insecure install
```

Using `-k`/`--insecure` makes Spack disable SSL checking when fetching from websites and from git.

Warning: This workaround should be used ONLY as a last resort! Wihout SSL certificate verification, spack and git will download from sites you wouldn't normally trust. The code you download and run may then be compromised! While this is not a major issue for archives that will be checksummed, it is especially problematic when downloading from name Git branches or tags, which relies entirely on trusting a certificate for security (no verification).

certificate for security (no verification).

2.6 Utilities Configuration

Although Spack does not need installation *per se*, it does rely on other packages to be available on its host system. If those packages are out of date or missing, then Spack will not work. Sometimes, an appeal to the system's package manager can fix such problems. If not, the solution is have Spack install the required packages, and then have Spack use them.

For example, if *curl* doesn't work, one could use the following steps to provide Spack a working *curl*:

```
$ spack install curl
$ spack load curl
```

or alternately:

```
$ spack module loads curl >>~/.bashrc
```

or if environment modules don't work:

```
$ export PATH=`spack location -i curl`/bin:$PATH
```

External commands are used by Spack in two places: within core Spack, and in the package recipes. The bootstrapping procedure for these two cases is somewhat different, and is treated separately below.

2.6.1 Core Spack Utilities

Core Spack uses the following packages, mainly to download and unpack source code, and to load generated environment modules: `curl`, `env`, `git`, `go`, `hg`, `svn`, `tar`, `unzip`, `patch`, `environment-modules`.

As long as the user's environment is set up to successfully run these programs from outside of Spack, they should work inside of Spack as well. They can generally be activated as in the *curl* example above; or some systems might already have an appropriate hand-built environment module that may be loaded. Either way works.

A few notes on specific programs in this list:

cURL, git, Mercurial, etc.

Spack depends on cURL to download tarballs, the format that most Spack-installed packages come in. Your system's cURL should always be able to download unencrypted `http://`. However, the cURL on some systems has problems with SSL-enabled `https://` URLs, due to outdated / insecure versions of OpenSSL on those systems. This will prevent Spack from installing any software requiring `https://` until a new cURL has been installed, using the technique above.

Warning: remember that if you install `curl` via Spack that it may rely on a user-space OpenSSL that is not upgraded regularly. It may fall out of date faster than your system OpenSSL.

Some packages use source code control systems as their download method: `git`, `hg`, `svn` and occasionally `go`. If you had to install a new `curl`, then chances are the system-supplied version of these other programs will also not work, because they also rely on OpenSSL. Once `curl` has been installed, you can similarly install the others.

Environment Modules

In order to use Spack's generated environment modules, you must have installed one of *Environment Modules* or *Lmod*. On many Linux distributions, this can be installed from the vendor's repository. For example: `yum install environment-modules` (Fedora/RHEL/CentOS). If your Linux distribution does not have Environment Modules, you can get it with Spack:

1. Consider using system tcl (as long as your system has Tcl version 8.0 or later): #) Identify its location using `which tclsh` #) Identify its version using `echo 'puts $tcl_version;exit 0' | tclsh` #) Add to `~/.spack/packages.yaml` and modify as appropriate:

```
    packages:
        tcl:
            paths:
                tcl@8.5: /usr
            version: [8.5]
            buildable: False
```

2. Install with:

```
$ spack install environment-modules
```

3. Activate with the following script (or apply the updates to your `.bashrc` file manually):

```
TMP=`tempfile`
echo >$TMP
MODULE_HOME=`spack location -i environment-modules`
MODULE_VERSION=`ls -1 $MODULE_HOME/Modules | head -1`
${MODULE_HOME}/Modules/${MODULE_VERSION}/bin/add.modules <$TMP
cp .bashrc $TMP
echo "MODULE_VERSION=${MODULE_VERSION}" > .bashrc
cat $TMP >>.bashrc
```

This adds to your `.bashrc` (or similar) files, enabling Environment Modules when you log in. Re-load your .bashrc (or log out and in again), and then test that the `module` command is found with:

```
$ module avail
```

2.6.2 Package Utilities

Spack may also encounter bootstrapping problems inside a package's `install()` method. In this case, Spack will normally be running inside a *sanitized build environment*. This includes all of the package's dependencies, but none of the environment Spack inherited from the user: if you load a module or modify `$PATH` before launching Spack, it will have no effect.

In this case, you will likely need to use the `--dirty` flag when running `spack install`, causing Spack to **not** sanitize the build environment. You are now responsible for making sure that environment does not do strange things to Spack or its installs.

Another way to get Spack to use its own version of something is to add that something to a package that needs it. For example:

```
depends_on('binutils', type='build')
```

This is considered best practice for some common build dependencies, such as `autotools` (if the `autoreconf` command is needed) and `cmake` — `cmake` especially, because different packages require a different version of CMake.

binutils

Sometimes, strange error messages can happen while building a package. For example, `ld` might crash. Or one receives a message like:

```
ld: final link failed: Nonrepresentable section on output
```

or:

```
ld: .../_fftpackmodule.o: unrecognized relocation (0x2a) in section `.text'
```

These problems are often caused by an outdated `binutils` on your system. Unlike CMake or Autotools, adding `depends_on('binutils')` to every package is not considered a best practice because every package written in C/C++/Fortran would need it. A potential workaround is to load a recent `binutils` into your environment and use the `--dirty` flag.

2.7 Spack on Cray

Spack differs slightly when used on a Cray system. The architecture spec can differentiate between the front-end and back-end processor and operating system. For example, on Edison at NERSC, the back-end target processor is "Ivy Bridge", so you can specify to use the back-end this way:

```
$ spack install zlib target=ivybridge
```

You can also use the operating system to build against the back-end:

```
$ spack install zlib os=CNL10
```

Notice that the name includes both the operating system name and the major version number concatenated together.

Alternatively, if you want to build something for the front-end, you can specify the front-end target processor. The processor for a login node on Edison is "Sandy bridge" so we specify on the command line like so:

```
$ spack install zlib target=sandybridge
```

And the front-end operating system is:

```
$ spack install zlib os=SuSE11
```

2.7.1 Cray compiler detection

Spack can detect compilers using two methods. For the front-end, we treat everything the same. The difference lies in back-end compiler detection. Back-end compiler detection is made via the Tcl module avail command. Once it detects the compiler it writes the appropriate PrgEnv and compiler module name to compilers.yaml and sets the paths to each compiler with Cray's compiler wrapper names (i.e. cc, CC, ftn). During build time, Spack will load the correct PrgEnv and compiler module and will call appropriate wrapper.

The compilers.yaml config file will also differ. There is a modules section that is filled with the compiler's Programming Environment and module name. On other systems, this field is empty []:

```
- compiler:
    modules:
      - PrgEnv-intel
      - intel/15.0.109
```

As mentioned earlier, the compiler paths will look different on a Cray system. Since most compilers are invoked using cc, CC and ftn, the paths for each compiler are replaced with their respective Cray compiler wrapper names:

```
paths:
  cc: cc
  cxx: CC
  f77: ftn
  fc: ftn
```

As opposed to an explicit path to the compiler executable. This allows Spack to call the Cray compiler wrappers during build time.

For more on compiler configuration, check out *Compiler configuration*.

Spack sets the default Cray link type to dynamic, to better match other other platforms. Individual packages can enable static linking (which is the default outside of Spack on cray systems) using the `-static` flag.

2.7.2 Setting defaults and using Cray modules

If you want to use default compilers for each PrgEnv and also be able to load cray external modules, you will need to set up a `packages.yaml`.

Here's an example of an external configuration for cray modules:

```
packages:
  mpi:
    modules:
      mpich@7.3.1%gcc@5.2.0 arch=cray_xc-haswell-CNL10: cray-mpich
      mpich@7.3.1%intel@16.0.0.109 arch=cray_xc-haswell-CNL10: cray-mpich
```

This tells Spack that for whatever package that depends on mpi, load the cray-mpich module into the environment. You can then be able to use whatever environment variables, libraries, etc, that are brought into the environment via module load.

You can set the default compiler that Spack can use for each compiler type. If you want to use the Cray defaults, then set them under `all:` in packages.yaml. In the compiler field, set the compiler specs in your order of preference. Whenever you build with that compiler type, Spack will concretize to that version.

Here is an example of a full packages.yaml used at NERSC

```
packages:
  mpi:
    modules:
      mpich@7.3.1%gcc@5.2.0 arch=cray_xc-CNL10-ivybridge: cray-mpich
      mpich@7.3.1%intel@16.0.0.109 arch=cray_xc-SuSE11-ivybridge: cray-mpich
    buildable: False
  netcdf:
    modules:
      netcdf@4.3.3.1%gcc@5.2.0 arch=cray_xc-CNL10-ivybridge: cray-netcdf
      netcdf@4.3.3.1%intel@16.0.0.109 arch=cray_xc-CNL10-ivybridge: cray-netcdf
    buildable: False
  hdf5:
```

```
    modules:
      hdf5@1.8.14%gcc@5.2.0 arch=cray_xc-CNL10-ivybridge: cray-hdf5
      hdf5@1.8.14%intel@16.0.0.109 arch=cray_xc-CNL10-ivybridge: cray-hdf5
    buildable: False
  all:
    compiler: [gcc@5.2.0, intel@16.0.0.109]
```

Here we tell spack that whenever we want to build with gcc use version 5.2.0 or if we want to build with intel compilers, use version 16.0.0.109. We add a spec for each compiler type for each cray modules. This ensures that for each compiler on our system we can use that external module.

For more on external packages check out the section *External Packages*.

CHAPTER 3

Basic usage

The `spack` command has many *subcommands*. You'll only need a small subset of them for typical usage.

Note that Spack colorizes output. `less -R` should be used with Spack to maintain this colorization. E.g.:

```
$ spack find | less -R
```

It is recommend that the following be put in your `.bashrc` file:

```
alias less='less -R'
```

3.1 Listing available packages

To install software with Spack, you need to know what software is available. You can see a list of available package names at the package-list webpage, or using the `spack list` command.

3.1.1 `spack list`

The `spack list` command prints out a list of all of the packages Spack can install:

```
$ spack list
ack                  go                       muster               py-prettytable       r-pkgmaker
activeharmony        go-bootstrap             mvapich2             py-proj              r-plotrix
adept-utils          gobject-introspection    mxml                 py-protobuf          r-plyr
adios                googletest               nag                  py-py2cairo          r-png
adol-c               gperf                    nano                 py-py2neo            r-praise
allinea-forge        gperftools               nasm                 py-pychecker         r-proto
allinea-reports      grackle                  nauty                py-pycparser         r-quantmod
antlr                graphlib                 nccmp                py-pydatalog         r-quantreg
ape                  graphviz                 ncdu                 py-pyelftools        r-R6
apex                 grib-api                 nco                  py-pygments          r-randomforest
apr                  gromacs                  ncurses              py-pygobject         r-raster
apr-util             gsl                      ncview               py-pygtk             r-rcolorbrewer
armadillo            gtkplus                  ndiff                py-pylint            r-rcpp
arpack               gts                      netcdf               py-pypar             r-rcppeigen
arpack-ng            guile                    netcdf-cxx           py-pyparsing         r-registry
asciidoc             h5hut                    netcdf-cxx4          py-pyqt              r-reshape2
astyle               harfbuzz                 netcdf-fortran       py-pyside            r-rgooglemaps
atk                  harminv                  netgauge             py-pytables          r-rjava
atlas                hdf                      netlib-lapack        py-python-daemon     r-rjson
atop                 hdf5                     netlib-scalapack     py-pytz              r-rjsonio
```

autoconf	hdf5-blosc	nettle	py-pyyaml	r-rmysql
automaded	hepmc	nextflow	py-restview	r-rngtools
automake	heppdt	ninja	py-rpy2	r-rodbc
bamtools	hmmer	numdiff	py-scientificpython	r-roxygen2
bash	hoomd-blue	nwchem	py-scikit-image	r-rpostgresql
bbcp	hpl	ocaml	py-scikit-learn	r-rsqlite
bcftools	hpx5	oce	py-scipy	r-rstan
bdw-gc	htslib	octave	py-setuptools	r-rstudioapi
bear	hub	octave-splines	py-shiboken	r-sandwich
bedtools2	hwloc	octopus	py-sip	r-scales
bertini	hydra	ompss	py-six	r-shiny
bib2xhtml	hypre	ompt-openmp	py-sncosmo	r-sp
binutils	ibmisc	opari2	py-snowballstemmer	r-sparsem
bison	icu4c	openblas	py-sphinx	r-stanheaders
bliss	ilmbase	opencoarrays	py-sphinx-rtd-theme	r-stringi
blitz	ImageMagick	opencv	py-SQLAlchemy	r-stringr
boost	intel	openexr	py-storm	r-survey
bowtie2	intel-parallel-studio	openjpeg	py-sympy	r-survival
boxlib	intltool	openmpi	py-tappy	r-tarifx
bpp-core	ior	openspeedshop	py-tuiview	r-testthat
bpp-phyl	ipopt	openssl	py-twisted	r-thdata
bpp-seq	ipp	opium	py-unittest2	r-threejs
bpp-suite	isl	osu-micro-benchmarks	py-unittest2py3k	r-tibble
bwa	itstool	otf	py-urwid	r-tidyr
bzip2	jasper	otf2	py-virtualenv	r-ttr
c-blosc	jdk	p4est	py-wcsaxes	r-vcd
cairo	jemalloc	panda	py-wheel	r-visnetwork
caliper	jpeg	pango	py-xlrd	r-whisker
callpath	jsoncpp	papi	py-yapf	r-withr
cantera	judy	paradiseo	py-yt	r-xlconnect
cask	julia	parallel	python	r-xlconnectjar
cblas	kdiff3	parallel-netcdf	qhull	r-xlsx
cbtf	kealib	paraver	qrupdate	r-xlsxjars
cbtf-argonavis	kripke	paraview	qt	r-xml
cbtf-krell	launchmon	parmetis	qt-creator	r-xtable
cbtf-lanl	lcms	parmgridgen	qthreads	r-xts
cdd	leveldb	parpack	R	r-yaml
cddlib	libaio	patchelf	r-abind	r-zoo
cdo	libarchive	pcre	r-assertthat	raja
cereal	libatomic-ops	pcre2	r-base64enc	ravel
cfitsio	libcerf	pdt	r-bh	readline
cgal	libcircle	perl	r-BiocGenerics	root
cgm	libctl	petsc	r-boot	rose
cgns	libdrm	pexsi	r-brew	rsync
charm	libdwarf	pgi	r-car	ruby
cityhash	libedit	pidx	r-caret	rust
cleverleaf	libelf	pixman	r-chron	rust-bindgen
clhep	libepoxy	pkg-config	r-class	SAMRAI
cloog	libevent	plumed	r-cluster	samtools
cmake	libffi	pmgr_collective	r-codetools	sbt
cmocka	libgcrypt	pngwriter	r-colorspace	scalasca
cnmem	libgd	polymake	r-crayon	scons
coreutils	libgpg-error	porta	r-cubature	scorep
cp2k	libgtextutils	postgresql	r-curl	scotch
cppcheck	libhio	ppl	r-datatable	scr
cram	libiconv	prank	r-dbi	screen
cryptopp	libjpeg-turbo	proj	r-devtools	sed
cscope	libjson-c	protobuf	r-diagrammer	seqtk

cube	libmesh	psi4	r-dichromat	serf
cuda	libmng	py-3to2	r-digest	silo
curl	libmonitor	py-alabaster	r-doparallel	slepc
czmq	libNBC	py-argcomplete	r-dplyr	snappy
daal	libpciaccess	py-astroid	r-dt	sparsehash
dakota	libpng	py-astropy	r-dygraphs	spindle
damselfly	libpthread-stubs	py-autopep8	r-e1071	spot
datamash	libsigsegv	py-babel	r-filehash	sqlite
dbus	libsodium	py-basemap	r-foreach	stat
dealii	libsplash	py-beautifulsoup4	r-foreign	stream
dia	libtermkey	py-biopython	r-gdata	subversion
docbook-xml	libtiff	py-blessings	r-geosphere	suite-sparse
doxygen	libtool	py-bottleneck	r-ggmap	sundials
dri2proto	libunistring	py-cclib	r-ggplot2	superlu
dtcmp	libunwind	py-cffi	r-ggvis	superlu-dist
dyninst	libuuid	py-coverage	r-git2r	superlu-mt
eigen	libuv	py-csvkit	r-glmnet	swiftsim
elfutils	libvterm	py-cycler	r-googlevis	swig
elk	libxau	py-cython	r-gridbase	sympol
elpa	libxc	py-dask	r-gridextra	szip
emacs	libxcb	py-dateutil	r-gtable	tar
environment-modules	libxml2	py-dbf	r-gtools	task
espresso	libxpm	py-decorator	r-htmltools	taskd
exodusii	libxshmfence	py-docutils	r-htmlwidgets	tau
exonerate	libxslt	py-emcee	r-httpuv	tbb
expat	libxsmm	py-epydoc	r-httr	tcl
extrae	llvm	py-flake8	r-igraph	tetgen
exuberant-ctags	llvm-lld	py-funcsigs	r-influencer	texinfo
fastx_toolkit	lmdb	py-genders	r-inline	texlive
fenics	lmod	py-genshi	r-irlba	the_platinum_s
ferret	lrslib	py-gnuplot	r-iterators	the_silver_sea
fftw	lrzip	py-h5py	r-jpeg	thrift
fish	lua	py-imagesize	r-jsonlite	tk
flex	lua-luafilesystem	py-iminuit	r-labeling	tmux
fltk	lua-luaposix	py-ipython	r-lattice	tmuxinator
flux	LuaJIT	py-jdcal	r-lazyeval	tree
foam-extend	lwgrp	py-jinja2	r-leaflet	triangle
fontconfig	lwm2	py-lockfile	r-lme4	trilinos
freetype	lz4	py-logilab-common	r-lmtest	turbomole
gasnet	lzma	py-mako	r-lubridate	uberftp
gcc	lzo	py-markupsafe	r-magic	udunits2
gdal	m4	py-matplotlib	r-magrittr	uncrustify
gdb	mafft	py-meep	r-mapproj	unibilium
gdk-pixbuf	mariadb	py-mistune	r-maps	unison
geant4	matio	py-mock	r-maptools	unixodbc
geos	mbedtls	py-mpi4py	r-markdown	util-linux
gettext	meep	py-mpmath	r-mass	valgrind
gflags	memaxes	py-mx	r-matrix	vim
ghostscript	mercurial	py-mysqldb1	r-matrixmodels	visit
ghostscript-fonts	mesa	py-nestle	r-memoise	vtk
giflib	metis	py-netcdf	r-mgcv	wannier90
git	mfem	py-networkx	r-mime	wget
git-lfs	Mitos	py-nose	r-minqa	wx
gl2ps	mkl	py-numexpr	r-multcomp	wxpropgrid
glib	moab	py-numpy	r-munsell	xcb-proto
glm	mpc	py-openpyxl	r-mvtnorm	xerces-c
global	mpe2	py-pandas	r-ncdf4	xorg-util-macr
globus_toolkit	mpfr	py-pbr	r-networkd3	xproto

```
glog                mpibash             py-pep8             r-nlme              xrootd
glpk                mpich               py-periodictable    r-nloptr            xz
gmake               mpileaks            py-pexpect          r-nmf               yasm
gmp                 mrnet               py-phonopy          r-nnet              zeromq
gmsh                msgpack-c           py-pil              r-np                zfp
gnu-prolog          mumps               py-pillow           r-openssl           zlib
gnuplot             munge               py-ply              r-packrat           zoltan
gnutls              muparser            py-pmw              r-pbkrtest          zsh
```

The packages are listed by name in alphabetical order. If you specify a pattern to match, it will follow this set of rules. A pattern with no wildcards, `*` or `?`, will be treated as though it started and ended with `*`, so `util` is equivalent to `*util*`. A pattern with no capital letters will be treated as case-insensitive. You can also add the `-i` flag to specify a case insensitive search, or `-d` to search the description of the package in addition to the name. Some examples:

All packages whose names contain "sql" case insensitive:

```
$ spack list sql
postgresql  py-mysqldb1  r-rmysql  r-rpostgresql  r-rsqlite  sqlite
```

All packages whose names start with a capital M:

```
$ spack list 'M*'
Mitos
```

All packages whose names or descriptions contain Documentation:

```
$ spack list -d Documentation
py-sphinx  r-rcpp
```

All packages whose names contain documentation case insensitive:

```
$ spack list -d documentation
doxygen  gflags  py-alabaster  py-docutils  py-epydoc  r-ggplot2  r-roxygen2  r-stanheaders  texinfo
```

3.1.2 spack info

To get more information on a particular package from *spack list*, use *spack info*. Just supply the name of a package:

```
$ spack info mpich
Package:    mpich
Homepage:   http://www.mpich.org

Safe versions:
    3.2      http://www.mpich.org/static/downloads/3.2/mpich-3.2.tar.gz
    3.1.4    http://www.mpich.org/static/downloads/3.1.4/mpich-3.1.4.tar.gz
    3.1.3    http://www.mpich.org/static/downloads/3.1.3/mpich-3.1.3.tar.gz
    3.1.2    http://www.mpich.org/static/downloads/3.1.2/mpich-3.1.2.tar.gz
    3.1.1    http://www.mpich.org/static/downloads/3.1.1/mpich-3.1.1.tar.gz
    3.1      http://www.mpich.org/static/downloads/3.1/mpich-3.1.tar.gz
    3.0.4    http://www.mpich.org/static/downloads/3.0.4/mpich-3.0.4.tar.gz

Variants:
    Name     Default   Description

    hydra    on        Build the hydra process manager
    pmi      on        Build with PMI support
    verbs    off       Build support for OpenFabrics verbs.
```

```
Build Dependencies:
    None

Link Dependencies:
    None

Run Dependencies:
    None

Virtual packages:
    mpich@3: provides mpi@:3.0
    mpich@1: provides mpi@:1.3

Description:
    MPICH is a high performance and widely portable implementation of the
    Message Passing Interface (MPI) standard.
```

Most of the information is self-explanatory. The *safe versions* are versions that Spack knows the checksum for, and it will use the checksum to verify that these versions download without errors or viruses.

Dependencies and *virtual dependencies* are described in more detail later.

3.1.3 spack versions

To see *more* available versions of a package, run `spack versions`. For example:

```
$ spack versions libelf
==> Safe versions (already checksummed):
  0.8.13  0.8.12
==> Remote versions (not yet checksummed):
  0.8.11  0.8.10  0.8.9  0.8.8  0.8.7  0.8.6  0.8.5  0.8.4  0.8.3  0.8.2  0.8.0  0.7.0  0.6.4  0.5.2
```

There are two sections in the output. *Safe versions* are versions for which Spack has a checksum on file. It can verify that these versions are downloaded correctly.

In many cases, Spack can also show you what versions are available out on the web—these are *remote versions*. Spack gets this information by scraping it directly from package web pages. Depending on the package and how its releases are organized, Spack may or may not be able to find remote versions.

3.2 Installing and uninstalling

3.2.1 spack install

`spack install` will install any package shown by `spack list`. For example, To install the latest version of the `mpileaks` package, you might type this:

```
$ spack install mpileaks
```

If `mpileaks` depends on other packages, Spack will install the dependencies first. It then fetches the `mpileaks` tarball, expands it, verifies that it was downloaded without errors, builds it, and installs it in its own directory under `$SPACK_ROOT/opt`. You'll see a number of messages from spack, a lot of build output, and a message that the packages is installed:

```
$ spack install mpileaks
==> Installing mpileaks
==> mpich is already installed in /home/gamblin2/spack/opt/linux-debian7-x86_64/gcc@4.4.7/mpich@3.0.4
==> callpath is already installed in /home/gamblin2/spack/opt/linux-debian7-x86_64/gcc@4.4.7/callpath
==> adept-utils is already installed in /home/gamblin2/spack/opt/linux-debian7-x86_64/gcc@4.4.7/adept
==> Trying to fetch from https://github.com/hpc/mpileaks/releases/download/v1.0/mpileaks-1.0.tar.gz
######################################################################## 100.0%
==> Staging archive: /home/gamblin2/spack/var/spack/stage/mpileaks@1.0%gcc@4.4.7 arch=linux-debian7-x
==> Created stage in /home/gamblin2/spack/var/spack/stage/mpileaks@1.0%gcc@4.4.7 arch=linux-debian7-x
==> No patches needed for mpileaks.
==> Building mpileaks.

... build output ...

==> Successfully installed mpileaks.
  Fetch: 2.16s.  Build: 9.82s.  Total: 11.98s.
[+] /home/gamblin2/spack/opt/linux-debian7-x86_64/gcc@4.4.7/mpileaks@1.0-59f6ad23
```

The last line, with the `[+]`, indicates where the package is installed.

3.2.2 Building a specific version

Spack can also build *specific versions* of a package. To do this, just add `@` after the package name, followed by a version:

```
$ spack install mpich@3.0.4
```

Any number of versions of the same package can be installed at once without interfering with each other. This is good for multi-user sites, as installing a version that one user needs will not disrupt existing installations for other users.

In addition to different versions, Spack can customize the compiler, compile-time options (variants), compiler flags, and platform (for cross compiles) of an installation. Spack is unique in that it can also configure the *dependencies* a package is built with. For example, two configurations of the same version of a package, one built with boost 1.39.0, and the other version built with version 1.43.0, can coexist.

This can all be done on the command line using the *spec* syntax. Spack calls the descriptor used to refer to a particular package configuration a **spec**. In the commands above, `mpileaks` and `mpileaks@3.0.4` are both valid *specs*. We'll talk more about how you can use them to customize an installation in *Specs & dependencies*.

3.2.3 `spack uninstall`

To uninstall a package, type `spack uninstall <package>`. This will ask the user for confirmation, and in case will completely remove the directory in which the package was installed.

```
$ spack uninstall mpich
```

If there are still installed packages that depend on the package to be uninstalled, spack will refuse to uninstall it.

To uninstall a package and every package that depends on it, you may give the `--dependents` option.

```
$ spack uninstall --dependents mpich
```

will display a list of all the packages that depend on `mpich` and, upon confirmation, will uninstall them in the right order.

A command like

```
$ spack uninstall mpich
```

may be ambiguous if multiple `mpich` configurations are installed. For example, if both `mpich@3.0.2` and `mpich@3.1` are installed, `mpich` could refer to either one. Because it cannot determine which one to uninstall, Spack will ask you either to provide a version number to remove the ambiguity or use the `--all` option to uninstall all of the matching packages.

You may force uninstall a package with the `--force` option

```
$ spack uninstall --force mpich
```

but you risk breaking other installed packages. In general, it is safer to remove dependent packages *before* removing their dependencies or use the `--dependents` option.

3.2.4 Non-Downloadable Tarballs

The tarballs for some packages cannot be automatically downloaded by Spack. This could be for a number of reasons:

1. The author requires users to manually accept a license agreement before downloading (`jdk` and `galahad`).
2. The software is proprietary and cannot be downloaded on the open Internet.

To install these packages, one must create a mirror and manually add the tarballs in question to it (see *Mirrors*):

1. Create a directory for the mirror. You can create this directory anywhere you like, it does not have to be inside `~/.spack`:

   ```
   $ mkdir ~/.spack/manual_mirror
   ```

2. Register the mirror with Spack by creating `~/.spack/mirrors.yaml`:

   ```
   mirrors:
     manual: file:///home/me/.spack/manual_mirror
   ```

3. Put your tarballs in it. Tarballs should be named `<package>/<package>-<version>.tar.gz`. For example:

   ```
   $ ls -l manual_mirror/galahad

   -rw-------. 1 me me 11657206 Jun 21 19:25 galahad-2.60003.tar.gz
   ```

4. Install as usual:

   ```
   $ spack install galahad
   ```

3.3 Seeing installed packages

We know that `spack list` shows you the names of available packages, but how do you figure out which are already installed?

3.3.1 `spack find`

`spack find` shows the *specs* of installed packages. A spec is like a name, but it has a version, compiler, architecture, and build options associated with it. In spack, you can have many installations of the same package with different specs.

Running `spack find` with no arguments lists installed packages:

```
$ spack find
==> 74 installed packages.
-- linux-debian7-x86_64 / gcc@4.4.7 -------------------------------
ImageMagick@6.8.9-10  libdwarf@20130729  py-dateutil@2.4.0
adept-utils@1.0       libdwarf@20130729  py-ipython@2.3.1
atk@2.14.0            libelf@0.8.12      py-matplotlib@1.4.2
boost@1.55.0          libelf@0.8.13      py-nose@1.3.4
bzip2@1.0.6           libffi@3.1         py-numpy@1.9.1
cairo@1.14.0          libmng@2.0.2       py-pygments@2.0.1
callpath@1.0.2        libpng@1.6.16      py-pyparsing@2.0.3
cmake@3.0.2           libtiff@4.0.3      py-pyside@1.2.2
dbus@1.8.6            libtool@2.4.2      py-pytz@2014.10
dbus@1.9.0            libxcb@1.11        py-setuptools@11.3.1
dyninst@8.1.2         libxml2@2.9.2      py-six@1.9.0
fontconfig@2.11.1     libxml2@2.9.2      python@2.7.8
freetype@2.5.3        llvm@3.0           qhull@1.0
gdk-pixbuf@2.31.2     memaxes@0.5        qt@4.8.6
glib@2.42.1           mesa@8.0.5         qt@5.4.0
graphlib@2.0.0        mpich@3.0.4        readline@6.3
gtkplus@2.24.25       mpileaks@1.0       sqlite@3.8.5
harfbuzz@0.9.37       mrnet@4.1.0        stat@2.1.0
hdf5@1.8.13           ncurses@5.9        tcl@8.6.3
icu@54.1              netcdf@4.3.3       tk@src
jpeg@9a               openssl@1.0.1h     vtk@6.1.0
launchmon@1.0.1       pango@1.36.8       xcb-proto@1.11
lcms@2.6              pixman@0.32.6      xz@5.2.0
libdrm@2.4.33         py-dateutil@2.4.0  zlib@1.2.8

-- linux-debian7-x86_64 / gcc@4.9.2 -------------------------------
libelf@0.8.10  mpich@3.0.4
```

Packages are divided into groups according to their architecture and compiler. Within each group, Spack tries to keep the view simple, and only shows the version of installed packages.

`spack find` can filter the package list based on the package name, spec, or a number of properties of their installation status. For example, missing dependencies of a spec can be shown with `--missing`, packages which were explicitly installed with `spack install <package>` can be singled out with `--explicit` and those which have been pulled in only as dependencies with `--implicit`.

In some cases, there may be different configurations of the *same* version of a package installed. For example, there are two installations of `libdwarf@20130729` above. We can look at them in more detail using `spack find --deps`, and by asking only to show `libdwarf` packages:

```
$ spack find --deps libdwarf
==> 2 installed packages.
-- linux-debian7-x86_64 / gcc@4.4.7 -------------------------------
    libdwarf@20130729-d9b90962
        ^libelf@0.8.12
    libdwarf@20130729-b52fac98
        ^libelf@0.8.13
```

Now we see that the two instances of `libdwarf` depend on *different* versions of `libelf`: 0.8.12 and 0.8.13. This view can become complicated for packages with many dependencies. If you just want to know whether two packages' dependencies differ, you can use `spack find --long`:

```
$ spack find --long libdwarf
==> 2 installed packages.
```

```
-- linux-debian7-x86_64 / gcc@4.4.7 ------------------------------
libdwarf@20130729-d9b90962  libdwarf@20130729-b52fac98
```

Now the `libdwarf` installs have hashes after their names. These are hashes over all of the dependencies of each package. If the hashes are the same, then the packages have the same dependency configuration.

If you want to know the path where each package is installed, you can use `spack find --paths`:

```
$ spack find --paths
==> 74 installed packages.
-- linux-debian7-x86_64 / gcc@4.4.7 ------------------------------
    ImageMagick@6.8.9-10  /home/gamblin2/spack/opt/linux-debian7-x86_64/gcc@4.4.7/ImageMagick@6.8.9-1
    adept-utils@1.0       /home/gamblin2/spack/opt/linux-debian7-x86_64/gcc@4.4.7/adept-utils@1.0-5ac
    atk@2.14.0            /home/gamblin2/spack/opt/linux-debian7-x86_64/gcc@4.4.7/atk@2.14.0-3d09ac09
    boost@1.55.0          /home/gamblin2/spack/opt/linux-debian7-x86_64/gcc@4.4.7/boost@1.55.0
    bzip2@1.0.6           /home/gamblin2/spack/opt/linux-debian7-x86_64/gcc@4.4.7/bzip2@1.0.6
    cairo@1.14.0          /home/gamblin2/spack/opt/linux-debian7-x86_64/gcc@4.4.7/cairo@1.14.0-fcc2ab
    callpath@1.0.2        /home/gamblin2/spack/opt/linux-debian7-x86_64/gcc@4.4.7/callpath@1.0.2-5dce
...
```

And, finally, you can restrict your search to a particular package by supplying its name:

```
$ spack find --paths libelf
-- linux-debian7-x86_64 / gcc@4.4.7 ------------------------------
    libelf@0.8.11  /home/gamblin2/spack/opt/linux-debian7-x86_64/gcc@4.4.7/libelf@0.8.11
    libelf@0.8.12  /home/gamblin2/spack/opt/linux-debian7-x86_64/gcc@4.4.7/libelf@0.8.12
    libelf@0.8.13  /home/gamblin2/spack/opt/linux-debian7-x86_64/gcc@4.4.7/libelf@0.8.13
```

`spack find` actually does a lot more than this. You can use *specs* to query for specific configurations and builds of each package. If you want to find only libelf versions greater than version 0.8.12, you could say:

```
$ spack find libelf@0.8.12:
-- linux-debian7-x86_64 / gcc@4.4.7 ------------------------------
    libelf@0.8.12  libelf@0.8.13
```

Finding just the versions of libdwarf built with a particular version of libelf would look like this:

```
$ spack find --long libdwarf ^libelf@0.8.12
==> 1 installed packages.
-- linux-debian7-x86_64 / gcc@4.4.7 ------------------------------
libdwarf@20130729-d9b90962
```

We can also search for packages that have a certain attribute. For example, `spack find libdwarf +debug` will show only installations of libdwarf with the 'debug' compile-time option enabled.

The full spec syntax is discussed in detail in *Specs & dependencies*.

3.4 Specs & dependencies

We know that `spack install`, `spack uninstall`, and other commands take a package name with an optional version specifier. In Spack, that descriptor is called a *spec*. Spack uses specs to refer to a particular build configuration (or configurations) of a package. Specs are more than a package name and a version; you can use them to specify the compiler, compiler version, architecture, compile options, and dependency options for a build. In this section, we'll go over the full syntax of specs.

Here is an example of a much longer spec than we've seen thus far:

```
mpileaks @1.2:1.4 %gcc@4.7.5 +debug -qt arch=bgq_os ^callpath @1.1 %gcc@4.7.2
```

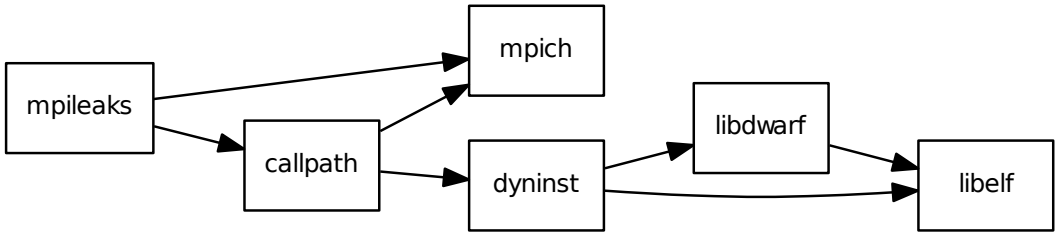
If provided to `spack install`, this will install the `mpileaks` library at some version between `1.2` and `1.4` (inclusive), built using `gcc` at version 4.7.5 for the Blue Gene/Q architecture, with debug options enabled, and without Qt support. Additionally, it says to link it with the `callpath` library (which it depends on), and to build callpath with `gcc` 4.7.2. Most specs will not be as complicated as this one, but this is a good example of what is possible with specs.

More formally, a spec consists of the following pieces:

- Package name identifier (`mpileaks` above)
- `@` Optional version specifier (`@1.2:1.4`)
- `%` Optional compiler specifier, with an optional compiler version (`gcc` or `gcc@4.7.3`)
- `+` or `-` or `~` Optional variant specifiers (`+debug`, `-qt`, or `~qt`) for boolean variants
- `name=<value>` Optional variant specifiers that are not restricted to boolean variants
- `name=<value>` Optional compiler flag specifiers. Valid flag names are `cflags`, `cxxflags`, `fflags`, `cppflags`, `ldflags`, and `ldlibs`.
- `target=<value> os=<value>` Optional architecture specifier (`target=haswell os=CNL10`)
- `^` Dependency specs (`^callpath@1.1`)

There are two things to notice here. The first is that specs are recursively defined. That is, each dependency after `^` is a spec itself. The second is that everything is optional *except* for the initial package name identifier. Users can be as vague or as specific as they want about the details of building packages, and this makes spack good for beginners and experts alike.

To really understand what's going on above, we need to think about how software is structured. An executable or a library (these are generally the artifacts produced by building software) depends on other libraries in order to run. We can represent the relationship between a package and its dependencies as a graph. Here is the full dependency graph for `mpileaks`:



Each box above is a package and each arrow represents a dependency on some other package. For example, we say that the package `mpileaks` *depends on* `callpath` and `mpich`. `mpileaks` also depends *indirectly* on `dyninst`, `libdwarf`, and `libelf`, in that these libraries are dependencies of `callpath`. To install `mpileaks`, Spack has to build all of these packages. Dependency graphs in Spack have to be acyclic, and the *depends on* relationship is directional, so this is a *directed, acyclic graph* or *DAG*.

The package name identifier in the spec is the root of some dependency DAG, and the DAG itself is implicit. Spack knows the precise dependencies among packages, but users do not need to know the full DAG structure. Each `^` in the full spec refers to some dependency of the root package. Spack will raise an error if you supply a name after `^` that the root does not actually depend on (e.g. `mpileaks ^emacs@23.3`).

Spack further simplifies things by only allowing one configuration of each package within any single build. Above, both `mpileaks` and `callpath` depend on `mpich`, but `mpich` appears only once in the DAG. You cannot build an `mpileaks` version that depends on one version of `mpich` *and* on a `callpath` version that depends on some *other* version of `mpich`. In general, such a configuration would likely behave unexpectedly at runtime, and Spack enforces this to ensure a consistent runtime environment.

The point of specs is to abstract this full DAG from Spack users. If a user does not care about the DAG at all, she can refer to mpileaks by simply writing `mpileaks`. If she knows that `mpileaks` indirectly uses `dyninst` and she wants a particular version of `dyninst`, then she can refer to `mpileaks ^dyninst@8.1`. Spack will fill in the rest when it parses the spec; the user only needs to know package names and minimal details about their relationship.

When spack prints out specs, it sorts package names alphabetically to normalize the way they are displayed, but users do not need to worry about this when they write specs. The only restriction on the order of dependencies within a spec is that they appear *after* the root package. For example, these two specs represent exactly the same configuration:

```
mpileaks ^callpath@1.0 ^libelf@0.8.3
mpileaks ^libelf@0.8.3 ^callpath@1.0
```

You can put all the same modifiers on dependency specs that you would put on the root spec. That is, you can specify their versions, compilers, variants, and architectures just like any other spec. Specifiers are associated with the nearest package name to their left. For example, above, `@1.1` and `%gcc@4.7.2` associates with the `callpath` package, while `@1.2:1.4`, `%gcc@4.7.5`, `+debug`, `-qt`, and `target=haswell os=CNL10` all associate with the `mpileaks` package.

In the diagram above, `mpileaks` depends on `mpich` with an unspecified version, but packages can depend on other packages with *constraints* by adding more specifiers. For example, `mpileaks` could depend on `mpich@1.2:` if it can only build with version `1.2` or higher of `mpich`.

Below are more details about the specifiers that you can add to specs.

3.4.1 Version specifier

A version specifier comes somewhere after a package name and starts with `@`. It can be a single version, e.g. `@1.0`, `@3`, or `@1.2a7`. Or, it can be a range of versions, such as `@1.0:1.5` (all versions between `1.0` and `1.5`, inclusive). Version ranges can be open, e.g. `:3` means any version up to and including `3`. This would include `3.4` and `3.4.2`. `4.2:` means any version above and including `4.2`. Finally, a version specifier can be a set of arbitrary versions, such as `@1.0,1.5,1.7` (`1.0`, `1.5`, or `1.7`). When you supply such a specifier to `spack install`, it constrains the set of versions that Spack will install.

If the version spec is not provided, then Spack will choose one according to policies set for the particular spack installation. If the spec is ambiguous, i.e. it could match multiple versions, Spack will choose a version within the spec's constraints according to policies set for the particular Spack installation.

Details about how versions are compared and how Spack determines if one version is less than another are discussed in the developer guide.

3.4.2 Compiler specifier

A compiler specifier comes somewhere after a package name and starts with `%`. It tells Spack what compiler(s) a particular package should be built with. After the `%` should come the name of some registered Spack compiler. This might include `gcc`, or `intel`, but the specific compilers available depend on the site. You can run `spack compilers` to get a list; more on this below.

The compiler spec can be followed by an optional *compiler version*. A compiler version specifier looks exactly like a package version specifier. Version specifiers will associate with the nearest package name or compiler specifier to their left in the spec.

If the compiler spec is omitted, Spack will choose a default compiler based on site policies.

3.4.3 Variants

Variants are named options associated with a particular package. They are optional, as each package must provide default values for each variant it makes available. Variants can be specified using a flexible parameter syntax `name=<value>`. For example, `spack install libelf debug=True` will install libelf build with debug flags. The names of particular variants available for a package depend on what was provided by the package author. `spack info <package>` will provide information on what build variants are available.

For compatibility with earlier versions, variants which happen to be boolean in nature can be specified by a syntax that represents turning options on and off. For example, in the previous spec we could have supplied `libelf +debug` with the same effect of enabling the debug compile time option for the libelf package.

Depending on the package a variant may have any default value. For `libelf` here, `debug` is `False` by default, and we turned it on with `debug=True` or `+debug`. If a variant is `True` by default you can turn it off by either adding `-name` or `~name` to the spec.

There are two syntaxes here because, depending on context, `~` and `-` may mean different things. In most shells, the following will result in the shell performing home directory substitution:

```
mpileaks ~debug   # shell may try to substitute this!
mpileaks~debug    # use this instead
```

If there is a user called `debug`, the `~` will be incorrectly expanded. In this situation, you would want to write `libelf -debug`. However, `-` can be ambiguous when included after a package name without spaces:

```
mpileaks-debug      # wrong!
mpileaks -debug     # right
```

Spack allows the `-` character to be part of package names, so the above will be interpreted as a request for the `mpileaks-debug` package, not a request for `mpileaks` built without `debug` options. In this scenario, you should write `mpileaks~debug` to avoid ambiguity.

When spack normalizes specs, it prints them out with no spaces boolean variants using the backwards compatibility syntax and uses only `~` for disabled boolean variants. We allow `-` and spaces on the command line is provided for convenience and legibility.

3.4.4 Compiler Flags

Compiler flags are specified using the same syntax as non-boolean variants, but fulfill a different purpose. While the function of a variant is set by the package, compiler flags are used by the compiler wrappers to inject flags into the compile line of the build. Additionally, compiler flags are inherited by dependencies. `spack install libdwarf cppflags=\"-g\"` will install both libdwarf and libelf with the `-g` flag injected into their compile line.

Notice that the value of the compiler flags must be escape quoted on the command line. From within python files, the same spec would be specified `libdwarf cppflags="-g"`. This is necessary because of how the shell handles the quote symbols.

The six compiler flags are injected in the order of implicit make commands in GNU Autotools. If all flags are set, the order is `$cppflags $cflags|$cxxflags $ldflags <command> $ldlibs` for C and C++ and `$fflags $cppflags $ldflags <command> $ldlibs` for Fortran.

3.4.5 Architecture specifiers

The architecture can be specified by using the reserved words `target` and/or `os` (`target=x86-64 os=debian7`). You can also use the triplet form of platform, operating system and processor.

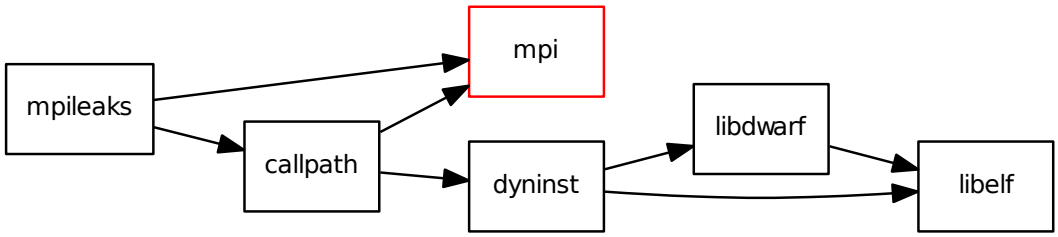
```
$ spack install libelf arch=cray-CNL10-haswell
```

Users on non-Cray systems won't have to worry about specifying the architecture. Spack will autodetect what kind of operating system is on your machine as well as the processor. For more information on how the architecture can be used on Cray machines, see *Spack on Cray*

3.5 Virtual dependencies

The dependence graph for `mpileaks` we saw above wasn't *quite* accurate. `mpileaks` uses MPI, which is an interface that has many different implementations. Above, we showed `mpileaks` and `callpath` depending on `mpich`, which is one *particular* implementation of MPI. However, we could build either with another implementation, such as `openmpi` or `mvapich`.

Spack represents interfaces like this using *virtual dependencies*. The real dependency DAG for `mpileaks` looks like this:



Notice that `mpich` has now been replaced with `mpi`. There is no *real* MPI package, but some packages *provide* the MPI interface, and these packages can be substituted in for `mpi` when `mpileaks` is built.

You can see what virtual packages a particular package provides by getting info on it:

```
$ spack info mpich
Package:    mpich
Homepage:   http://www.mpich.org

Safe versions:
    3.2      http://www.mpich.org/static/downloads/3.2/mpich-3.2.tar.gz
    3.1.4    http://www.mpich.org/static/downloads/3.1.4/mpich-3.1.4.tar.gz
    3.1.3    http://www.mpich.org/static/downloads/3.1.3/mpich-3.1.3.tar.gz
    3.1.2    http://www.mpich.org/static/downloads/3.1.2/mpich-3.1.2.tar.gz
    3.1.1    http://www.mpich.org/static/downloads/3.1.1/mpich-3.1.1.tar.gz
    3.1      http://www.mpich.org/static/downloads/3.1/mpich-3.1.tar.gz
    3.0.4    http://www.mpich.org/static/downloads/3.0.4/mpich-3.0.4.tar.gz

Variants:
    Name     Default   Description
```

```
    hydra    on         Build the hydra process manager
    pmi      on         Build with PMI support
    verbs    off        Build support for OpenFabrics verbs.

Build Dependencies:
    None

Link Dependencies:
    None

Run Dependencies:
    None

Virtual packages:
    mpich@3: provides mpi@:3.0
    mpich@1: provides mpi@:1.3

Description:
    MPICH is a high performance and widely portable implementation of the
    Message Passing Interface (MPI) standard.
```

Spack is unique in that its virtual packages can be versioned, just like regular packages. A particular version of a package may provide a particular version of a virtual package, and we can see above that `mpich` versions `1` and above provide all `mpi` interface versions up to `1`, and `mpich` versions `3` and above provide `mpi` versions up to `3`. A package can *depend on* a particular version of a virtual package, e.g. if an application needs MPI-2 functions, it can depend on `mpi@2:` to indicate that it needs some implementation that provides MPI-2 functions.

3.5.1 Constraining virtual packages

When installing a package that depends on a virtual package, you can opt to specify the particular provider you want to use, or you can let Spack pick. For example, if you just type this:

```
$ spack install mpileaks
```

Then spack will pick a provider for you according to site policies. If you really want a particular version, say `mpich`, then you could run this instead:

```
$ spack install mpileaks ^mpich
```

This forces spack to use some version of `mpich` for its implementation. As always, you can be even more specific and require a particular `mpich` version:

```
$ spack install mpileaks ^mpich@3
```

The `mpileaks` package in particular only needs MPI-1 commands, so any MPI implementation will do. If another package depends on `mpi@2` and you try to give it an insufficient MPI implementation (e.g., one that provides only `mpi@:1`), then Spack will raise an error. Likewise, if you try to plug in some package that doesn't provide MPI, Spack will raise an error.

3.5.2 Specifying Specs by Hash

Complicated specs can become cumbersome to enter on the command line, especially when many of the qualifications are necessary to distinguish between similar installs, for example when using the `uninstall` command. To avoid this, when referencing an existing spec, Spack allows you to reference specs by their hash. We previously discussed the spec hash that Spack computes. In place of a spec in any command, substitute `/<hash>` where `<hash>` is any

amount from the beginning of a spec hash. If the given spec hash is sufficient to be unique, Spack will replace the reference with the spec to which it refers. Otherwise, it will prompt for a more qualified hash.

Note that this will not work to reinstall a depencency uninstalled by `spack uninstall --force`.

3.5.3 `spack providers`

You can see what packages provide a particular virtual package using `spack providers`. If you wanted to see what packages provide `mpi`, you would just run:

```
$ spack providers mpi
    intel-parallel-studio@cluster:+mpi  mpich@3:      mvapich2@2.0:  openmpi@1.7.5:
    mpich@1:                            mvapich2@1.9  openmpi@1.6.5  openmpi@2.0.0:
```

And if you *only* wanted to see packages that provide MPI-2, you would add a version specifier to the spec:

```
$ spack providers mpi@2
    intel-parallel-studio@cluster:+mpi  mvapich2@1.9   openmpi@1.6.5   openmpi@2.0.0:
    mpich@3:                            mvapich2@2.0:  openmpi@1.7.5:
```

Notice that the package versions that provide insufficient MPI versions are now filtered out.

3.6 Integration with module systems

Spack provides some integration with Environment Modules to make it easier to use the packages it installs. If your system does not already have Environment Modules, see *Environment Modules*.

Note: Spack also supports Dotkit, which is used by some systems. If you system does not already have a module system installed, you should use Environment Modules or LMod.

3.6.1 Spack and module systems

You can enable shell support by sourcing some files in the `/share/spack` directory.

For `bash` or `ksh`, run:

```
export SPACK_ROOT=/path/to/spack
. $SPACK_ROOT/share/spack/setup-env.sh
```

For `csh` and `tcsh` run:

```
setenv SPACK_ROOT /path/to/spack
source $SPACK_ROOT/share/spack/setup-env.csh
```

You can put the above code in your `.bashrc` or `.cshrc`, and Spack's shell support will be available on the command line.

When you install a package with Spack, it automatically generates a module file that lets you add the package to your environment.

Currently, Spack supports the generation of Environment Modules and Dotkit. Generated module files for each of these systems can be found in these directories:

```
$SPACK_ROOT/share/spack/modules
$SPACK_ROOT/share/spack/dotkit
```

The directories are automatically added to your `MODULEPATH` and `DK_NODE` environment variables when you enable Spack's *shell support*.

3.6.2 Using Modules & Dotkits

If you have shell support enabled you should be able to run either `module avail` or `use -l spack` to see what modules/dotkits have been installed. Here is sample output of those programs, showing lots of installed packages.

```
$ module avail

------- /home/gamblin2/spack/share/spack/modules/linux-debian7-x86_64 --------
adept-utils@1.0%gcc@4.4.7-5adef8da   libelf@0.8.13%gcc@4.4.7
automaded@1.0%gcc@4.4.7-d9691bb0     libelf@0.8.13%intel@15.0.0
boost@1.55.0%gcc@4.4.7               mpc@1.0.2%gcc@4.4.7-559607f5
callpath@1.0.1%gcc@4.4.7-5dce4318    mpfr@3.1.2%gcc@4.4.7
dyninst@8.1.2%gcc@4.4.7-b040c20e     mpich@3.0.4%gcc@4.4.7
gcc@4.9.1%gcc@4.4.7-93ab98c5         mpich@3.0.4%gcc@4.9.0
gmp@6.0.0a%gcc@4.4.7                 mrnet@4.1.0%gcc@4.4.7-72b7881d
graphlib@2.0.0%gcc@4.4.7             netgauge@2.4.6%gcc@4.9.0-27912b7b
launchmon@1.0.1%gcc@4.4.7            stat@2.1.0%gcc@4.4.7-51101207
libNBC@1.1.1%gcc@4.9.0-27912b7b      sundials@2.5.0%gcc@4.9.0-27912b7b
libdwarf@20130729%gcc@4.4.7-b52fac98
```

```
$ use -l spack

spack ----------
  adept-utils@1.0%gcc@4.4.7-5adef8da - adept-utils @1.0
  automaded@1.0%gcc@4.4.7-d9691bb0 - automaded @1.0
  boost@1.55.0%gcc@4.4.7 - boost @1.55.0
  callpath@1.0.1%gcc@4.4.7-5dce4318 - callpath @1.0.1
  dyninst@8.1.2%gcc@4.4.7-b040c20e - dyninst @8.1.2
  gmp@6.0.0a%gcc@4.4.7 - gmp @6.0.0a
  libNBC@1.1.1%gcc@4.9.0-27912b7b - libNBC @1.1.1
  libdwarf@20130729%gcc@4.4.7-b52fac98 - libdwarf @20130729
  libelf@0.8.13%gcc@4.4.7 - libelf @0.8.13
  libelf@0.8.13%intel@15.0.0 - libelf @0.8.13
  mpc@1.0.2%gcc@4.4.7-559607f5 - mpc @1.0.2
  mpfr@3.1.2%gcc@4.4.7 - mpfr @3.1.2
  mpich@3.0.4%gcc@4.4.7 - mpich @3.0.4
  mpich@3.0.4%gcc@4.9.0 - mpich @3.0.4
  netgauge@2.4.6%gcc@4.9.0-27912b7b - netgauge @2.4.6
  sundials@2.5.0%gcc@4.9.0-27912b7b - sundials @2.5.0
```

The names here should look familiar, they're the same ones from `spack find`. You *can* use the names here directly. For example, you could type either of these commands to load the callpath module:

```
$ use callpath@1.0.1%gcc@4.4.7-5dce4318
```

```
$ module load callpath@1.0.1%gcc@4.4.7-5dce4318
```

Neither of these is particularly pretty, easy to remember, or easy to type. Luckily, Spack has its own interface for using modules and dotkits. You can use the same spec syntax you're used to:

Environment Modules	Dotkit
`spack load <spec>`	`spack use <spec>`
`spack unload <spec>`	`spack unuse <spec>`

And you can use the same shortened names you use everywhere else in Spack. For example, this will add the `mpich` package built with `gcc` to your path:

```
$ spack install mpich %gcc@4.4.7

# ... wait for install ...

$ spack use mpich %gcc@4.4.7
Prepending: mpich@3.0.4%gcc@4.4.7 (ok)
$ which mpicc
~/src/spack/opt/linux-debian7-x86_64/gcc@4.4.7/mpich@3.0.4/bin/mpicc
```

Or, similarly with modules, you could type:

```
$ spack load mpich %gcc@4.4.7
```

These commands will add appropriate directories to your `PATH`, `MANPATH`, `CPATH`, and `LD_LIBRARY_PATH`. When you no longer want to use a package, you can type unload or unuse similarly:

```
$ spack unload mpich %gcc@4.4.7  # modules
$ spack unuse  mpich %gcc@4.4.7  # dotkit
```

Note: These `use`, `unuse`, `load`, and `unload` subcommands are only available if you have enabled Spack's shell support *and* you have dotkit or modules installed on your machine.

3.6.3 Ambiguous module names

If a spec used with load/unload or use/unuse is ambiguous (i.e. more than one installed package matches it), then Spack will warn you:

```
$ spack load libelf
==> Error: Multiple matches for spec libelf.  Choose one:
libelf@0.8.13%gcc@4.4.7 arch=linux-debian7-x86_64
libelf@0.8.13%intel@15.0.0 arch=linux-debian7-x86_64
```

You can either type the `spack load` command again with a fully qualified argument, or you can add just enough extra constraints to identify one package. For example, above, the key differentiator is that one `libelf` is built with the Intel compiler, while the other used `gcc`. You could therefore just type:

```
$ spack load libelf %intel
```

To identify just the one built with the Intel compiler.

3.6.4 Module files generation and customization

Environment Modules and Dotkit files are generated when packages are installed, and are placed in the following directories under the Spack root:

```
$SPACK_ROOT/share/spack/modules
$SPACK_ROOT/share/spack/dotkit
```

The content that gets written in each module file can be customized in two ways:

1. overriding part of the `spack.Package` API within a `package.py`
2. writing dedicated configuration files

3.6.5 Override `Package` API

There are currently two methods in `spack.Package` that may affect the content of module files:

```
def setup_environment(self, spack_env, run_env):
    """Set up the compile and runtime environments for a package."""
    pass
```

```
def setup_dependent_environment(self, spack_env, run_env, dependent_spec):
    """Set up the environment of packages that depend on this one"""
    pass
```

As briefly stated in the comments, the first method lets you customize the module file content for the package you are currently writing, the second allows for modifications to your dependees module file. In both cases one needs to fill `run_env` with the desired list of environment modifications.

Example : `builtin/packages/python/package.py`

The `python` package that comes with the `builtin` Spack repository overrides `setup_dependent_environment` in the following way:

```
def setup_dependent_environment(self, spack_env, run_env, extension_spec):
    if extension_spec.package.extends(self.spec):
        run_env.prepend_path('PYTHONPATH', os.path.join(extension_spec.prefix, self.site_packages_di
```

to insert the appropriate `PYTHONPATH` modifications in the module files of python packages.

3.6.6 Recursive Modules

In some cases, it is desirable to load not just a module, but also all the modules it depends on. This is not required for most modules because Spack builds binaries with RPATH support. However, not all packages use RPATH to find their dependencies: this can be true in particular for Python extensions, which are currently *not* built with RPATH.

Scripts to load modules recursively may be made with the command:

```
$ spack module loads --dependencies <spec>
```

An equivalent alternative is:

```
$ source <( spack module loads --dependencies <spec> )
```

Warning: The `spack load` command does not currently accept the `--dependencies` flag. Use `spack module loads` instead, for now.

3.6.7 Module Commands for Shell Scripts

Although Spack is flexible, the `module` command is much faster. This could become an issue when emitting a series of `spack load` commands inside a shell script. By adding the `--shell` flag, `spack module find` may also be used to generate code that can be cut-and-pasted into a shell script. For example:

```
$ spack module loads --dependencies py-numpy git
# bzip2@1.0.6%gcc@4.9.3=linux-x86_64
module load bzip2-1.0.6-gcc-4.9.3-ktnrhkrmbbtlvnagfatrarzjojmkvzsx
# ncurses@6.0%gcc@4.9.3=linux-x86_64
module load ncurses-6.0-gcc-4.9.3-kaazyneh3bjkfnalunchyqtygoe2mncv
# zlib@1.2.8%gcc@4.9.3=linux-x86_64
module load zlib-1.2.8-gcc-4.9.3-v3ufwaahjnviyvgjcelo36nywx2ufj7z
# sqlite@3.8.5%gcc@4.9.3=linux-x86_64
module load sqlite-3.8.5-gcc-4.9.3-a3eediswgd5f3rmto7g3szoew5nhehbr
# readline@6.3%gcc@4.9.3=linux-x86_64
module load readline-6.3-gcc-4.9.3-se6r3lsycrwxyhreg4lqirp6xixxejh3
# python@3.5.1%gcc@4.9.3=linux-x86_64
module load python-3.5.1-gcc-4.9.3-5q5rsrtjld4u6jiicuvtnx52m7tfhegi
# py-setuptools@20.5%gcc@4.9.3=linux-x86_64
module load py-setuptools-20.5-gcc-4.9.3-4qr2suj6p6glepnedmwhl4f62x64wxw2
# py-nose@1.3.7%gcc@4.9.3=linux-x86_64
module load py-nose-1.3.7-gcc-4.9.3-pwhtjw2dvdvfzjwuuztkzr7b4l6zepli
# openblas@0.2.17%gcc@4.9.3+shared=linux-x86_64
module load openblas-0.2.17-gcc-4.9.3-pw6rmlom7apfsnjtzfttyayzc7nx5e7y
# py-numpy@1.11.0%gcc@4.9.3+blas+lapack=linux-x86_64
module load py-numpy-1.11.0-gcc-4.9.3-mulodttw5pcyjufva4htsktwty4qd52r
# curl@7.47.1%gcc@4.9.3=linux-x86_64
module load curl-7.47.1-gcc-4.9.3-ohz3fwsepm3b462p5lnaquv7op7naqbi
# autoconf@2.69%gcc@4.9.3=linux-x86_64
module load autoconf-2.69-gcc-4.9.3-bkibjqhgqm5e3o423ogfv2y3o6h2uoq4
# cmake@3.5.0%gcc@4.9.3~doc+ncurses+openssl~qt=linux-x86_64
module load cmake-3.5.0-gcc-4.9.3-x7xnsklmgwla3ubfgzppamtbqk5rwn7t
# expat@2.1.0%gcc@4.9.3=linux-x86_64
module load expat-2.1.0-gcc-4.9.3-6pkz2ucnk2e62imwakejjvbv6egncppd
# git@2.8.0-rc2%gcc@4.9.3+curl+expat=linux-x86_64
module load git-2.8.0-rc2-gcc-4.9.3-3bib4hqtnv5xjjoq5ugt3inblt4xrgkd
```

The script may be further edited by removing unnecessary modules.

3.6.8 Module Prefixes

On some systems, modules are automatically prefixed with a certain string; `spack module loads` needs to know about that prefix when it issues `module load` commands. Add the `--prefix` option to your `spack module loads` commands if this is necessary.

For example, consider the following on one system:

..code-block:: console

$ module avail linux-SuSE11-x86_64/antlr-2.7.7-gcc-5.3.0-bdpl46y

$ spack module loads antlr # WRONG! # antlr@2.7.7%gcc@5.3.0~csharp+cxx~java~python arch=linux-SuSE11-x86_64 module load antlr-2.7.7-gcc-5.3.0-bdpl46y

$ spack module loads –prefix linux-SuSE11-x86_64/ antlr # antlr@2.7.7%gcc@5.3.0~csharp+cxx~java~python arch=linux-SuSE11-x86_64 module load linux-SuSE11-x86_64/antlr-2.7.7-gcc-5.3.0-bdpl46y

3.6.9 Configuration files

Another way of modifying the content of module files is writing a `modules.yaml` configuration file. Following usual Spack conventions, this file can be placed either at *site* or *user* scope.

The default site configuration reads:

```
# -------------------------------------------------------------------------
# This is the default configuration for Spack's module file generation.
#
# Settings here are versioned with Spack and are intended to provide
# sensible defaults out of the box. Spack maintainers should edit this
# file to keep it current.
#
# Users can override these settings by editing the following files.
#
# Per-spack-instance settings (overrides defaults):
#   $SPACK_ROOT/etc/spack/modules.yaml
#
# Per-user settings (overrides default and site settings):
#   ~/.spack/modules.yaml
# -------------------------------------------------------------------------
modules:
  enable:
    - tcl
    - dotkit
  prefix_inspections:
    bin:
      - PATH
    man:
      - MANPATH
    share/man:
      - MANPATH
    lib:
      - LIBRARY_PATH
      - LD_LIBRARY_PATH
    lib64:
      - LIBRARY_PATH
      - LD_LIBRARY_PATH
    include:
      - CPATH
    lib/pkgconfig:
      - PKG_CONFIG_PATH
    lib64/pkgconfig:
      - PKG_CONFIG_PATH
    '':
      - CMAKE_PREFIX_PATH
```

It basically inspects the installation prefixes for the existence of a few folders and, if they exist, it prepends a path to a given list of environment variables.

For each module system that can be enabled a finer configuration is possible:

```
modules:
  tcl:
    # contains environment modules specific customizations
  dotkit:
    # contains dotkit specific customizations
```

The structure under the `tcl` and `dotkit` keys is almost equal, and will be showcased in the following by some examples.

Select module files by spec constraints

Using spec syntax it's possible to have different customizations for different groups of module files.

Considering :

```
modules:
  tcl:
    all: # Default addition for every package
      environment:
        set:
          BAR: 'bar'
    ^openmpi:: # A double ':' overrides previous rules
      environment:
        set:
          BAR: 'baz'
    zlib:
      environment:
        prepend_path:
          LD_LIBRARY_PATH: 'foo'
    zlib%gcc@4.8:
      environment:
        unset:
        - FOOBAR
```

what will happen is that:

- every module file will set `BAR=bar`
- unless the associated spec satisfies `^openmpi` in which case `BAR=baz`
- any spec that satisfies `zlib` will additionally prepend `foo` to `LD_LIBRARY_PATH`
- any spec that satisfies `zlib%gcc@4.8` will additionally unset `FOOBAR`

Note:

Order does matter The modifications associated with the `all` keyword are always evaluated first, no matter where they appear in the configuration file. All the other spec constraints are instead evaluated top to bottom.

Filter modifications out of module files

Modifications to certain environment variables in module files are generated by default. Suppose you would like to avoid having `CPATH` and `LIBRARY_PATH` modified by your dotkit modules. Then :

```
modules:
  dotkit:
    all:
      filter:
        # Exclude changes to any of these variables
        environment_blacklist: ['CPATH', 'LIBRARY_PATH']
```

will generate dotkit module files that will not contain modifications to either `CPATH` or `LIBRARY_PATH` and environment module files that instead will contain those modifications.

Autoload dependencies

The following lines in `modules.yaml`:

```
modules:
  tcl:
    all:
      autoload: 'direct'
```

will produce environment module files that will automatically load their direct dependencies.

Note:

Allowed values for `autoload` statements Allowed values for `autoload` statements are either `none`, `direct` or `all`. In `tcl` configuration it is possible to use the option `prerequisites` that accepts the same values and will add `prereq` statements instead of automatically loading other modules.

Blacklist or whitelist the generation of specific module files

Sometimes it is desirable not to generate module files, a common use case being not providing the users with software built using the system compiler.

A configuration file like:

```
modules:
  tcl:
    whitelist: ['gcc', 'llvm']  # Whitelist will have precedence over blacklist
    blacklist: ['%gcc@4.4.7']   # Assuming gcc@4.4.7 is the system compiler
```

will skip module file generation for anything that satisfies `%gcc@4.4.7`, with the exception of specs that satisfy `gcc` or `llvm`.

Customize the naming scheme and insert conflicts

A configuration file like:

```
modules:
  tcl:
    naming_scheme: '{name}/{version}-{compiler.name}-{compiler.version}'
    all:
      conflict: ['{name}', 'intel/14.0.1']
```

will create module files that will conflict with `intel/14.0.1` and with the base directory of the same module, effectively preventing the possibility to load two or more versions of the same software at the same time.

Note:

Tokens available for the naming scheme currently only the tokens shown in the example are available to construct the naming scheme

---

Note: The `conflict` option is `tcl` specific

3.6.10 Regenerating Module files

Module and dotkit files are generated when packages are installed, and are placed in the directory `share/spack/modules` under the Spack root. The command `spack refresh` will regenerate them all without re-building the packages; for example, if module format or options have changed:

```
$ spack module refresh
==> Regenerating tcl module files.
==> Regenerating dotkit module files.
```

3.7 Extensions & Python support

Spack's installation model assumes that each package will live in its own install prefix. However, certain packages are typically installed *within* the directory hierarchy of other packages. For example, modules in interpreted languages like Python are typically installed in the `$prefix/lib/python-2.7/site-packages` directory.

Spack has support for this type of installation as well. In Spack, a package that can live inside the prefix of another package is called an *extension*. Suppose you have Python installed like so:

```
$ spack find python
==> 1 installed packages.
-- linux-debian7-x86_64 / gcc@4.4.7 --------------------------------
python@2.7.8
```

3.7.1 `spack extensions`

You can find extensions for your Python installation like this:

```
$ spack extensions python
==> python@2.7.8%gcc@4.4.7 arch=linux-debian7-x86_64-703c7a96
==> 36 extensions:
geos           py-ipython     py-pexpect    py-pyside            py-sip
py-basemap     py-libxml2     py-pil        py-pytz              py-six
py-biopython   py-mako        py-pmw        py-rpy2              py-sympy
py-cython      py-matplotlib  py-pychecker  py-scientificpython  py-virtualenv
py-dateutil    py-mpi4py      py-pygments   py-scikit-learn
py-epydoc      py-mx          py-pylint     py-scipy
py-gnuplot     py-nose        py-pyparsing  py-setuptools
py-h5py        py-numpy       py-pyqt       py-shiboken

==> 12 installed:
-- linux-debian7-x86_64 / gcc@4.4.7 --------------------------------
py-dateutil@2.4.0    py-nose@1.3.4       py-pyside@1.2.2
py-dateutil@2.4.0    py-numpy@1.9.1      py-pytz@2014.10
py-ipython@2.3.1     py-pygments@2.0.1   py-setuptools@11.3.1
py-matplotlib@1.4.2  py-pyparsing@2.0.3  py-six@1.9.0

==> None activated.
```

The extensions are a subset of what's returned by `spack list`, and they are packages like any other. They are installed into their own prefixes, and you can see this with `spack find --paths`:

```
$ spack find --paths py-numpy
==> 1 installed packages.
```

```
-- linux-debian7-x86_64 / gcc@4.4.7 ------------------------------
    py-numpy@1.9.1  /g/g21/gamblin2/src/spack/opt/linux-debian7-x86_64/gcc@4.4.7/py-numpy@1.9.1-66733
```

However, even though this package is installed, you cannot use it directly when you run `python`:

```
$ spack load python
$ python
Python 2.7.8 (default, Feb 17 2015, 01:35:25)
[GCC 4.4.7 20120313 (Red Hat 4.4.7-11)] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ImportError: No module named numpy
>>>
```

3.7.2 Extensions & Environment Modules

There are two ways to get `numpy` working in Python. The first is to use *Integration with module systems*. You can simply `use` or `load` the module for the extension, and it will be added to the `PYTHONPATH` in your current shell.

For tcl modules:

```
$ spack load python
$ spack load py-numpy
```

or, for dotkit:

```
$ spack use python
$ spack use py-numpy
```

Now `import numpy` will succeed for as long as you keep your current session open.

3.7.3 Activating Extensions

It is often desirable to have certain packages *always* available as part of a Python installation. Spack offers a more permanent solution for this case. Instead of requiring users to load particular environment modules, you can *activate* the package within the Python installation:

3.7.4 `spack activate`

```
$ spack activate py-numpy
==> Activated extension py-setuptools@11.3.1%gcc@4.4.7 arch=linux-debian7-x86_64-3c74eb69 for python@
==> Activated extension py-nose@1.3.4%gcc@4.4.7 arch=linux-debian7-x86_64-5f70f816 for python@2.7.8%g
==> Activated extension py-numpy@1.9.1%gcc@4.4.7 arch=linux-debian7-x86_64-66733244 for python@2.7.8%
```

Several things have happened here. The user requested that `py-numpy` be activated in the `python` installation it was built with. Spack knows that `py-numpy` depends on `py-nose` and `py-setuptools`, so it activated those packages first. Finally, once all dependencies were activated in the `python` installation, `py-numpy` was activated as well.

If we run `spack extensions` again, we now see the three new packages listed as activated:

```
$ spack extensions python
==> python@2.7.8%gcc@4.4.7  arch=linux-debian7-x86_64-703c7a96
==> 36 extensions:
geos          py-ipython     py-pexpect    py-pyside            py-sip
py-basemap    py-libxml2     py-pil        py-pytz              py-six
py-biopython  py-mako        py-pmw        py-rpy2              py-sympy
py-cython     py-matplotlib  py-pychecker  py-scientificpython  py-virtualenv
py-dateutil   py-mpi4py      py-pygments   py-scikit-learn
py-epydoc     py-mx          py-pylint     py-scipy
py-gnuplot    py-nose        py-pyparsing  py-setuptools
py-h5py       py-numpy       py-pyqt       py-shiboken

==> 12 installed:
-- linux-debian7-x86_64 / gcc@4.4.7 --------------------------------
py-dateutil@2.4.0    py-nose@1.3.4       py-pyside@1.2.2
py-dateutil@2.4.0    py-numpy@1.9.1      py-pytz@2014.10
py-ipython@2.3.1     py-pygments@2.0.1   py-setuptools@11.3.1
py-matplotlib@1.4.2  py-pyparsing@2.0.3  py-six@1.9.0

==> 3 currently activated:
-- linux-debian7-x86_64 / gcc@4.4.7 --------------------------------
py-nose@1.3.4  py-numpy@1.9.1  py-setuptools@11.3.1
```

Now, when a user runs python, `numpy` will be available for import *without* the user having to explicitly loaded. `python@2.7.8` now acts like a system Python installation with `numpy` installed inside of it.

Spack accomplishes this by symbolically linking the *entire* prefix of the `py-numpy` into the prefix of the `python` package. To the python interpreter, it looks like `numpy` is installed in the `site-packages` directory.

The only limitation of activation is that you can only have a *single* version of an extension activated at a time. This is because multiple versions of the same extension would conflict if symbolically linked into the same prefix. Users who want a different version of a package can still get it by using environment modules, but they will have to explicitly load their preferred version.

3.7.5 `spack activate -f`

If, for some reason, you want to activate a package *without* its dependencies, you can use `spack activate -f`:

```
$ spack activate -f py-numpy
==> Activated extension py-numpy@1.9.1%gcc@4.4.7 arch=linux-debian7-x86_64-66733244 for python@2.7.8
```

3.7.6 `spack deactivate`

We've seen how activating an extension can be used to set up a default version of a Python module. Obviously, you may want to change that at some point. `spack deactivate` is the command for this. There are several variants:

- `spack deactivate <extension>` will deactivate a single extension. If another activated extension depends on this one, Spack will warn you and exit with an error.
- `spack deactivate --force <extension>` deactivates an extension regardless of packages that depend on it.
- `spack deactivate --all <extension>` deactivates an extension and all of its dependencies. Use `--force` to disregard dependents.
- `spack deactivate --all <extendee>` deactivates *all* activated extensions of a package. For example, to deactivate *all* python extensions, use:

```
$ spack deactivate --all python
```

3.8 Filesystem requirements

Spack currently needs to be run from a filesystem that supports `flock` locking semantics. Nearly all local filesystems and recent versions of NFS support this, but parallel filesystems may be mounted without `flock` support enabled. You can determine how your filesystems are mounted with `mount -p`. The output for a Lustre filesystem might look like this:

```
$ mount -l | grep lscratch
pilsner-mds1-lnet0@o2ib100:/lsd on /p/lscratchd type lustre (rw,nosuid,noauto,_netdev,lazystatfs,floc
porter-mds1-lnet0@o2ib100:/lse on /p/lscratche type lustre (rw,nosuid,noauto,_netdev,lazystatfs,flock
```

Note the `flock` option on both Lustre mounts. If you do not see this or a similar option for your filesystem, you may need ot ask your system administrator to enable `flock`.

This issue typically manifests with the error below:

```
$ ./spack find
Traceback (most recent call last):
File "./spack", line 176, in <module>
  main()
File "./spack", line 154,' in main
  return_val = command(parser, args)
File "./spack/lib/spack/spack/cmd/find.py", line 170, in find
  specs = set(spack.installed_db.query(\**q_args))
File "./spack/lib/spack/spack/database.py", line 551, in query
  with self.read_transaction():
File "./spack/lib/spack/spack/database.py", line 598, in __enter__
  if self._enter() and self._acquire_fn:
File "./spack/lib/spack/spack/database.py", line 608, in _enter
  return self._db.lock.acquire_read(self._timeout)
File "./spack/lib/spack/llnl/util/lock.py", line 103, in acquire_read
  self._lock(fcntl.LOCK_SH, timeout)   # can raise LockError.
File "./spack/lib/spack/llnl/util/lock.py", line 64, in _lock
  fcntl.lockf(self._fd, op | fcntl.LOCK_NB)
IOError: [Errno 38] Function not implemented
```

A nicer error message is TBD in future versions of Spack.

3.9 Getting Help

3.9.1 `spack help`

If you don't find what you need here, the `help` subcommand will print out out a list of *all* of spack's options and subcommands:

```
$ spack help
usage: spack [-h] [-d] [-D] [-k] [-m] [-p] [-v] [-V] SUBCOMMAND ...

Spack: the Supercomputing PACKage Manager.

spec expressions:
  PACKAGE [CONSTRAINTS]
```

```
    CONSTRAINTS:
      [0;36m@version[0m
      [0;32m%compiler  @compiler_version[0m
      [0;94m+variant[0m
      [0;31m-variant[0m or [0;31m~variant[0m
      [0;35m=architecture[0m
      [^DEPENDENCY [CONSTRAINTS] ...]

positional arguments:
  SUBCOMMAND
    activate      Activate a package extension.
    arch          Print the architecture for this machine
    bootstrap     Create a new installation of spack in another prefix
    cd            cd to spack directories in the shell.
    checksum      Checksum available versions of a package.
    clean         Remove build stage and source tarball for packages.
    compiler      Manage compilers
    compilers     List available compilers. Same as 'spack compiler list'.
    config        Get and set configuration options.
    create        Create a new package file from an archive URL
    deactivate    Deactivate a package extension.
    debug         Debugging commands for troubleshooting Spack.
    dependents    Show installed packages that depend on another.
    diy           Do-It-Yourself: build from an existing source directory.
    doc           Run pydoc from within spack.
    edit          Open package files in $EDITOR
    env           Run a command with the install environment for a spec.
    extensions    List extensions for package.
    fetch         Fetch archives for packages
    find          Find installed spack packages
    graph         Generate graphs of package dependency relationships.
    help          Get help on spack and its commands
    info          Get detailed information on a particular package
    install       Build and install packages
    list          List available spack packages
    load          Add package to environment using modules.
    location      Print out locations of various directories used by Spack
    md5           Calculate md5 checksums for files/urls.
    mirror        Manage mirrors.
    module        Manipulate module files
    package-list  Print a list of all packages in reStructuredText.
    patch         Patch expanded archive sources in preparation for install
    pkg           Query packages associated with particular git revisions.
    providers     List packages that provide a particular virtual package
    purge         Remove temporary build files and/or downloaded archives
    python        Launch an interpreter as spack would launch a command
    reindex       Rebuild Spack's package database.
    repo          Manage package source repositories.
    restage       Revert checked out package source code.
    setup         Create a configuration script and module, but don't build.
    spec          print out abstract and concrete versions of a spec.
    stage         Expand downloaded archive in preparation for install
    test          Run unit tests
    test-install  Run package install as a unit test, output formatted results.
    uninstall     Remove an installed package
    unload        Remove package from environment using module.
    unuse         Remove package from environment using dotkit.
    url-parse     Show parsing of a URL, optionally spider web for versions.
```

```
    urls          Inspect urls used by packages in spack.
    use           Add package to environment using dotkit.
    versions      List available versions of a package
    view          Produce a single-rooted directory view of a spec.

optional arguments:
  -h, --help      show this help message and exit
  -d, --debug     Write out debug logs during compile
  -D, --pdb       Run spack under the pdb debugger
  -k, --insecure  Do not check ssl certificates when downloading.
  -m, --mock      Use mock packages instead of real ones.
  -p, --profile   Profile execution using cProfile.
  -v, --verbose   Print additional output during builds
  -V, --version   show program's version number and exit
```

Adding an argument, e.g. `spack help <subcommand>`, will print out usage information for a particular subcommand:

```
$ spack help install
usage: spack install [-h] [-i] [-j JOBS] [--keep-prefix] [--keep-stage] [-n]
                     [-v] [--fake] [--dirty] [--run-tests]
                     ...

positional arguments:
  packages              specs of packages to install

optional arguments:
  -h, --help            show this help message and exit
  -i, --ignore-dependencies
                        Do not try to install dependencies of requested
                        packages.
  -j JOBS, --jobs JOBS  Explicitly set number of make jobs. Default is #cpus.
  --keep-prefix         Don't remove the install prefix if installation fails.
  --keep-stage          Don't remove the build stage if installation succeeds.
  -n, --no-checksum     Do not check packages against checksum
  -v, --verbose         Display verbose build output while installing.
  --fake                Fake install. Just remove prefix and create a fake
                        file.
  --dirty               Install a package *without* cleaning the environment.
  --run-tests           Run tests during installation of a package.
```

Alternately, you can use `spack --help` in place of `spack help`, or `spack <subcommand> --help` to get help on a particular subcommand.

CHAPTER 4

Workflows

The process of using Spack involves building packages, running binaries from those packages, and developing software that depends on those packages. For example, one might use Spack to build the `netcdf` package, use `spack load` to run the `ncdump` binary, and finally, write a small C program to read/write a particular NetCDF file.

Spack supports a variety of workflows to suit a variety of situations and user preferences, there is no single way to do all these things. This chapter demonstrates different workflows that have been developed, pointing out the pros and cons of them.

4.1 Definitions

First some basic definitions.

4.1.1 Package, Concrete Spec, Installed Package

In Spack, a package is an abstract recipe to build one piece of software. Spack packages may be used to build, in principle, any version of that software with any set of variants. Examples of packages include `curl` and `zlib`.

A package may be *instantiated* to produce a concrete spec; one possible realization of a particular package, out of combinatorially many other realizations. For example, here is a concrete spec instantiated from `curl`:

```
curl@7.50.1%gcc@5.3.0 arch=linux-SuSE11-x86_64
    ^openssl@system%gcc@5.3.0 arch=linux-SuSE11-x86_64
    ^zlib@1.2.8%gcc@5.3.0 arch=linux-SuSE11-x86_64
```

Spack's core concretization algorithm generates concrete specs by instantiating packages from its repo, based on a set of "hints", including user input and the `packages.yaml` file. This algorithm may be accessed at any time with the `spack spec` command. For example:

```
$ spack spec curl
  curl@7.50.1%gcc@5.3.0 arch=linux-SuSE11-x86_64
      ^openssl@system%gcc@5.3.0 arch=linux-SuSE11-x86_64
      ^zlib@1.2.8%gcc@5.3.0 arch=linux-SuSE11-x86_64
```

Every time Spack installs a package, that installation corresponds to a concrete spec. Only a vanishingly small fraction of possible concrete specs will be installed at any one Spack site.

4.1.2 Consistent Sets

A set of Spack specs is said to be *consistent* if each package is only instantiated one way within it — that is, if two specs in the set have the same package, then they must also have the same version, variant, compiler, etc. For example, the following set is consistent:

```
curl@7.50.1%gcc@5.3.0 arch=linux-SuSE11-x86_64
    ^openssl@system%gcc@5.3.0 arch=linux-SuSE11-x86_64
    ^zlib@1.2.8%gcc@5.3.0 arch=linux-SuSE11-x86_64
zlib@1.2.8%gcc@5.3.0 arch=linux-SuSE11-x86_64
```

The following set is not consistent:

```
curl@7.50.1%gcc@5.3.0 arch=linux-SuSE11-x86_64
    ^openssl@system%gcc@5.3.0 arch=linux-SuSE11-x86_64
    ^zlib@1.2.8%gcc@5.3.0 arch=linux-SuSE11-x86_64
zlib@1.2.7%gcc@5.3.0 arch=linux-SuSE11-x86_64
```

The compatibility of a set of installed packages determines what may be done with it. It is always possible to `spack load` any set of installed packages, whether or not they are consistent, and run their binaries from the command line. However, a set of installed packages can only be linked together in one binary if it is consistent.

If the user produces a series of `spack spec` or `spack load` commands, in general there is no guarantee of consistency between them. Spack's concretization procedure guarantees that the results of any *single* `spack spec` call will be consistent. Therefore, the best way to ensure a consistent set of specs is to create a Spack package with dependencies, and then instantiate that package. We will use this technique below.

4.2 Building Packages

Suppose you are tasked with installing a set of software packages on a system in order to support one application – both a core application program, plus software to prepare input and analyze output. The required software might be summed up as a series of `spack install` commands placed in a script. If needed, this script can always be run again in the future. For example:

```
#!/bin/sh
spack install modele-utils
spack install emacs
spack install ncview
spack install nco
spack install modele-control
spack install py-numpy
```

In most cases, this script will not correctly install software according to your specific needs: choices need to be made for variants, versions and virtual dependency choices may be needed. It *is* possible to specify these choices by extending specs on the command line; however, the same choices must be specified repeatedly. For example, if you wish to use `openmpi` to satisfy the `mpi` dependency, then `^openmpi` will have to appear on *every* `spack install` line that uses MPI. It can get repetitive fast.

Customizing Spack installation options is easier to do in the `~/.spack/packages.yaml` file. In this file, you can specify preferred versions and variants to use for packages. For example:

```
packages:
    python:
        version: [3.5.1]
    modele-utils:
        version: [cmake]
```

```
    everytrace:
        version: [develop]
    eigen:
        variants: ~suitesparse
    netcdf:
        variants: +mpi

    all:
        compiler: [gcc@5.3.0]
        providers:
            mpi: [openmpi]
            blas: [openblas]
            lapack: [openblas]
```

This approach will work as long as you are building packages for just one application.

4.2.1 Multiple Applications

Suppose instead you're building multiple inconsistent applications. For example, users want package A to be built with `openmpi` and package B with `mpich` — but still share many other lower-level dependencies. In this case, a single `packages.yaml` file will not work. Plans are to implement *per-project* `packages.yaml` files. In the meantime, one could write shell scripts to switch `packages.yaml` between multiple versions as needed, using symlinks.

4.2.2 Combinatorial Sets of Installs

Suppose that you are now tasked with systematically building many incompatible versions of packages. For example, you need to build `petsc` 9 times for 3 different MPI implementations on 3 different compilers, in order to support user needs. In this case, you will need to either create 9 different `packages.yaml` files; or more likely, create 9 different `spack install` command lines with the correct options in the spec. Here is a real-life example of this kind of usage:

```
#!/bin/sh
#

compilers=(
    %gcc
    %intel
    %pgi
)

mpis=(
    openmpi+psm~verbs
    openmpi~psm+verbs
    mvapich2+psm~mrail
    mvapich2~psm+mrail
    mpich+verbs
)

for compiler in "${compilers[@]}"
do
    # Serial installs
    spack install szip           $compiler
    spack install hdf            $compiler
    spack install hdf5           $compiler
    spack install netcdf         $compiler
```

```
    spack install netcdf-fortran $compiler
    spack install ncview         $compiler

    # Parallel installs
    for mpi in "${mpis[@]}"
    do
        spack install $mpi            $compiler
        spack install hdf5~cxx+mpi    $compiler ^$mpi
        spack install parallel-netcdf $compiler ^$mpi
    done
done
```

4.3 Running Binaries from Packages

Once Spack packages have been built, the next step is to use them. As with building packages, there are many ways to use them, depending on the use case.

4.3.1 Find and Run

The simplest way to run a Spack binary is to find it and run it! In many cases, nothing more is needed because Spack builds binaries with RPATHs. Spack installation directories may be found with `spack location -i` commands. For example:

```
$ spack location -i cmake
/home/me/spack2/opt/spack/linux-SuSE11-x86_64/gcc-5.3.0/cmake-3.6.0-7cxrynb6esss6jognj23ak55fgxkwtx7
```

This gives the root of the Spack package; relevant binaries may be found within it. For example:

```
$ CMAKE=`spack location -i cmake`/bin/cmake
```

Standard UNIX tools can find binaries as well. For example:

```
$ find ~/spack2/opt -name cmake | grep bin
/home/me/spack2/opt/spack/linux-SuSE11-x86_64/gcc-5.3.0/cmake-3.6.0-7cxrynb6esss6jognj23ak55fgxkwtx7/
```

These methods are suitable, for example, for setting up build processes or GUIs that need to know the location of particular tools. However, other more powerful methods are generally preferred for user environments.

4.3.2 Spack-Generated Modules

Suppose that Spack has been used to install a set of command-line programs, which users now wish to use. One can in principle put a number of `spack load` commands into `.bashrc`, for example, to load a set of Spack-generated modules:

```
spack load modele-utils
spack load emacs
spack load ncview
spack load nco
spack load modele-control
```

Although simple load scripts like this are useful in many cases, they have some drawbacks:

1. The set of modules loaded by them will in general not be consistent. They are a decent way to load commands to be called from command shells. See below for better ways to assemble a consistent set of packages for building application programs.
2. The `spack spec` and `spack install` commands use a sophisticated concretization algorithm that chooses the "best" among several options, taking into account `packages.yaml` file. The `spack load` and `spack module loads` commands, on the other hand, are not very smart: if the user-supplied spec matches more than one installed package, then `spack module loads` will fail. This may change in the future. For now, the workaround is to be more specific on any `spack module loads` lines that fail.

Generated Load Scripts

Another problem with using *spack load* is, it is slow; a typical user environment could take several seconds to load, and would not be appropriate to put into `.bashrc` directly. It is preferable to use a series of `spack module loads` commands to pre-compute which modules to load. These can be put in a script that is run whenever installed Spack packages change. For example:

```
#!/bin/sh
#
# Generate module load commands in ~/env/spackenv

cat <<EOF | /bin/sh >$HOME/env/spackenv
FIND='spack module loads --prefix linux-SuSE11-x86_64/'

\$FIND modele-utils
\$FIND emacs
\$FIND ncview
\$FIND nco
\$FIND modele-control
EOF
```

The output of this file is written in `~/env/spackenv`:

```
# binutils@2.25%gcc@5.3.0+gold~krellpatch~libiberty arch=linux-SuSE11-x86_64
module load linux-SuSE11-x86_64/binutils-2.25-gcc-5.3.0-6w5d2t4
# python@2.7.12%gcc@5.3.0~tk~ucs4 arch=linux-SuSE11-x86_64
module load linux-SuSE11-x86_64/python-2.7.12-gcc-5.3.0-2azoju2
# ncview@2.1.7%gcc@5.3.0 arch=linux-SuSE11-x86_64
module load linux-SuSE11-x86_64/ncview-2.1.7-gcc-5.3.0-uw3knq2
# nco@4.5.5%gcc@5.3.0 arch=linux-SuSE11-x86_64
module load linux-SuSE11-x86_64/nco-4.5.5-gcc-5.3.0-7aqmimu
# modele-control@develop%gcc@5.3.0 arch=linux-SuSE11-x86_64
module load linux-SuSE11-x86_64/modele-control-develop-gcc-5.3.0-7rddsij
# zlib@1.2.8%gcc@5.3.0 arch=linux-SuSE11-x86_64
module load linux-SuSE11-x86_64/zlib-1.2.8-gcc-5.3.0-fe5onbi
# curl@7.50.1%gcc@5.3.0 arch=linux-SuSE11-x86_64
module load linux-SuSE11-x86_64/curl-7.50.1-gcc-5.3.0-4vlev55
# hdf5@1.10.0-patch1%gcc@5.3.0+cxx~debug+fortran+mpi+shared~szip~threadsafe arch=linux-SuSE11-x86_64
module load linux-SuSE11-x86_64/hdf5-1.10.0-patch1-gcc-5.3.0-pwnsr4w
# netcdf@4.4.1%gcc@5.3.0~hdf4+mpi arch=linux-SuSE11-x86_64
module load linux-SuSE11-x86_64/netcdf-4.4.1-gcc-5.3.0-rl5canv
# netcdf-fortran@4.4.4%gcc@5.3.0 arch=linux-SuSE11-x86_64
module load linux-SuSE11-x86_64/netcdf-fortran-4.4.4-gcc-5.3.0-stdk2xq
# modele-utils@cmake%gcc@5.3.0+aux+diags+ic arch=linux-SuSE11-x86_64
module load linux-SuSE11-x86_64/modele-utils-cmake-gcc-5.3.0-idyjul5
# everytrace@develop%gcc@5.3.0+fortran+mpi arch=linux-SuSE11-x86_64
module load linux-SuSE11-x86_64/everytrace-develop-gcc-5.3.0-p5wmb25
```

Users may now put `source ~/env/spackenv` into `.bashrc`.

Note: Some module systems put a prefix on the names of modules created by Spack. For example, that prefix is `linux-SuSE11-x86_64/` in the above case. If a prefix is not needed, you may omit the `--prefix` flag from `spack module loads`.

Transitive Dependencies

In the script above, each `spack module loads` command generates a *single* `module load` line. Transitive dependencies do not usually need to be loaded, only modules the user needs in in `$PATH`. This is because Spack builds binaries with RPATH. Spack's RPATH policy has some nice features:

1. Modules for multiple inconsistent applications may be loaded simultaneously. In the above example (Multiple Applications), package A and package B can coexist together in the user's $PATH, even though they use different MPIs.
2. RPATH eliminates a whole class of strange errors that can happen in non-RPATH binaries when the wrong `LD_LIBRARY_PATH` is loaded.
3. Recursive module systems such as LMod are not necessary.
4. Modules are not needed at all to execute binaries. If a path to a binary is known, it may be executed. For example, the path for a Spack-built compiler can be given to an IDE without requiring the IDE to load that compiler's module.

Unfortunately, Spack's RPATH support does not work in all case. For example:

1. Software comes in many forms — not just compiled ELF binaries, but also as interpreted code in Python, R, JVM bytecode, etc. Those systems almost universally use an environment variable analogous to `LD_LIBRARY_PATH` to dynamically load libraries.
2. Although Spack generally builds binaries with RPATH, it does not currently do so for compiled Python extensions (for example, `py-numpy`). Any libraries that these extensions depend on (`blas` in this case, for example) must be specified in the `LD_LIBRARY_PATH`.‘
3. In some cases, Spack-generated binaries end up without a functional RPATH for no discernible reason.

In cases where RPATH support doesn't make things "just work," it can be necessary to load a module's dependencies as well as the module itself. This is done by adding the `--dependencies` flag to the `spack module loads` command. For example, the following line, added to the script above, would be used to load SciPy, along with Numpy, core Python, BLAS/LAPACK and anything else needed:

```
spack module loads --dependencies py-scipy
```

4.3.3 Extension Packages

Extensions may be used as an alternative to loading Python (and similar systems) packages directly. If extensions are activated, then `spack load python` will also load all the extensions activated for the given `python`. This reduces the need for users to load a large number of modules.

However, Spack extensions have two potential drawbacks:

1. Activated packages that involve compiled C extensions may still need their dependencies to be loaded manually. For example, `spack load openblas` might be required to make `py-numpy` work.
2. Extensions "break" a core feature of Spack, which is that multiple versions of a package can co-exist side-by-side. For example, suppose you wish to run a Python package in two different environments but the same basic

Python — one with `py-numpy@1.7` and one with `py-numpy@1.8`. Spack extensions will not support this potential debugging use case.

4.3.4 Dummy Packages

As an alternative to a series of `module load` commands, one might consider dummy packages as a way to create a *consistent* set of packages that may be loaded as one unit. The idea here is pretty simple:

1. Create a package (say, `mydummy`) with no URL and no `install()` method, just dependencies.
2. Run `spack install mydummy` to install.

An advantage of this method is the set of packages produced will be consistent. This means that you can reliably build software against it. A disadvantage is the set of packages will be consistent; this means you cannot load up two applications this way if they are not consistent with each other.

4.3.5 Filesystem Views

Filesystem views offer an alternative to environment modules, another way to assemble packages in a useful way and load them into a user's environment.

A filesystem view is a single directory tree that is the union of the directory hierarchies of a number of installed packages; it is similar to the directory hiearchy that might exist under `/usr/local`. The files of the view's installed packages are brought into the view by symbolic or hard links, referencing the original Spack installation.

When software is built and installed, absolute paths are frequently "baked into" the software, making it non-relocatable. This happens not just in RPATHs, but also in shebangs, configuration files, and assorted other locations.

Therefore, programs run out of a Spack view will typically still look in the original Spack-installed location for shared libraries and other resources. This behavior is not easily changed; in general, there is no way to know where absolute paths might be written into an installed package, and how to relocate it. Therefore, the original Spack tree must be kept in place for a filesystem view to work, even if the view is built with hardlinks.

Using Filesystem Views

A filesystem view is created, and packages are linked in, by the `spack view` command's `symlink` and `hardlink` sub-commands. The `spack view remove` command can be used to unlink some or all of the filesystem view.

The following example creates a filesystem view based on an installed `cmake` package and then removes from the view the files in the `cmake` package while retaining its dependencies.

```
$ spack view --verbose symlink myview cmake@3.5.2
==> Linking package: "ncurses"
==> Linking package: "zlib"
==> Linking package: "openssl"
==> Linking package: "cmake"

$ ls myview/
bin  doc  etc  include  lib  share

$ ls myview/bin/
captoinfo  clear  cpack     ctest    infotocap        openssl  tabs  toe   tset
ccmake     cmake  c_rehash  infocmp  ncurses6-config  reset    tic   tput

$ spack view --verbose --dependencies false rm myview cmake@3.5.2
==> Removing package: "cmake"
```

```
$ ls myview/bin/
captoinfo  c_rehash  infotocap        openssl  tabs  toe   tset
clear      infocmp   ncurses6-config  reset    tic   tput
```

Note: If the set of packages being included in a view is inconsistent, then it is possible that two packages will provide the same file. Any conflicts of this type are handled on a first-come-first-served basis, and a warning is printed.

Note: When packages are removed from a view, empty directories are purged.

Fine-Grain Control

The `--exclude` and `--dependencies` option flags allow for fine-grained control over which packages and dependencies do or not get included in a view. For example, suppose you are developing the `appsy` package. You wish to build against a view of all `appsy` dependencies, but not `appsy` itself:

```
$ spack view symlink --dependencies yes --exclude appsy appsy
```

Alternately, you wish to create a view whose purpose is to provide binary executables to end users. You only need to include applications they might want, and not those applications' dependencies. In this case, you might use:

```
$ spack view symlink --dependencies no cmake
```

Hybrid Filesystem Views

Although filesystem views are usually created by Spack, users are free to add to them by other means. For example, imagine a filesystem view, created by Spack, that looks something like:

```
/path/to/MYVIEW/bin/programA -> /path/to/spack/.../bin/programA
/path/to/MYVIEW/lib/libA.so -> /path/to/spack/.../lib/libA.so
```

Now, the user may add to this view by non-Spack means; for example, by running a classic install script. For example:

```
$ tar -xf B.tar.gz
$ cd B/
$ ./configure --prefix=/path/to/MYVIEW \
            --with-A=/path/to/MYVIEW
$ make && make install
```

The result is a hybrid view:

```
/path/to/MYVIEW/bin/programA -> /path/to/spack/.../bin/programA
/path/to/MYVIEW/bin/programB
/path/to/MYVIEW/lib/libA.so -> /path/to/spack/.../lib/libA.so
/path/to/MYVIEW/lib/libB.so
```

In this case, real files coexist, interleaved with the "view" symlinks. At any time one can delete `/path/to/MYVIEW` or use `spack view` to manage it surgically. None of this will affect the real Spack install area.

4.3.6 Discussion: Running Binaries

Modules, extension packages and filesystem views are all ways to assemble sets of Spack packages into a useful environment. They are all semantically similar, in that conflicting installed packages cannot simultaneously be loaded, activated or included in a view.

With all of these approaches, there is no guarantee that the environment created will be consistent. It is possible, for example, to simultaneously load application A that uses OpenMPI and application B that uses MPICH. Both applications will run just fine in this inconsistent environment because they rely on RPATHs, not the environment, to find their dependencies.

In general, environments set up using modules vs. views will work similarly. Both can be used to set up ephemeral or long-lived testing/development environments. Operational differences between the two approaches can make one or the other preferable in certain environments:

- Filesystem views do not require environment module infrastructure. Although Spack can install `environment-modules`, users might be hostile to its use. Filesystem views offer a good solution for sysadmins serving users who just "want all the stuff I need in one place" and don't want to hear about Spack.
- Although modern build systems will find dependencies wherever they might be, some applications with hand-built make files expect their dependencies to be in one place. One common problem is makefiles that assume that `netcdf` and `netcdf-fortran` are installed in the same tree. Or, one might use an IDE that requires tedious configuration of dependency paths; and it's easier to automate that administration in a view-building script than in the IDE itself. For all these cases, a view will be preferable to other ways to assemble an environment.
- On systems with I-node quotas, modules might be preferable to views and extension packages.
- Views and activated extensions maintain state that is semantically equivalent to the information in a `spack module loads` script. Administrators might find things easier to maintain without the added "heavyweight" state of a view.

CHAPTER 5

Developing Software with Spack

For any project, one needs to assemble an environment of that application's dependencies. You might consider loading a series of modules or creating a filesystem view. This approach, while obvious, has some serious drawbacks:

1. There is no guarantee that an environment created this way will be consistent. Your application could end up with dependency A expecting one version of MPI, and dependency B expecting another. The linker will not be happy...
2. Suppose you need to debug a package deep within your software DAG. If you build that package with a manual environment, then it becomes difficult to have Spack auto-build things that depend on it. That could be a serious problem, depending on how deep the package in question is in your dependency DAG.
3. At its core, Spack is a sophisticated concretization algorithm that matches up packages with appropriate dependencies and creates a *consistent* environment for the package it's building. Writing a list of `spack load` commands for your dependencies is at least as hard as writing the same list of `depends_on()` declarations in a Spack package. But it makes no use of Spack concretization and is more error-prone.
4. Spack provides an automated, systematic way not just to find a packages's dependencies — but also to build other packages on top. Any Spack package can become a dependency for another Spack package, offering a powerful vision of software re-use. If you build your package A outside of Spack, then your ability to use it as a building block for other packages in an automated way is diminished: other packages depending on package A will not be able to use Spack to fulfill that dependency.
5. If you are reading this manual, you probably love Spack. You're probably going to write a Spack package for your software so prospective users can install it with the least amount of pain. Why should you go to additional work to find dependencies in your development environment? Shouldn't Spack be able to help you build your software based on the package you've already written?

In this section, we show how Spack can be used in the software development process to greatest effect, and how development packages can be seamlessly integrated into the Spack ecosystem. We will show how this process works by example, assuming the software you are creating is called `mylib`.

5.1 Write the CMake Build

For now, the techniques in this section only work for CMake-based projects, although they could be easily extended to other build systems in the future. We will therefore assume you are using CMake to build your project.

The `CMakeLists.txt` file should be written as normal. A few caveats:

1. Your project should produce binaries with RPATHs. This will ensure that they work the same whether built manually or automatically by Spack. For example:

```
# enable @rpath in the install name for any shared library being built
# note: it is planned that a future version of CMake will enable this by default
set(CMAKE_MACOSX_RPATH 1)

# Always use full RPATH
# http://www.cmake.org/Wiki/CMake_RPATH_handling
# http://www.kitware.com/blog/home/post/510

# use, i.e. don't skip the full RPATH for the build tree
SET(CMAKE_SKIP_BUILD_RPATH  FALSE)

# when building, don't use the install RPATH already
# (but later on when installing)
SET(CMAKE_BUILD_WITH_INSTALL_RPATH FALSE)

# add the automatically determined parts of the RPATH
# which point to directories outside the build tree to the install RPATH
SET(CMAKE_INSTALL_RPATH_USE_LINK_PATH TRUE)

# the RPATH to be used when installing, but only if it's not a system directory
LIST(FIND CMAKE_PLATFORM_IMPLICIT_LINK_DIRECTORIES "${CMAKE_INSTALL_PREFIX}/lib" isSystemDir)
IF("${isSystemDir}" STREQUAL "-1")
   SET(CMAKE_INSTALL_RPATH "${CMAKE_INSTALL_PREFIX}/lib")
ENDIF("${isSystemDir}" STREQUAL "-1")
```

2. Spack provides a CMake variable called `SPACK_TRANSITIVE_INCLUDE_PATH`, which contains the `include/` directory for all of your project's transitive dependencies. It can be useful if your project `#include``s files from package B, which ``#include` files from package C, but your project only lists project B as a dependency. This works in traditional single-tree build environments, in which B and C's include files live in the same place. In order to make it work with Spack as well, you must add the following to `CMakeLists.txt`. It will have no effect when building without Spack:

```
    # Include all the transitive dependencies determined by Spack.
    # If we're not running with Spack, this does nothing...
    include_directories($ENV{SPACK_TRANSITIVE_INCLUDE_PATH})
```

> **Note:** Note that this feature is controversial and could break with future versions of GNU ld. The best practice is to make sure anything you `#include` is listed as a dependency in your CMakeLists.txt (and Spack package).

5.2 Write the Spack Package

The Spack package also needs to be written, in tandem with setting up the build (for example, CMake). The most important part of this task is declaring dependencies. Here is an example of the Spack package for the `mylib` package (ellipses for brevity):

```
class Mylib(CMakePackage):
    """Misc. reusable utilities used by Myapp."""

    homepage = "https://github.com/citibeth/mylib"
    url = "https://github.com/citibeth/mylib/tarball/123"

    version('0.1.2', '3a6acd70085e25f81b63a7e96c504ef9')
    version('develop', git='https://github.com/citibeth/mylib.git',
        branch='develop')
```

```
    variant('everytrace', default=False,
            description='Report errors through Everytrace')
    ...

    extends('python')

    depends_on('eigen')
    depends_on('everytrace', when='+everytrace')
    depends_on('proj', when='+proj')
    ...
    depends_on('cmake', type='build')
    depends_on('doxygen', type='build')

    def configure_args(self):
        spec = self.spec
        return [
            '-DUSE_EVERYTRACE=%s' % ('YES' if '+everytrace' in spec else 'NO'),
            '-DUSE_PROJ4=%s' % ('YES' if '+proj' in spec else 'NO'),
            ...
            '-DUSE_UDUNITS2=%s' % ('YES' if '+udunits2' in spec else 'NO'),
            '-DUSE_GTEST=%s' % ('YES' if '+googletest' in spec else 'NO')]
```

This is a standard Spack package that can be used to install `mylib` in a production environment. The list of dependencies in the Spack package will generally be a repeat of the list of CMake dependencies. This package also has some features that allow it to be used for development:

1. It subclasses `CMakePackage` instead of `Package`. This eliminates the need to write an `install()` method, which is defined in the superclass. Instead, one just needs to write the `configure_args()` method. That method should return the arguments needed for the `cmake` command (beyond the standard CMake arguments, which Spack will include already). These arguments are typically used to turn features on/off in the build.
2. It specifies a non-checksummed version `develop`. Running `spack install mylib@develop` the `@develop` version will install the latest version off the develop branch. This method of download is useful for the developer of a project while it is in active development; however, it should only be used by developers who control and trust the repository in question!
3. The `url`, `url_for_version()` and `homepage` attributes are not used in development. Don't worry if you don't have any, or if they are behind a firewall.

5.3 Build with Spack

Now that you have a Spack package, you can use Spack to find its dependencies automatically. For example:

```
$ cd mylib
$ spack setup mylib@local
```

The result will be a file `spconfig.py` in the top-level `mylib/` directory. It is a short script that calls CMake with the dependencies and options determined by Spack — similar to what happens in `spack install`, but now written out in script form. From a developer's point of view, you can think of `spconfig.py` as a stand-in for the `cmake` command.

Note: You can invent any "version" you like for the `spack setup` command.

Note: Although `spack setup` does not build your package, it does create and install a module file, and mark in

---

the database that your package has been installed. This can lead to errors, of course, if you don't subsequently install your package. Also... you will need to `spack uninstall` before you run `spack setup` again.

You can now build your project as usual with CMake:

```
$ mkdir build; cd build
$ ../spconfig.py ..   # Instead of cmake ..
$ make
$ make install
```

Once your `make install` command is complete, your package will be installed, just as if you'd run `spack install`. Except you can now edit, re-build and re-install as often as needed, without checking into Git or downloading tarballs.

Note: The build you get this way will be *almost* the same as the build from `spack install`. The only difference is, you will not be using Spack's compiler wrappers. This difference has not caused problems in our experience, as long as your project sets RPATHs as shown above. You DO use RPATHs, right?

5.4 Build Other Software

Now that you've built `mylib` with Spack, you might want to build another package that depends on it — for example, `myapp`. This is accomplished easily enough:

```
$ spack install myapp ^mylib@local
```

Note that auto-built software has now been installed *on top of* manually-built software, without breaking Spack's "web." This property is useful if you need to debug a package deep in the dependency hierarchy of your application. It is a *big* advantage of using `spack setup` to build your package's environment.

If you feel your software is stable, you might wish to install it with `spack install` and skip the source directory. You can just use, for example:

```
$ spack install mylib@develop
```

5.5 Release Your Software

You are now ready to release your software as a tarball with a numbered version, and a Spack package that can build it. If you're hosted on GitHub, this process will be a bit easier.

1. Put tag(s) on the version(s) in your GitHub repo you want to be release versions. For example, a tag `v0.1.0` for version 0.1.0.
2. Set the `url` in your `package.py` to download a tarball for the appropriate version. GitHub will give you a tarball for any commit in the repo, if you tickle it the right way. For example:

```
    url = 'https://github.com/citibeth/mylib/tarball/v0.1.2'
```

3. Use Spack to determine your version's hash, and cut'n'paste it into your `package.py`:

```
    $ spack checksum mylib 0.1.2
    ==> Found 1 versions of mylib
      0.1.2     https://github.com/citibeth/mylib/tarball/v0.1.2
```

```
How many would you like to checksum? (default is 5, q to abort)
==> Downloading...
==> Trying to fetch from https://github.com/citibeth/mylib/tarball/v0.1.2
######################################################################## 100.0%
==> Checksummed new versions of mylib:
      version('0.1.2', '3a6acd70085e25f81b63a7e96c504ef9')
```

4. You should now be able to install released version 0.1.2 of your package with:

```
$ spack install mylib@0.1.2
```

5. There is no need to remove the *develop* version from your package. Spack concretization will always prefer numbered version to non-numeric versions. Users will only get it if they ask for it.

5.6 Distribute Your Software

Once you've released your software, other people will want to build it; and you will need to tell them how. In the past, that has meant a few paragraphs of pros explaining which dependencies to install. But now you use Spack, and those instructions are written in executable Python code. But your software has many dependencies, and you know Spack is the best way to install it:

1. First, you will want to fork Spack's `develop` branch. Your aim is to provide a stable version of Spack that you KNOW will install your software. If you make changes to Spack in the process, you will want to submit pull requests to Spack core.
2. Add your software's `package.py` to that fork. You should submit a pull request for this as well, unless you don't want the public to know about your software.
3. Prepare instructions that read approximately as follows:
   (a) Download Spack from your forked repo.
   (b) Install Spack; see *Getting Started*.
   (c) Set up an appropriate `packages.yaml` file. You should tell your users to include in this file whatever versions/variants are needed to make your software work correctly (assuming those are not already in your `packages.yaml`).
   (d) Run `spack install mylib`.
   (e) Run this script to generate the `module load` commands or filesystem view needed to use this software.
4. Be aware that your users might encounter unexpected bootstrapping issues on their machines, especially if they are running on older systems. The *Getting Started* section should cover this, but there could always be issues.

5.7 Other Build Systems

`spack setup` currently only supports CMake-based builds, in packages that subclass `CMakePackage`. The intent is that this mechanism should support a wider range of build systems; for example, GNU Autotools. Someone well-versed in Autotools is needed to develop this patch and test it out.

Python Distutils is another popular build system that should get `spack setup` support. For non-compiled languages like Python, `spack diy` may be used. Even better is to put the source directory directly in the user's `PYTHONPATH`. Then, edits in source files are immediately available to run without any install process at all!

5.8 Conclusion

The `spack setup` development workflow provides better automation, flexibility and safety than workflows relying on environment modules or filesystem views. However, it has some drawbacks:

1. It currently works only with projects that use the CMake build system. Support for other build systems is not hard to build, but will require a small amount of effort for each build system to be supported. It might not work well with some IDEs.
2. It only works with packages that sub-class `StagedPackage`. Currently, most Spack packages do not. Converting them is not hard; but must be done on a package-by-package basis.
3. It requires that users are comfortable with Spack, as they integrate Spack explicitly in their workflow. Not all users are willing to do this.

CHAPTER 6

Upstream Bug Fixes

It is not uncommon to discover a bug in an upstream project while trying to build with Spack. Typically, the bug is in a package that serves a dependency to something else. This section describes procedure to work around and ultimately resolve these bugs, while not delaying the Spack user's main goal.

6.1 Buggy New Version

Sometimes, the old version of a package works fine, but a new version is buggy. For example, it was once found that Adios did not build with hdf5@1.10. If the old version of `hdf5` will work with `adios`, the suggested procedure is:

1. Revert `adios` to the old version of `hdf5`. Put in its `adios/package.py`:

```
# Adios does not build with HDF5 1.10
# See: https://github.com/LLNL/spack/issues/1683
depends_on('hdf5@:1.9')
```

2. Determine whether the problem is with `hdf5` or `adios`, and report the problem to the appropriate upstream project. In this case, the problem was with `adios`.
3. Once a new version of `adios` comes out with the bugfix, modify `adios/package.py` to reflect it:

```
# Adios up to v1.10.0 does not build with HDF5 1.10
# See: https://github.com/LLNL/spack/issues/1683
depends_on('hdf5@:1.9', when='@:1.10.0')
depends_on('hdf5', when='@1.10.1:')
```

6.2 No Version Works

Sometimes, *no* existing versions of a dependency work for a build. This typically happens when developing a new project: only then does the developer notice that existing versions of a dependency are all buggy, or the non-buggy versions are all missing a critical feature.

In the long run, the upstream project will hopefully fix the bug and release a new version. But that could take a while, even if a bugfix has already been pushed to the project's repository. In the meantime, the Spack user needs things to work.

The solution is to create an unofficial Spack release of the project, as soon as the bug is fixed in *some* repository. A study of the Git history of `py-proj/package.py` is instructive here:

1. On April 1, an initial bugfix was identified for the PyProj project and a pull request submitted to PyProj. Because the upstream authors had not yet fixed the bug, the `py-proj` Spack package downloads from a forked repository, set up by the package's author. A non-numeric version number is used to make it easy to upgrade the package without recomputing checksums; however, this is an untrusted download method and should not be distributed. The package author has now become, temporarily, a maintainer of the upstream project:

```
# We need the benefits of this PR
# https://github.com/jswhit/pyproj/pull/54
version('citibeth-latlong2',
    git='https://github.com/citibeth/pyproj.git',
    branch='latlong2')
```

2. By May 14, the upstream project had accepted a pull request with the required bugfix. At this point, the forked repository was deleted. However, the upstream project still had not released a new version with a bugfix. Therefore, a Spack-only release was created by specifying the desired hash in the main project repository. The version number `@1.9.5.1.1` was chosen for this "release" because it's a descendent of the officially released version `@1.9.5.1`. This is a trusted download method, and can be released to the Spack community:

```
# This is not a tagged release of pyproj.
# The changes in this "version" fix some bugs, especially with Python3 use.
version('1.9.5.1.1', 'd035e4bc704d136db79b43ab371b27d2',
    url='https://www.github.com/jswhit/pyproj/tarball/0be612cc9f972e38b50a90c946a9b353e2ab140f')
```

Note: It would have been simpler to use Spack's Git download method, which is also a trusted download in this case:

```
# This is not a tagged release of pyproj.
# The changes in this "version" fix some bugs, especially with Python3 use.
version('1.9.5.1.1',
     git='https://github.com/jswhit/pyproj.git',
     commit='0be612cc9f972e38b50a90c946a9b353e2ab140f')
```

Note: In this case, the upstream project fixed the bug in its repository in a relatively timely manner. If that had not been the case, the numbered version in this step could have been released from the forked repository.

---

3. The author of the Spack package has now become an unofficial release engineer for the upstream project. Depending on the situation, it may be advisable to put `preferred=True` on the latest *officially released* version.

4. As of August 31, the upstream project still had not made a new release with the bugfix. In the meantime, Spack-built `py-proj` provides the bugfix needed by packages depending on it. As long as this works, there is no particular need for the upstream project to make a new official release.

5. If the upstream project releases a new official version with the bugfix, then the unofficial `version()` line should be removed from the Spack package.

6.3 Patches

Spack's source patching mechanism provides another way to fix bugs in upstream projects. This has advantages and disadvantages compared to the procedures above.

Advantages:

1. It can fix bugs in existing released versions, and (probably) future releases as well.

2. It is lightweight, does not require a new fork to be set up.

Disadvantages:

1. It is harder to develop and debug a patch, vs. a branch in a repository. The user loses the automation provided by version control systems.
2. Although patches of a few lines work OK, large patch files can be hard to create and maintain.

CHAPTER 7

Configuration

7.1 Temporary space

Warning: Temporary space configuration will eventually be moved to configuration files, but currently these settings are in `lib/spack/spack/__init__.py`

By default, Spack will try to do all of its building in temporary space. There are two main reasons for this. First, Spack is designed to run out of a user's home directory, and on may systems the home directory is network mounted and potentially not a very fast filesystem. We create build stages in a temporary directory to avoid this. Second, many systems impose quotas on home directories, and `/tmp` or similar directories often have more available space. This helps conserve space for installations in users' home directories.

You can customize temporary directories by editing `lib/spack/spack/__init__.py`. Specifically, find this part of the file:

```
# Whether to build in tmp space or directly in the stage_path.
# If this is true, then spack will make stage directories in
# a tmp filesystem, and it will symlink them into stage_path.
use_tmp_stage = True

# Locations to use for staging and building, in order of preference
# Use a %u to add a username to the stage paths here, in case this
# is a shared filesystem.  Spack will use the first of these paths
# that it can create.
tmp_dirs = ['/nfs/tmp2/%u/spack-stage',
            '/var/tmp/%u/spack-stage',
            '/tmp/%u/spack-stage']
```

The `use_tmp_stage` variable controls whether Spack builds **directly** inside the `var/spack/` directory. Normally, Spack will try to find a temporary directory for a build, then it *symlinks* that temporary directory into `var/spack/` so that you can keep track of what temporary directories Spack is using.

The `tmp_dirs` variable is a list of paths Spack should search when trying to find a temporary directory. They can optionally contain a `%u`, which will substitute the current user's name into the path. The list is searched in order, and Spack will create a temporary stage in the first directory it finds to which it has write access. Add more elements to the list to indicate where your own site's temporary directory is.

7.2 External Packages

Spack can be configured to use externally-installed packages rather than building its own packages. This may be desirable if machines ship with system packages, such as a customized MPI that should be used instead of Spack building its own MPI.

External packages are configured through the `packages.yaml` file found in a Spack installation's `etc/spack/` or a user's `~/.spack/` directory. Here's an example of an external configuration:

```
packages:
  openmpi:
    paths:
      openmpi@1.4.3%gcc@4.4.7 arch=linux-x86_64-debian7: /opt/openmpi-1.4.3
      openmpi@1.4.3%gcc@4.4.7 arch=linux-x86_64-debian7+debug: /opt/openmpi-1.4.3-debug
      openmpi@1.6.5%intel@10.1 arch=linux-x86_64-debian7: /opt/openmpi-1.6.5-intel
```

This example lists three installations of OpenMPI, one built with gcc, one built with gcc and debug information, and another built with Intel. If Spack is asked to build a package that uses one of these MPIs as a dependency, it will use the the pre-installed OpenMPI in the given directory. Packages.yaml can also be used to specify modules

Each `packages.yaml` begins with a `packages:` token, followed by a list of package names. To specify externals, add a `paths` or `modules` token under the package name, which lists externals in a `spec: /path` or `spec: module-name` format. Each spec should be as well-defined as reasonably possible. If a package lacks a spec component, such as missing a compiler or package version, then Spack will guess the missing component based on its most-favored packages, and it may guess incorrectly.

Each package version and compilers listed in an external should have entries in Spack's packages and compiler configuration, even though the package and compiler may not every be built.

The packages configuration can tell Spack to use an external location for certain package versions, but it does not restrict Spack to using external packages. In the above example, if an OpenMPI 1.8.4 became available Spack may choose to start building and linking with that version rather than continue using the pre-installed OpenMPI versions.

To prevent this, the `packages.yaml` configuration also allows packages to be flagged as non-buildable. The previous example could be modified to be:

```
packages:
  openmpi:
    paths:
      openmpi@1.4.3%gcc@4.4.7 arch=linux-x86_64-debian7: /opt/openmpi-1.4.3
      openmpi@1.4.3%gcc@4.4.7 arch=linux-x86_64-debian7+debug: /opt/openmpi-1.4.3-debug
      openmpi@1.6.5%intel@10.1 arch=linux-x86_64-debian7: /opt/openmpi-1.6.5-intel
    buildable: False
```

The addition of the `buildable` flag tells Spack that it should never build its own version of OpenMPI, and it will instead always rely on a pre-built OpenMPI. Similar to `paths`, `buildable` is specified as a property under a package name.

If an external module is specified as not buildable, then Spack will load the external module into the build environment which can be used for linking.

The `buildable` does not need to be paired with external packages. It could also be used alone to forbid packages that may be buggy or otherwise undesirable.

7.2.1 False Paths for System Packages

Sometimes, the externally-installed package one wishes to use with Spack comes with the Operating System and is installed in a standard place — `/usr`, for example. Many other packages are there as well. If Spack adds it to build

paths, then some packages might pick up dependencies from `/usr` than the intended Spack version.

In order to avoid this problem, it is advisable to specify a fake path in `packages.yaml`, thereby preventing Spack from adding the real path to compiler command lines. This will work becuase compilers normally search standard system paths, even if they are not on the command line. For example:

```
packages:
    # Recommended for security reasons
    # Do not install OpenSSL as non-root user.
    openssl:
        paths:
            openssl@system: /false/path
        version: [system]
        buildable: False
```

7.2.2 Extracting System Packages

In some cases, using false paths for system packages will not work. Some builds need to run binaries out of their dependencies, not just access their libraries: the build needs to know the real location of the system package.

In this case, one can create a Spack-like single-package tree by creating symlinks to the files related to just that package. Depending on the OS, it is possible to obtain a list of the files in a single OS-installed package. For example, on RedHat / Fedora:

```
$ repoquery --list openssl-devel
...
/usr/lib/libcrypto.so
/usr/lib/libssl.so
/usr/lib/pkgconfig/libcrypto.pc
/usr/lib/pkgconfig/libssl.pc
/usr/lib/pkgconfig/openssl.pc
...
```

Spack currently does not provide an automated way to create a symlink tree to these files.

7.3 Concretization Preferences

Spack can be configured to prefer certain compilers, package versions, depends_on, and variants during concretization. The preferred configuration can be controlled via the `~/.spack/packages.yaml` file for user configuations, or the `etc/spack/packages.yaml` site configuration.

Here's an example packages.yaml file that sets preferred packages:

```
packages:
  opencv:
    compiler: [gcc@4.9]
    variants: +debug
  gperftools:
    version: [2.2, 2.4, 2.3]
  all:
    compiler: [gcc@4.4.7, gcc@4.6:, intel, clang, pgi]
    providers:
      mpi: [mvapich, mpich, openmpi]
```

At a high level, this example is specifying how packages should be concretized. The opencv package should prefer using gcc 4.9 and be built with debug options. The gperftools package should prefer version 2.2 over 2.4. Every

package on the system should prefer mvapich for its MPI and gcc 4.4.7 (except for opencv, which overrides this by preferring gcc 4.9). These options are used to fill in implicit defaults. Any of them can be overwritten on the command line if explicitly requested.

Each packages.yaml file begins with the string `packages:` and package names are specified on the next level. The special string `all` applies settings to each package. Underneath each package name is one or more components: `compiler`, `variants`, `version`, or `providers`. Each component has an ordered list of spec `constraints`, with earlier entries in the list being preferred over later entries.

Sometimes a package installation may have constraints that forbid the first concretization rule, in which case Spack will use the first legal concretization rule. Going back to the example, if a user requests gperftools 2.3 or later, then Spack will install version 2.4 as the 2.4 version of gperftools is preferred over 2.3.

An explicit concretization rule in the preferred section will always take preference over unlisted concretizations. In the above example, xlc isn't listed in the compiler list. Every listed compiler from gcc to pgi will thus be preferred over the xlc compiler.

The syntax for the `provider` section differs slightly from other concretization rules. A provider lists a value that packages may `depend_on` (e.g, mpi) and a list of rules for fulfilling that dependency.

CHAPTER 8

Mirrors

Some sites may not have access to the internet for fetching packages. These sites will need a local repository of tarballs from which they can get their files. Spack has support for this with *mirrors*. A mirror is a URL that points to a directory, either on the local filesystem or on some server, containing tarballs for all of Spack's packages.

Here's an example of a mirror's directory structure:

```
mirror/
    cmake/
        cmake-2.8.10.2.tar.gz
    dyninst/
        dyninst-8.1.1.tgz
        dyninst-8.1.2.tgz
    libdwarf/
        libdwarf-20130126.tar.gz
        libdwarf-20130207.tar.gz
        libdwarf-20130729.tar.gz
    libelf/
        libelf-0.8.12.tar.gz
        libelf-0.8.13.tar.gz
    libunwind/
        libunwind-1.1.tar.gz
    mpich/
        mpich-3.0.4.tar.gz
    mvapich2/
        mvapich2-1.9.tgz
```

The structure is very simple. There is a top-level directory. The second level directories are named after packages, and the third level contains tarballs for each package, named after each package.

Note: Archives are **not** named exactly the way they were in the package's fetch URL. They have the form `<name>-<version>.<extension>`, where `<name>` is Spack's name for the package, `<version>` is the version of the tarball, and `<extension>` is whatever format the package's fetch URL contains.

In order to make mirror creation reasonably fast, we copy the tarball in its original format to the mirror directory, but we do not standardize on a particular compression algorithm, because this would potentially require expanding and re-compressing each archive.

8.1 spack mirror

Mirrors are managed with the `spack mirror` command. The help for `spack mirror` looks like this:

```
$ spack help mirror
usage: spack mirror [-h] [-n] SUBCOMMAND ...

positional arguments:
  SUBCOMMAND
    create           Create a directory to be used as a spack mirror, and fill
                     it with package archives.
    add              Add a mirror to Spack.
    remove (rm)      Remove a mirror by name.
    list             Print out available mirrors to the console.

optional arguments:
  -h, --help         show this help message and exit
  -n, --no-checksum  Do not check fetched packages against checksum
```

The `create` command actually builds a mirror by fetching all of its packages from the internet and checksumming them.

The other three commands are for managing mirror configuration. They control the URL(s) from which Spack downloads its packages.

8.2 spack mirror create

You can create a mirror using the `spack mirror create` command, assuming you're on a machine where you can access the internet.

The command will iterate through all of Spack's packages and download the safe ones into a directory structure like the one above. Here is what it looks like:

```
$ spack mirror create libelf libdwarf
==> Created new mirror in spack-mirror-2014-06-24
==> Trying to fetch from http://www.mr511.de/software/libelf-0.8.13.tar.gz
######################################################################   81.6%
==> Checksum passed for libelf@0.8.13
==> Added libelf@0.8.13
==> Trying to fetch from http://www.mr511.de/software/libelf-0.8.12.tar.gz
######################################################################   98.6%
==> Checksum passed for libelf@0.8.12
==> Added libelf@0.8.12
==> Trying to fetch from http://www.prevanders.net/libdwarf-20130207.tar.gz
######################################################################   97.3%
==> Checksum passed for libdwarf@20130207
==> Added libdwarf@20130207
==> Trying to fetch from http://www.prevanders.net/libdwarf-20130126.tar.gz
######################################################################   78.9%
==> Checksum passed for libdwarf@20130126
==> Added libdwarf@20130126
==> Trying to fetch from http://www.prevanders.net/libdwarf-20130729.tar.gz
######################################################################   84.7%
==> Added libdwarf@20130729
==> Added spack-mirror-2014-06-24/libdwarf/libdwarf-20130729.tar.gz to mirror
==> Added python@2.7.8.
```

```
==> Successfully updated mirror in spack-mirror-2015-02-24.
  Archive stats:
    0    already present
    5    added
    0    failed to fetch.
```

Once this is done, you can tar up the `spack-mirror-2014-06-24` directory and copy it over to the machine you want it hosted on.

8.2.1 Custom package sets

Normally, `spack mirror create` downloads all the archives it has checksums for. If you want to only create a mirror for a subset of packages, you can do that by supplying a list of package specs on the command line after `spack mirror create`. For example, this command:

```
$ spack mirror create libelf@0.8.12: boost@1.44:
```

Will create a mirror for libelf versions greater than or equal to 0.8.12 and boost versions greater than or equal to 1.44.

8.2.2 Mirror files

If you have a *very* large number of packages you want to mirror, you can supply a file with specs in it, one per line:

```
$ cat specs.txt
libdwarf
libelf@0.8.12:
boost@1.44:
boost@1.39.0
...
$ spack mirror create -f specs.txt
...
```

This is useful if there is a specific suite of software managed by your site.

8.3 `spack mirror add`

Once you have a mirror, you need to let spack know about it. This is relatively simple. First, figure out the URL for the mirror. If it's a file, you can use a file URL like this one:

```
file:///Users/gamblin2/spack-mirror-2014-06-24
```

That points to the directory on the local filesystem. If it were on a web server, you could use a URL like this one:

https://example.com/some/web-hosted/directory/spack-mirror-2014-06-24

Spack will use the URL as the root for all of the packages it fetches. You can tell your Spack installation to use that mirror like this:

```
$ spack mirror add local_filesystem file:///Users/gamblin2/spack-mirror-2014-06-24
```

Each mirror has a name so that you can refer to it again later.

8.4 spack mirror list

To see all the mirrors Spack knows about, run `spack mirror list`:

```
$ spack mirror list
local_filesystem    file:///Users/gamblin2/spack-mirror-2014-06-24
```

8.5 spack mirror remove

To remove a mirror by name, run:

```
$ spack mirror remove local_filesystem
$ spack mirror list
==> No mirrors configured.
```

8.6 Mirror precedence

Adding a mirror really adds a line in `~/.spack/mirrors.yaml`:

```
mirrors:
  local_filesystem: file:///Users/gamblin2/spack-mirror-2014-06-24
  remote_server: https://example.com/some/web-hosted/directory/spack-mirror-2014-06-24
```

If you want to change the order in which mirrors are searched for packages, you can edit this file and reorder the sections. Spack will search the topmost mirror first and the bottom-most mirror last.

8.7 Local Default Cache

Spack caches resources that are downloaded as part of installs. The cache is a valid spack mirror: it uses the same directory structure and naming scheme as other Spack mirrors (so it can be copied anywhere and referenced with a URL like other mirrors). The mirror is maintained locally (within the Spack installation directory) at `var/spack/cache/`. It is always enabled (and is always searched first when attempting to retrieve files for an installation) but can be cleared with *purge*; the cache directory can also be deleted manually without issue.

Caching includes retrieved tarball archives and source control repositories, but only resources with an associated digest or commit ID (e.g. a revision number for SVN) will be cached.

CHAPTER 9

Command index

This is an alphabetical list of commands with links to the places they appear in the documentation.

- *spack activate*
- *spack cd*
- *spack checksum*
- *spack clean*
- *spack compiler add*
- *spack compiler find*
- *spack compiler info*
- *spack compilers*
- *spack create*
- *spack deactivate*
- *spack edit*
- *spack edit –force*
- *spack env*
- *spack extensions*
- *spack fetch*
- *spack find*
- *spack graph*
- *spack help*
- *spack info*
- *spack install*
- *spack list*
- *spack location*
- *spack mirror*
- *spack mirror add*
- *spack mirror create*
- *spack mirror list*
- *spack mirror remove*
- *spack –profile*
- *spack patch*
- *spack providers*
- *spack purge*
- *spack restage*
- *spack spec*
- *spack stage*
- *spack uninstall*
- *spack versions*

CHAPTER 10

Packaging Guide

This guide is intended for developers or administrators who want to package software so that Spack can install it. It assumes that you have at least some familiarity with Python, and that you've read the *basic usage guide*, especially the part about *specs*.

There are two key parts of Spack:

1. **Specs**: expressions for describing builds of software, and
2. **Packages**: Python modules that describe how to build software according to a spec.

Specs allow a user to describe a *particular* build in a way that a package author can understand. Packages allow a the packager to encapsulate the build logic for different versions, compilers, options, platforms, and dependency combinations in one place. Essentially, a package translates a spec into build logic.

Packages in Spack are written in pure Python, so you can do anything in Spack that you can do in Python. Python was chosen as the implementation language for two reasons. First, Python is becoming ubiquitous in the scientific software community. Second, it's a modern language and has many powerful features to help make package writing easy.

10.1 Creating & editing packages

10.1.1 `spack create`

The `spack create` command creates a directory with the package name and generates a `package.py` file with a boilerplate package template from a URL. The URL should point to a tarball or other software archive. In most cases, `spack create` plus a few modifications is all you need to get a package working.

Here's an example:

```
$ spack create http://www.cmake.org/files/v2.8/cmake-2.8.12.1.tar.gz
```

Spack examines the tarball URL and tries to figure out the name of the package to be created. Once the name is determined a directory in the appropriate repository is created with that name. Spack prefers, but does not require, that names be lower case so the directory name will be lower case when `spack create` generates it. In cases where it is desired to have mixed case or upper case simply rename the directory. Spack also tries to determine what version strings look like for this package. Using this information, it will try to find *additional* versions by spidering the package's webpage. If it finds multiple versions, Spack prompts you to tell it how many versions you want to download and checksum:

```
$ spack create http://www.cmake.org/files/v2.8/cmake-2.8.12.1.tar.gz
==> This looks like a URL for cmake version 2.8.12.1.
==> Creating template for package cmake
```

```
==> Found 18 versions of cmake.
  2.8.12.1  http://www.cmake.org/files/v2.8/cmake-2.8.12.1.tar.gz
  2.8.12    http://www.cmake.org/files/v2.8/cmake-2.8.12.tar.gz
  2.8.11.2  http://www.cmake.org/files/v2.8/cmake-2.8.11.2.tar.gz
  ...
  2.8.0     http://www.cmake.org/files/v2.8/cmake-2.8.0.tar.gz

Include how many checksums in the package file? (default is 5, q to abort)
```

Spack will automatically download the number of tarballs you specify (starting with the most recent) and checksum each of them.

You do not *have* to download all of the versions up front. You can always choose to download just one tarball initially, and run *spack checksum* later if you need more.

Note: If `spack create` fails to detect the package name correctly, you can try supplying it yourself, e.g.:

```
$ spack create --name cmake  http://www.cmake.org/files/v2.8/cmake-2.8.12.1.tar.gz
```

If it fails entirely, you can get minimal boilerplate by using *spack edit –force*, or you can manually create a directory and `package.py` file for the package in `var/spack/repos/builtin/packages`.

Note: Spack can fetch packages from source code repositories, but, `spack create` will *not* currently create a boilerplate package from a repository URL. You will need to use *spack edit –force* and manually edit the `version()` directives to fetch from a repo. See *Fetching from VCS repositories* for details.

Let's say you download 3 tarballs:

```
Include how many checksums in the package file? (default is 5, q to abort) 3
==> Downloading...
==> Fetching http://www.cmake.org/files/v2.8/cmake-2.8.12.1.tar.gz
######################################################################    98.6%
==> Fetching http://www.cmake.org/files/v2.8/cmake-2.8.12.tar.gz
######################################################################    96.7%
==> Fetching http://www.cmake.org/files/v2.8/cmake-2.8.11.2.tar.gz
######################################################################    95.2%
```

Now Spack generates boilerplate code and opens a new `package.py` file in your favorite `$EDITOR`:

```
1  #
2  # This is a template package file for Spack.  We've put "FIXME"
3  # next to all the things you'll want to change. Once you've handled
4  # them, you can save this file and test your package like this:
5  #
6  #     spack install cmake
7  #
8  # You can edit this file again by typing:
9  #
10 #     spack edit cmake
11 #
12 # See the Spack documentation for more information on packaging.
13 # If you submit this package back to Spack as a pull request,
14 # please first remove this boilerplate and all FIXME comments.
15 #
16 from spack import *
17
```

```
18
19  class Cmake(Package):
20      """FIXME: Put a proper description of your package here."""
21
22      # FIXME: Add a proper url for your package's homepage here.
23      homepage = "http://www.example.com"
24      url      = "http://www.cmake.org/files/v2.8/cmake-2.8.12.1.tar.gz"
25
26      version('2.8.12.1', '9d38cd4e2c94c3cea97d0e2924814acc')
27      version('2.8.12',   '105bc6d21cc2e9b6aff901e43c53afea')
28      version('2.8.11.2', '6f5d7b8e7534a5d9e1a7664ba63cf882')
29
30      # FIXME: Add dependencies if this package requires them.
31      # depends_on("foo")
32
33      def install(self, spec, prefix):
34          # FIXME: Modify the configure line to suit your build system here.
35          configure("--prefix=" + prefix)
36
37          # FIXME: Add logic to build and install here
38          make()
39          make("install")
```

The tedious stuff (creating the class, checksumming archives) has been done for you.

In the generated package, the download `url` attribute is already set. All the things you still need to change are marked with `FIXME` labels. You can delete the commented instructions between the license and the first import statement after reading them. The rest of the tasks you need to do are as follows:

1. Add a description.

   Immediately inside the package class is a *docstring* in triple-quotes (`"""`). It's used to generate the description shown when users run `spack info`.

2. Change the `homepage` to a useful URL.

   The `homepage` is displayed when users run `spack info` so that they can learn about packages.

3. Add `depends_on()` calls for the package's dependencies.

   `depends_on` tells Spack that other packages need to be built and installed before this one. See *Dependencies*.

4. Get the `install()` method working.

   The `install()` method implements the logic to build a package. The code should look familiar; it is designed to look like a shell script. Specifics will differ depending on the package, and *implementing the install method* is covered in detail later.

Before going into details, we'll cover a few more basics.

10.1.2 `spack edit`

One of the easiest ways to learn to write packages is to look at existing ones. You can edit a package file by name with the `spack edit` command:

```
$ spack edit cmake
```

So, if you used `spack create` to create a package, then saved and closed the resulting file, you can get back to it with `spack edit`. The `cmake` package actually lives in

`$SPACK_ROOT/var/spack/repos/builtin/packages/cmake/package.py`, but this provides a much simpler shortcut and saves you the trouble of typing the full path.

If you try to edit a package that doesn't exist, Spack will recommend using `spack create` or `spack edit --force`:

```
$ spack edit foo
==> Error: No package 'foo'.  Use spack create, or supply -f/--force to edit a new file.
```

10.1.3 `spack edit --force`

`spack edit --force` can be used to create a new, minimal boilerplate package:

```
$ spack edit -f foo
```

Unlike `spack create`, which infers names and versions, and which actually downloads the tarball and checksums it for you, `spack edit --force` has no such fanciness. It will substitute dummy values for you to fill in yourself:

```
1  from spack import *
2
3  class Foo(Package):
4      """Description"""
5
6      homepage = "http://www.example.com"
7      url      = "http://www.example.com/foo-1.0.tar.gz"
8
9      version('1.0', '0123456789abcdef0123456789abcdef')
10
11     def install(self, spec, prefix):
12         configure("--prefix=%s" % prefix)
13         make()
14         make("install")
```

This is useful when `spack create` cannot figure out the name and version of your package from the archive URL.

10.2 Naming & directory structure

This section describes how packages need to be named, and where they live in Spack's directory structure. In general, *spack create* and *spack edit* handle creating package files for you, so you can skip most of the details here.

10.2.1 `var/spack/repos/builtin/packages`

A Spack installation directory is structured like a standard UNIX install prefix (`bin`, `lib`, `include`, `var`, `opt`, etc.). Most of the code for Spack lives in `$SPACK_ROOT/lib/spack`. Packages themselves live in `$SPACK_ROOT/var/spack/repos/builtin/packages`.

If you `cd` to that directory, you will see directories for each package:

```
$ cd $SPACK_ROOT/var/spack/repos/builtin/packages && ls
ImageMagick
LuaJIT
Mitos
R
SAMRAI
ack
```

```
activeharmony
adept-utils
adios
adol-c
...
```

Each directory contains a file called `package.py`, which is where all the python code for the package goes. For example, the `libelf` package lives in:

```
$SPACK_ROOT/var/spack/repos/builtin/packages/libelf/package.py
```

Alongside the `package.py` file, a package may contain extra directories or files (like patches) that it needs to build.

10.2.2 Package Names

Packages are named after the directory containing `package.py`. It is preferred, but not required, that the directory, and thus the package name, are lower case. So, `libelf`'s `package.py` lives in a directory called `libelf`. The `package.py` file defines a class called `Libelf`, which extends Spack's `Package` class. For example, here is `$SPACK_ROOT/var/spack/repos/builtin/packages/libelf/package.py`:

```
from spack import *

class Libelf(Package):
    """ ... description ... """
    homepage = ...
    url = ...
    version(...)
    depends_on(...)

    def install():
        ...
```

The **directory name** (`libelf`) determines the package name that users should provide on the command line. e.g., if you type any of these:

```
$ spack install libelf
$ spack install libelf@0.8.13
```

Spack sees the package name in the spec and looks for `libelf/package.py` in `var/spack/repos/builtin/packages`. Likewise, if you say `spack install py-numpy`, then Spack looks for `py-numpy/package.py`.

Spack uses the directory name as the package name in order to give packagers more freedom in naming their packages. Package names can contain letters, numbers, dashes, and underscores. Using a Python identifier (e.g., a class name or a module name) would make it difficult to support these options. So, you can name a package `3proxy` or `_foo` and Spack won't care. It just needs to see that name in the package spec.

10.2.3 Package class names

Spack loads `package.py` files dynamically, and it needs to find a special class name in the file for the load to succeed. The **class name** (`Libelf` in our example) is formed by converting words separated by - or _ in the file name to camel case. If the name starts with a number, we prefix the class name with _. Here are some examples:

Module Name	Class Name
`foo_bar`	`FooBar`
`docbook-xml`	`DocbookXml`
`FooBar`	`Foobar`
`3proxy`	`_3proxy`

In general, you won't have to remember this naming convention because *spack create* and *spack edit* handle the details for you.

10.3 Trusted Downloads

Spack verifies that the source code it downloads is not corrupted or compromised; or at least, that it is the same version the author of the Spack package saw when the package was created. If Spack uses a download method it can verify, we say the download method is *trusted*. Trust is important for *all downloads*: Spack has no control over the security of the various sites from which it downloads source code, and can never assume that any particular site hasn't been compromised.

Trust is established in different ways for different download methods. For the most common download method — a single-file tarball — the tarball is checksummed. Git downloads using `commit=` are trusted implicitly, as long as a hash is specified.

Spack also provides untrusted download methods: tarball URLs may be supplied without a checksum, or Git downloads may specify a branch or tag instead of a hash. If the user does not control or trust the source of an untrusted download, it is a security risk. Unless otherwise specified by the user for special cases, Spack should by default use *only* trusted download methods.

Unfortunately, Spack does not currently provide that guarantee. It does provide the following mechanisms for safety:

1. By default, Spack will only install a tarball package if it has a checksum and that checksum matches. You can override this with `spack install --no-checksum`.
2. Numeric versions are almost always tarball downloads, whereas non-numeric versions not named `develop` frequently download untrusted branches or tags from a version control system. As long as a package has at least one numeric version, and no non-numeric version named `develop`, Spack will prefer it over any non-numeric versions.

10.3.1 Checksums

For tarball downloads, Spack can currently support checksums using the MD5, SHA-1, SHA-224, SHA-256, SHA-384, and SHA-512 algorithms. It determines the algorithm to use based on the hash length.

10.4 Package Version Numbers

Most Spack versions are numeric, a tuple of integers; for example, `apex@0.1`, `ferret@6.96` or `py-netcdf@1.2.3.1`. Spack knows how to compare and sort numeric versions.

Some Spack versions involve slight extensions of numeric syntax; for example, `py-sphinx-rtd-theme@0.1.10a0`. In this case, numbers are always considered to be "newer" than letters. This is for consistency with *RPM <https://bugzilla.redhat.com/show_bug.cgi?id=50977>*.

Spack versions may also be arbitrary non-numeric strings; any string here will suffice; for example, `@develop`, `@master`, `@local`. The following rules determine the sort order of numeric vs. non-numeric versions:

1. The non-numeric versions `@develop` is considered greatest (newest).

2. Numeric versions are all less than `@develop` version, and are sorted numerically.
3. All other non-numeric versions are less than numeric versions, and are sorted alphabetically.

The logic behind this sort order is two-fold:

1. Non-numeric versions are usually used for special cases while developing or debugging a piece of software. Keeping most of them less than numeric versions ensures that Spack choose numeric versions by default whenever possible.
2. The most-recent development version of a package will usually be newer than any released numeric versions. This allows the `develop` version to satisfy dependencies like `depends_on(abc, when="@x.y.z:")`

10.4.1 Date Versions

If you wish to use dates as versions, it is best to use the format `@date-yyyy-mm-dd`. This will ensure they sort in the correct order. If you want your date versions to be numeric (assuming they don't conflict with other numeric versions), you can use just `yyyy.mm.dd`.

Alternately, you might use a hybrid release-version / date scheme. For example, `@1.3.2016.08.31` would mean the version from the `1.3` branch, as of August 31, 2016.

10.5 Adding new versions

The most straightforward way to add new versions to your package is to add a line like this in the package class:

```
class Foo(Package):
    url = 'http://example.com/foo-1.0.tar.gz'
    version('8.2.1', '4136d7b4c04df68b686570afa26988ac')
    ...
```

Versions should be listed with the newest version first.

10.5.1 Version URLs

By default, each version's URL is extrapolated from the `url` field in the package. For example, Spack is smart enough to download version `8.2.1.` of the `Foo` package above from `http://example.com/foo-8.2.1.tar.gz`.

If spack *cannot* extrapolate the URL from the `url` field by default, you can write your own URL generation algorithm in place of the `url` declaration. For example:

```
class Foo(Package):
    version('8.2.1', '4136d7b4c04df68b686570afa26988ac')
    ...
    def url_for_version(self, version):
        return 'http://example.com/version_%s/foo-%s.tar.gz' \
            % (version, version)
    ...
```

If a URL cannot be derived systematically, you can add an explicit URL for a particular version:

```
version('8.2.1', '4136d7b4c04df68b686570afa26988ac',
        url='http://example.com/foo-8.2.1-special-version.tar.gz')
```

For the URL above, you might have to add an explicit URL because the version can't simply be substituted in the original `url` to construct the new one for `8.2.1`.

When you supply a custom URL for a version, Spack uses that URL *verbatim* and does not perform extrapolation.

10.5.2 Skipping the expand step

Spack normally expands archives automatically after downloading them. If you want to skip this step (e.g., for self-extracting executables and other custom archive types), you can add `expand=False` to a `version` directive.

```
version('8.2.1', '4136d7b4c04df68b686570afa26988ac',
        url='http://example.com/foo-8.2.1-special-version.tar.gz', expand=False)
```

When `expand` is set to `False`, Spack sets the current working directory to the directory containing the downloaded archive before it calls your `install` method. Within `install`, the path to the downloaded archive is available as `self.stage.archive_file`.

Here is an example snippet for packages distributed as self-extracting archives. The example sets permissions on the downloaded file to make it executable, then runs it with some arguments.

```
def install(self, spec, prefix):
    set_executable(self.stage.archive_file)
    installer = Executable(self.stage.archive_file)
    installer('--prefix=%s' % prefix, 'arg1', 'arg2', 'etc.')
```

10.5.3 `spack md5`

If you have one or more files to checksum, you can use the `spack md5` command to do it:

```
$ spack md5 foo-8.2.1.tar.gz foo-8.2.2.tar.gz
==> 2 MD5 checksums:
4136d7b4c04df68b686570afa26988ac  foo-8.2.1.tar.gz
1586b70a49dfe05da5fcc29ef239dce0  foo-8.2.2.tar.gz
```

`spack md5` also accepts one or more URLs and automatically downloads the files for you:

```
$ spack md5 http://example.com/foo-8.2.1.tar.gz
==> Trying to fetch from http://example.com/foo-8.2.1.tar.gz
######################################################################## 100.0%
==> 1 MD5 checksum:
4136d7b4c04df68b686570afa26988ac  foo-8.2.1.tar.gz
```

Doing this for lots of files, or whenever a new package version is released, is tedious. See `spack checksum` below for an automated version of this process.

10.5.4 `spack checksum`

If you want to add new versions to a package you've already created, this is automated with the `spack checksum` command. Here's an example for `libelf`:

```
$ spack checksum libelf
==> Found 16 versions of libelf.
  0.8.13    http://www.mr511.de/software/libelf-0.8.13.tar.gz
  0.8.12    http://www.mr511.de/software/libelf-0.8.12.tar.gz
  0.8.11    http://www.mr511.de/software/libelf-0.8.11.tar.gz
  0.8.10    http://www.mr511.de/software/libelf-0.8.10.tar.gz
  0.8.9     http://www.mr511.de/software/libelf-0.8.9.tar.gz
  0.8.8     http://www.mr511.de/software/libelf-0.8.8.tar.gz
```

```
  0.8.7     http://www.mr511.de/software/libelf-0.8.7.tar.gz
  0.8.6     http://www.mr511.de/software/libelf-0.8.6.tar.gz
  0.8.5     http://www.mr511.de/software/libelf-0.8.5.tar.gz
  ...
  0.5.2     http://www.mr511.de/software/libelf-0.5.2.tar.gz

How many would you like to checksum? (default is 5, q to abort)
```

This does the same thing that `spack create` does, but it allows you to go back and add new versions easily as you need them (e.g., as they're released). It fetches the tarballs you ask for and prints out a list of `version` commands ready to copy/paste into your package file:

```
==> Checksummed new versions of libelf:
    version('0.8.13', '4136d7b4c04df68b686570afa26988ac')
    version('0.8.12', 'e21f8273d9f5f6d43a59878dc274fec7')
    version('0.8.11', 'e931910b6d100f6caa32239849947fbf')
    version('0.8.10', '9db4d36c283d9790d8fa7df1f4d7b4d9')
```

By default, Spack will search for new tarball downloads by scraping the parent directory of the tarball you gave it. So, if your tarball is at `http://example.com/downloads/foo-1.0.tar.gz`, Spack will look in `http://example.com/downloads/` for links to additional versions. If you need to search another path for download links, you can supply some extra attributes that control how your package finds new versions. See the documentation on *attribute_list_url* and *attribute_list_depth*.

Note:

- This command assumes that Spack can extrapolate new URLs from an existing URL in the package, and that Spack can find similar URLs on a webpage. If that's not possible, e.g. if the package's developers don't name their tarballs consistently, you'll need to manually add `version` calls yourself.
- For `spack checksum` to work, Spack needs to be able to `import` your package in Python. That means it can't have any syntax errors, or the `import` will fail. Use this once you've got your package in working order.

10.6 Fetching from VCS repositories

For some packages, source code is provided in a Version Control System (VCS) repository rather than in a tarball. Spack can fetch packages from VCS repositories. Currently, Spack supports fetching with *Git*, *Mercurial (hg)*, and *Subversion (SVN)*.

To fetch a package from a source repository, you add a `version()` call to your package with parameters indicating the repository URL and any branch, tag, or revision to fetch. See below for the parameters you'll need for each VCS system.

10.6.1 Git

Git fetching is enabled with the following parameters to `version`:

- `git`: URL of the git repository.
- `tag`: name of a tag to fetch.
- `branch`: name of a branch to fetch.
- `commit`: SHA hash (or prefix) of a commit to fetch.

- `submodules`: Also fetch submodules when checking out this repository.

Only one of `tag`, `branch`, or `commit` can be used at a time.

Default branch To fetch a repository's default branch:

```
class Example(Package):
    ...
    version('develop', git='https://github.com/example-project/example.git')
```

This download method is untrusted, and is not recommended.

Tags To fetch from a particular tag, use the `tag` parameter along with `git`:

```
version('1.0.1', git='https://github.com/example-project/example.git',
        tag='v1.0.1')
```

This download method is untrusted, and is not recommended.

Branches To fetch a particular branch, use `branch` instead:

```
version('experimental', git='https://github.com/example-project/example.git',
        branch='experimental')
```

This download method is untrusted, and is not recommended.

Commits Finally, to fetch a particular commit, use `commit`:

```
version('2014-10-08', git='https://github.com/example-project/example.git',
        commit='9d38cd4e2c94c3cea97d0e2924814acc')
```

This doesn't have to be a full hash; you can abbreviate it as you'd expect with git:

```
version('2014-10-08', git='https://github.com/example-project/example.git',
        commit='9d38cd')
```

This download method *is trusted*. It is the recommended way to securely download from a Git repository.

It may be useful to provide a saner version for commits like this, e.g. you might use the date as the version, as done above. Or you could just use the abbreviated commit hash. It's up to the package author to decide what makes the most sense.

Submodules

You can supply `submodules=True` to cause Spack to fetch submodules along with the repository at fetch time.

```
version('1.0.1', git='https://github.com/example-project/example.git',
        tag='v1.0.1', submdoules=True)
```

GitHub

If a project is hosted on GitHub, *any* valid Git branch, tag or hash may be downloaded as a tarball. This is accomplished simply by constructing an appropriate URL. Spack can checksum any package downloaded this way, thereby producing a trusted download. For example, the following downloads a particular hash, and then applies a checksum.

```
version('1.9.5.1.1', 'd035e4bc704d136db79b43ab371b27d2',
    url='https://www.github.com/jswhit/pyproj/tarball/0be612cc9f972e38b50a90c946a9b353e2ab140f')
```

10.6.2 Mercurial

Fetching with mercurial works much like git, but you use the `hg` parameter.

Default Add the `hg` parameter with no `revision`:

```
version('develop', hg='https://jay.grs.rwth-aachen.de/hg/example')
```

This download method is untrusted, and is not recommended.

Revisions Add `hg` and `revision` parameters:

```
version('1.0', hg='https://jay.grs.rwth-aachen.de/hg/example',
        revision='v1.0')
```

This download method is untrusted, and is not recommended.

Unlike `git`, which has special parameters for different types of revisions, you can use `revision` for branches, tags, and commits when you fetch with Mercurial.

As with git, you can fetch these versions using the `spack install example@<version>` command-line syntax.

10.6.3 Subversion

To fetch with subversion, use the `svn` and `revision` parameters:

Fetching the head Simply add an `svn` parameter to `version`:

```
version('develop', svn='https://outreach.scidac.gov/svn/libmonitor/trunk')
```

This download method is untrusted, and is not recommended.

Fetching a revision To fetch a particular revision, add a `revision` to the version call:

```
version('develop', svn='https://outreach.scidac.gov/svn/libmonitor/trunk',
        revision=128)
```

This download method is untrusted, and is not recommended.

Subversion branches are handled as part of the directory structure, so you can check out a branch or tag by changing the `url`.

10.7 Expanding additional resources in the source tree

Some packages (most notably compilers) provide optional features if additional resources are expanded within their source tree before building. In Spack it is possible to describe such a need with the `resource` directive :

```
resource(
   name='cargo',
   git='https://github.com/rust-lang/cargo.git',
   tag='0.10.0',
   destination='cargo'
)
```

Based on the keywords present among the arguments the appropriate `FetchStrategy` will be used for the resource. The keyword `destination` is relative to the source root of the package and should point to where the resource is to be expanded.

10.8 Automatic caching of files fetched during installation

Spack maintains a cache (described *here*) which saves files retrieved during package installations to avoid re-downloading in the case that a package is installed with a different specification (but the same version) or reinstalled on account of a change in the hashing scheme.

10.9 Licensed software

In order to install licensed software, Spack needs to know a few more details about a package. The following class attributes should be defined.

10.9.1 `license_required`

Boolean. If set to `True`, this software requires a license. If set to `False`, all of the following attributes will be ignored. Defaults to `False`.

10.9.2 `license_comment`

String. Contains the symbol used by the license manager to denote a comment. Defaults to `#`.

10.9.3 `license_files`

List of strings. These are files that the software searches for when looking for a license. All file paths must be relative to the installation directory. More complex packages like Intel may require multiple licenses for individual components. Defaults to the empty list.

10.9.4 `license_vars`

List of strings. Environment variables that can be set to tell the software where to look for a license if it is not in the usual location. Defaults to the empty list.

10.9.5 `license_url`

String. A URL pointing to license setup instructions for the software. Defaults to the empty string.

For example, let's take a look at the package for the PGI compilers.

```
# Licensing
license_required = True
license_comment  = '#'
license_files    = ['license.dat']
license_vars     = ['PGROUPD_LICENSE_FILE', 'LM_LICENSE_FILE']
license_url      = 'http://www.pgroup.com/doc/pgiinstall.pdf'
```

As you can see, PGI requires a license. Its license manager, FlexNet, uses the `#` symbol to denote a comment. It expects the license file to be named `license.dat` and to be located directly in the installation prefix. If you would like the installation file to be located elsewhere, simply set `PGROUPD_LICENSE_FILE` or `LM_LICENSE_FILE` after installation. For further instructions on installation and licensing, see the URL provided.

Let's walk through a sample PGI installation to see exactly what Spack is and isn't capable of. Since PGI does not provide a download URL, it must be downloaded manually. It can either be added to a mirror or located in the current directory when `spack install pgi` is run. See *Mirrors* for instructions on setting up a mirror.

After running `spack install pgi`, the first thing that will happen is Spack will create a global license file located at `$SPACK_ROOT/etc/spack/licenses/pgi/license.dat`. It will then open up the file using the editor set in `$EDITOR`, or vi if unset. It will look like this:

```
# A license is required to use pgi.
#
# The recommended solution is to store your license key in this global
# license file. After installation, the following symlink(s) will be
# added to point to this file (relative to the installation prefix):
#
#   license.dat
#
# Alternatively, use one of the following environment variable(s):
#
#   PGROUPD_LICENSE_FILE
#   LM_LICENSE_FILE
#
# If you choose to store your license in a non-standard location, you may
# set one of these variable(s) to the full pathname to the license file, or
# port@host if you store your license keys on a dedicated license server.
# You will likely want to set this variable in a module file so that it
# gets loaded every time someone tries to use pgi.
#
# For further information on how to acquire a license, please refer to:
#
#   http://www.pgroup.com/doc/pgiinstall.pdf
#
# You may enter your license below.
```

You can add your license directly to this file, or tell FlexNet to use a license stored on a separate license server. Here is an example that points to a license server called licman1:

```
SERVER licman1.mcs.anl.gov 00163eb7fba5 27200
USE_SERVER
```

If your package requires the license to install, you can reference the location of this global license using `self.global_license_file`. After installation, symlinks for all of the files given in `license_files` will be created, pointing to this global license. If you install a different version or variant of the package, Spack will automatically detect and reuse the already existing global license.

If the software you are trying to package doesn't rely on license files, Spack will print a warning message, letting the user know that they need to set an environment variable or pointing them to installation documentation.

10.10 Patches

Depending on the host architecture, package version, known bugs, or other issues, you may need to patch your software to get it to build correctly. Like many other package systems, spack allows you to store patches alongside your package files and apply them to source code after it's downloaded.

10.10.1 `patch`

You can specify patches in your package file with the `patch()` directive. `patch` looks like this:

```
class Mvapich2(Package):
    ...
    patch('ad_lustre_rwcontig_open_source.patch', when='@1.9:')
```

The first argument can be either a URL or a filename. It specifies a patch file that should be applied to your source. If the patch you supply is a filename, then the patch needs to live within the spack source tree. For example, the patch above lives in a directory structure like this:

```
$SPACK_ROOT/var/spack/repos/builtin/packages/
    mvapich2/
        package.py
        ad_lustre_rwcontig_open_source.patch
```

If you supply a URL instead of a filename, the patch will be fetched from the URL and then applied to your source code.

Warning: It is generally better to use a filename rather than a URL for your patch. Patches fetched from URLs are not currently checksummed, and adding checksums for them is tedious for the package builder. File patches go into the spack repository, which gives you git's integrity guarantees. URL patches may be removed in a future spack version.

`patch` can take two options keyword arguments. They are:

when

If supplied, this is a spec that tells spack when to apply the patch. If the installed package spec matches this spec, the patch will be applied. In our example above, the patch is applied when mvapich is at version `1.9` or higher.

level

This tells spack how to run the `patch` command. By default, the level is 1 and spack runs `patch -p 1`. If level is 2, spack will run `patch -p 2`, and so on.

A lot of people are confused by level, so here's a primer. If you look in your patch file, you may see something like this:

```
1  --- a/src/mpi/romio/adio/ad_lustre/ad_lustre_rwcontig.c 2013-12-10 12:05:44.806417000 -0800
2  +++ b/src/mpi/romio/adio/ad_lustre/ad_lustre_rwcontig.c 2013-12-10 11:53:03.295622000 -0800
3  @@ -8,7 +8,7 @@
4    *   Copyright (C) 2008 Sun Microsystems, Lustre group
5    \*/
6
7  -#define _XOPEN_SOURCE 600
8  +//#define _XOPEN_SOURCE 600
9   #include <stdlib.h>
10  #include <malloc.h>
11  #include "ad_lustre.h"
```

Lines 1-2 show paths with synthetic `a/` and `b/` prefixes. These are placeholders for the two `mvapich2` source directories that `diff` compared when it created the patch file. This is git's default behavior when creating patch files, but other programs may behave differently.

`-p1` strips off the first level of the prefix in both paths, allowing the patch to be applied from the root of an expanded mvapich2 archive. If you set level to `2`, it would strip off `src`, and so on.

It's generally easier to just structure your patch file so that it applies cleanly with `-p1`, but if you're using a patch you didn't create yourself, `level` can be handy.

10.10.2 Patch functions

In addition to supplying patch files, you can write a custom function to patch a package's source. For example, the `py-pyside` package contains some custom code for tweaking the way the PySide build handles `RPATH`:

```
    def patch(self):
        """Undo PySide RPATH handling and add Spack RPATH."""
        # Figure out the special RPATH
        pypkg = self.spec['python'].package
        rpath = self.rpath
        rpath.append(os.path.join(
            self.prefix, pypkg.site_packages_dir, 'PySide'))

        # Add Spack's standard CMake args to the sub-builds.
        # They're called BY setup.py so we have to patch it.
        filter_file(
            r'OPTION_CMAKE,',
            r'OPTION_CMAKE, ' + (
                '"-DCMAKE_INSTALL_RPATH_USE_LINK_PATH=FALSE", '
                '"-DCMAKE_INSTALL_RPATH=%s",' % ':'.join(rpath)),
            'setup.py')

        # PySide tries to patch ELF files to remove RPATHs
        # Disable this and go with the one we set.
        if self.spec.satisfies('@1.2.4:'):
            rpath_file = 'setup.py'
        else:
            rpath_file = 'pyside_postinstall.py'

        filter_file(r'(^\s*)(rpath_cmd\(.*\))', r'\1#\2', rpath_file)

        # TODO: rpath handling for PySide 1.2.4 still doesn't work.
        # PySide can't find the Shiboken library, even though it comes
        # bundled with it and is installed in the same directory.

        # PySide does not provide official support for
        # Python 3.5, but it should work fine
        filter_file("'Programming Language :: Python :: 3.4'",
                    "'Programming Language :: Python :: 3.4',\r\n        "
                    "'Programming Language :: Python :: 3.5'",
                    "setup.py")
```

A `patch` function, if present, will be run after patch files are applied and before `install()` is run.

You could put this logic in `install()`, but putting it in a patch function gives you some benefits. First, spack ensures that the `patch()` function is run once per code checkout. That means that if you run install, hit ctrl-C, and run install again, the code in the patch function is only run once. Also, you can tell Spack to run only the patching part of the build using the *spack patch* command.

10.11 Handling RPATHs

Spack installs each package in a way that ensures that all of its dependencies are found when it runs. It does this using RPATHs. An RPATH is a search path, stored in a binary (an

executable or library), that tells the dynamic loader where to find its dependencies at run-time. You may be familiar with LD_LIBRARY_PATH on Linux or *DYLD_LIBRARY_PATH <https://developer.apple.com/library/mac/documentation/Darwin/Reference/ManPages/man1/dyld.1.html>* on Mac OS X. RPATH is similar to these paths, in that it tells the loader where to find libraries. Unlike them, it is embedded in the binary and not set in each user's environment.

RPATHs in Spack are handled in one of three ways:

1. For most packages, RPATHs are handled automatically using Spack's *compiler wrappers*. These wrappers are set in standard variables like `CC`, `CXX`, `F77`, and `FC`, so most build systems (autotools and many gmake systems) pick them up and use them.
2. CMake also respects Spack's compiler wrappers, but many CMake builds have logic to overwrite RPATHs when binaries are installed. Spack provides the `std_cmake_args` variable, which includes parameters necessary for CMake build use the right installation RPATH. It can be used like this when `cmake` is invoked:

```
    class MyPackage(Package):
        ...
        def install(self, spec, prefix):
            cmake('..', *std_cmake_args)
            make()
            make('install')
```

3. If you need to modify the build to add your own RPATHs, you can use the `self.rpath` property of your package, which will return a list of all the RPATHs that Spack will use when it links. You can see this how this is used in the *PySide example* above.

10.12 Finding new versions

You've already seen the `homepage` and `url` package attributes:

```
from spack import *


class Mpich(Package):
    """MPICH is a high performance and widely portable implementation of
       the Message Passing Interface (MPI) standard."""
    homepage = "http://www.mpich.org"
    url      = "http://www.mpich.org/static/downloads/3.0.4/mpich-3.0.4.tar.gz"
```

These are class-level attributes used by Spack to show users information about the package, and to determine where to download its source code.

Spack uses the tarball URL to extrapolate where to find other tarballs of the same package (e.g. in *spack checksum*, but this does not always work. This section covers ways you can tell Spack to find tarballs elsewhere.

10.12.1 `list_url`

When spack tries to find available versions of packages (e.g. with *spack checksum*), it spiders the parent directory of the tarball in the `url` attribute. For example, for libelf, the url is:

```
url = "http://www.mr511.de/software/libelf-0.8.13.tar.gz"
```

Here, Spack spiders `http://www.mr511.de/software/` to find similar tarball links and ultimately to make a list of available versions of `libelf`.

For many packages, the tarball's parent directory may be unlistable, or it may not contain any links to source code archives. In fact, many times additional package downloads aren't even available in the same directory as the download URL.

For these, you can specify a separate `list_url` indicating the page to search for tarballs. For example, `libdwarf` has the homepage as the `list_url`, because that is where links to old versions are:

```
class Libdwarf(Package):
    homepage = "http://www.prevanders.net/dwarf.html"
    url      = "http://www.prevanders.net/libdwarf-20130729.tar.gz"
    list_url = homepage
```

10.12.2 `list_depth`

`libdwarf` and many other packages have a listing of available versions on a single webpage, but not all do. For example, `mpich` has a tarball URL that looks like this:

```
url = "http://www.mpich.org/static/downloads/3.0.4/mpich-3.0.4.tar.gz"
```

But its downloads are in many different subdirectories of `http://www.mpich.org/static/downloads/`. So, we need to add a `list_url` *and* a `list_depth` attribute:

```
class Mpich(Package):
    homepage   = "http://www.mpich.org"
    url        = "http://www.mpich.org/static/downloads/3.0.4/mpich-3.0.4.tar.gz"
    list_url   = "http://www.mpich.org/static/downloads/"
    list_depth = 2
```

By default, Spack only looks at the top-level page available at `list_url`. `list_depth` tells it to follow up to 2 levels of links from the top-level page. Note that here, this implies two levels of subdirectories, as the `mpich` website is structured much like a filesystem. But `list_depth` really refers to link depth when spidering the page.

10.13 Parallel builds

By default, Spack will invoke `make()` with a `-j <njobs>` argument, so that builds run in parallel. It figures out how many jobs to run by determining how many cores are on the host machine. Specifically, it uses the number of CPUs reported by Python's multiprocessing.cpu_count().

If a package does not build properly in parallel, you can override this setting by adding `parallel = False` to your package. For example, OpenSSL's build does not work in parallel, so its package looks like this:

```
class Openssl(Package):
    homepage = "http://www.openssl.org"
    url      = "http://www.openssl.org/source/openssl-1.0.1h.tar.gz"

    version('1.0.1h', '8d6d684a9430d5cc98a62a5d8fbda8cf')
    depends_on("zlib")

    parallel = False
```

Similarly, you can disable parallel builds only for specific make commands, as `libdwarf` does:

```
class Libelf(Package):
    ...

    def install(self, spec, prefix):
```

```
        configure("--prefix=" + prefix,
                  "--enable-shared",
                  "--disable-dependency-tracking",
                  "--disable-debug")
        make()

        # The mkdir commands in libelf's install can fail in parallel
        make("install", parallel=False)
```

The first make will run in parallel here, but the second will not. If you set `parallel` to `False` at the package level, then each call to `make()` will be sequential by default, but packagers can call `make(parallel=True)` to override it.

10.14 Dependencies

We've covered how to build a simple package, but what if one package relies on another package to build? How do you express that in a package file? And how do you refer to the other package in the build script for your own package?

Spack makes this relatively easy. Let's take a look at the `libdwarf` package to see how it's done:

```
class Libdwarf(Package):
    homepage = "http://www.prevanders.net/dwarf.html"
    url      = "http://www.prevanders.net/libdwarf-20130729.tar.gz"
    list_url = homepage

    version('20130729', '4cc5e48693f7b93b7aa0261e63c0e21d')
    ...

    depends_on("libelf")

    def install(self, spec, prefix):
        ...
```

10.14.1 `depends_on()`

The highlighted `depends_on('libelf')` call tells Spack that it needs to build and install the `libelf` package before it builds `libdwarf`. This means that in your `install()` method, you are guaranteed that `libelf` has been built and installed successfully, so you can rely on it for your libdwarf build.

10.14.2 Dependency specs

`depends_on` doesn't just take the name of another package. It takes a full spec. This means that you can restrict the versions or other configuration options of `libelf` that `libdwarf` will build with. Here's an example. Suppose that in the `libdwarf` package you write:

```
depends_on("libelf@0.8:")
```

Now `libdwarf` will require a version of `libelf` version `0.8` or higher in order to build. If some versions of `libelf` are installed but they are all older than this, then Spack will build a new version of `libelf` that satisfies the spec's version constraint, and it will build `libdwarf` with that one. You could just as easily provide a version range:

```
depends_on("libelf@0.8.2:0.8.4:")
```

Or a requirement for a particular variant or compiler flags:

```
depends_on("libelf@0.8+debug")
depends_on('libelf debug=True')
depends_on('libelf cppflags="-fPIC"')
```

Both users *and* package authors can use the same spec syntax to refer to different package configurations. Users use the spec syntax on the command line to find installed packages or to install packages with particular constraints, and package authors can use specs to describe relationships between packages.

Additionally, dependencies may be specified for specific use cases:

```
depends_on("cmake", type="build")
depends_on("libelf", type=("build", "link"))
depends_on("python", type="run")
```

The dependency types are:

- **"build"**: made available during the project's build. The package will be added to `PATH`, the compiler include paths, and `PYTHONPATH`. Other projects which depend on this one will not have these modified (building project X doesn't need project Y's build dependencies).
- **"link"**: the project is linked to by the project. The package will be added to the current package's `rpath`.
- **"run"**: the project is used by the project at runtime. The package will be added to `PATH` and `PYTHONPATH`.

Additional hybrid dependency types are (note the lack of quotes):

- **<not specified>**: `type` assumed to be `("build", "link")`. This is the common case for compiled language usage.
- **alldeps**: All dependency types. **Note:** No quotes here
- **nolink**: Equal to `("build", "run")`, for use by dependencies that are not expressed via a linker (e.g., Python or Lua module loading). **Note:** No quotes here

Dependency Formulas

This section shows how to write appropriate `depends_on()` declarations for some common cases.

- Python 2 only: `depends_on('python@:2.8')`
- Python 2.7 only: `depends_on('python@2.7:2.8')`
- Python 3 only: `depends_on('python@3:')`

10.14.3 `setup_dependent_environment()`

Spack provides a mechanism for dependencies to provide variables that can be used in their dependents' build. Any package can declare a `setup_dependent_environment()` function, and this function will be called before the `install()` method of any dependent packages. This allows dependencies to set up environment variables and other properties to be used by dependents.

The function declaration should look like this:

```
class Qt(Package):
    ...
    def setup_dependent_environment(self, module, spec, dep_spec):
        """Dependencies of Qt find it using the QTDIR environment variable."""
        os.environ['QTDIR'] = self.prefix
```

Here, the Qt package sets the `QTDIR` environment variable so that packages that depend on a particular Qt installation will find it.

The arguments to this function are:

- **module**: the module of the dependent package, where global properties can be assigned.
- **spec**: the spec of the *dependency package* (the one the function is called on).
- **dep_spec**: the spec of the dependent package (i.e. dep_spec depends on spec).

A good example of using these is in the Python package:

```
    def setup_dependent_environment(self, spack_env, run_env, extension_spec):
        """Set PYTHONPATH to include site-packages dir for the
        extension and any other python extensions it depends on."""
        pythonhome = self.prefix
        spack_env.set('PYTHONHOME', pythonhome)

        python_paths = []
        for d in extension_spec.traverse(deptype=nolink, deptype_query='run'):
            if d.package.extends(self.spec):
                python_paths.append(join_path(d.prefix,
                                              self.site_packages_dir))

        pythonpath = ':'.join(python_paths)
        spack_env.set('PYTHONPATH', pythonpath)

        # For run time environment set only the path for
        # extension_spec and prepend it to PYTHONPATH
        if extension_spec.package.extends(self.spec):
            run_env.prepend_path('PYTHONPATH', join_path(
                extension_spec.prefix, self.site_packages_dir))
```

The first thing that happens here is that the `python` command is inserted into module scope of the dependent. This allows most python packages to have a very simple install method, like this:

```
def install(self, spec, prefix):
    python('setup.py', 'install', '--prefix={0}'.format(prefix))
```

Python's `setup_dependent_environment` method also sets up some other variables, creates a directory, and sets up the `PYTHONPATH` so that dependent packages can find their dependencies at build time.

10.15 Extensions

Spack's support for package extensions is documented extensively in *Extensions & Python support*. This section documents how to make your own extendable packages and extensions.

To support extensions, a package needs to set its `extendable` property to `True`, e.g.:

```
class Python(Package):
    ...
    extendable = True
    ...
```

To make a package into an extension, simply add simply add an `extends` call in the package definition, and pass it the name of an extendable package:

```
class PyNumpy(Package):
    ...
    extends('python')
    ...
```

Now, the `py-numpy` package can be used as an argument to `spack activate`. When it is activated, all the files in its prefix will be symbolically linked into the prefix of the python package.

Some packages produce a Python extension, but are only compatible with Python 3, or with Python 2. In those cases, a `depends_on()` declaration should be made in addition to the `extends()` declaration:

```
class Icebin(Package):
    extends('python', when='+python')
    depends_on('python@3:', when='+python')
```

Many packages produce Python extensions for *some* variants, but not others: they should extend `python` only if the appropriate variant(s) are selected. This may be accomplished with conditional `extends()` declarations:

```
class FooLib(Package):
    variant('python', default=True, description= \
        'Build the Python extension Module')
    extends('python', when='+python')
    ...
```

Sometimes, certain files in one package will conflict with those in another, which means they cannot both be activated (symlinked) at the same time. In this case, you can tell Spack to ignore those files when it does the activation:

```
class PySncosmo(Package):
    ...
    # py-sncosmo binaries are duplicates of those from py-astropy
    extends('python', ignore=r'bin/.*')
    depends_on('py-astropy')
    ...
```

The code above will prevent everything in the `$prefix/bin/` directory from being linked in at activation time.

Note: You can call *either* `depends_on` or `extends` on any one package, but not both. For example you cannot both `depends_on('python')` and `extends(python)` in the same package. `extends` implies `depends_on`.

10.15.1 Activation & deactivation

Spack's `Package` class has default `activate` and `deactivate` implementations that handle symbolically linking extensions' prefixes into the directory of the parent package. However, extendable packages can override these methods to add custom activate/deactivate logic of their own. For example, the `activate` and `deactivate` methods in the Python class use the symbolic linking, but they also handle details surrounding Python's `.pth` files, and other aspects of Python packaging.

Spack's extensions mechanism is designed to be extensible, so that other packages (like Ruby, R, Perl, etc.) can provide their own custom extension management logic, as they may not handle modules the same way that Python does.

Let's look at Python's activate function:

```
    def activate(self, ext_pkg, **args):
        ignore = self.python_ignore(ext_pkg, args)
        args.update(ignore=ignore)

```

```
        super(Python, self).activate(ext_pkg, **args)

        exts = spack.install_layout.extension_map(self.spec)
        exts[ext_pkg.name] = ext_pkg.spec
        self.write_easy_install_pth(exts)
```

This function is called on the *extendee* (Python). It first calls `activate` in the superclass, which handles symlinking the extension package's prefix into this package's prefix. It then does some special handling of the `easy-install.pth` file, part of Python's setuptools.

Deactivate behaves similarly to activate, but it unlinks files:

```
    def deactivate(self, ext_pkg, **args):
        args.update(ignore=self.python_ignore(ext_pkg, args))
        super(Python, self).deactivate(ext_pkg, **args)

        exts = spack.install_layout.extension_map(self.spec)
        # Make deactivate idempotent
        if ext_pkg.name in exts:
            del exts[ext_pkg.name]
            self.write_easy_install_pth(exts)
```

Both of these methods call some custom functions in the Python package. See the source for Spack's Python package for details.

10.15.2 Activation arguments

You may have noticed that the `activate` function defined above takes keyword arguments. These are the keyword arguments from `extends()`, and they are passed to both activate and deactivate.

This capability allows an extension to customize its own activation by passing arguments to the extendee. Extendees can likewise implement custom `activate()` and `deactivate()` functions to suit their needs.

The only keyword argument supported by default is the `ignore` argument, which can take a regex, list of regexes, or a predicate to determine which files *not* to symlink during activation.

10.16 Virtual dependencies

In some cases, more than one package can satisfy another package's dependency. One way this can happen is if a package depends on a particular *interface*, but there are multiple *implementations* of the interface, and the package could be built with any of them. A *very* common interface in HPC is the Message Passing Interface (MPI), which is used in many large-scale parallel applications.

MPI has several different implementations (e.g., MPICH, OpenMPI, and MVAPICH) and scientific applications can be built with any one of them. Complicating matters, MPI does not have a standardized ABI, so a package built with one implementation cannot simply be relinked with another implementation. Many package managers handle interfaces like this by requiring many similar package files, e.g., `foo`, `foo-mvapich`, `foo-mpich`, but Spack avoids this explosion of package files by providing support for *virtual dependencies*.

10.16.1 `provides`

In Spack, `mpi` is handled as a *virtual package*. A package like `mpileaks` can depend on it just like any other package, by supplying a `depends_on` call in the package definition. For example:

```
class Mpileaks(Package):
    homepage = "https://github.com/hpc/mpileaks"
    url = "https://github.com/hpc/mpileaks/releases/download/v1.0/mpileaks-1.0.tar.gz"

    version('1.0', '8838c574b39202a57d7c2d68692718aa')

    depends_on("mpi")
    depends_on("adept-utils")
    depends_on("callpath")
```

Here, `callpath` and `adept-utils` are concrete packages, but there is no actual package file for `mpi`, so we say it is a *virtual* package. The syntax of `depends_on`, is the same for both. If we look inside the package file of an MPI implementation, say MPICH, we'll see something like this:

```
class Mpich(Package):
    provides('mpi')
    ...
```

The `provides("mpi")` call tells Spack that the `mpich` package can be used to satisfy the dependency of any package that `depends_on('mpi')`.

10.16.2 Versioned Interfaces

Just as you can pass a spec to `depends_on`, so can you pass a spec to `provides` to add constraints. This allows Spack to support the notion of *versioned interfaces*. The MPI standard has gone through many revisions, each with new functions added, and each revision of the standard has a version number. Some packages may require a recent implementation that supports MPI-3 functions, but some MPI versions may only provide up to MPI-2. Others may need MPI 2.1 or higher. You can indicate this by adding a version constraint to the spec passed to `provides`:

```
provides("mpi@:2")
```

Suppose that the above `provides` call is in the `mpich2` package. This says that `mpich2` provides MPI support *up to* version 2, but if a package `depends_on("mpi@3")`, then Spack will *not* build that package with `mpich2`.

10.16.3 `provides when`

The same package may provide different versions of an interface depending on *its* version. Above, we simplified the `provides` call in `mpich` to make the explanation easier. In reality, this is how `mpich` calls `provides`:

```
provides('mpi@:3', when='@3:')
provides('mpi@:1', when='@1:')
```

The `when` argument to `provides` allows you to specify optional constraints on the *providing* package, or the *provider*. The provider only provides the declared virtual spec when *it* matches the constraints in the when clause. Here, when `mpich` is at version 3 or higher, it provides MPI up to version 3. When `mpich` is at version 1 or higher, it provides the MPI virtual package at version 1.

The `when` qualifier ensures that Spack selects a suitably high version of `mpich` to satisfy some other package that `depends_on` a particular version of MPI. It will also prevent a user from building with too low a version of `mpich`. For example, suppose the package `foo` declares this:

```
class Foo(Package):
    ...
    depends_on('mpi@2')
```

Suppose a user invokes `spack install` like this:

```
$ spack install foo ^mpich@1.0
```

Spack will fail with a constraint violation, because the version of MPICH requested is too low for the `mpi` requirement in `foo`.

10.17 Abstract & concrete specs

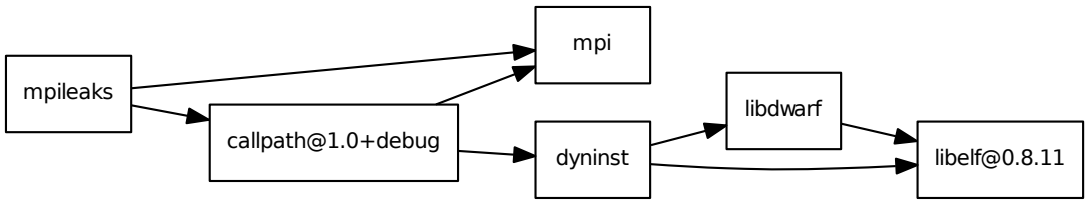
Now that we've seen how spec constraints can be specified *on the command line* and within package definitions, we can talk about how Spack puts all of this information together. When you run this:

```
$ spack install mpileaks ^callpath@1.0+debug ^libelf@0.8.11
```

Spack parses the command line and builds a spec from the description. The spec says that `mpileaks` should be built with the `callpath` library at 1.0 and with the debug option enabled, and with `libelf` version 0.8.11. Spack will also look at the `depends_on` calls in all of these packages, and it will build a spec from that. The specs from the command line and the specs built from package descriptions are then combined, and the constraints are checked against each other to make sure they're satisfiable.

What we have after this is done is called an *abstract spec*. An abstract spec is partially specified. In other words, it could describe more than one build of a package. Spack does this to make things easier on the user: they should only have to specify as much of the package spec as they care about. Here's an example partial spec DAG, based on the constraints above:

```
mpileaks
    ^callpath@1.0+debug
        ^dyninst
            ^libdwarf
                ^libelf@0.8.11
        ^mpi
```

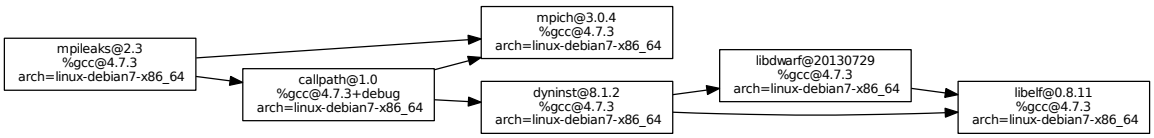


This diagram shows a spec DAG output as a tree, where successive levels of indentation represent a depends-on relationship. In the above DAG, we can see some packages annotated with their constraints, and some packages with no annotations at all. When there are no annotations, it means the user doesn't care what configuration of that package is built, just so long as it works.

10.17.1 Concretization

An abstract spec is useful for the user, but you can't install an abstract spec. Spack has to take the abstract spec and "fill in" the remaining unspecified parts in order to install. This process is called **concretization**. Concretization happens in between the time the user runs `spack install` and the time the `install()` method is called. The concretized version of the spec above might look like this:

```
mpileaks@2.3%gcc@4.7.3 arch=linux-debian7-x86_64
    ^callpath@1.0%gcc@4.7.3+debug arch=linux-debian7-x86_64
        ^dyninst@8.1.2%gcc@4.7.3 arch=linux-debian7-x86_64
            ^libdwarf@20130729%gcc@4.7.3 arch=linux-debian7-x86_64
                ^libelf@0.8.11%gcc@4.7.3 arch=linux-debian7-x86_64
        ^mpich@3.0.4%gcc@4.7.3 arch=linux-debian7-x86_64
```



Here, all versions, compilers, and platforms are filled in, and there is a single version (no version ranges) for each package. All decisions about configuration have been made, and only after this point will Spack call the `install()` method for your package.

Concretization in Spack is based on certain selection policies that tell Spack how to select, e.g., a version, when one is not specified explicitly. Concretization policies are discussed in more detail in *Configuration*. Sites using Spack can customize them to match the preferences of their own users.

10.17.2 `spack spec`

For an arbitrary spec, you can see the result of concretization by running `spack spec`. For example:

```
$ spack spec dyninst@8.0.1
dyninst@8.0.1
    ^libdwarf
        ^libelf

dyninst@8.0.1%gcc@4.7.3 arch=linux-debian7-x86_64
    ^libdwarf@20130729%gcc@4.7.3 arch=linux-debian7-x86_64
        ^libelf@0.8.13%gcc@4.7.3 arch=linux-debian7-x86_64
```

This is useful when you want to know exactly what Spack will do when you ask for a particular spec.

10.17.3 Concretization Policies

A user may have certain preferences for how packages should be concretized on their system. For example, one user may prefer packages built with OpenMPI and the Intel compiler. Another user may prefer packages be built with MVAPICH and GCC.

See the *Concretization Preferences* section for more details.

10.18 Inconsistent Specs

Suppose a user needs to install package C, which depends on packages A and B. Package A builds a library with a Python2 extension, and package B builds a library with a Python3 extension. Packages A and B cannot be loaded together in the same Python runtime:

```
class A(Package):
    variant('python', default=True, 'enable python bindings')
    depends_on('python@2.7', when='+python')
    def install(self, spec, prefix):
        # do whatever is necessary to enable/disable python
        # bindings according to variant

class B(Package):
    variant('python', default=True, 'enable python bindings')
    depends_on('python@3.2:', when='+python')
    def install(self, spec, prefix):
        # do whatever is necessary to enable/disable python
        # bindings according to variant
```

Package C needs to use the libraries from packages A and B, but does not need either of the Python extensions. In this case, package C should simply depend on the `~python` variant of A and B:

```
class C(Package):
    depends_on('A~python')
    depends_on('B~python')
```

This may require that A or B be built twice, if the user wishes to use the Python extensions provided by them: once for `+python` and once for `~python`. Other than using a little extra disk space, that solution has no serious problems.

10.19 Implementing the `install` method

The last element of a package is its `install()` method. This is where the real work of installation happens, and it's the main part of the package you'll need to customize for each piece of software.

```
1     def install(self, spec, prefix):
2         configure("--prefix=" + prefix,
3                   "--enable-shared",
4                   "--disable-dependency-tracking",
5                   "--disable-debug")
6         make()
7
8         # The mkdir commands in libelf's install can fail in parallel
9         make("install", parallel=False)
```

`install` takes a `spec`: a description of how the package should be built, and a `prefix`: the path to the directory where the software should be installed.

Spack provides wrapper functions for `configure` and `make` so that you can call them in a similar way to how you'd call a shell command. In reality, these are Python functions. Spack provides these functions to make writing packages more natural. See the section on *shell wrappers*.

Now that the metadata is out of the way, we can move on to the `install()` method. When a user runs `spack install`, Spack fetches an archive for the correct version of the software, expands the archive, and sets the current working directory to the root directory of the expanded archive. It then instantiates a package object and calls the `install()` method.

The `install()` signature looks like this:

```
class Foo(Package):
    def install(self, spec, prefix):
        ...
```

The parameters are as follows:

self For those not used to Python instance methods, this is the package itself. In this case it's an instance of `Foo`, which extends `Package`. For API docs on Package objects, see *Package*.

spec This is the concrete spec object created by Spack from an abstract spec supplied by the user. It describes what should be installed. It will be of type *Spec*.

prefix This is the path that your install method should copy build targets into. It acts like a string, but it's actually its own special type, *Prefix*.

`spec` and `prefix` are passed to `install` for convenience. `spec` is also available as an attribute on the package (`self.spec`), and `prefix` is actually an attribute of `spec` (`spec.prefix`).

As mentioned in *The install environment*, you will usually not need to refer to dependencies explicitly in your package file, as the compiler wrappers take care of most of the heavy lifting here. There will be times, though, when you need to refer to the install locations of dependencies, or when you need to do something different depending on the version, compiler, dependencies, etc. that your package is built with. These parameters give you access to this type of information.

10.20 The install environment

In general, you should not have to do much differently in your install method than you would when installing a package on the command line. In fact, you may need to do *less* than you would on the command line.

Spack tries to set environment variables and modify compiler calls so that it *appears* to the build system that you're building with a standard system install of everything. Obviously that's not going to cover *all* build systems, but it should make it easy to port packages to Spack if they use a standard build system. Usually with autotools or cmake, building and installing is easy. With builds that use custom Makefiles, you may need to add logic to modify the makefiles.

The remainder of the section covers the way Spack's build environment works.

10.20.1 Environment variables

Spack sets a number of standard environment variables that serve two purposes:

1. Make build systems use Spack's compiler wrappers for their builds.
2. Allow build systems to find dependencies more easily

The Compiler environment variables that Spack sets are:

Variable	Purpose
`CC`	C compiler
`CXX`	C++ compiler
`F77`	Fortran 77 compiler
`FC`	Fortran 90 and above compiler

All of these are standard variables respected by most build systems. If your project uses `Autotools` or `CMake`, then it should pick them up automatically when you run `configure` or `cmake` in the `install()` function. Many traditional builds using GNU Make and BSD make also respect these variables, so they may work with these systems.

If your build system does *not* automatically pick these variables up from the environment, then you can simply pass them on the command line or use a patch as part of your build process to get the correct compilers into the project's build system. There are also some file editing commands you can use – these are described later in the *section on file manipulation*.

In addition to the compiler variables, these variables are set before entering `install()` so that packages can locate dependencies easily:

PATH	Set to point to /bin directories of dependencies
CMAKE_PREFIX_PATH	Path to dependency prefixes for CMake
PKG_CONFIG_PATH	Path to any pkgconfig directories for dependencies
PYTHONPATH	Path to site-packages dir of any python dependencies

PATH is set up to point to dependencies /bin directories so that you can use tools installed by dependency packages at build time. For example, $MPICH_ROOT/bin/mpicc is frequently used by dependencies of mpich.

CMAKE_PREFIX_PATH contains a colon-separated list of prefixes where cmake will search for dependency libraries and headers. This causes all standard CMake find commands to look in the paths of your dependencies, so you *do not* have to manually specify arguments like -DDEPENDENCY_DIR=/path/to/dependency to cmake. More on this is in the CMake documentation.

PKG_CONFIG_PATH is for packages that attempt to discover dependencies using the GNU pkg-config tool. It is similar to CMAKE_PREFIX_PATH in that it allows a build to automatically discover its dependencies.

If you want to see the environment that a package will build with, or if you want to run commands in that environment to test them out, you can use the *spack env* command, documented below.

10.20.2 Compiler interceptors

As mentioned, CC, CXX, F77, and FC are set to point to Spack's compiler wrappers. These are simply called cc, c++, f77, and f90, and they live in $SPACK_ROOT/lib/spack/env.

$SPACK_ROOT/lib/spack/env is added first in the PATH environment variable when install() runs so that system compilers are not picked up instead.

All of these compiler wrappers point to a single compiler wrapper script that figures out which *real* compiler it should be building with. This comes either from spec concretization or from a user explicitly asking for a particular compiler using, e.g., %intel on the command line.

In addition to invoking the right compiler, the compiler wrappers add flags to the compile line so that dependencies can be easily found. These flags are added for each dependency, if they exist:

Compile-time library search paths * -L$dep_prefix/lib * -L$dep_prefix/lib64

Runtime library search paths (RPATHs) * $rpath_flag$dep_prefix/lib * $rpath_flag$dep_prefix/lib64

Include search paths * -I$dep_prefix/include

An example of this would be the libdwarf build, which has one dependency: libelf. Every call to cc in the libdwarf build will have -I$LIBELF_PREFIX/include, -L$LIBELF_PREFIX/lib, and $rpath_flag$LIBELF_PREFIX/lib inserted on the command line. This is done transparently to the project's build system, which will just think it's using a system where libelf is readily available. Because of this, you **do not** have to insert extra -I, -L, etc. on the command line.

Another useful consequence of this is that you often do *not* have to add extra parameters on the configure line to get autotools to find dependencies. The libdwarf install method just calls configure like this:

```
configure("--prefix=" + prefix)
```

Because of the -L and -I arguments, configure will successfully find libdwarf.h and libdwarf.so, without the packager having to provide --with-libdwarf=/path/to/libdwarf on the command line.

Note: For most compilers, $rpath_flag is -Wl,-rpath,. However, NAG passes its flags to GCC instead of passing them directly to the linker. Therefore, its $rpath_flag is doubly

wrapped: `-Wl,-Wl„-rpath,.` `$rpath_flag` can be overriden on a compiler specific basis in `lib/spack/spack/compilers/$compiler.py`.

The compiler wrappers also pass the compiler flags specified by the user from the command line (`cflags`, `cxxflags`, `fflags`, `cppflags`, `ldflags`, and/or `ldlibs`). They do not override the canonical autotools flags with the same names (but in ALL-CAPS) that may be passed into the build by particularly challenging package scripts.

10.20.3 Compiler flags

In rare circumstances such as compiling and running small unit tests, a package developer may need to know what are the appropriate compiler flags to enable features like `OpenMP`, `c++11`, `c++14` and alike. To that end the compiler classes in `spack` implement the following **properties**: `openmp_flag`, `cxx11_flag`, `cxx14_flag`, which can be accessed in a package by `self.compiler.cxx11_flag` and alike. Note that the implementation is such that if a given compiler version does not support this feature, an error will be produced. Therefore package developers can also use these properties to assert that a compiler supports the requested feature. This is handy when a package supports additional variants like

```
variant('openmp', default=True, description="Enable OpenMP support.")
```

10.20.4 Message Parsing Interface (MPI)

It is common for high performance computing software/packages to use `MPI`. As a result of conretization, a given package can be built using different implementations of MPI such as `Openmpi`, `MPICH` or `IntelMPI`. In some scenarios, to configure a package, one has to provide it with appropriate MPI compiler wrappers such as `mpicc`, `mpic++`. However different implementations of `MPI` may have different names for those wrappers. In order to make package's `install()` method indifferent to the choice `MPI` implementation, each package which implements `MPI` sets up `self.spec.mpicc`, `self.spec.mpicxx`, `self.spec.mpifc` and `self.spec.mpif77` to point to `C`, `C++`, `Fortran 90` and `Fortran 77` `MPI` wrappers. Package developers are advised to use these variables, for example `self.spec['mpi'].mpicc` instead of hard-coding `join_path(self.spec['mpi'].prefix.bin, 'mpicc')` for the reasons outlined above.

10.20.5 Blas and Lapack libraries

Different packages provide implementation of `Blas` and `Lapack` routines. The names of the resulting static and/or shared libraries differ from package to package. In order to make the `install()` method independent of the choice of `Blas` implementation, each package which provides it sets up `self.spec.blas_libs` to point to the correct `Blas` libraries. The same applies to packages which provide `Lapack`. Package developers are advised to use these variables, for example `spec['blas'].blas_libs.joined()` instead of hard-coding `join_path(spec['blas'].prefix.lib, 'libopenblas.so')`.

10.20.6 Forking `install()`

To give packagers free reign over their install environment, Spack forks a new process each time it invokes a package's `install()` method. This allows packages to have their own completely sandboxed build environment, without impacting other jobs that the main Spack process runs. Packages are free to change the environment or to modify Spack internals, because each `install()` call has its own dedicated process.

10.21 Failing the build

Sometimes you don't want a package to successfully install unless some condition is true. You can explicitly cause the build to fail from `install()` by raising an `InstallError`, for example:

```
if spec.architecture.startswith('darwin'):
    raise InstallError('This package does not build on Mac OS X!')
```

10.22 Prefix objects

Spack passes the `prefix` parameter to the install method so that you can pass it to `configure`, `cmake`, or some other installer, e.g.:

```
configure('--prefix=' + prefix)
```

For the most part, prefix objects behave exactly like strings. For packages that do not have their own install target, or for those that implement it poorly (like `libdwarf`), you may need to manually copy things into particular directories under the prefix. For this, you can refer to standard subdirectories without having to construct paths yourself, e.g.:

```
def install(self, spec, prefix):
    mkdirp(prefix.bin)
    install('foo-tool', prefix.bin)

    mkdirp(prefix.include)
    install('foo.h', prefix.include)

    mkdirp(prefix.lib)
    install('libfoo.a', prefix.lib)
```

Most of the standard UNIX directory names are attributes on the `prefix` object. Here is a full list:

Prefix Attribute	Location
`prefix.bin`	`$prefix/bin`
`prefix.sbin`	`$prefix/sbin`
`prefix.etc`	`$prefix/etc`
`prefix.include`	`$prefix/include`
`prefix.lib`	`$prefix/lib`
`prefix.lib64`	`$prefix/lib64`
`prefix.libexec`	`$prefix/libexec`
`prefix.share`	`$prefix/share`
`prefix.doc`	`$prefix/doc`
`prefix.info`	`$prefix/info`
`prefix.man`	`$prefix/man`
`prefix.man[1-8]`	`$prefix/man/man[1-8]`
`prefix.share_man`	`$prefix/share/man`
`prefix.share_man[1-8]`	`$prefix/share/man[1-8]`

10.23 Spec objects

When `install` is called, most parts of the build process are set up for you. The correct version's tarball has been downloaded and expanded. Environment variables like `CC` and `CXX` are set to point to the correct compiler and

version. An install prefix has already been selected and passed in as `prefix`. In most cases this is all you need to get `configure`, `cmake`, or another install working correctly.

There will be times when you need to know more about the build configuration. For example, some software requires that you pass special parameters to `configure`, like `--with-libelf=/path/to/libelf` or `--with-mpich`. You might also need to supply special compiler flags depending on the compiler. All of this information is available in the spec.

10.23.1 Testing spec constraints

You can test whether your spec is configured a certain way by using the `satisfies` method. For example, if you want to check whether the package's version is in a particular range, you can use specs to do that, e.g.:

```
configure_args = [
    '--prefix={0}'.format(prefix)
]

if spec.satisfies('@1.2:1.4'):
    configure_args.append("CXXFLAGS='-DWITH_FEATURE'")

configure(*configure_args)
```

This works for compilers, too:

```
if spec.satisfies('%gcc'):
    configure_args.append('CXXFLAGS="-g3 -O3"')
if spec.satisfies('%intel'):
    configure_args.append('CXXFLAGS="-xSSE2 -fast"')
```

Or for combinations of spec constraints:

```
if spec.satisfies('@1.2%intel'):
    tty.error("Version 1.2 breaks when using Intel compiler!")
```

You can also do similar satisfaction tests for dependencies:

```
if spec.satisfies('^dyninst@8.0'):
    configure_args.append('CXXFLAGS=-DSPECIAL_DYNINST_FEATURE')
```

This could allow you to easily work around a bug in a particular dependency version.

You can use `satisfies()` to test for particular dependencies, e.g. `foo.satisfies('^openmpi@1.2')` or `foo.satisfies('^mpich')`, or you can use Python's built-in `in` operator:

```
if 'libelf' in spec:
    print "this package depends on libelf"
```

This is useful for virtual dependencies, as you can easily see what implementation was selected for this build:

```
if 'openmpi' in spec:
    configure_args.append('--with-openmpi')
elif 'mpich' in spec:
    configure_args.append('--with-mpich')
elif 'mvapich' in spec:
    configure_args.append('--with-mvapich')
```

It's also a bit more concise than satisfies. The difference between the two functions is that `satisfies()` tests whether spec constraints overlap at all, while `in` tests whether a spec or any of its dependencies satisfy the provided spec.

10.23.2 Accessing Dependencies

You may need to get at some file or binary that's in the prefix of one of your dependencies. You can do that by sub-scripting the spec:

```
my_mpi = spec['mpi']
```

The value in the brackets needs to be some package name, and spec needs to depend on that package, or the operation will fail. For example, the above code will fail if the `spec` doesn't depend on `mpi`. The value returned and assigned to `my_mpi`, is itself just another `Spec` object, so you can do all the same things you would do with the package's own spec:

```
mpicc = join_path(my_mpi.prefix.bin, 'mpicc')
```

10.23.3 Multimethods and `@when`

Spack allows you to make multiple versions of instance functions in packages, based on whether the package's spec satisfies particular criteria.

The `@when` annotation lets packages declare multiple versions of methods like `install()` that depend on the package's spec. For example:

```
class SomePackage(Package):
    ...

    def install(self, prefix):
        # Do default install

    @when('arch=chaos_5_x86_64_ib')
    def install(self, prefix):
        # This will be executed instead of the default install if
        # the package's sys_type() is chaos_5_x86_64_ib.

    @when('arch=linux-debian7-x86_64')
    def install(self, prefix):
        # This will be executed if the package's sys_type() is
        # linux-debian7-x86_64.
```

In the above code there are three versions of `install()`, two of which are specialized for particular platforms. The version that is called depends on the architecture of the package spec.

Note that this works for methods other than install, as well. So, if you only have part of the install that is platform specific, you could do something more like this:

```
class SomePackage(Package):
   ...
    # virtual dependence on MPI.
    # could resolve to mpich, mpich2, OpenMPI
    depends_on('mpi')

    def setup(self):
        # do nothing in the default case
        pass

    @when('^openmpi')
    def setup(self):
        # do something special when this is built with OpenMPI for
        # its MPI implementations.
```

```
    def install(self, prefix):
        # Do common install stuff
        self.setup()
        # Do more common install stuff
```

You can write multiple `@when` specs that satisfy the package's spec, for example:

```
class SomePackage(Package):
    ...
    depends_on('mpi')

    def setup_mpi(self):
        # the default, called when no @when specs match
        pass

    @when('^mpi@3:')
    def setup_mpi(self):
        # this will be called when mpi is version 3 or higher
        pass

    @when('^mpi@2:')
    def setup_mpi(self):
        # this will be called when mpi is version 2 or higher
        pass

    @when('^mpi@1:')
    def setup_mpi(self):
        # this will be called when mpi is version 1 or higher
        pass
```

In situations like this, the first matching spec, in declaration order will be called. As before, if no `@when` spec matches, the default method (the one without the `@when` decorator) will be called.

Warning: The default version of decorated methods must **always** come first. Otherwise it will override all of the platform-specific versions. There's not much we can do to get around this because of the way decorators work.

10.24 Shell command functions

Recall the install method from `libelf`:

```
    def install(self, spec, prefix):
        configure("--prefix=" + prefix,
                  "--enable-shared",
                  "--disable-dependency-tracking",
                  "--disable-debug")
        make()

        # The mkdir commands in libelf's install can fail in parallel
        make("install", parallel=False)
```

Normally in Python, you'd have to write something like this in order to execute shell commands:

```
import subprocess
subprocess.check_call('configure', '--prefix={0}'.format(prefix))
```

We've tried to make this a bit easier by providing callable wrapper objects for some shell commands. By default, `configure`, `cmake`, and `make` wrappers are are provided, so you can call them more naturally in your package files.

If you need other commands, you can use `which` to get them:

```
sed = which('sed')
sed('s/foo/bar/', filename)
```

The `which` function will search the `PATH` for the application.

Callable wrappers also allow spack to provide some special features. For example, in Spack, `make` is parallel by default, and Spack figures out the number of cores on your machine and passes an appropriate value for `-j<numjobs>` when it calls `make` (see the `parallel` *package attribute* <*attribute_parallel*>). In a package file, you can supply a keyword argument, `parallel=False`, to the `make` wrapper to disable parallel make. In the `libelf` package, this allows us to avoid race conditions in the library's build system.

10.25 Sanity checking an installation

By default, Spack assumes that a build has failed if nothing is written to the install prefix, and that it has succeeded if anything (a file, a directory, etc.) is written to the install prefix after `install()` completes.

Consider a simple autotools build like this:

```
def install(self, spec, prefix):
    configure("--prefix={0}".format(prefix))
    make()
    make("install")
```

If you are using using standard autotools or CMake, `configure` and `make` will not write anything to the install prefix. Only `make install` writes the files, and only once the build is already complete. Not all builds are like this. Many builds of scientific software modify the install prefix *before* `make install`. Builds like this can falsely report that they were successfully installed if an error occurs before the install is complete but after files have been written to the `prefix`.

10.25.1 `sanity_check_is_file` and `sanity_check_is_dir`

You can optionally specify *sanity checks* to deal with this problem. Add properties like this to your package:

```
class MyPackage(Package):
    ...

    sanity_check_is_file = ['include/libelf.h']
    sanity_check_is_dir  = [lib]

    def install(self, spec, prefix):
        configure("--prefix=" + prefix)
        make()
        make("install")
```

Now, after `install()` runs, Spack will check whether `$prefix/include/libelf.h` exists and is a file, and whether `$prefix/lib` exists and is a directory. If the checks fail, then the build will fail and the install prefix will be removed. If they succeed, Spack considers the build successful and keeps the prefix in place.

10.26 File manipulation functions

Many builds are not perfect. If a build lacks an install target, or if it does not use systems like CMake or autotools, which have standard ways of setting compilers and options, you may need to edit files or install some files yourself to get them working with Spack.

You can do this with standard Python code, and Python has rich libraries with functions for file manipulation and filtering. Spack also provides a number of convenience functions of its own to make your life even easier. These functions are described in this section.

All of the functions in this section can be included by simply running:

```
from spack import *
```

This is already part of the boilerplate for packages created with `spack create` or `spack edit`.

10.26.1 Filtering functions

`filter_file(regex, repl, *filenames, **kwargs)` Works like `sed` but with Python regular expression syntax. Takes a regular expression, a replacement, and a set of files. `repl` can be a raw string or a callable function. If it is a raw string, it can contain `\1`, `\2`, etc. to refer to capture groups in the regular expression. If it is a callable, it is passed the Python `MatchObject` and should return a suitable replacement string for the particular match.

Examples:

1. Filtering a Makefile to force it to use Spack's compiler wrappers:

```
filter_file(r'^CC\s*=.*',  spack_cc,  'Makefile')
filter_file(r'^CXX\s*=.*', spack_cxx, 'Makefile')
filter_file(r'^F77\s*=.*', spack_f77, 'Makefile')
filter_file(r'^FC\s*=.*',  spack_fc,  'Makefile')
```

2. Replacing `#!/usr/bin/perl` with `#!/usr/bin/env perl` in `bib2xhtml`:

```
filter_file(r'#!/usr/bin/perl',
            '#!/usr/bin/env perl', join_path(prefix.bin, 'bib2xhtml'))
```

3. Switching the compilers used by `mpich`'s MPI wrapper scripts from `cc`, etc. to the compilers used by the Spack build:

```
filter_file('CC="cc"', 'CC="%s"' % self.compiler.cc,
            join_path(prefix.bin, 'mpicc'))

filter_file('CXX="c++"', 'CXX="%s"' % self.compiler.cxx,
            join_path(prefix.bin, 'mpicxx'))
```

`change_sed_delimiter(old_delim, new_delim, *filenames)` Some packages, like TAU, have a build system that can't install into directories with, e.g. '@' in the name, because they use hard-coded `sed` commands in their build.

`change_sed_delimiter` finds all `sed` search/replace commands and change the delimiter. e.g., if the file contains commands that look like `s///`, you can use this to change them to `s@@@`.

Example of changing `s///` to `s@@@` in TAU:

```
change_sed_delimiter('@', ';', 'configure')
change_sed_delimiter('@', ';', 'utils/FixMakefile')
change_sed_delimiter('@', ';', 'utils/FixMakefile.sed.default')
```

10.26.2 File functions

ancestor(dir, n=1) Get the n[th] ancestor of the directory `dir`.

can_access(path) True if we can read and write to the file at `path`. Same as native python `os.access(file_name, os.R_OK|os.W_OK)`.

install(src, dest) Install a file to a particular location. For example, install a header into the `include` directory under the install `prefix`:

```
install('my-header.h', join_path(prefix.include))
```

join_path(prefix, *args) Like `os.path.join`, this joins paths using the OS path separator. However, this version allows an arbitrary number of arguments, so you can string together many path components.

mkdirp(*paths) Create each of the directories in `paths`, creating any parent directories if they do not exist.

working_dir(dirname, kwargs) This is a Python Context Manager that makes it easier to work with subdirectories in builds. You use this with the Python `with` statement to change into a working directory, and when the with block is done, you change back to the original directory. Think of it as a safe `pushd` / `popd` combination, where `popd` is guaranteed to be called at the end, even if exceptions are thrown.

Example usage:

1. The `libdwarf` build first runs `configure` and `make` in a subdirectory called `libdwarf`. It then implements the installation code itself. This is natural with `working_dir`:

```
with working_dir('libdwarf'):
    configure("--prefix=" + prefix, "--enable-shared")
    make()
    install('libdwarf.a',  prefix.lib)
```

2. Many CMake builds require that you build "out of source", that is, in a subdirectory. You can handle creating and `cd`'ing to the subdirectory like the LLVM package does:

```
with working_dir('spack-build', create=True):
    cmake('..',
          '-DLLVM_REQUIRES_RTTI=1',
          '-DPYTHON_EXECUTABLE=/usr/bin/python',
          '-DPYTHON_INCLUDE_DIR=/usr/include/python2.6',
          '-DPYTHON_LIBRARY=/usr/lib64/libpython2.6.so',
          *std_cmake_args)
    make()
    make("install")
```

The `create=True` keyword argument causes the command to create the directory if it does not exist.

touch(path) Create an empty file at `path`.

10.27 Coding Style Guidelines

The following guidelines are provided, in the interests of making Spack packages work in a consistent manner:

10.27.1 Variant Names

Spack packages with variants similar to already-existing Spack packages should use the same name for their variants. Standard variant names are:

Name	Default	Description
shared	True	Build shared libraries
static	True	Build static libraries
mpi	True	Use MPI
python	False	Build Python extension

If specified in this table, the corresponding default should be used when declaring a variant.

10.27.2 Version Lists

Spack packages should list supported versions with the newest first.

10.27.3 Special Versions

The following *special* version names may be used when building a package:

`@system`

Indicates a hook to the OS-installed version of the package. This is useful, for example, to tell Spack to use the OS-installed version in `packages.yaml`:

```
openssl:
  paths:
    openssl@system: /usr
  buildable: False
```

Certain Spack internals look for the `@system` version and do appropriate things in that case.

`@local`

Indicates the version was built manually from some source tree of unknown provenance (see `spack setup`).

10.28 Packaging workflow commands

When you are building packages, you will likely not get things completely right the first time.

The `spack install` command performs a number of tasks before it finally installs each package. It downloads an archive, expands it in a temporary directory, and only then gives control to the package's `install()` method. If the build doesn't go as planned, you may want to clean up the temporary directory, or if the package isn't downloading properly, you might want to run *only* the `fetch` stage of the build.

A typical package workflow might look like this:

```
$ spack edit mypackage
$ spack install mypackage
... build breaks! ...
$ spack clean mypackage
$ spack edit mypackage
$ spack install mypackage
... repeat clean/install until install works ...
```

Below are some commands that will allow you some finer-grained control over the install process.

10.28.1 spack fetch

The first step of `spack install`. Takes a spec and determines the correct download URL to use for the requested package version, then downloads the archive, checks it against an MD5 checksum, and stores it in a staging directory if the check was successful. The staging directory will be located under `$SPACK_HOME/var/spack`.

When run after the archive has already been downloaded, `spack fetch` is idempotent and will not download the archive again.

10.28.2 spack stage

The second step in `spack install` after `spack fetch`. Expands the downloaded archive in its temporary directory, where it will be built by `spack install`. Similar to `fetch`, if the archive has already been expanded, `stage` is idempotent.

10.28.3 spack patch

After staging, Spack applies patches to downloaded packages, if any have been specified in the package file. This command will run the install process through the fetch, stage, and patch phases. Spack keeps track of whether patches have already been applied and skips this step if they have been. If Spack discovers that patches didn't apply cleanly on some previous run, then it will restage the entire package before patching.

10.28.4 spack restage

Restores the source code to pristine state, as it was before building.

Does this in one of two ways:

1. If the source was fetched as a tarball, deletes the entire build directory and re-expands the tarball.
2. If the source was checked out from a repository, this deletes the build directory and checks it out again.

10.28.5 spack clean

Cleans up temporary files for a particular package, by deleting the expanded/checked out source code *and* any downloaded archive. If `fetch`, `stage`, or `install` are run again after this, Spack's build process will start from scratch.

10.28.6 spack purge

Cleans up all of Spack's temporary and cached files. This can be used to recover disk space if temporary files from interrupted or failed installs accumulate in the staging area.

When called with `--stage` or `--all` (or without arguments, in which case the default is `--all`) this removes all staged files; this is equivalent to running `spack clean` for every package you have fetched or staged.

When called with `--cache` or `--all` this will clear all resources *cached* during installs.

10.28.7 Keeping the stage directory on success

By default, `spack install` will delete the staging area once a package has been successfully built and installed. Use `--keep-stage` to leave the build directory intact:

```
$ spack install --keep-stage <spec>
```

This allows you to inspect the build directory and potentially debug the build. You can use `purge` or `clean` later to get rid of the unwanted temporary files.

10.28.8 Keeping the install prefix on failure

By default, `spack install` will delete any partially constructed install prefix if anything fails during `install()`. If you want to keep the prefix anyway (e.g. to diagnose a bug), you can use `--keep-prefix`:

```
$ spack install --keep-prefix <spec>
```

Note that this may confuse Spack into thinking that the package has been installed properly, so you may need to use `spack uninstall --force` to get rid of the install prefix before you build again:

```
$ spack uninstall --force <spec>
```

10.29 Graphing dependencies

10.29.1 `spack graph`

Spack provides the `spack graph` command for graphing dependencies. The command by default generates an ASCII rendering of a spec's dependency graph. For example:

```
$ spack graph mpileaks
o  mpileaks
|\
| |\
| o |  callpath
|/| |
| |\|
| |\ \
| | |\ \
| | | o |  dyninst
| | |/| |
| |/|/| |
| | | | o  adept-utils
| |_|_|/|
|/| | |/
| | | o  boost
| | | |\
| | | o |  zlib
| | |  /
o | | |  openmpi
o | | |  hwloc
o | | |  libpciaccess
 / / /
| o |  libdwarf
|/ /
o |  libelf
 /
o  bzip2
```

At the top is the root package in the DAG, with dependency edges emerging from it. On a color terminal, the edges are colored by which dependency they lead to.

```
$ spack graph --deptype=all mpileaks
o  mpileaks
|\
| |\
| o |  callpath
|/| |
| |\|
| |\ \
| | |\ \
| | | |\ \
| | | o | |  dyninst
| | |/| | |
| |/|/| | |
| | | |\| |
| | | | | o  adept-utils
| |_|_|_|/|
|/| | | |/|
| | | |/|/
| | | | o  cmake
| | | | |\
| | | | | |\
| | | | | | |\
| | | | | | | |\
| | | | | | | | |\
| | | | | | | | | |\
| | | | | | | | o | | |  libarchive
| | | | | |_|_|/| | | |
| | | | |/| |_|/| | | |
| | | | | |/| |/| | | |
| | | | | | |/| | | | |
| | | | | | | | |\| | |
| | | | | | | | |\ \ \ \
| | | | | | | | | |\ \ \ \
| | | | | | | | | | |\ \ \ \
| | | | | | | | | | | |\ \ \ \
| | | | | | | | | | | | |\ \ \ \
| | | | | | | | | | | | | | |_|/
| | | | | | | | | | | | | |/| |
| | | | | | | | | | | | | | o  curl
| | | | | |_|_|_|_|_|_|_|_|_|/|
| | | | |/| | |_|_|_|_|_|_|_|/
| | | | | | |/| | | | | | | |
| | | | | | o | | | | | | | |  openssl
| | | | | |/ / / / / / / / /
| | | | |/| | | | | | | | |
| | | | | | | | | | | o | |  libxml2
| | | | | |_|_|_|_|_|/| | |
| | | | |/| |_|_|_|_|/ / /
| | | | | |/| | | | | | |
| | | o | | | | | | | | |  boost
| | | |\| | | | | | | | |
| | | | |_|_|_|_|_|_|/ /
| | | |/| | | | | | | |
| | | | o | | | | | | |  zlib
| | | |  / / / / / / /
| | | | o | | | | | |  xz
| | | |  / / / / / /
```

```
o | | | | | | | | |  openmpi
| | | | | o | | | |  nettle
| | | | o | | | | |  ncurses
| | | |  / / / / /
o | | | | | | | |  hwloc
o | | | | | | | |  libpciaccess
o | | | | | | | |  libtool
| | | | o | | | |  gmp
| |_|_|/ / / / /
|/| | | | | | |
o | | | | | | |  m4
| | | | o | | |  lzo
| | | |  / / /
| | | | o | |  lzma
| | | |  / /
| | | | o |  lz4
| | | |  /
o | | | |  libsigsegv
 / / / /
| o | |  libdwarf
|/ / /
o | |  libelf
 / /
| o  expat
|
o  bzip2
```

The `deptype` argument tells Spack what types of dependencies to graph. By default it includes link and run dependencies but not build dependencies. Supplying `--deptype=all` will show the build dependencies as well. This is equivalent to `--deptype=build,link,run`. Options for `deptype` include:

- Any combination of `build`, `link`, and `run` separated by commas.
- `nobuild`, `nolink`, `norun` to omit one type.
- `all` or `alldeps` for all types of dependencies.

You can also use `spack graph` to generate graphs in the widely used Dot format. For example:

```
$ spack graph --dot mpileaks
digraph G {
  labelloc = "b"
  rankdir = "TB"
  ranksep = "5"
node[
     fontname=Monaco,
     penwidth=2,
     fontsize=12,
     margin=.1,
     shape=box,
     fillcolor=lightblue,
     style="rounded,filled"]

  "okmcnkbi566dnpgswy3o5fzhcnm37aul" [label="mpileaks-okmcnkb"]
  "hgro7j6kls3kusklxszximkknrwvholv" [label="adept-utils-hgro7j6"]
  "2zkydrm2mmxhbubxvwvaleit5l2jgn5k" [label="boost-2zkydrm"]
  "ykdjqd4wbklyofwszkotdhcjn6bhnuii" [label="bzip2-ykdjqd4"]
  "bds4iesjpb62krtxlwbirx66degfrtwn" [label="zlib-bds4ies"]
  "dhkbno5zs3cg5stcmnfturyhqvfbsf4y" [label="openmpi-dhkbno5"]
  "uofzaakjqqx5kqtzm6ss52hei7qn7w2s" [label="hwloc-uofzaak"]
```

```
  "agw4feb42kwzk34z5jgnqmt74s7gr463" [label="libpciaccess-agw4feb"]
  "2fbmwvtojpz4232o2ba7sdpd7nkgxddm" [label="callpath-2fbmwvt"]
  "bcw4iyeu6s3eatnlzo7auzsdi4zflrxq" [label="dyninst-bcw4iye"]
  "ks2fyujjpfydci7xpwjar3zfepoyais5" [label="libdwarf-ks2fyuj"]
  "jldavpu2djm4w4ewjzougoflgqqlplu3" [label="libelf-jldavpu"]

  "ks2fyujjpfydci7xpwjar3zfepoyais5" -> "jldavpu2djm4w4ewjzougoflgqqlplu3"
  "2fbmwvtojpz4232o2ba7sdpd7nkgxddm" -> "hgro7j6kls3kusklxszximkknrwvholv"
  "bcw4iyeu6s3eatnlzo7auzsdi4zflrxq" -> "ks2fyujjpfydci7xpwjar3zfepoyais5"
  "okmcnkbi566dnpgswy3o5fzhcnm37aul" -> "dhkbno5zs3cg5stcmnfturyhqvfbsf4y"
  "2fbmwvtojpz4232o2ba7sdpd7nkgxddm" -> "jldavpu2djm4w4ewjzougoflgqqlplu3"
  "2fbmwvtojpz4232o2ba7sdpd7nkgxddm" -> "dhkbno5zs3cg5stcmnfturyhqvfbsf4y"
  "2zkydrm2mmxhbubxvwvaleit5l2jgn5k" -> "ykdjqd4wbklyofwszkotdhcjn6bhnuii"
  "bcw4iyeu6s3eatnlzo7auzsdi4zflrxq" -> "2zkydrm2mmxhbubxvwvaleit5l2jgn5k"
  "2zkydrm2mmxhbubxvwvaleit5l2jgn5k" -> "bds4iesjpb62krtxlwbirx66degfrtwn"
  "bcw4iyeu6s3eatnlzo7auzsdi4zflrxq" -> "jldavpu2djm4w4ewjzougoflgqqlplu3"
  "okmcnkbi566dnpgswy3o5fzhcnm37aul" -> "hgro7j6kls3kusklxszximkknrwvholv"
  "dhkbno5zs3cg5stcmnfturyhqvfbsf4y" -> "uofzaakjqqx5kqtzm6ss52hei7qn7w2s"
  "okmcnkbi566dnpgswy3o5fzhcnm37aul" -> "2fbmwvtojpz4232o2ba7sdpd7nkgxddm"
  "hgro7j6kls3kusklxszximkknrwvholv" -> "2zkydrm2mmxhbubxvwvaleit5l2jgn5k"
  "hgro7j6kls3kusklxszximkknrwvholv" -> "dhkbno5zs3cg5stcmnfturyhqvfbsf4y"
  "uofzaakjqqx5kqtzm6ss52hei7qn7w2s" -> "agw4feb42kwzk34z5jgnqmt74s7gr463"
  "2fbmwvtojpz4232o2ba7sdpd7nkgxddm" -> "ks2fyujjpfydci7xpwjar3zfepoyais5"
  "2fbmwvtojpz4232o2ba7sdpd7nkgxddm" -> "bcw4iyeu6s3eatnlzo7auzsdi4zflrxq"
}
```

This graph can be provided as input to other graphing tools, such as those in Graphviz.

10.30 Interactive shell support

Spack provides some limited shell support to make life easier for packagers. You can enable these commands by sourcing a setup file in the `share/spack` directory. For `bash` or `ksh`, run:

```
export SPACK_ROOT=/path/to/spack
. $SPACK_ROOT/share/spack/setup-env.sh
```

For `csh` and `tcsh` run:

```
setenv SPACK_ROOT /path/to/spack
source $SPACK_ROOT/share/spack/setup-env.csh
```

`spack cd` will then be available.

10.30.1 `spack cd`

`spack cd` allows you to quickly cd to pertinent directories in Spack. Suppose you've staged a package but you want to modify it before you build it:

```
$ spack stage libelf
==> Trying to fetch from http://www.mr511.de/software/libelf-0.8.13.tar.gz
######################################################################## 100.0%
==> Staging archive: /Users/gamblin2/src/spack/var/spack/stage/libelf@0.8.13%gcc@4.8.3 arch=linux-deb
==> Created stage in /Users/gamblin2/src/spack/var/spack/stage/libelf@0.8.13%gcc@4.8.3 arch=linux-deb
$ spack cd libelf
$ pwd
/Users/gamblin2/src/spack/var/spack/stage/libelf@0.8.13%gcc@4.8.3 arch=linux-debian7-x86_64/libelf-0.
```

`spack cd` here changed the current working directory to the directory containing the expanded `libelf` source code. There are a number of other places you can cd to in the spack directory hierarchy:

```
$ spack cd -h
usage: spack cd [-h] [-m | -r | -i | -p | -P | -s | -S | -b] ...

positional arguments:
  spec                spec of package to fetch directory for.

optional arguments:
  -h, --help          show this help message and exit
  -m, --module-dir    Spack python module directory.
  -r, --spack-root    Spack installation root.
  -i, --install-dir   Install prefix for spec (spec need not be installed).
  -p, --package-dir   Directory enclosing a spec's package.py file.
  -P, --packages      Top-level packages directory for Spack.
  -s, --stage-dir     Stage directory for a spec.
  -S, --stages        Top level Stage directory.
  -b, --build-dir     Checked out or expanded source directory for a spec
                      (requires it to be staged first).
```

Some of these change directory into package-specific locations (stage directory, install directory, package directory) and others change to core spack locations. For example, `spack cd -m` will take you to the main python source directory of your spack install.

10.30.2 spack env

`spack env` functions much like the standard unix `env` command, but it takes a spec as an argument. You can use it to see the environment variables that will be set when a particular build runs, for example:

```
$ spack env mpileaks@1.1%intel
```

This will display the entire environment that will be set when the `mpileaks@1.1%intel` build runs.

To run commands in a package's build environment, you can simply provide them after the spec argument to `spack env`:

```
$ spack cd mpileaks@1.1%intel
$ spack env mpileaks@1.1%intel ./configure
```

This will cd to the build directory and then run `configure` in the package's build environment.

10.30.3 spack location

`spack location` is the same as `spack cd` but it does not require shell support. It simply prints out the path you ask for, rather than cd'ing to it. In bash, this:

```
$ cd $(spack location -b <spec>)
```

is the same as:

```
$ spack cd -b <spec>
```

`spack location` is intended for use in scripts or makefiles that need to know where packages are installed. e.g., in a makefile you might write:

```
DWARF_PREFIX = $(spack location -i libdwarf)
CXXFLAGS += -I$DWARF_PREFIX/include
CXXFLAGS += -L$DWARF_PREFIX/lib
```

10.30.4 Build System Configuration Support

Imagine a developer creating a CMake or Autotools-based project in a local directory, which depends on libraries A-Z. Once Spack has installed those dependencies, one would like to run `cmake` with appropriate command line and environment so CMake can find them. The `spack setup` command does this conveniently, producing a CMake configuration that is essentially the same as how Spack *would have* configured the project. This can be demonstrated with a usage example:

```
$ cd myproject
$ spack setup myproject@local
$ mkdir build; cd build
$ ../spconfig.py ..
$ make
$ make install
```

Notes:

- Spack must have `myproject/package.py` in its repository for this to work.
- `spack setup` produces the executable script `spconfig.py` in the local directory, and also creates the module file for the package. `spconfig.py` is normally run from the user's out-of-source build directory.
- The version number given to `spack setup` is arbitrary, just like `spack diy`. `myproject/package.py` does not need to have any valid downloadable versions listed (typical when a project is new).
- spconfig.py produces a CMake configuration that *does not* use the Spack wrappers. Any resulting binaries *will not* use RPATH, unless the user has enabled it. This is recommended for development purposes, not production.
- `spconfig.py` is human readable, and can serve as a developer reference of what dependencies are being used.
- `make install` installs the package into the Spack repository, where it may be used by other Spack packages.
- CMake-generated makefiles re-run CMake in some circumstances. Use of `spconfig.py` breaks this behavior, requiring the developer to manually re-run `spconfig.py` when a `CMakeLists.txt` file has changed.

10.30.5 CMakePackage

In order to enable `spack setup` functionality, the author of `myproject/package.py` must subclass from `CMakePackage` instead of the standard `Package` superclass. Because CMake is standardized, the packager does not need to tell Spack how to run `cmake; make; make install`. Instead the packager only needs to create (optional) methods `configure_args()` and `configure_env()`, which provide the arguments (as a list) and extra environment variables (as a dict) to provide to the `cmake` command. Usually, these will translate variant flags into CMake definitions. For example:

```
def configure_args(self):
    spec = self.spec
    return [
        '-DUSE_EVERYTRACE=%s' % ('YES' if '+everytrace' in spec else 'NO'),
        '-DBUILD_PYTHON=%s' % ('YES' if '+python' in spec else 'NO'),
        '-DBUILD_GRIDGEN=%s' % ('YES' if '+gridgen' in spec else 'NO'),
        '-DBUILD_COUPLER=%s' % ('YES' if '+coupler' in spec else 'NO'),
        '-DUSE_PISM=%s' % ('YES' if '+pism' in spec else 'NO')
    ]
```

If needed, a packager may also override methods defined in `StagedPackage` (see below).

10.30.6 StagedPackage

`CMakePackage` is implemented by subclassing the `StagedPackage` superclass, which breaks down the standard `Package.install()` method into several sub-stages: `setup`, `configure`, `build` and `install`. Details:

- Instead of implementing the standard `install()` method, package authors implement the methods for the sub-stages `install_setup()`, `install_configure()`, `install_build()`, and `install_install()`.
- The `spack install` command runs the sub-stages `configure`, `build` and `install` in order. (The `setup` stage is not run by default; see below).
- The `spack setup` command runs the sub-stages `setup` and a dummy install (to create the module file).
- The sub-stage install methods take no arguments (other than `self`). The arguments `spec` and `prefix` to the standard `install()` method may be accessed via `self.spec` and `self.prefix`.

10.30.7 GNU Autotools

The `setup` functionality is currently only available for CMake-based packages. Extending this functionality to GNU Autotools-based packages would be easy (and should be done by a developer who actively uses Autotools). Packages that use non-standard build systems can gain `setup` functionality by subclassing `StagedPackage` directly.

CHAPTER 11

Developer Guide

This guide is intended for people who want to work on Spack itself. If you just want to develop packages, see the *Packaging Guide*.

It is assumed that you've read the *Basic usage* and *Packaging Guide* sections, and that you're familiar with the concepts discussed there. If you're not, we recommend reading those first.

11.1 Overview

Spack is designed with three separate roles in mind:

1. **Users**, who need to install software *without* knowing all the details about how it is built.
2. **Packagers** who know how a particular software package is built and encode this information in package files.
3. **Developers** who work on Spack, add new features, and try to make the jobs of packagers and users easier.

Users could be end users installing software in their home directory, or administrators installing software to a shared directory on a shared machine. Packagers could be administrators who want to automate software builds, or application developers who want to make their software more accessible to users.

As you might expect, there are many types of users with different levels of sophistication, and Spack is designed to accommodate both simple and complex use cases for packages. A user who only knows that he needs a certain package should be able to type something simple, like `spack install <package name>`, and get the package that he wants. If a user wants to ask for a specific version, use particular compilers, or build several versions with different configurations, then that should be possible with a minimal amount of additional specification.

This gets us to the two key concepts in Spack's software design:

1. **Specs**: expressions for describing builds of software, and
2. **Packages**: Python modules that build software according to a spec.

A package is a template for building particular software, and a spec as a descriptor for one or more instances of that template. Users express the configuration they want using a spec, and a package turns the spec into a complete build.

The obvious difficulty with this design is that users under-specify what they want. To build a software package, the package object needs a *complete* specification. In Spack, if a spec describes only one instance of a package, then we say it is **concrete**. If a spec could describes many instances, (i.e. it is under-specified in one way or another), then we say it is **abstract**.

Spack's job is to take an *abstract* spec from the user, find a *concrete* spec that satisfies the constraints, and hand the task of building the software off to the package object. The rest of this document describes all the pieces that come together to make that happen.

11.2 Directory Structure

So that you can familiarize yourself with the project, we'll start with a high level view of Spack's directory structure:

```
spack/                  <- installation root
   bin/
      spack             <- main spack executable

   etc/
      spack/            <- Spack config files.
                           Can be overridden by files in ~/.spack.

   var/
      spack/            <- build & stage directories
          repos/             <- contains package repositories
             builtin/        <- pkg repository that comes with Spack
                repo.yaml    <- descriptor for the builtin repository
                packages/    <- directories under here contain packages
          cache/        <- saves resources downloaded during installs

   opt/
      spack/            <- packages are installed here

   lib/
      spack/
         docs/          <- source for this documentation
         env/           <- compiler wrappers for build environment

         external/      <- external libs included in Spack distro
         llnl/          <- some general-use libraries

         spack/         <- spack module; contains Python code
            cmd/        <- each file in here is a spack subcommand
            compilers/  <- compiler description files
            test/       <- unit test modules
            util/       <- common code
```

Spack is designed so that it could live within a standard UNIX directory hierarchy, so `lib`, `var`, and `opt` all contain a `spack` subdirectory in case Spack is installed alongside other software. Most of the interesting parts of Spack live in `lib/spack`.

Spack has *one* directory layout and there is no install process. Most Python programs don't look like this (they use distutils, `setup.py`, etc.) but we wanted to make Spack *very* easy to use. The simple layout spares users from the need to install Spack into a Python environment. Many users don't have write access to a Python installation, and installing an entire new instance of Python to bootstrap Spack would be very complicated. Users should not have to install install a big, complicated package to use the thing that's supposed to spare them from the details of big, complicated packages. The end result is that Spack works out of the box: clone it and add `bin` to your PATH and you're ready to go.

11.3 Code Structure

This section gives an overview of the various Python modules in Spack, grouped by functionality.

11.3.1 Package-related modules

spack.package Contains the *Package* class, which is the superclass for all packages in Spack. Methods on Package implement all phases of the *package lifecycle* and manage the build process.

spack.packages Contains all of the packages in Spack and methods for managing them. Functions like packages.get and class_name_for_package_name handle mapping package module names to class names and dynamically instantiating packages by name from module files.

spack.relations *Relations* are relationships between packages, like depends_on and provides. See *Dependencies* and *Virtual dependencies*.

spack.multimethod Implementation of the *@when* decorator, which allows *multimethods* in packages.

11.3.2 Spec-related modules

spack.spec Contains *Spec* and SpecParser. Also implements most of the logic for normalization and concretization of specs.

spack.parse Contains some base classes for implementing simple recursive descent parsers: *Parser* and *Lexer*. Used by SpecParser.

spack.concretize Contains *DefaultConcretizer* implementation, which allows site administrators to change Spack's *Concretization Policies*.

spack.version Implements a simple *Version* class with simple comparison semantics. Also implements *VersionRange* and *VersionList*. All three are comparable with each other and offer union and intersection operations. Spack uses these classes to compare versions and to manage version constraints on specs. Comparison semantics are similar to the LooseVersion class in distutils and to the way RPM compares version strings.

spack.compilers Submodules contains descriptors for all valid compilers in Spack. This is used by the build system to set up the build environment.

> **Warning:** Not yet implemented. Currently has two compiler descriptions, but compilers aren't fully integrated with the build process yet.

spack.architecture architecture.sys_type is used to determine the host architecture while building.

> **Warning:** Not yet implemented. Should eventually have architecture descriptions for cross-compiling.

11.3.3 Build environment

spack.stage Handles creating temporary directories for builds.

spack.compilation This contains utility functions used by the compiler wrapper script, cc.

spack.directory_layout Classes that control the way an installation directory is laid out. Create more implementations of this to change the hierarchy and naming scheme in $spack_prefix/opt

11.3.4 Spack Subcommands

spack.cmd Each module in this package implements a Spack subcommand. See *writing commands* for details.

11.3.5 Unit tests

spack.test Implements Spack's test suite. Add a module and put its name in the test suite in `__init__.py` to add more unit tests.

spack.test.mock_packages This is a fake package hierarchy used to mock up packages for Spack's test suite.

11.3.6 Other Modules

spack.globals Includes global settings for Spack. the default policy classes for things like *temporary space* and *concretization*.

spack.tty Basic output functions for all of the messages Spack writes to the terminal.

spack.color Implements a color formatting syntax used by `spack.tty`.

spack.url URL parsing, for deducing names and versions of packages from tarball URLs.

spack.util In this package are a number of utility modules for the rest of Spack.

spack.error *SpackError*, the base class for Spack's exception hierarchy.

11.4 Spec objects

11.5 Package objects

Most spack commands look something like this:

1. Parse an abstract spec (or specs) from the command line,
2. *Normalize* the spec based on information in package files,
3. *Concretize* the spec according to some customizable policies,
4. Instantiate a package based on the spec, and
5. Call methods (e.g., `install()`) on the package object.

The information in Package files is used at all stages in this process.

Conceptually, packages are overloaded. They contain:

11.6 Stage objects

11.7 Writing commands

11.8 Unit tests

11.9 Unit testing

11.10 Developer commands

11.10.1 `spack doc`

11.10.2 `spack test`

11.11 Profiling

Spack has some limited built-in support for profiling, and can report statistics using standard Python timing tools. To use this feature, supply `-p` to Spack on the command line, before any subcommands.

11.11.1 `spack --profile`

`spack --profile` output looks like this:

```
$ spack --profile graph dyninst
o  dyninst
|\
| |\
| | o  boost
| | |\
| | o |  zlib
| | |  /
| o |  libdwarf
|/ /
o |  libelf
 /
o  bzip2
         5687153 function calls (5419955 primitive calls) in 7.813 seconds

   Ordered by: internal time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
88718/66872    0.499    0.000    1.636    0.000 spec.py:839(traverse_with_deptype)
    27431    0.411    0.000    0.411    0.000 {posix.stat}
    28812    0.394    0.000    0.908    0.000 spec.py:2532(spec)
870338/870337    0.319    0.000    0.321    0.000 {isinstance}
  418/416    0.300    0.001    0.856    0.002 inspect.py:472(getmodule)
    28810    0.290    0.000    4.066    0.000 spec.py:543(__init__)
149235/38886    0.254    0.000    0.518    0.000 spec.py:220(canonical_deptype)
    28812    0.247    0.000    2.714    0.000 spec.py:2454(do_parse)
...
```

The bottom of the output shows the top most time consuming functions, slowest on top. The profiling support is from Python's built-in tool, cProfile.

CHAPTER 12

spack package

12.1 Subpackages

12.1.1 spack.cmd package

Subpackages

spack.cmd.common package

Submodules

spack.cmd.common.arguments module

spack.cmd.common.arguments.**add_common_arguments**(*parser*, *list_of_arguments*)

Module contents

Submodules

spack.cmd.activate module

spack.cmd.activate.**activate**(*parser*, *args*)

spack.cmd.activate.**setup_parser**(*subparser*)

spack.cmd.arch module

spack.cmd.arch.**arch**(*parser*, *args*)

spack.cmd.bootstrap module

spack.cmd.bootstrap.**bootstrap**(*parser*, *args*)

spack.cmd.bootstrap.**get_origin_info**(*remote*)

spack.cmd.bootstrap.**setup_parser**(*subparser*)

spack.cmd.cd module

spack.cmd.cd.**cd**(*parser*, *args*)

spack.cmd.cd.**setup_parser**(*subparser*)
This is for decoration – spack cd is used through spack's shell support. This allows spack cd to print a descriptive help message when called with -h.

spack.cmd.checksum module

spack.cmd.checksum.**checksum**(*parser*, *args*)

spack.cmd.checksum.**get_checksums**(*versions*, *urls*, ***kwargs*)

spack.cmd.checksum.**setup_parser**(*subparser*)

spack.cmd.clean module

spack.cmd.clean.**clean**(*parser*, *args*)

spack.cmd.clean.**setup_parser**(*subparser*)

spack.cmd.compiler module

spack.cmd.compiler.**compiler**(*parser*, *args*)

spack.cmd.compiler.**compiler_find**(*args*)
Search either $PATH or a list of paths OR MODULES for compilers and add them to Spack's configuration.

spack.cmd.compiler.**compiler_info**(*args*)
Print info about all compilers matching a spec.

spack.cmd.compiler.**compiler_list**(*args*)

spack.cmd.compiler.**compiler_remove**(*args*)

spack.cmd.compiler.**setup_parser**(*subparser*)

spack.cmd.compilers module

spack.cmd.compilers.**compilers**(*parser*, *args*)

spack.cmd.compilers.**setup_parser**(*subparser*)

spack.cmd.config module

spack.cmd.config.**config**(*parser*, *args*)

spack.cmd.config.**config_edit**(*args*)

spack.cmd.config.**config_get**(*args*)

spack.cmd.config.**setup_parser**(*subparser*)

spack.cmd.create module

class spack.cmd.create.**BuildSystemGuesser**
Bases: object

spack.cmd.create.**create**(*parser*, *args*)

spack.cmd.create.**fetch_tarballs**(*url*, *name*, *version*)
Try to find versions of the supplied archive by scraping the web. Prompts the user to select how many to download if many are found.

spack.cmd.create.**find_repository**(*spec*, *args*)

spack.cmd.create.**guess_name_and_version**(*url*, *args*)

spack.cmd.create.**make_version_calls**(*ver_hash_tuples*)
Adds a version() call to the package for each version found.

spack.cmd.create.**setup_parser**(*subparser*)

spack.cmd.deactivate module

spack.cmd.deactivate.**deactivate**(*parser*, *args*)

spack.cmd.deactivate.**setup_parser**(*subparser*)

spack.cmd.debug module

spack.cmd.debug.**create_db_tarball**(*args*)

spack.cmd.debug.**debug**(*parser*, *args*)

spack.cmd.debug.**setup_parser**(*subparser*)

spack.cmd.dependents module

spack.cmd.dependents.**dependents**(*parser*, *args*)

spack.cmd.dependents.**setup_parser**(*subparser*)

spack.cmd.diy module

spack.cmd.diy.**diy**(*self*, *args*)

spack.cmd.diy.**setup_parser**(*subparser*)

spack.cmd.doc module

spack.cmd.doc.**doc**(*parser*, *args*)

spack.cmd.doc.**setup_parser**(*subparser*)

spack.cmd.edit module

spack.cmd.edit.**edit**(*parser*, *args*)

spack.cmd.edit.**edit_package**(*name*, *repo_path*, *namespace*, *force=False*)

spack.cmd.edit.**setup_parser**(*subparser*)

spack.cmd.env module

spack.cmd.env.**env**(*parser*, *args*)

spack.cmd.env.**setup_parser**(*subparser*)

spack.cmd.extensions module

spack.cmd.extensions.**extensions**(*parser*, *args*)

spack.cmd.extensions.**setup_parser**(*subparser*)

spack.cmd.fetch module

spack.cmd.fetch.**fetch**(*parser*, *args*)

spack.cmd.fetch.**setup_parser**(*subparser*)

spack.cmd.find module

spack.cmd.find.**find**(*parser*, *args*)

spack.cmd.find.**query_arguments**(*args*)

spack.cmd.find.**setup_parser**(*subparser*)

spack.cmd.graph module

spack.cmd.graph.**graph**(*parser*, *args*)

spack.cmd.graph.**setup_parser**(*subparser*)

spack.cmd.help module

spack.cmd.help.**help**(*parser*, *args*)

spack.cmd.help.**setup_parser**(*subparser*)

spack.cmd.info module

spack.cmd.info.**info**(*parser*, *args*)

spack.cmd.info.**padder**(*str_list*, *extra=0*)
Return a function to pad elements of a list.

spack.cmd.info.**print_text_info**(*pkg*)
Print out a plain text description of a package.

spack.cmd.info.**setup_parser**(*subparser*)

spack.cmd.install module

spack.cmd.install.**install**(*parser*, *args*)

spack.cmd.install.**setup_parser**(*subparser*)

spack.cmd.list module

spack.cmd.list.**list**(*parser*, *args*)

spack.cmd.list.**setup_parser**(*subparser*)

spack.cmd.load module

spack.cmd.load.**load**(*parser*, *args*)

spack.cmd.load.**setup_parser**(*subparser*)
Parser is only constructed so that this prints a nice help message with -h.

spack.cmd.location module

spack.cmd.location.**location**(*parser*, *args*)

spack.cmd.location.**setup_parser**(*subparser*)

spack.cmd.md5 module

spack.cmd.md5.**compute_md5_checksum**(*url*)

spack.cmd.md5.**md5**(*parser*, *args*)

spack.cmd.md5.**setup_parser**(*subparser*)

spack.cmd.mirror module

spack.cmd.mirror.**mirror**(*parser*, *args*)

spack.cmd.mirror.**mirror_add**(*args*)
Add a mirror to Spack.

spack.cmd.mirror.**mirror_create**(*args*)
Create a directory to be used as a spack mirror, and fill it with package archives.

spack.cmd.mirror.**mirror_list**(*args*)
Print out available mirrors to the console.

spack.cmd.mirror.**mirror_remove**(*args*)
Remove a mirror by name.

spack.cmd.mirror.**setup_parser**(*subparser*)

spack.cmd.module module

exception spack.cmd.module.**MultipleMatches**
Bases: exceptions.Exception

exception spack.cmd.module.**NoMatch**
Bases: exceptions.Exception

spack.cmd.module.**find**(*mtype*, *specs*, *args*)
Look at all installed packages and see if the spec provided matches any. If it does, check whether there is a module file of type <mtype> there, and print out the name that the user should type to use that package's module.

spack.cmd.module.**loads**(*mtype*, *specs*, *args*)
Prompt the list of modules associated with a list of specs

spack.cmd.module.**module**(*parser*, *args*)

spack.cmd.module.**refresh**(*mtype*, *specs*, *args*)
Regenerate module files for item in specs

spack.cmd.module.**rm**(*mtype*, *specs*, *args*)
Deletes module files associated with items in specs

spack.cmd.module.**setup_parser**(*subparser*)

spack.cmd.module.**subcommand**(*subparser_name*)
Registers a function in the callbacks dictionary

spack.cmd.package_list module

spack.cmd.package_list.**github_url**(*pkg*)
Link to a package file on github.

spack.cmd.package_list.**package_list**(*parser*, *args*)

spack.cmd.package_list.**print_rst_package_list**()
Print out information on all packages in restructured text.

spack.cmd.package_list.**rst_table**(*elts*)
Print out a RST-style table.

spack.cmd.patch module

spack.cmd.patch.**patch**(*parser*, *args*)

spack.cmd.patch.**setup_parser**(*subparser*)

spack.cmd.pkg module

spack.cmd.pkg.**diff_packages**(*rev1*, *rev2*)

spack.cmd.pkg.**get_git**()

spack.cmd.pkg.**list_packages**(*rev*)

spack.cmd.pkg.**pkg**(*parser*, *args*)

spack.cmd.pkg.**pkg_add**(*args*)

spack.cmd.pkg.**pkg_added**(*args*)
Show packages added since a commit.

spack.cmd.pkg.**pkg_diff**(*args*)
Compare packages available in two different git revisions.

spack.cmd.pkg.**pkg_list**(*args*)
List packages associated with a particular spack git revision.

spack.cmd.pkg.**pkg_removed**(*args*)
Show packages removed since a commit.

spack.cmd.pkg.**setup_parser**(*subparser*)

spack.cmd.providers module

spack.cmd.providers.**providers**(*parser*, *args*)

spack.cmd.providers.**setup_parser**(*subparser*)

spack.cmd.purge module

spack.cmd.purge.**purge**(*parser*, *args*)

spack.cmd.purge.**setup_parser**(*subparser*)

spack.cmd.python module

spack.cmd.python.**python**(*parser*, *args*)

spack.cmd.python.**setup_parser**(*subparser*)

spack.cmd.reindex module

spack.cmd.reindex.**reindex**(*parser*, *args*)

spack.cmd.repo module

spack.cmd.repo.**repo**(*parser*, *args*)

spack.cmd.repo.**repo_add**(*args*)
Add a package source to Spack's configuration.

spack.cmd.repo.**repo_create**(*args*)
Create a new package repository.

spack.cmd.repo.**repo_list**(*args*)
Show registered repositories and their namespaces.

spack.cmd.repo.**repo_remove**(*args*)
Remove a repository from Spack's configuration.

spack.cmd.repo.**setup_parser**(*subparser*)

spack.cmd.restage module

spack.cmd.restage.**restage**(*parser*, *args*)

spack.cmd.restage.**setup_parser**(*subparser*)

spack.cmd.setup module

spack.cmd.setup.**setup**(*self*, *args*)

spack.cmd.setup.**setup_parser**(*subparser*)

spack.cmd.spec module

spack.cmd.spec.**setup_parser**(*subparser*)

spack.cmd.spec.**spec**(*parser*, *args*)

spack.cmd.stage module

spack.cmd.stage.**setup_parser**(*subparser*)

spack.cmd.stage.**stage**(*parser*, *args*)

spack.cmd.test module

class spack.cmd.test.**MockCache**
Bases: object

fetcher(*targetPath*, *digest*, ***kwargs*)

store(*copyCmd*, *relativeDst*)

class spack.cmd.test.**MockCacheFetcher**
Bases: object

fetch()

set_stage(*stage*)

spack.cmd.test.**setup_parser**(*subparser*)

spack.cmd.test.**test**(*parser*, *args*)

spack.cmd.test_install module

class spack.cmd.test_install.**TestCase**(*classname*, *name*, *time=None*)
Bases: object

results = {0: None, 1: 'failure', 2: 'skipped', 3: 'error'}

set_result(*result_type*, *message=None*, *error_type=None*, *text=None*)

class spack.cmd.test_install.**TestResult**
Bases: object

ERRORED = 3

FAILED = 1

PASSED = 0

SKIPPED = 2

class spack.cmd.test_install.**TestSuite**(*filename*)
Bases: object

append(*item*)

spack.cmd.test_install.**failed_dependencies**(*spec*)

spack.cmd.test_install.**fetch_log**(*path*)

spack.cmd.test_install.**get_filename**(*args*, *top_spec*)

spack.cmd.test_install.**get_top_spec_or_die**(*args*)

spack.cmd.test_install.**install_single_spec**(*spec*, *number_of_jobs*)

spack.cmd.test_install.**setup_parser**(*subparser*)

spack.cmd.test_install.**test_install**(*parser*, *args*)

spack.cmd.uninstall module

spack.cmd.uninstall.**concretize_specs**(*specs*, *allow_multiple_matches=False*, *force=False*)
Returns a list of specs matching the non necessarily concretized specs given from cli

Args: specs: list of specs to be matched against installed packages allow_multiple_matches : if True multiple matches are admitted

Return: list of specs

spack.cmd.uninstall.**do_uninstall**(*specs*, *force*)
Uninstalls all the specs in a list.

Args: specs: list of specs to be uninstalled force: force uninstallation (boolean)

spack.cmd.uninstall.**installed_dependents**(*specs*)
Returns a dictionary that maps a spec with a list of its installed dependents

Args: specs: list of specs to be checked for dependents

Returns: dictionary of installed dependents

spack.cmd.uninstall.**setup_parser**(*subparser*)

spack.cmd.uninstall.**uninstall**(*parser*, *args*)

spack.cmd.unload module

spack.cmd.unload.**setup_parser**(*subparser*)
Parser is only constructed so that this prints a nice help message with -h.

spack.cmd.unload.**unload**(*parser*, *args*)

spack.cmd.unuse module

spack.cmd.unuse.**setup_parser**(*subparser*)
Parser is only constructed so that this prints a nice help message with -h.

spack.cmd.unuse.**unuse**(*parser*, *args*)

spack.cmd.url_parse module

spack.cmd.url_parse.**print_name_and_version**(*url*)

spack.cmd.url_parse.**setup_parser**(*subparser*)

spack.cmd.url_parse.**url_parse**(*parser*, *args*)

spack.cmd.urls module

spack.cmd.urls.**setup_parser**(*subparser*)

spack.cmd.urls.**urls**(*parser*, *args*)

spack.cmd.use module

spack.cmd.use.**setup_parser**(*subparser*)
Parser is only constructed so that this prints a nice help message with -h.

spack.cmd.use.**use**(*parser*, *args*)

spack.cmd.versions module

spack.cmd.versions.**setup_parser**(*subparser*)

spack.cmd.versions.**versions**(*parser*, *args*)

spack.cmd.view module

Produce a "view" of a Spack DAG.

A "view" is file hierarchy representing the union of a number of Spack-installed package file hierarchies. The union is formed from:

- specs resolved from the package names given by the user (the seeds)
- all depenencies of the seeds unless user specifies *–no-depenencies*
- less any specs with names matching the regular expressions given by *–exclude*

The *view* can be built and tore down via a number of methods (the "actions"):

- symlink :: a file system view which is a directory hierarchy that is the union of the hierarchies of the installed packages in the DAG where installed files are referenced via symlinks.
- hardlink :: like the symlink view but hardlinks are used.
- statlink :: a view producing a status report of a symlink or hardlink view.

The file system view concept is imspired by Nix, implemented by brett.viren@gmail.com ca 2016.

spack.cmd.view.**assuredir**(*path*)
Assure path exists as a directory

spack.cmd.view.**check_one**(*spec*, *path*, *verbose=False*)
Check status of view in path against spec

spack.cmd.view.**filter_exclude**(*specs*, *exclude*)
Filter specs given sequence of exclude regex

spack.cmd.view.**flatten**(*seeds*, *descend=True*)
Normalize and flattend seed specs and descend hiearchy

spack.cmd.view.**link_one**(*spec*, *path*, *link=<built-in function symlink>*, *verbose=False*)
Link all files in *spec* into directory *path*.

spack.cmd.view.**purge_empty_directories**(*path*)
Ascend up from the leaves accessible from *path* and remove empty directories.

spack.cmd.view.**relative_to**(*prefix*, *path*)
Return end of *path* relative to *prefix*

spack.cmd.view.**remove_one**(*spec*, *path*, *verbose=False*)
Remove any files found in *spec* from *path* and purge empty directories.

spack.cmd.view.**setup_parser**(*sp*)

spack.cmd.view.**transform_path**(*spec*, *path*, *prefix=None*)
Return the a relative path corresponding to given path spec.prefix

spack.cmd.view.**view**(*parser*, *args*)
Produce a view of a set of packages.

spack.cmd.view.**visitor_add**(*specs*, *args*)
Symlink all files found in specs

spack.cmd.view.**visitor_check**(*specs*, *args*)
Give status of view in args.path relative to specs

spack.cmd.view.**visitor_hard**(*specs*, *args*)
Hardlink all files found in specs

spack.cmd.view.**visitor_hardlink**(*specs*, *args*)
Hardlink all files found in specs

spack.cmd.view.**visitor_remove**(*specs*, *args*)
Remove all files and directories found in specs from args.path

spack.cmd.view.**visitor_rm**(*specs*, *args*)
Remove all files and directories found in specs from args.path

spack.cmd.view.**visitor_soft**(*specs*, *args*)
Symlink all files found in specs

spack.cmd.view.**visitor_statlink**(*specs*, *args*)
Give status of view in args.path relative to specs

spack.cmd.view.**visitor_status**(*specs*, *args*)
Give status of view in args.path relative to specs

spack.cmd.view.**visitor_symlink**(*specs*, *args*)
Symlink all files found in specs

Module contents

spack.cmd.**ask_for_confirmation**(*message*)

spack.cmd.**disambiguate_spec**(*spec*)

spack.cmd.**display_specs**(*specs*, ***kwargs*)

spack.cmd.**elide_list**(*line_list*, *max_num=10*)

Takes a long list and limits it to a smaller number of elements, replacing intervening elements with '...'. For example:

```
elide_list([1,2,3,4,5,6], 4)
```

gives:

```
[1, 2, 3, '...', 6]
```

spack.cmd.**get_cmd_function_name**(*name*)

spack.cmd.**get_command**(*name*)

Imports the command's function from a module and returns it.

spack.cmd.**get_module**(*name*)

Imports the module for a particular command name and returns it.

spack.cmd.**gray_hash**(*spec*, *length*)

spack.cmd.**parse_specs**(*args*, ***kwargs*)

Convenience function for parsing arguments from specs. Handles common exceptions and dies if there are errors.

12.1.2 spack.compilers package

Submodules

spack.compilers.cce module

class spack.compilers.cce.**Cce**(*cspec*, *operating_system*, *paths*, *modules=[]*, *alias=None*, ***kwargs*)

Bases: *spack.compiler.Compiler*

Cray compiler environment compiler.

PrgEnv = 'PrgEnv-cray'

PrgEnv_compiler = 'cce'

cc_names = ['cc']

cxx_names = ['CC']

classmethod default_version(*comp*)

f77_names = ['ftn']

fc_names = ['ftn']

link_paths = {'cc': 'cc', 'cxx': 'c++', 'f77': 'f77', 'fc': 'fc'}

suffixes = ['-mp-\\d\\.\\d']

spack.compilers.clang module

class spack.compilers.clang.**Clang**(*cspec*, *operating_system*, *paths*, *modules=[]*, *alias=None*, ***kwargs*)

Bases: *spack.compiler.Compiler*

cc_names = ['clang']

cxx11_flag

cxx_names = ['clang++']

classmethod default_version(*comp*)

The '–version' option works for clang compilers. On most platforms, output looks like this:

```
clang version 3.1 (trunk 149096)
Target: x86_64-unknown-linux-gnu
Thread model: posix
```

On Mac OS X, it looks like this:

```
Apple LLVM version 7.0.2 (clang-700.1.81)
Target: x86_64-apple-darwin15.2.0
Thread model: posix
```

f77_names = []

fc_names = []

is_apple

link_paths = {'cc': 'clang/clang', 'cxx': 'clang/clang++', 'f77': 'f77', 'fc': 'f90'}

openmp_flag

spack.compilers.gcc module

class spack.compilers.gcc.**Gcc**(*cspec*, *operating_system*, *paths*, *modules=[]*, *alias=None*, ***kwargs*)

Bases: *spack.compiler.Compiler*

PrgEnv = 'PrgEnv-gnu'

PrgEnv_compiler = 'gcc'

cc_names = ['gcc']

cxx11_flag

cxx14_flag

cxx_names = ['g++']

f77_names = ['gfortran']

classmethod f77_version(*f77*)

fc_names = ['gfortran']

classmethod fc_version(*fc*)

link_paths = {'cc': 'gcc/gcc', 'cxx': 'gcc/g++', 'f77': 'gcc/gfortran', 'fc': 'gcc/gfortran'}

openmp_flag

stdcxx_libs

suffixes = ['-mp-\\d\\.\\d', '-\\d\\.\\d', '-\\d']

spack.compilers.intel module

class spack.compilers.intel.**Intel**(*cspec*, *operating_system*, *paths*, *modules=[]*, *alias=None*, ***kwargs*)

Bases: *spack.compiler.Compiler*

PrgEnv = 'PrgEnv-intel'

PrgEnv_compiler = 'intel'

cc_names = ['icc']

cxx11_flag

cxx_names = ['icpc']

classmethod default_version(*comp*)

The '–version' option seems to be the most consistent one for intel compilers. Output looks like this:

```
icpc (ICC) 12.1.5 20120612
Copyright (C) 1985-2012 Intel Corporation.  All rights reserved.
```

or:

```
ifort (IFORT) 12.1.5 20120612
Copyright (C) 1985-2012 Intel Corporation.  All rights reserved.
```

f77_names = ['ifort']

fc_names = ['ifort']

link_paths = {'cc': 'intel/icc', 'cxx': 'intel/icpc', 'f77': 'intel/ifort', 'fc': 'intel/ifort'}

openmp_flag

stdcxx_libs

spack.compilers.nag module

class spack.compilers.nag.**Nag**(*cspec*, *operating_system*, *paths*, *modules=[]*, *alias=None*, ***kwargs*)

Bases: *spack.compiler.Compiler*

cc_names = []

cxx11_flag

cxx_names = []

classmethod default_version(*comp*)

The '-V' option works for nag compilers. Output looks like this:

```
NAG Fortran Compiler Release 6.0(Hibiya) Build 1037
Product NPL6A60NA for x86-64 Linux
```

f77_names = ['nagfor']

f77_rpath_arg

fc_names = ['nagfor']

fc_rpath_arg

link_paths = {'cc': 'cc', 'cxx': 'c++', 'f77': 'nag/nagfor', 'fc': 'nag/nagfor'}

openmp_flag

spack.compilers.pgi module

class spack.compilers.pgi.**Pgi**(*cspec*, *operating_system*, *paths*, *modules=[]*, *alias=None*, ***kwargs*)
Bases: *spack.compiler.Compiler*

PrgEnv = 'PrgEnv-pgi'

PrgEnv_compiler = 'pgi'

cc_names = ['pgcc']

cxx11_flag

cxx_names = ['pgc++', 'pgCC']

classmethod default_version(*comp*)
The '-V' option works for all the PGI compilers. Output looks like this:

```
pgcc 15.10-0 64-bit target on x86-64 Linux -tp sandybridge
The Portland Group - PGI Compilers and Tools
Copyright (c) 2015, NVIDIA CORPORATION.  All rights reserved.
```

f77_names = ['pgfortran', 'pgf77']

fc_names = ['pgfortran', 'pgf95', 'pgf90']

link_paths = {'cc': 'pgi/pgcc', 'cxx': 'pgi/pgc++', 'f77': 'pgi/pgfortran', 'fc': 'pgi/pgfortran'}

openmp_flag

spack.compilers.xl module

class spack.compilers.xl.**Xl**(*cspec*, *operating_system*, *paths*, *modules=[]*, *alias=None*, ***kwargs*)
Bases: *spack.compiler.Compiler*

cc_names = ['xlc', 'xlc_r']

cxx11_flag

cxx_names = ['xlC', 'xlC_r', 'xlc++', 'xlc++_r']

classmethod default_version(*comp*)
The '-qversion' is the standard option fo XL compilers. Output looks like this:

```
IBM XL C/C++ for Linux, V11.1 (5724-X14)
Version: 11.01.0000.0000
```

or:

```
IBM XL Fortran for Linux, V13.1 (5724-X16)
Version: 13.01.0000.0000
```

or:

```
IBM XL C/C++ for AIX, V11.1 (5724-X13)
Version: 11.01.0000.0009
```

or:

```
        IBM XL C/C++ Advanced Edition for Blue Gene/P, V9.0
        Version: 09.00.0000.0017
```

f77_names = ['xlf', 'xlf_r']

classmethod f77_version(*f77*)

fc_names = ['xlf90', 'xlf90_r', 'xlf95', 'xlf95_r', 'xlf2003', 'xlf2003_r', 'xlf2008', 'xlf2008_r']

classmethod fc_version(*fc*)
The fortran and C/C++ versions of the XL compiler are always two units apart. By this we mean that the fortran release that goes with XL C/C++ 11.1 is 13.1. Having such a difference in version number is confusing spack quite a lot. Most notably if you keep the versions as is the default xl compiler will only have fortran and no C/C++. So we associate the Fortran compiler with the version associated to the C/C++ compiler. One last stumble. Version numbers over 10 have at least a .1 those under 10 a .0. There is no xlf 9.x or under currently available. BG/P and BG/L can such a compiler mix and possibly older version of AIX and linux on power.

link_paths = {'cc': 'xl/xlc', 'cxx': 'xl/xlc++', 'f77': 'xl/xlf', 'fc': 'xl/xlf90'}

openmp_flag

Module contents

This module contains functions related to finding compilers on the system and configuring Spack to use multiple compilers.

exception spack.compilers.**CompilerSpecInsufficientlySpecificError**(*compiler_spec*)
Bases: *spack.error.SpackError*

exception spack.compilers.**InvalidCompilerConfigurationError**(*compiler_spec*)
Bases: *spack.error.SpackError*

exception spack.compilers.**NoCompilerForSpecError**(*compiler_spec*, *target*)
Bases: *spack.error.SpackError*

exception spack.compilers.**NoCompilersError**
Bases: *spack.error.SpackError*

spack.compilers.**add_compilers_to_config**(*compilers*, *scope=None*, *init_config=True*)
Add compilers to the config for the specified architecture.

Arguments:

- compilers: a list of Compiler objects.
- scope: configuration scope to modify.

spack.compilers.**all_compiler_types**()

spack.compilers.**all_compilers**(*scope=None*, *init_config=True*)

spack.compilers.**all_compilers_config**(*scope=None*, *init_config=True*)
Return a set of specs for all the compiler versions currently available to build with. These are instances of CompilerSpec.

spack.compilers.**all_os_classes**()
Return the list of classes for all operating systems available on this platform

spack.compilers.**class_for_compiler_name**(*compiler_name*)
Given a compiler module name, get the corresponding Compiler class.

spack.compilers.**compiler_for_spec**(*cspec_like*, **args*, ***kwargs*)

spack.compilers.**compilers_for_spec**(*cspec_like*, **args*, ***kwargs*)

spack.compilers.**default_compiler**()

spack.compilers.**find**(*cspec_like*, **args*, ***kwargs*)

spack.compilers.**find_compilers**(**paths*)
Return a list of compilers found in the suppied paths. This invokes the find_compilers() method for each operating system associated with the host platform, and appends the compilers detected to a list.

spack.compilers.**get_compiler_config**(*scope=None*, *init_config=True*)
Return the compiler configuration for the specified architecture.

spack.compilers.**remove_compiler_from_config**(*cspec_like*, **args*, ***kwargs*)

spack.compilers.**supported**(*cspec_like*, **args*, ***kwargs*)

spack.compilers.**supported_compilers**()
Return a set of names of compilers supported by Spack.

See available_compilers() to get a list of all the available versions of supported compilers.

12.1.3 spack.hooks package

Submodules

spack.hooks.dotkit module

spack.hooks.dotkit.**post_install**(*pkg*)

spack.hooks.dotkit.**post_uninstall**(*pkg*)

spack.hooks.extensions module

spack.hooks.extensions.**pre_uninstall**(*pkg*)

spack.hooks.licensing module

spack.hooks.licensing.**post_install**(*pkg*)
This hook symlinks local licenses to the global license for licensed software.

spack.hooks.licensing.**pre_install**(*pkg*)
This hook handles global license setup for licensed software.

spack.hooks.licensing.**set_up_license**(*pkg*)
Prompt the user, letting them know that a license is required.

For packages that rely on license files, a global license file is created and opened for editing.

For packages that rely on environment variables to point to a license, a warning message is printed.

For all other packages, documentation on how to set up a license is printed.

spack.hooks.licensing.**symlink_license**(*pkg*)
Create local symlinks that point to the global license file.

spack.hooks.licensing.**write_license_file**(*pkg*, *license_path*)
Writes empty license file.

Comments give suggestions on alternative methods of installing a license.

spack.hooks.lmodmodule module

spack.hooks.lmodmodule.**post_install**(*pkg*)

spack.hooks.lmodmodule.**post_uninstall**(*pkg*)

spack.hooks.sbang module

spack.hooks.sbang.**filter_shebang**(*path*)
Adds a second shebang line, using sbang, at the beginning of a file.

spack.hooks.sbang.**filter_shebangs_in_directory**(*directory*, *filenames=None*)

spack.hooks.sbang.**post_install**(*pkg*)
This hook edits scripts so that they call /bin/bash $spack_prefix/bin/sbang instead of something longer than the shebang limit.

spack.hooks.sbang.**shebang_too_long**(*path*)
Detects whether a file has a shebang line that is too long.

spack.hooks.tclmodule module

spack.hooks.tclmodule.**post_install**(*pkg*)

spack.hooks.tclmodule.**post_uninstall**(*pkg*)

Module contents

This package contains modules with hooks for various stages in the Spack install process. You can add modules here and they'll be executed by package at various times during the package lifecycle.

Each hook is just a function that takes a package as a parameter. Hooks are not executed in any particular order.

Currently the following hooks are supported:

- pre_install()
- post_install()
- pre_uninstall()
- post_uninstall()

This can be used to implement support for things like module systems (e.g. modules, dotkit, etc.) or to add other custom features.

class spack.hooks.**HookRunner**(*hook_name*)
Bases: object

12.1.4 spack.operating_systems package

Submodules

spack.operating_systems.cnl module

class spack.operating_systems.cnl.**Cnl**
Bases: *spack.architecture.OperatingSystem*

Compute Node Linux (CNL) is the operating system used for the Cray XC series super computers. It is a very stripped down version of GNU/Linux. Any compilers found through this operating system will be used with modules. If updated, user must make sure that version and name are updated to indicate that OS has been upgraded (or downgraded)

find_compiler(*cmp_cls*, **paths*)

find_compilers(**paths*)

spack.operating_systems.linux_distro module

class spack.operating_systems.linux_distro.**LinuxDistro**
Bases: *spack.architecture.OperatingSystem*

This class will represent the autodetected operating system for a Linux System. Since there are many different flavors of Linux, this class will attempt to encompass them all through autodetection using the python module platform and the method platform.dist()

spack.operating_systems.mac_os module

class spack.operating_systems.mac_os.**MacOs**
Bases: *spack.architecture.OperatingSystem*

This class represents the macOS operating system. This will be auto detected using the python platform.mac_ver. The macOS platform will be represented using the major version operating system name, i.e el capitan, yosemite...etc.

Module contents

12.1.5 spack.platforms package

Submodules

spack.platforms.bgq module

class spack.platforms.bgq.**Bgq**
Bases: *spack.architecture.Platform*

back_end = 'powerpc'

default = 'powerpc'

classmethod detect()

front_end = 'power7'

priority = 30

spack.platforms.cray module

class spack.platforms.cray.**Cray**
Bases: *spack.architecture.Platform*

classmethod detect()

priority = 10

classmethod setup_platform_environment(*pkg*, *env*)
Change the linker to default dynamic to be more similar to linux/standard linker behavior

spack.platforms.darwin module

class spack.platforms.darwin.**Darwin**
Bases: *spack.architecture.Platform*

back_end = 'x86_64'

default = 'x86_64'

classmethod detect()

front_end = 'x86_64'

priority = 89

spack.platforms.linux module

class spack.platforms.linux.**Linux**
Bases: *spack.architecture.Platform*

classmethod detect()

priority = 90

spack.platforms.test module

class spack.platforms.test.**Test**
Bases: *spack.architecture.Platform*

back_end = 'x86_64'

back_os = 'CNL10'

default = 'x86_64'

default_os = 'CNL10'

classmethod detect()

front_end = 'x86_32'

priority = 1000000

Module contents

12.1.6 spack.schema package

Submodules

spack.schema.compilers module

Schema for compiler configuration files.

spack.schema.mirrors module

Schema for mirror configuration files.

spack.schema.modules module

Schema for mirror configuration files.

spack.schema.packages module

Schema for packages.yaml configuration files.

spack.schema.repos module

Schema for repository configuration files.

spack.schema.targets module

Schema for target configuration files.

Module contents

This module contains jsonschema files for all of Spack's YAML formats.

12.1.7 spack.test package

Subpackages

spack.test.cmd package

Submodules

spack.test.cmd.find module

class spack.test.cmd.find.**FindTest**(*methodName='runTest'*)

Bases: unittest.case.TestCase

test_query_arguments()

spack.test.cmd.module module

class spack.test.cmd.module.**TestModule**(*methodName='runTest'*)

Bases: *spack.test.mock_database.MockDatabase*

test_module_common_operations()

spack.test.cmd.test_compiler_cmd module

class spack.test.cmd.test_compiler_cmd.**CompilerCmdTest**(*methodName='runTest'*)

Bases: *spack.test.mock_packages_test.MockPackagesTest*

Test compiler commands for add and remove

test_compiler_add()

test_compiler_remove()

class spack.test.cmd.test_compiler_cmd.**MockArgs**(*add_paths=[]*, *scope=None*, *compiler_spec=None*, *all=None*)

Bases: object

spack.test.cmd.test_compiler_cmd.**make_mock_compiler**()

Make a directory containing a fake, but detectable compiler.

spack.test.cmd.test_install module

class spack.test.cmd.test_install.**MockArgs**(*package*)

Bases: object

class spack.test.cmd.test_install.**MockPackage**(*spec*, *buildLogPath*)

Bases: object

do_install(**args*, ***kwargs*)

class spack.test.cmd.test_install.**MockPackageDb**(*init=None*)

Bases: object

get(*spec*)

class spack.test.cmd.test_install.**MockSpec**(*name*, *version*, *hashStr=None*)

Bases: object

dag_hash()

dependencies(*deptype=None*)

dependents(*deptype=None*)

short_spec

traverse(*order=None*)

class spack.test.cmd.test_install.**TestInstallTest**(*methodName='runTest'*)

Bases: unittest.case.TestCase

Tests test-install where X->Y

setUp()

tearDown()

test_dependency_already_installed()

test_installing_both()

spack.test.cmd.test_install.**mock_fetch_log**(*path*)

spack.test.cmd.test_install.**mock_open**(**args*, ***kwds*)

spack.test.cmd.uninstall module

class spack.test.cmd.uninstall.**MockArgs**(*packages*, *all=False*, *force=False*, *dependents=False*)

Bases: object

class spack.test.cmd.uninstall.**TestUninstall**(*methodName='runTest'*)
Bases: *spack.test.mock_database.MockDatabase*

test_uninstall()

Module contents

Submodules

spack.test.architecture module

Test checks if the architecture class is created correctly and also that the functions are looking for the correct architecture name

class spack.test.architecture.**ArchitectureTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

setUp()

tearDown()

test_boolness()

test_dict_functions_for_architecture()

test_platform()

test_user_back_end_input()
Test when user inputs backend that both the backend target and backend operating system match

test_user_defaults()

test_user_front_end_input()
Test when user inputs just frontend that both the frontend target and frontend operating system match

test_user_input_combination()

spack.test.build_system_guess module

class spack.test.build_system_guess.**InstallTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

Tests the build system guesser in spack create

check_archive(*filename*, *system*)

setUp()

tearDown()

test_R()

test_autotools()

test_cmake()

test_python()

test_scons()

test_unknown()

spack.test.cc module

This test checks that the Spack cc compiler wrapper is parsing arguments correctly.

class spack.test.cc.**CompilerTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

check_cc(*command*, *args*, *expected*)

check_cpp(*command*, *args*, *expected*)

check_cxx(*command*, *args*, *expected*)

check_fc(*command*, *args*, *expected*)

check_ld(*command*, *args*, *expected*)

setUp()

tearDown()

test_all_deps()
Ensure includes and RPATHs for all deps are added.

test_as_mode()

test_ccld_mode()

test_cpp_mode()

test_dep_include()
Ensure a single dependency include directory is added.

test_dep_lib()
Ensure a single dependency RPATH is added.

test_dep_rpath()
Ensure RPATHs for root package are added.

test_flags()

test_ld_deps()
Ensure no (extra) -I args or -Wl, are passed in ld mode.

test_ld_deps_reentrant()
Make sure ld -r is handled correctly on OS's where it doesn't support rpaths.

test_ld_mode()

test_vcheck_mode()

spack.test.concretize module

class spack.test.concretize.**ConcretizeTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

check_concretize(*abstract_spec*)

check_spec(*abstract*, *concrete*)

test_compiler_child()

test_compiler_inheritance()

test_concretize_dag()

test_concretize_no_deps()

test_concretize_preferred_version()

test_concretize_two_virtuals()
Test a package with multiple virtual dependencies.

test_concretize_two_virtuals_with_dual_provider()
Test a package with multiple virtual dependencies and force a provider that provides both.

test_concretize_two_virtuals_with_dual_provider_and_a_conflict()
Test a package with multiple virtual dependencies and force a provider that provides both, and another conflicting package that provides one.

test_concretize_two_virtuals_with_one_bound()
Test a package with multiple virtual dependencies and one preset.

test_concretize_two_virtuals_with_two_bound()
Test a package with multiple virtual deps and two of them preset.

test_concretize_variant()

test_concretize_with_provides_when()
Make sure insufficient versions of MPI are not in providers list when we ask for some advanced version.

test_concretize_with_restricted_virtual()

test_concretize_with_virtual()

test_conretize_compiler_flags()

test_external_and_virtual()

test_external_package()

test_external_package_module()

test_find_spec_children()

test_find_spec_none()

test_find_spec_parents()
Tests the spec finding logic used by concretization.

test_find_spec_self()

test_find_spec_sibling()

test_my_dep_depends_on_provider_of_my_virtual_dep()

test_nobuild_package()

test_virtual_is_fully_expanded_for_callpath()

test_virtual_is_fully_expanded_for_mpileaks()

spack.test.concretize_preferences module

class spack.test.concretize_preferences.**ConcretizePreferencesTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

Test concretization preferences are being applied correctly.

assert_variant_values(*spec*, ***variants*)

concretize(*abstract_spec*)

setUp()
Create config section to store concretization preferences

tearDown()

test_preferred_compilers()
Test preferred compilers are applied correctly

test_preferred_providers()
Test preferred providers of virtual packages are applied correctly

test_preferred_variants()
Test preferred variants are applied correctly

test_preferred_versions()
Test preferred package versions are applied correctly

update_packages(*pkgname*, *section*, *value*)
Update config and reread package list

spack.test.config module

class spack.test.config.**ConfigTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

check_config(*comps*, **compiler_names*)
Check that named compilers in comps match Spack's config.

setUp()

tearDown()

test_write_key_in_memory()

test_write_key_to_disk()

test_write_list_in_memory()

test_write_to_same_priority_file()

spack.test.database module

These tests check the database is functioning properly, both in memory and in its file

class spack.test.database.**DatabaseTest**(*methodName='runTest'*)
Bases: *spack.test.mock_database.MockDatabase*

test_005_db_exists()
Make sure db cache file exists after creating.

test_010_all_install_sanity()
Ensure that the install layout reflects what we think it does.

test_015_write_and_read()

test_020_db_sanity()
Make sure query() returns what's actually in the db.

test_025_reindex()
Make sure reindex works and ref counts are valid.

test_030_db_sanity_from_another_process()

test_040_ref_counts()
Ensure that we got ref counts right when we read the DB.

test_050_basic_query()
Ensure querying database is consistent with what is installed.

test_060_remove_and_add_root_package()

test_070_remove_and_add_dependency_package()

test_080_root_ref_counts()

test_090_non_root_ref_counts()

test_100_no_write_with_exception_on_remove()

test_110_no_write_with_exception_on_install()

spack.test.directory_layout module

This test verifies that the Spack directory layout works properly.

class spack.test.directory_layout.**DirectoryLayoutTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

Tests that a directory layout works correctly and produces a consistent install path.

setUp()

tearDown()

test_find()
Test that finding specs within an install layout works.

test_handle_unknown_package()
This test ensures that spack can at least do *some* operations with packages that are installed but that it does not know about. This is actually not such an uncommon scenario with spack; it can happen when you switch from a git branch where you're working on a new package.

This test ensures that the directory layout stores enough information about installed packages' specs to uninstall or query them again if the package goes away.

test_read_and_write_spec()
This goes through each package in spack and creates a directory for it. It then ensures that the spec for the directory's installed package can be read back in consistently, and finally that the directory can be removed by the directory layout.

spack.test.environment module

class spack.test.environment.**EnvironmentTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

setUp()

tearDown()

test_extend()

test_extra_arguments()

test_path_manipulation()

test_set()

test_set_path()

test_source_files()

test_unset()

spack.test.file_cache module

Test Spack's FileCache.

class spack.test.file_cache.**FileCacheTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

Ensure that a file cache can properly write to a file and recover its contents.

setUp()

tearDown()

test_remove()
Test removing an entry from the cache.

test_write_and_read_cache_file()
Test writing then reading a cached file.

test_write_and_write_cache_file()
Test two write transactions on a cached file.

spack.test.git_fetch module

class spack.test.git_fetch.**GitFetchTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

Tests fetching from a dummy git repository.

assert_rev(*rev*)
Check that the current git revision is equal to the supplied rev.

setUp()
Create a git repository with master and two other branches, and one tag, so that we can experiment on it.

tearDown()
Destroy the stage space used by this test.

test_fetch_branch()
Test fetching a branch.

test_fetch_commit()
Test fetching a particular commit.

test_fetch_master()
Test a default git checkout with no commit or tag specified.

test_fetch_tag()
Test fetching a tag.

try_fetch(*rev*, *test_file*, *args*)
Tries to:

1. Fetch the repo using a fetch strategy constructed with supplied args.
2. Check if the test_file is in the checked out repository.

3.Assert that the repository is at the revision supplied.

4.Add and remove some files, then reset the repo, and ensure it's all there again.

spack.test.hg_fetch module

class spack.test.hg_fetch.**HgFetchTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

Tests fetching from a dummy hg repository.

setUp()
Create a hg repository with master and two other branches, and one tag, so that we can experiment on it.

tearDown()
Destroy the stage space used by this test.

test_fetch_default()
Test a default hg checkout with no commit or tag specified.

test_fetch_rev0()
Test fetching a branch.

try_fetch(*rev*, *test_file*, *args*)
Tries to:

1.Fetch the repo using a fetch strategy constructed with supplied args.

2.Check if the test_file is in the checked out repository.

3.Assert that the repository is at the revision supplied.

4.Add and remove some files, then reset the repo, and ensure it's all there again.

spack.test.install module

class spack.test.install.**InstallTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

Tests install and uninstall on a trivial package.

fake_fetchify(*pkg*)
Fake the URL for a package so it downloads from a file.

setUp()

tearDown()

test_install_and_uninstall()

test_install_environment()

spack.test.library_list module

class spack.test.library_list.**LibraryListTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

setUp()

test_add()

test_flags()

test_get_item()

test_joined_and_str()

test_paths_manipulation()

test_repr()

spack.test.link_tree module

class spack.test.link_tree.**LinkTreeTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

Tests Spack's LinkTree class.

check_dir(*filename*)

check_file_link(*filename*)

setUp()

tearDown()

test_ignore()

test_merge_to_existing_directory()

test_merge_to_new_directory()

test_merge_with_empty_directories()

spack.test.lock module

These tests ensure that our lock works correctly.

class spack.test.lock.**LockTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

acquire_read(*barrier*)

acquire_write(*barrier*)

multiproc_test(**functions*)
Order some processes using simple barrier synchronization.

setUp()

tearDown()

test_complex_acquire_and_release_chain()

test_read_lock_timeout_on_write()

test_read_lock_timeout_on_write_2()

test_read_lock_timeout_on_write_3()

test_transaction()

test_transaction_with_context_manager()

test_transaction_with_context_manager_and_exception()

test_transaction_with_exception()

test_write_lock_timeout_on_read()

test_write_lock_timeout_on_read_2()

test_write_lock_timeout_on_read_3()

test_write_lock_timeout_on_write()

test_write_lock_timeout_on_write_2()

test_write_lock_timeout_on_write_3()

test_write_lock_timeout_with_multiple_readers_2_1()

test_write_lock_timeout_with_multiple_readers_2_2()

test_write_lock_timeout_with_multiple_readers_3_1()

test_write_lock_timeout_with_multiple_readers_3_2()

timeout_read(*barrier*)

timeout_write(*barrier*)

spack.test.make_executable module

Tests for Spack's built-in parallel make support.

This just tests whether the right args are getting passed to make.

class spack.test.make_executable.**MakeExecutableTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

setUp()

tearDown()

test_make_explicit()

test_make_normal()

test_make_one_job()

test_make_parallel_disabled()

test_make_parallel_false()

test_make_parallel_precedence()

spack.test.mirror module

class spack.test.mirror.**MirrorTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

check_mirror()

setUp()
Sets up a mock package and a mock repo for each fetch strategy, to ensure that the mirror can create archives for each of them.

set_up_package(*name*, *MockRepoClass*, *url_attr*)
Set up a mock package to be mirrored. Each package needs us to:

1. Set up a mock repo/archive to fetch from.
2. Point the package's version args at that repo.

tearDown()
Destroy all the stages created by the repos in setup.

test_all_mirror()

test_git_mirror()

test_hg_mirror()

test_svn_mirror()

test_url_mirror()

spack.test.mock_database module

class spack.test.mock_database.**MockDatabase**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

setUp()

tearDown()

spack.test.mock_packages_test module

class spack.test.mock_packages_test.**MockPackagesTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

cleanmock()
Restore the real packages path after any test.

initmock()

setUp()

set_pkg_dep(*pkg_name*, *spec*, *deptypes=('build', 'link', 'run')*)
Alters dependence information for a package.

Adds a dependency on <spec> to pkg. Use this to mock up constraints.

tearDown()

spack.test.mock_repo module

class spack.test.mock_repo.**MockArchive**
Bases: *spack.test.mock_repo.MockRepo*

Creates a very simple archive directory with a configure script and a makefile that installs to a prefix. Tars it up into an archive.

class spack.test.mock_repo.**MockGitRepo**
Bases: *spack.test.mock_repo.MockVCSRepo*

rev_hash(*rev*)

class spack.test.mock_repo.**MockHgRepo**
Bases: *spack.test.mock_repo.MockVCSRepo*

get_rev()
Get current mercurial revision.

class spack.test.mock_repo.**MockRepo**(*stage_name*, *repo_name*)
Bases: object

destroy()
Destroy resources associated with this mock repo.

class spack.test.mock_repo.**MockSvnRepo**
Bases: *spack.test.mock_repo.MockVCSRepo*

class spack.test.mock_repo.**MockVCSRepo**(*stage_name*, *repo_name*)
Bases: *spack.test.mock_repo.MockRepo*

spack.test.modules module

class spack.test.modules.**DotkitTests**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

configuration_dotkit = {'enable': ['dotkit'], 'dotkit': {'all': {'prerequisites': 'direct'}}}

get_modulefile_content(*spec*)

setUp()

tearDown()

test_dotkit()

class spack.test.modules.**HelperFunctionsTests**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

test_inspect_path()

test_update_dictionary_extending_list()

class spack.test.modules.**LmodTests**(*methodName='runTest'*)
Bases: *spack.test.modules.ModuleFileGeneratorTests*

configuration_alter_environment = {'enable': ['lmod'], 'lmod': {'all': {'filter': {'environment_blacklist': ['CM

configuration_autoload_all = {'enable': ['lmod'], 'lmod': {'all': {'autoload': 'all'}}}

configuration_autoload_direct = {'enable': ['lmod'], 'lmod': {'all': {'autoload': 'direct'}}}

configuration_blacklist = {'enable': ['lmod'], 'lmod': {'blacklist': ['callpath'], 'all': {'autoload': 'direct'}}}

factory
alias of LmodModule

test_alter_environment()

test_autoload()

test_blacklist()

test_simple_case()

class spack.test.modules.**ModuleFileGeneratorTests**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

Base class to test module file generators. Relies on child having defined a 'factory' attribute to create an instance of the generator to be tested.

get_modulefile_content(*spec*)

setUp()

tearDown()

class spack.test.modules.**TclTests**(*methodName='runTest'*)
Bases: *spack.test.modules.ModuleFileGeneratorTests*

configuration_alter_environment = {'tcl': {'all': {'filter': {'environment_blacklist': ['CMAKE_PREFIX_PATH

configuration_autoload_all = {'tcl': {'all': {'autoload': 'all'}}, 'enable': ['tcl']}

configuration_autoload_direct = {'tcl': {'all': {'autoload': 'direct'}}, 'enable': ['tcl']}

configuration_blacklist = {'tcl': {'blacklist': ['callpath', 'mpi'], 'whitelist': ['zmpi'], 'all': {'autoload': 'direct'}}

configuration_conflicts = {'tcl': {'all': {'conflict': ['{name}', 'intel/14.0.1']}, 'naming_scheme': '{name}/{version

configuration_prerequisites_all = {'tcl': {'all': {'prerequisites': 'all'}}, 'enable': ['tcl']}

configuration_prerequisites_direct = {'tcl': {'all': {'prerequisites': 'direct'}}, 'enable': ['tcl']}

configuration_suffix = {'tcl': {'mpileaks': {'suffixes': {'~debug': 'bar', '+debug': 'foo'}}}, 'enable': ['tcl']}

configuration_wrong_conflicts = {'tcl': {'all': {'conflict': ['{name}/{compiler.name}']}, 'naming_scheme': '{nam

factory
alias of TclModule

test_alter_environment()

test_autoload()

test_blacklist()

test_conflicts()

test_prerequisites()

test_simple_case()

test_suffixes()

spack.test.modules.**mock_open**(**args*, ***kwds*)

spack.test.multimethod module

Test for multi_method dispatch.

class spack.test.multimethod.**MultiMethodTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

test_default_works()

test_dependency_match()

test_mpi_version()

test_no_version_match()

test_one_version_match()

test_target_match()

test_undefined_mpi_version()

test_version_overlap()

test_virtual_dep_match()

spack.test.namespace_trie module

class spack.test.namespace_trie.**NamespaceTrieTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

setUp()

test_add_multiple()

test_add_none_multiple()

test_add_none_single()

test_add_single()

test_add_three()

spack.test.optional_deps module

class spack.test.optional_deps.**ConcretizeTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

check_normalize(*spec_string*, *expected*)

test_chained_mpi()

test_default_variant()

test_multiple_conditionals()

test_normalize_simple_conditionals()

test_transitive_chain()

spack.test.package_sanity module

This test does sanity checks on Spack's builtin package database.

class spack.test.package_sanity.**PackageSanityTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

check_db()
Get all packages in a DB to make sure they work.

test_get_all_mock_packages()
Get the mock packages once each too.

test_get_all_packages()
Get all packages once and make sure that works.

test_url_versions()
Check URLs for regular packages, if they are explicitly defined.

spack.test.packages module

class spack.test.packages.**PackagesTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

test_import_class_from_package()

test_import_module_from_package()

test_import_namespace_container_modules()

test_import_package()

test_import_package_as()

test_load_package()

test_nonexisting_package_filename()

test_package_class_names()

test_package_filename()

test_package_name()

spack.test.pattern module

class spack.test.pattern.**CompositeTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

setUp()

test_composite_from_interface()

test_composite_from_method_list()

test_error_conditions()

spack.test.provider_index module

Tests for provider index cache files.

Tests assume that mock packages provide this:

```
{'blas':   {
     blas: set([netlib-blas, openblas, openblas-with-lapack])},
 'lapack': {lapack: set([netlib-lapack, openblas-with-lapack])},
 'mpi': {mpi@:1: set([mpich@:1]),
                  mpi@:2.0: set([mpich2]),
                  mpi@:2.1: set([mpich2@1.1:]),
                  mpi@:2.2: set([mpich2@1.2:]),
                  mpi@:3: set([mpich@3:]),
                  mpi@:10.0: set([zmpi])},
  'stuff': {stuff: set([externalvirtual])}}
```

class spack.test.provider_index.**ProviderIndexTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

test_copy()

test_equal()

test_mpi_providers()

test_providers_for_simple()

test_yaml_round_trip()

spack.test.python_version module

This test ensures that all Spack files are Python version 2.6 or less.

Spack was originally 2.7, but enough systems in 2014 are still using 2.6 on their frontend nodes that we need 2.6 to get adopted.

class spack.test.python_version.**PythonVersionTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

check_python_versions(**files*)

package_py_files()

pyfiles(**search_paths*)

test_core_module_compatibility()

test_package_module_compatibility()

spack.test.sbang module

Test that Spack's shebang filtering works correctly.

class spack.test.sbang.**SbangTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

setUp()

tearDown()

test_shebang_handles_non_writable_files()

test_shebang_handling()

spack.test.spec_dag module

These tests check Spec DAG operations using dummy packages. You can find the dummy packages here:

```
spack/lib/spack/spack/test/mock_packages
```

class spack.test.spec_dag.**SpecDagTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

check_links(*spec_to_check*)

test_conflicting_package_constraints()

test_conflicting_spec_constraints()

test_contains()

test_copy_concretized()

test_copy_normalized()

test_copy_simple()

test_dependents_and_dependencies_are_correct()

test_deptype_traversal()

test_deptype_traversal_full()

test_deptype_traversal_run()

test_deptype_traversal_with_builddeps()

test_equal()

test_invalid_dep()

test_normalize_a_lot()

test_normalize_mpileaks()

test_normalize_twice()
Make sure normalize can be run twice on the same spec, and that it is idempotent.

test_normalize_with_virtual_package()

test_normalize_with_virtual_spec()

test_postorder_edge_traversal()

test_postorder_node_traversal()

test_postorder_path_traversal()

test_preorder_edge_traversal()

test_preorder_node_traversal()

test_preorder_path_traversal()

test_unsatisfiable_architecture()

test_unsatisfiable_compiler()

test_unsatisfiable_compiler_version()

test_unsatisfiable_version()

test_using_ordered_dict()
Checks that dicts are ordered

Necessary to make sure that dag_hash is stable across python versions and processes.

spack.test.spec_semantics module

class spack.test.spec_semantics.**SpecSematicsTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

This tests satisfies(), constrain() and other semantic operations on specs.

check_constrain(*expected*, *spec*, *constraint*)

check_constrain_changed(*spec*, *constraint*)

check_constrain_not_changed(*spec*, *constraint*)

check_invalid_constraint(*spec*, *constraint*)

check_satisfies(*spec*, *anon_spec*, *concrete=False*)

check_unsatisfiable(*spec*, *anon_spec*, *concrete=False*)

test_constrain_architecture()

test_constrain_changed()

test_constrain_compiler()

test_constrain_compiler_flags()

test_constrain_dependency_changed()

test_constrain_dependency_not_changed()

test_constrain_not_changed()

test_constrain_variants()

test_dep_index()

test_invalid_constraint()

test_satisfies()

test_satisfies_architecture()

test_satisfies_compiler()

test_satisfies_compiler_version()

test_satisfies_dependencies()

test_satisfies_dependency_versions()

test_satisfies_matching_compiler_flag()

test_satisfies_matching_variant()

test_satisfies_namespace()

test_satisfies_namespaced_dep()
Ensure spec from same or unspecified namespace satisfies namespace constraint.

test_satisfies_unconstrained_compiler_flag()

test_satisfies_unconstrained_variant()

test_satisfies_virtual()

test_satisfies_virtual_dep_with_virtual_constraint()
Ensure we can satisfy virtual constraints when there are multiple vdep providers in the specs.

test_satisfies_virtual_dependencies()

test_satisfies_virtual_dependency_versions()

test_self_index()

test_spec_contains_deps()

test_unsatisfiable_compiler_flag()

test_unsatisfiable_compiler_flag_mismatch()

test_unsatisfiable_variant_mismatch()

test_unsatisfiable_variants()

test_virtual_index()

spack.test.spec_syntax module

class spack.test.spec_syntax.**SpecSyntaxTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

check_lex(*tokens*, *spec*)
Check that the provided spec parses to the provided token list.

check_parse(*expected*, *spec=None*, *remove_arch=True*)
Assert that the provided spec is able to be parsed.

If this is called with one argument, it assumes that the string is canonical (i.e., no spaces and ~ instead of - for variants) and that it will convert back to the string it came from.

If this is called with two arguments, the first argument is the expected canonical form and the second is a non-canonical input to be parsed.

test_ambiguous()

test_canonicalize()

test_dependencies_with_versions()

test_duplicate_compiler()

test_duplicate_depdendence()

test_duplicate_variant()

test_full_specs()

test_minimal_spaces()

test_package_names()

test_parse_errors()

test_simple_dependence()

test_spaces_between_dependences()

test_spaces_between_options()

test_way_too_many_spaces()

spack.test.spec_yaml module

Test YAML serialization for specs.

YAML format preserves DAG informatoin in the spec.

class spack.test.spec_yaml.**SpecYamlTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

check_yaml_round_trip(*spec*)

test_ambiguous_version_spec()

test_concrete_spec()

test_normal_spec()

test_simple_spec()

test_yaml_subdag()

spack.test.stage module

Test that the Stage class works correctly.

class spack.test.stage.**StageTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

check_chdir(*stage*, *stage_name*)

check_chdir_to_source(*stage*, *stage_name*)

check_destroy(*stage*, *stage_name*)
Figure out whether a stage was destroyed correctly.

check_expand_archive(*stage*, *stage_name*)

check_fetch(*stage*, *stage_name*)

check_setup(*stage*, *stage_name*)
Figure out whether a stage was set up correctly.

get_stage_path(*stage*, *stage_name*)
Figure out where a stage should be living. This depends on whether it's named.

setUp()
This sets up a mock archive to fetch, and a mock temp space for use by the Stage class. It doesn't actually create the Stage – that is done by individual tests.

tearDown()
Blows away the test environment directory.

test_chdir()

test_expand_archive()

test_expand_archive_with_chdir()

test_fetch()

test_keep_exceptions()

test_keep_without_exceptions()

test_no_keep_with_exceptions()

test_no_keep_without_exceptions()

test_restage()

test_setup_and_destroy_name_with_tmp()

test_setup_and_destroy_name_without_tmp()

test_setup_and_destroy_no_name_with_tmp()

test_setup_and_destroy_no_name_without_tmp()

spack.test.stage.**use_tmp**(**args*, ***kwds*)
Allow some test code to be executed with spack.use_tmp_stage set to a certain value. Context manager makes sure it's reset on failure.

spack.test.svn_fetch module

class spack.test.svn_fetch.**SvnFetchTest**(*methodName='runTest'*)
Bases: *spack.test.mock_packages_test.MockPackagesTest*

Tests fetching from a dummy git repository.

assert_rev(*rev*)
Check that the current revision is equal to the supplied rev.

setUp()
Create an svn repository with two revisions.

tearDown()
Destroy the stage space used by this test.

test_fetch_default()
Test a default checkout and make sure it's on rev 1

test_fetch_r1()
Test fetching an older revision (0).

try_fetch(*rev*, *test_file*, *args*)
Tries to:

1. Fetch the repo using a fetch strategy constructed with supplied args.
2. Check if the test_file is in the checked out repository.
3. Assert that the repository is at the revision supplied.
4. Add and remove some files, then reset the repo, and ensure it's all there again.

spack.test.tally_plugin module

class spack.test.tally_plugin.**Tally**
Bases: nose.plugins.base.Plugin

addError(*test*, *err*)

addFailure(*test*, *err*)

addSuccess(*test*)

configure(*options*, *conf*)

finalize(*result*)

name = 'tally'

numberOfTestsRun
Excludes skipped tests

options(*parser*, *env={'LANG': 'C.UTF-8', 'READTHEDOCS_PROJECT': 'spack-citibeth-fork', 'COLIFY_SIZE': '25x120', 'READTHEDOCS': 'True', 'SPACK_ROOT': '../../..', 'APPDIR': '/app', 'DEBIAN_FRONTEND': 'noninteractive', 'OLDPWD': '/', 'HOSTNAME': 'build-4476002-project-60229-spack-citibeth-fork', 'PWD': '/home/docs/checkouts/readthedocs.org/user_builds/spack-citibeth-fork/checkouts/efischer-docs2/lib/spack/docs', 'BIN_PATH': '/home/docs/checkouts/readthedocs.org/user_builds/spack-citibeth-fork/envs/efischer-docs2/bin', 'READTHEDOCS_VERSION': 'efischer-docs2', 'PATH': '/home/docs/checkouts/readthedocs.org/user_builds/spack-citibeth-fork/envs/efischer-docs2/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:../../../bin', 'HOME': '/home/docs'}*)

spack.test.url_extrapolate module

Tests ability of spack to extrapolate URL versions from existing versions.

class spack.test.url_extrapolate.**UrlExtrapolateTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

check_url(*base*, *version*, *new_url*)

test_dyninst_version()

test_gcc()

test_github_raw()

test_libdwarf_version()

test_libelf_version()

test_mpileaks_version()

test_partial_version_prefix()

test_scalasca_partial_version()

spack.test.url_parse module

This file has a bunch of versions tests taken from the excellent version detection in Homebrew.

class spack.test.url_parse.**UrlParseTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

assert_not_detected(*string*)

check(*name*, *v*, *string*, ***kwargs*)

test_angband_version_style()

test_another_erlang_version_style()

test_apache_version_style()

test_astyle_verson_style()

test_boost_version_style()

test_dash_rc_style()

test_dash_version_dash_style()

test_debian_style_1()

test_debian_style_2()

test_erlang_version_style()

test_fann_version()

test_gcc_version()

test_gcc_version_precedence()

test_github_raw_url()

test_gloox_beta_style()

test_haxe_version()

test_hdf5_version()

test_iges_version()

test_imagemagick_style()

test_imap_version()

test_jpeg_style()

test_lame_version_style()

test_mpileaks_version()

test_mvapich2_19_version()

test_mvapich2_20_version()

test_new_github_style()

test_no_version()

test_noseparator_single_digit()

test_omega_version_style()

test_openssl_version()

test_p7zip_version_style()

test_pypy_version()

test_rc_style()

test_ruby_version_style()

test_scalasca_version()

test_sphinx_beta_style()

test_stable_suffix()

test_suite3270_version()

test_synergy_version()

test_version_all_dots()

test_version_developer_that_hates_us_format()

test_version_dos2unix()

test_version_github()

test_version_github_with_high_patch_number()

test_version_internal_dash()

test_version_regular()

test_version_single_digit()

test_version_sourceforge_download()

test_version_underscore_separator()

test_wwwoffle_version()

test_xaw3d_version()

test_yet_another_erlang_version_style()

test_yet_another_version()

spack.test.url_substitution module

This test does sanity checks on substituting new versions into URLs

class spack.test.url_substitution.**PackageSanityTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

test_hypre_url_substitution()

test_otf2_url_substitution()

spack.test.versions module

These version tests were taken from the RPM source code. We try to maintain compatibility with RPM's version semantics where it makes sense.

class spack.test.versions.**VersionsTest**(*methodName='runTest'*)

Bases: unittest.case.TestCase

assert_canonical(*canonical_list*, *version_list*)

assert_does_not_satisfy(*v1*, *v2*)

assert_in(*needle*, *haystack*)

assert_no_overlap(*v1*, *v2*)

assert_not_in(*needle*, *haystack*)

assert_overlaps(*v1*, *v2*)

assert_satisfies(*v1*, *v2*)

assert_ver_eq(*a*, *b*)

assert_ver_gt(*a*, *b*)

assert_ver_lt(*a*, *b*)

check_intersection(*expected*, *a*, *b*)

check_union(*expected*, *a*, *b*)

test_alpha()

test_alpha_beta()

test_alpha_with_dots()

test_basic_version_satisfaction()

test_basic_version_satisfaction_in_lists()

test_canonicalize_list()

test_close_numbers()

test_contains()

test_date_stamps()

test_double_alpha()

test_formatted_strings()

test_get_item()

test_in_list()

test_intersect_with_containment()

test_intersection()

test_lists_overlap()

test_num_alpha_with_no_separator()

test_nums_and_patch()

test_overlap_with_containment()

test_padded_numbers()

test_patch()

test_ranges_overlap()

test_rc_versions()

test_repr_and_str()

test_rpm_oddities()

test_satisfaction_with_lists()

test_three_segments()

test_two_segments()

test_underscores()

test_union_with_containment()

test_version_range_satisfaction()

test_version_range_satisfaction_in_lists()

test_version_ranges()

spack.test.yaml module

Test Spack's custom YAML format.

class spack.test.yaml.**YamlTest**(*methodName='runTest'*)
Bases: unittest.case.TestCase

setUp()

test_dict_order()

test_line_numbers()

test_parse()

Module contents

spack.test.**list_tests**()
Return names of all tests that can be run for Spack.

spack.test.**run**(*names*, *outputDir*, *verbose=False*)
Run tests with the supplied names. Names should be a list. If it's empty, run ALL of Spack's tests.

12.1.8 spack.util package

Submodules

spack.util.compression module

spack.util.compression.**allowed_archive**(*path*)

spack.util.compression.**decompressor_for**(*path*)
Get the appropriate decompressor for a path.

spack.util.compression.**extension**(*path*)
Get the archive extension for a path.

spack.util.compression.**strip_extension**(*path*)
Get the part of a path that does not include its compressed type extension.

spack.util.crypto module

class spack.util.crypto.**Checker**(*hexdigest*, ***kwargs*)
Bases: object

A checker checks files against one particular hex digest. It will automatically determine what hashing algorithm to used based on the length of the digest it's initialized with. e.g., if the digest is 32 hex characters long this will use md5.

Example: know your tarball should hash to 'abc123'. You want to check files against this. You would use this class like so:

```
hexdigest = 'abc123'
checker = Checker(hexdigest)
success = checker.check('downloaded.tar.gz')
```

After the call to check, the actual checksum is available in checker.sum, in case it's needed for error output.

You can trade read performance and memory usage by adjusting the block_size optional arg. By default it's a 1MB (2**20 bytes) buffer.

check(*filename*)
Read the file with the specified name and check its checksum against self.hexdigest. Return True if they match, False otherwise. Actual checksum is stored in self.sum.

hash_name
Get the name of the hash function this Checker is using.

spack.util.crypto.**checksum**(*hashlib_algo*, *filename*, ***kwargs*)
Returns a hex digest of the filename generated using an algorithm from hashlib.

spack.util.debug module

Debug signal handler: prints a stack trace and enters interpreter.

register_interrupt_handler() enables a ctrl-C handler that prints a stack trace and drops the user into an interpreter.

spack.util.debug.**debug_handler**(*sig*, *frame*)
Interrupt running process, and provide a python prompt for interactive debugging.

spack.util.debug.**register_interrupt_handler**()
Print traceback and enter an interpreter on Ctrl-C

spack.util.environment module

spack.util.environment.**dump_environment**(*path*)
Dump the current environment out to a file.

spack.util.environment.**env_flag**(*name*)

spack.util.environment.**get_path**(*name*)

spack.util.environment.**path_put_first**(*var_name*, *directories*)
Puts the provided directories first in the path, adding them if they're not already there.

spack.util.environment.**path_set**(*var_name*, *directories*)

spack.util.executable module

class spack.util.executable.**Executable**(*name*)
Bases: object

Class representing a program that can be run on the command line.

add_default_arg(*arg*)

command

spack.util.executable.**which**(*name*, ***kwargs*)
Finds an executable in the path like command-line which.

exception spack.util.executable.**ProcessError**(*msg*, *long_message=None*)
Bases: *spack.error.SpackError*

long_message

spack.util.multiproc module

This implements a parallel map operation but it can accept more values than multiprocessing.Pool.apply() can. For example, apply() will fail to pickle functions if they're passed indirectly as parameters.

spack.util.multiproc.**spawn**(*f*)

spack.util.multiproc.**parmap**(*f*, *X*)

class spack.util.multiproc.**Barrier**(*n*, *timeout=None*)
Simple reusable semaphore barrier.

Python 2.6 doesn't have multiprocessing barriers so we implement this.

See http://greenteapress.com/semaphores/downey08semaphores.pdf, p. 41.

wait()

spack.util.naming module

spack.util.naming.**mod_to_class**(*mod_name*)
Convert a name from module style to class name style. Spack mostly follows PEP-8:

- Module and package names use lowercase_with_underscores.
- Class names use the CapWords convention.

Regular source code follows these convetions. Spack is a bit more liberal with its Package names nad Compiler names:

- They can contain '-' as well as '_', but cannot start with '-'.
- They can start with numbers, e.g. "3proxy".

This function converts from the module convention to the class convention by removing _ and - and converting surrounding lowercase text to CapWords. If mod_name starts with a number, the class name returned will be prepended with '_' to make a valid Python identifier.

spack.util.naming.**spack_module_to_python_module**(*mod_name*)
Given a Spack module name, returns the name by which it can be imported in Python.

spack.util.naming.**valid_module_name**(*mod_name*)
Return whether mod_name is valid for use in Spack.

spack.util.naming.**valid_fully_qualified_module_name**(*mod_name*)
Return whether mod_name is a valid namespaced module name.

spack.util.naming.**validate_fully_qualified_module_name**(*mod_name*)
Raise an exception if mod_name is not a valid namespaced module name.

spack.util.naming.**validate_module_name**(*mod_name*)
Raise an exception if mod_name is not valid.

spack.util.naming.**possible_spack_module_names**(*python_mod_name*)
Given a Python module name, return a list of all possible spack module names that could correspond to it.

class spack.util.naming.**NamespaceTrie**(*separator='.'*)
Bases: object

class **Element**(*value*)
Bases: object

NamespaceTrie.**has_value**(*namespace*)
True if there is a value set for the given namespace.

NamespaceTrie.**is_leaf**(*namespace*)
True if this namespace has no children in the trie.

NamespaceTrie.**is_prefix**(*namespace*)
True if the namespace has a value, or if it's the prefix of one that does.

spack.util.pattern module

class spack.util.pattern.**Bunch**(***kwargs*)
Bases: object

Carries a bunch of named attributes (from Alex Martelli bunch)

spack.util.pattern.**composite**(*interface=None*, *method_list=None*, *container=<type 'list'>*)
Returns a class decorator that patches a class adding all the methods it needs to be a composite for a given interface.

Parameters

- **interface** – class exposing the interface to which the composite object must conform. Only non-private and non-special methods will be taken into account
- **method_list** – names of methods that should be part of the composite
- **container** – container for the composite object (default = list). Must fulfill the MutableSequence contract. The composite class will expose the container API to manage object composition

Returns class decorator

spack.util.prefix module

This file contains utilities to help with installing packages.

class spack.util.prefix.**Prefix**
Bases: str

This class represents an installation prefix, but provides useful attributes for referring to directories inside the prefix.

For example, you can do something like this:

```
prefix = Prefix('/usr')
print prefix.lib
print prefix.lib64
print prefix.bin
print prefix.share
print prefix.man4
```

This program would print:

/usr/lib /usr/lib64 /usr/bin /usr/share /usr/share/man/man4

Prefix objects behave identically to strings. In fact, they subclass str. So operators like + are legal:

print “foobar ” + prefix

This prints ‘foobar /usr”. All of this is meant to make custom installs easy.

spack.util.spack_yaml module

Enhanced YAML parsing for Spack.

- load() preserves YAML Marks on returned objects – this allows us to access file and line information later.
- Our load methods use ``OrderedDict class instead of YAML’s default unorderd dict.

spack.util.spack_yaml.**load**(**args*, ***kwargs*)
Load but modify the loader instance so that it will add __line__ atrributes to the returned object.

spack.util.spack_yaml.**dump**(**args*, ***kwargs*)

spack.util.string module

spack.util.string.**comma_and**(*sequence*)

spack.util.string.**comma_list**(*sequence*, *article=’‘*)

spack.util.string.**comma_or**(*sequence*)

spack.util.web module

class spack.util.web.**LinkParser**
Bases: HTMLParser.HTMLParser

This parser just takes an HTML page and strips out the hrefs on the links. Good enough for a really simple spider.

handle_starttag(*tag*, *attrs*)

spack.util.web.**find_versions_of_archive**(**archive_urls*, ***kwargs*)
Scrape web pages for new versions of a tarball.

Arguments:

archive_urls: URLs for different versions of a package. Typically these are just the tarballs from the package file itself. By default, this searches the parent directories of archives.

Keyword Arguments: list_url:

URL for a listing of archives. Spack wills scrape these pages for download links that look like the archive URL.

list_depth: Max depth to follow links on list_url pages.

spack.util.web.**spider**(*root_url*, ***kwargs*)
Gets web pages from a root URL. If depth is specified (e.g., depth=2), then this will also fetches pages linked from the root and its children up to depth.

This will spawn processes to fetch the children, for much improved performance over a sequential fetch.

Module contents

12.2 Submodules

12.3 spack.abi module

class spack.abi.**ABI**
Bases: object

This class provides methods to test ABI compatibility between specs. The current implementation is rather rough and could be improved.

architecture_compatible(*parent*, *child*)
Return true if parent and child have ABI compatible targets.

compatible(*parent*, *child*, ***kwargs*)
Returns true iff a parent and child spec are ABI compatible

compiler_compatible(*parent*, *child*, ***kwargs*)
Return true if compilers for parent and child are ABI compatible.

12.4 spack.architecture module

This module contains all the elements that are required to create an architecture object. These include, the target processor, the operating system, and the architecture platform (i.e. cray, darwin, linux, bgq, etc) classes.

On a multiple architecture machine, the architecture spec field can be set to build a package against any target and operating system that is present on the platform. On Cray platforms or any other architecture that has different front and back end environments, the operating system will determine the method of compiler detection.

There are two different types of compiler detection:

1. Through the $PATH env variable (front-end detection)
2. Through the tcl module system. (back-end detection)

Depending on which operating system is specified, the compiler will be detected using one of those methods.

For platforms such as linux and darwin, the operating system is autodetected and the target is set to be x86_64.

The command line syntax for specifying an architecture is as follows:

target=<Target name> os=<OperatingSystem name>

If the user wishes to use the defaults, either target or os can be left out of the command line and Spack will concretize using the default. These defaults are set in the 'platforms/' directory which contains the different subclasses for platforms. If the machine has multiple architectures, the user can also enter front-end, or fe or back-end or be. These settings will concretize to their respective front-end and back-end targets and operating systems. Additional platforms can be added by creating a subclass of Platform and adding it inside the platform directory.

Platforms are an abstract class that are extended by subclasses. If the user wants to add a new type of platform (such as cray_xe), they can create a subclass and set all the class attributes such as priority, front_target, back_target, front_os, back_os. Platforms also contain a priority class attribute. A lower number signifies higher priority. These numbers are arbitrarily set and can be changed though often there isn't much need unless a new platform is added and the user wants that to be detected first.

Targets are created inside the platform subclasses. Most architecture (like linux, and darwin) will have only one target (x86_64) but in the case of Cray machines, there is both a frontend and backend processor. The user can specify which targets are present on front-end and back-end architecture

Depending on the platform, operating systems are either auto-detected or are set. The user can set the front-end and back-end operating setting by the class attributes front_os and back_os. The operating system as described earlier, will be responsible for compiler detection.

class spack.architecture.**Arch**(*plat=None*, *os=None*, *target=None*)
Bases: object

Architecture is now a class to help with setting attributes.

TODO: refactor so that we don't need this class.

concrete

to_dict()

exception spack.architecture.**NoPlatformError**
Bases: *spack.error.SpackError*

class spack.architecture.**OperatingSystem**(*name*, *version*)
Bases: object

Operating System will be like a class similar to platform extended by subclasses for the specifics. Operating System will contain the compiler finding logic. Instead of calling two separate methods to find compilers we call find_compilers method for each operating system

find_compiler(*cmp_cls*, **path*)
Try to find the given type of compiler in the user's environment. For each set of compilers found, this returns compiler objects with the cc, cxx, f77, fc paths and the version filled in.

This will search for compilers with the names in cc_names, cxx_names, etc. and it will group them if they have common prefixes, suffixes, and versions. e.g., gcc-mp-4.7 would be grouped with g++-mp-4.7 and gfortran-mp-4.7.

find_compilers(**paths*)
Return a list of compilers found in the suppied paths. This invokes the find() method for each Compiler class, and appends the compilers detected to a list.

to_dict()

class spack.architecture.**Platform**(*name*)
Bases: object

Abstract class that each type of Platform will subclass. Will return a instance of it once it is returned

add_operating_system(*name*, *os_class*)
Add the operating_system class object into the platform.operating_sys dictionary

add_target(*name*, *target*)
Used by the platform specific subclass to list available targets. Raises an error if the platform specifies a name that is reserved by spack as an alias.

back_end = None

back_os = None

default = None

default_os = None

classmethod detect()
Subclass is responsible for implementing this method. Returns True if the Platform class detects that it is the current platform and False if it's not.

front_end = None

front_os = None

operating_system(*name*)

priority = None

classmethod setup_platform_environment(*pkg*, *env*)
Subclass can override this method if it requires any platform-specific build environment modifications.

target(*name*)
This is a getter method for the target dictionary that handles defaulting based on the values provided by default, front-end, and back-end. This can be overwritten by a subclass for which we want to provide further aliasing options.

class spack.architecture.**Target**(*name*, *module_name=None*)
Bases: object

Target is the processor of the host machine. The host machine may have different front-end and back-end targets, especially if it is a Cray machine. The target will have a name and also the module_name (e.g craype-compiler). Targets will also recognize which platform they came from using the set_platform method. Targets will have compiler finding strategies

spack.architecture.**arch_from_dict**(*d*)
Uses _platform_from_dict, _operating_system_from_dict, _target_from_dict helper methods to recreate the arch tuple from the dictionary read from a yaml file

12.5 spack.build_environment module

This module contains all routines related to setting up the package build environment. All of this is set up by package.py just before install() is called.

There are two parts to the build environment:

1. Python build environment (i.e. install() method)

This is how things are set up when install() is called. Spack takes advantage of each package being in its own module by adding a bunch of command-like functions (like configure(), make(), etc.) in the package's module scope. Ths allows package writers to call them all directly in Package.install() without writing 'self.' everywhere. No, this isn't Pythonic. Yes, it makes the code more readable and more like the shell script from which someone is likely porting.

2. Build execution environment

   This is the set of environment variables, like PATH, CC, CXX, etc. that control the build. There are also a number of environment variables used to pass information (like RPATHs and other information about dependencies) to Spack's compiler wrappers. All of these env vars are also set up here.

Skimming this module is a nice way to get acquainted with the types of calls you can make from within the install() function.

exception spack.build_environment.**InstallError**(*message*, *long_message=None*)
Bases: *spack.error.SpackError*

Raised when a package fails to install

class spack.build_environment.**MakeExecutable**(*name*, *jobs*)
Bases: *spack.util.executable.Executable*

Special callable executable object for make so the user can specify parallel or not on a per-invocation basis. Using 'parallel' as a kwarg will override whatever the package's global setting is, so you can either default to true or false and override particular calls.

Note that if the SPACK_NO_PARALLEL_MAKE env var is set it overrides everything.

spack.build_environment.**fork**(*pkg*, *function*, *dirty=False*)
Fork a child process to do part of a spack build.

Parameters

- **pkg** – pkg whose environemnt we should set up the forked process for.
- **function** – arg-less function to run in the child process.
- **dirty** – If True, do NOT clean the environment before building.

Usage:

```
def child_fun():
    # do stuff
build_env.fork(pkg, child_fun)
```

Forked processes are run with the build environment set up by spack.build_environment. This allows package authors to have full control over the environment, etc. without affecting other builds that might be executed in the same spack call.

If something goes wrong, the child process is expected to print the error and the parent process will exit with error as well. If things go well, the child exits and the parent carries on.

spack.build_environment.**get_path_from_module**(*mod*)
Inspects a TCL module for entries that indicate the absolute path at which the library supported by said module can be found.

spack.build_environment.**get_rpaths**(*pkg*)
Get a list of all the rpaths for a package.

spack.build_environment.**load_external_modules**(*pkg*)
traverse the spec list and find any specs that have external modules.

spack.build_environment.**load_module**(*mod*)
Takes a module name and removes modules until it is possible to load that module. It then loads the provided module. Depends on the modulecmd implementation of modules used in cray and lmod.

spack.build_environment.**parent_class_modules**(*cls*)
Get list of super class modules that are all descend from spack.Package

spack.build_environment.**set_build_environment_variables**(*pkg*, *env*, *dirty=False*)
This ensures a clean install environment when we build packages.

Arguments: dirty – skip unsetting the user's environment settings.

spack.build_environment.**set_compiler_environment_variables**(*pkg*, *env*)

spack.build_environment.**set_module_variables_for_package**(*pkg*, *module*)
Populate the module scope of install() with some useful functions. This makes things easier for package writers.

spack.build_environment.**setup_package**(*pkg*, *dirty=False*)
Execute all environment setup routines.

12.6 spack.compiler module

class spack.compiler.**Compiler**(*cspec*, *operating_system*, *paths*, *modules=[]*, *alias=None*, ***kwargs*)
Bases: object

This class encapsulates a Spack "compiler", which includes C, C++, and Fortran compilers. Subclasses should implement support for specific compilers, their possible names, arguments, and how to identify the particular type of compiler.

PrgEnv = None

PrgEnv_compiler = None

cc_names = []

cc_rpath_arg

classmethod cc_version(*cc*)

cxx11_flag

cxx14_flag

cxx_names = []

cxx_rpath_arg

classmethod cxx_version(*cxx*)

classmethod default_version(*cc*)
Override just this to override all compiler version functions.

f77_names = []

f77_rpath_arg

classmethod f77_version(*f77*)

fc_names = []

fc_rpath_arg

classmethod fc_version(*fc*)

openmp_flag

prefixes = []

suffixes = ['-.*']

version

spack.compiler.**get_compiler_version**(*compiler_path*, *version_arg*, *regex='(.*)'*)

12.7 spack.concretize module

Functions here are used to take abstract specs and make them concrete. For example, if a spec asks for a version between 1.8 and 1.9, these functions might take will take the most recent 1.9 version of the package available. Or, if the user didn't specify a compiler for a spec, then this will assign a compiler to the spec based on defaults or user preferences.

TODO: make this customizable and allow users to configure concretization policies.

class spack.concretize.**DefaultConcretizer**
Bases: object

This class doesn't have any state, it just provides some methods for concretization. You can subclass it to override just some of the default concretization strategies, or you can override all of them.

choose_virtual_or_external(*spec*)
Given a list of candidate virtual and external packages, try to find one that is most ABI compatible.

concretize_architecture(*spec*)
If the spec is empty provide the defaults of the platform. If the architecture is not a basestring, then check if either the platform, target or operating system are concretized. If any of the fields are changed then return True. If everything is concretized (i.e the architecture attribute is a namedtuple of classes) then return False. If the target is a string type, then convert the string into a concretized architecture. If it has no architecture and the root of the DAG has an architecture, then use the root otherwise use the defaults on the platform.

concretize_compiler(*spec*)
If the spec already has a compiler, we're done. If not, then take the compiler used for the nearest ancestor with a compiler spec and use that. If the ancestor's compiler is not concrete, then used the preferred compiler as specified in spackconfig.

Intuition: Use the spackconfig default if no package that depends on this one has a strict compiler requirement. Otherwise, try to build with the compiler that will be used by libraries that link to this one, to maximize compatibility.

concretize_compiler_flags(*spec*)
The compiler flags are updated to match those of the spec whose compiler is used, defaulting to no compiler flags in the spec. Default specs set at the compiler level will still be added later.

concretize_variants(*spec*)
If the spec already has variants filled in, return. Otherwise, add the user preferences from packages.yaml or the default variants from the package specification.

concretize_version(*spec*)
If the spec is already concrete, return. Otherwise take the preferred version from spackconfig, and default to the package's version if there are no available versions.

TODO: In many cases we probably want to look for installed versions of each package and use an installed version if we can link to it. The policy implemented here will tend to rebuild a lot of stuff becasue it will prefer a compiler in the spec to any compiler already- installed things were built with. There is likely some better policy that finds some middle ground between these two extremes.

exception spack.concretize.**NoBuildError**(*spec*)
Bases: *spack.error.SpackError*

Raised when a package is configured with the buildable option False, but no satisfactory external versions can be found

exception spack.concretize.**NoValidVersionError**(*spec*)
Bases: *spack.error.SpackError*

Raised when there is no way to have a concrete version for a particular spec.

exception spack.concretize.**UnavailableCompilerVersionError**(*compiler_spec*, *operating_system*)
Bases: *spack.error.SpackError*

Raised when there is no available compiler that satisfies a compiler spec.

spack.concretize.**cmp_specs**(*lhs*, *rhs*)

spack.concretize.**find_spec**(*spec*, *condition*)
Searches the dag from spec in an intelligent order and looks for a spec that matches a condition

12.8 spack.config module

This module implements Spack's configuration file handling.

12.8.1 Configuration file scopes

When Spack runs, it pulls configuration data from several config directories, each of which contains configuration files. In Spack, there are two configuration scopes:

1. `site`: Spack loads site-wide configuration options from `$(prefix)/etc/spack/`.
2. `user`: Spack next loads per-user configuration options from `~/.spack/`.

Spack may read configuration files from both of these locations. When configurations conflict, the user config options take precedence over the site configurations. Each configuration directory may contain several configuration files, such as compilers.yaml or mirrors.yaml.

12.8.2 Configuration file format

Configuration files are formatted using YAML syntax. This format is implemented by libyaml (included with Spack as an external module), and it's easy to read and versatile.

Config files are structured as trees, like this `compiler` section:

```
compilers:
  chaos_5_x86_64_ib:
     gcc@4.4.7:
       cc: /usr/bin/gcc
       cxx: /usr/bin/g++
       f77: /usr/bin/gfortran
       fc: /usr/bin/gfortran
  bgqos_0:
     xlc@12.1:
       cc: /usr/local/bin/mpixlc
       ...
```

In this example, entries like "compilers" and "xlc@12.1" are used to categorize entries beneath them in the tree. At the root of the tree, entries like "cc" and "cxx" are specified as name/value pairs.

`config.get_config()` returns these trees as nested dicts, but it strips the first level off. So, `config.get_config('compilers')` would return something like this for the above example:

```
{ 'chaos_5_x86_64_ib' :
    { 'gcc@4.4.7' :
        { 'cc' : '/usr/bin/gcc',
          'cxx' : '/usr/bin/g++'
          'f77' : '/usr/bin/gfortran'
          'fc' : '/usr/bin/gfortran' }
        }
    { 'bgqos_0' :
       { 'cc' : '/usr/local/bin/mpixlc' } }
```

Likewise, the `mirrors.yaml` file's first line must be `mirrors:`, but `get_config()` strips that off too.

12.8.3 Precedence

`config.py` routines attempt to recursively merge configuration across scopes. So if there are `compilers.py` files in both the site scope and the user scope, `get_config('compilers')` will return merged dictionaries of *all* the compilers available. If a user compiler conflicts with a site compiler, Spack will overwrite the site configuration wtih the user configuration. If both the user and site `mirrors.yaml` files contain lists of mirrors, then `get_config()` will return a concatenated list of mirrors, with the user config items first.

Sometimes, it is useful to *completely* override a site setting with a user one. To accomplish this, you can use *two* colons at the end of a key in a configuration file. For example, this:

```
compilers::
  chaos_5_x86_64_ib:
     gcc@4.4.7:
       cc: /usr/bin/gcc
       cxx: /usr/bin/g++
       f77: /usr/bin/gfortran
       fc: /usr/bin/gfortran
  bgqos_0:
     xlc@12.1:
       cc: /usr/local/bin/mpixlc
       ...
```

Will make Spack take compilers *only* from the user configuration, and the site configuration will be ignored.

exception `spack.config.`**`ConfigError`**(*message*, *long_message=None*)
Bases: *spack.error.SpackError*

exception `spack.config.`**`ConfigFileError`**(*message*, *long_message=None*)
Bases: *spack.config.ConfigError*

exception `spack.config.`**`ConfigFormatError`**(*validation_error*, *data*)
Bases: *spack.config.ConfigError*

Raised when a configuration format does not match its schema.

exception `spack.config.`**`ConfigSanityError`**(*validation_error*, *data*)
Bases: *spack.config.ConfigFormatError*

Same as ConfigFormatError, raised when config is written by Spack.

class spack.config.**ConfigScope**(*name*, *path*)
Bases: object

This class represents a configuration scope.

A scope is one directory containing named configuration files. Each file is a config "section" (e.g., mirrors, compilers, etc).

clear()
Empty cached config information.

get_section(*section*)

get_section_filename(*section*)

write_section(*section*)

spack.config.**clear_config_caches**()
Clears the caches for configuration files, which will cause them to be re-read upon the next request

spack.config.**extend_with_default**(*validator_class*)
Add support for the 'default' attr for properties and patternProperties.

jsonschema does not handle this out of the box – it only validates. This allows us to set default values for configs where certain fields are *None* b/c they're deleted or commented out.

spack.config.**get_config**(*section*, *scope=None*)
Get configuration settings for a section.

Strips off the top-level section name from the YAML dict.

spack.config.**get_config_filename**(*scope*, *section*)
For some scope and section, get the name of the configuration file

spack.config.**get_path**(*path*, *data*)

spack.config.**highest_precedence_scope**()
Get the scope with highest precedence (prefs will override others).

spack.config.**is_spec_buildable**(*spec*)
Return true if the spec pkgspec is configured as buildable

spack.config.**print_section**(*section*)
Print a configuration to stdout.

spack.config.**section_schemas** = {'mirrors': {'additionalProperties': False, 'patternProperties': {'mirrors:?': {'defaul
OrderedDict of config scopes keyed by name. Later scopes will override earlier scopes.

spack.config.**spec_externals**(*spec*)
Return a list of external specs (with external directory path filled in), one for each known external installation.

spack.config.**update_config**(*section*, *update_data*, *scope=None*)
Update the configuration file for a particular scope.

Overwrites contents of a section in a scope with update_data, then writes out the config file.

update_data should have the top-level section name stripped off (it will be re-added). Data itself can be a list, dict, or any other yaml-ish structure.

spack.config.**validate_scope**(*scope*)
Ensure that scope is valid, and return a valid scope if it is None.

This should be used by routines in config.py to validate scope name arguments, and to determine a default scope where no scope is specified.

spack.config.**validate_section**(*data*, *schema*)
Validate data read in from a Spack YAML file.

This leverages the line information (start_mark, end_mark) stored on Spack YAML structures.

spack.config.**validate_section_name**(*section*)
Exit if the section is not a valid section.

12.9 spack.database module

Spack's installation tracking database.

The database serves two purposes:

1. It implements a cache on top of a potentially very large Spack directory hierarchy, speeding up many operations that would otherwise require filesystem access.
2. It will allow us to track external installations as well as lost packages and their dependencies.

Prior ot the implementation of this store, a direcotry layout served as the authoritative database of packages in Spack. This module provides a cache and a sanity checking mechanism for what is in the filesystem.

exception spack.database.**CorruptDatabaseError**(*message*, *long_message=None*)
Bases: *spack.error.SpackError*

Raised when errors are found while reading the database.

class spack.database.**Database**(*root*, *db_dir=None*)
Bases: object

add(*spec_like*, **args*, ***kwargs*)

get_record(*spec_like*, **args*, ***kwargs*)

installed_extensions_for(*spec_like*, **args*, ***kwargs*)

missing(*spec*)

query(*query_spec=<built-in function any>*, *known=<built-in function any>*, *installed=True*, *explicit=<built-in function any>*)
Run a query on the database.

query_spec Queries iterate through specs in the database and return those that satisfy the supplied query_spec. If query_spec is *any*, This will match all specs in the database. If it is a spec, we'll evaluate spec.satisfies(query_spec).

The query can be constrained by two additional attributes:

known Possible values: True, False, any

Specs that are "known" are those for which Spack can locate a package.py file – i.e., Spack "knows" how to install them. Specs that are unknown may represent packages that existed in a previous version of Spack, but have since either changed their name or been removed.

installed Possible values: True, False, any

Specs for which a prefix exists are "installed". A spec that is NOT installed will be in the database if some other spec depends on it but its installation has gone away since Spack installed it.

TODO: Specs are a lot like queries. Should there be a wildcard spec object, and should specs have attributes like installed and known that can be queried? Or are these really special cases that only belong here?

query_one(*query_spec*, *known=<built-in function any>*, *installed=True*)
Query for exactly one spec that matches the query spec.

Raises an assertion error if more than one spec matches the query. Returns None if no installed package matches.

read_transaction(*timeout=60*)
Get a read lock context manager for use in a *with* block.

reindex(*directory_layout*)
Build database index from scratch based on a directory layout.

Locks the DB if it isn't locked already.

remove(*spec_like*, **args*, ***kwargs*)

write_transaction(*timeout=60*)
Get a write lock context manager for use in a *with* block.

class spack.database.**InstallRecord**(*spec*, *path*, *installed*, *ref_count=0*, *explicit=False*)
Bases: object

A record represents one installation in the DB.

The record keeps track of the spec for the installation, its install path, AND whether or not it is installed. We need the installed flag in case a user either:

1.blew away a directory, or

2.used spack uninstall -f to get rid of it

If, in either case, the package was removed but others still depend on it, we still need to track its spec, so we don't actually remove from the database until a spec has no installed dependents left.

classmethod from_dict(*spec*, *dictionary*)

to_dict()

exception spack.database.**InvalidDatabaseVersionError**(*expected*, *found*)
Bases: *spack.error.SpackError*

exception spack.database.**NonConcreteSpecAddError**(*message*, *long_message=None*)
Bases: *spack.error.SpackError*

Raised when attemptint to add non-concrete spec to DB.

12.10 spack.directives module

This package contains directives that can be used within a package.

Directives are functions that can be called inside a package definition to modify the package, for example:

class OpenMpi(Package): depends_on("hwloc") provides("mpi") ...

`provides` and `depends_on` are spack directives.

The available directives are:

- `version`
- `depends_on`
- `provides`
- `extends`

- patch
- variant
- resource

spack.directives.**depends_on**(**args*, ***kwargs*)
Creates a dict of deps with specs defining when they apply. This directive is to be used inside a Package definition to declare that the package requires other packages to be built first. @see The section "Dependency specs" in the Spack Packaging Guide.

spack.directives.**extends**(**args*, ***kwargs*)
Same as depends_on, but dependency is symlinked into parent prefix.

This is for Python and other language modules where the module needs to be installed into the prefix of the Python installation. Spack handles this by installing modules into their own prefix, but allowing ONE module version to be symlinked into a parent Python install at a time.

keyword arguments can be passed to extends() so that extension packages can pass parameters to the extendee's extension mechanism.

spack.directives.**provides**(**args*, ***kwargs*)
Allows packages to provide a virtual dependency. If a package provides 'mpi', other packages can declare that they depend on "mpi", and spack can use the providing package to satisfy the dependency.

spack.directives.**patch**(**args*, ***kwargs*)
Packages can declare patches to apply to source. You can optionally provide a when spec to indicate that a particular patch should only be applied when the package's spec meets certain conditions (e.g. a particular version).

spack.directives.**version**(**args*, ***kwargs*)
Adds a version and metadata describing how to fetch it. Metadata is just stored as a dict in the package's versions dictionary. Package must turn it into a valid fetch strategy later.

spack.directives.**variant**(**args*, ***kwargs*)
Define a variant for the package. Packager can specify a default value (on or off) as well as a text description.

spack.directives.**resource**(**args*, ***kwargs*)
Define an external resource to be fetched and staged when building the package. Based on the keywords present in the dictionary the appropriate FetchStrategy will be used for the resource. Resources are fetched and staged in their own folder inside spack stage area, and then moved into the stage area of the package that needs them.

List of recognized keywords:

- 'when' : (optional) represents the condition upon which the resource is needed
- 'destination' : (optional) path where to move the resource. This path must be relative to the main package stage area.
- 'placement' : (optional) gives the possibility to fine tune how the resource is moved into the main package stage area.

12.11 spack.directory_layout module

class spack.directory_layout.**DirectoryLayout**(*root*)
Bases: object

A directory layout is used to associate unique paths with specs. Different installations are going to want differnet layouts for their install, and they can use this to customize the nesting structure of spack installs.

add_extension(*spec*, *ext_spec*)
Add to the list of currently installed extensions.

all_specs()
To be implemented by subclasses to traverse all specs for which there is a directory within the root.

check_activated(*spec*, *ext_spec*)
Ensure that ext_spec can be removed from spec.

If not, raise NoSuchExtensionError.

check_extension_conflict(*spec*, *ext_spec*)
Ensure that ext_spec can be activated in spec.

If not, raise ExtensionAlreadyInstalledError or ExtensionConflictError.

check_installed(*spec*)
Checks whether a spec is installed.

Return the spec's prefix, if it is installed, None otherwise.

Raise an exception if the install is inconsistent or corrupt.

create_install_directory(*spec*)
Creates the installation directory for a spec.

extension_map(*spec*)
Get a dict of currently installed extension packages for a spec.

Dict maps { name : extension_spec } Modifying dict does not affect internals of this layout.

hidden_file_paths
Return a list of hidden files used by the directory layout.

Paths are relative to the root of an install directory.

If the directory layout uses no hidden files to maintain state, this should return an empty container, e.g. [] or (,).

path_for_spec(*spec*)
Return absolute path from the root to a directory for the spec.

relative_path_for_spec(*spec*)
Implemented by subclasses to return a relative path from the install root to a unique location for the provided spec.

remove_extension(*spec*, *ext_spec*)
Remove from the list of currently installed extensions.

remove_install_directory(*spec*)
Removes a prefix and any empty parent directories from the root. Raised RemoveFailedError if something goes wrong.

exception spack.directory_layout.**DirectoryLayoutError**(*message*, *long_msg=None*)
Bases: *spack.error.SpackError*

Superclass for directory layout errors.

exception spack.directory_layout.**ExtensionAlreadyInstalledError**(*spec*, *ext_spec*)
Bases: *spack.directory_layout.DirectoryLayoutError*

Raised when an extension is added to a package that already has it.

exception spack.directory_layout.**ExtensionConflictError**(*spec*, *ext_spec*, *conflict*)
Bases: *spack.directory_layout.DirectoryLayoutError*

Raised when an extension is added to a package that already has it.

exception spack.directory_layout.**InconsistentInstallDirectoryError**(*message*, *long_msg=None*)
Bases: *spack.directory_layout.DirectoryLayoutError*

Raised when a package seems to be installed to the wrong place.

exception spack.directory_layout.**InstallDirectoryAlreadyExistsError**(*path*)
Bases: *spack.directory_layout.DirectoryLayoutError*

Raised when create_install_directory is called unnecessarily.

exception spack.directory_layout.**InvalidExtensionSpecError**(*message*, *long_msg=None*)
Bases: *spack.directory_layout.DirectoryLayoutError*

Raised when an extension file has a bad spec in it.

exception spack.directory_layout.**NoSuchExtensionError**(*spec*, *ext_spec*)
Bases: *spack.directory_layout.DirectoryLayoutError*

Raised when an extension isn't there on deactivate.

exception spack.directory_layout.**RemoveFailedError**(*installed_spec*, *prefix*, *error*)
Bases: *spack.directory_layout.DirectoryLayoutError*

Raised when a DirectoryLayout cannot remove an install prefix.

exception spack.directory_layout.**SpecHashCollisionError**(*installed_spec*, *new_spec*)
Bases: *spack.directory_layout.DirectoryLayoutError*

Raised when there is a hash collision in an install layout.

exception spack.directory_layout.**SpecReadError**(*message*, *long_msg=None*)
Bases: *spack.directory_layout.DirectoryLayoutError*

Raised when directory layout can't read a spec.

class spack.directory_layout.**YamlDirectoryLayout**(*root*, ***kwargs*)
Bases: *spack.directory_layout.DirectoryLayout*

Lays out installation directories like this::

<install root>/

<platform-os-target>/

<compiler>-<compiler version>/ <name>-<version>-<variants>-<hash>

The hash here is a SHA-1 hash for the full DAG plus the build spec. TODO: implement the build spec.

To avoid special characters (like ~) in the directory name, only enabled variants are included in the install path. Disabled variants are omitted.

add_extension(*spec*, *ext_spec*)

all_specs()

build_env_path(*spec*)

build_log_path(*spec*)

build_packages_path(*spec*)

check_activated(*spec*, *ext_spec*)

check_extension_conflict(*spec*, *ext_spec*)

check_installed(*spec*)

create_install_directory(*spec*)

extension_file_path(*spec*)
Gets full path to an installed package's extension file

extension_map(*spec*)
Defensive copying version of _extension_map() for external API.

hidden_file_paths

metadata_path(*spec*)

read_spec(*path*)
Read the contents of a file and parse them as a spec

relative_path_for_spec(*spec*)

remove_extension(*spec*, *ext_spec*)

spec_file_path(*spec*)
Gets full path to spec file

specs_by_hash()

write_spec(*spec*, *path*)
Write a spec out to a file.

12.12 spack.environment module

class spack.environment.**AppendPath**(*name*, *value*, ***kwargs*)
Bases: *spack.environment.NameValueModifier*

execute()

class spack.environment.**EnvironmentModifications**(*other=None*)
Bases: object

Keeps track of requests to modify the current environment.

Each call to a method to modify the environment stores the extra information on the caller in the request: - 'filename' : filename of the module where the caller is defined - 'lineno': line number where the request occurred - 'context' : line of code that issued the request that failed

append_path(*name*, *path*, ***kwargs*)
Stores in the current object a request to append a path to a path list

Args: name: name of the path list in the environment path: path to be appended

apply_modifications()
Applies the modifications and clears the list

clear()
Clears the current list of modifications

extend(*other*)

static from_sourcing_files(**args*, ***kwargs*)
Creates an instance of EnvironmentModifications that, if executed, has the same effect on the environment as sourcing the files passed as parameters

Parameters ***args** – list of files to be sourced

Return type instance of EnvironmentModifications

group_by_name()
Returns a dict of the modifications grouped by variable name

Returns: dict mapping the environment variable name to the modifications to be done on it

prepend_path(*name*, *path*, ***kwargs*)
Same as *append_path*, but the path is pre-pended

Args: name: name of the path list in the environment path: path to be pre-pended

remove_path(*name*, *path*, ***kwargs*)
Stores in the current object a request to remove a path from a path list

Args: name: name of the path list in the environment path: path to be removed

set(*name*, *value*, ***kwargs*)
Stores in the current object a request to set an environment variable

Args: name: name of the environment variable to be set value: value of the environment variable

set_path(*name*, *elts*, ***kwargs*)
Stores a request to set a path generated from a list.

Args: name: name o the environment variable to be set. elts: elements of the path to set.

unset(*name*, ***kwargs*)
Stores in the current object a request to unset an environment variable

Args: name: name of the environment variable to be set

class spack.environment.**NameModifier**(*name*, ***kwargs*)
Bases: object

update_args(***kwargs*)

class spack.environment.**NameValueModifier**(*name*, *value*, ***kwargs*)
Bases: object

update_args(***kwargs*)

class spack.environment.**PrependPath**(*name*, *value*, ***kwargs*)
Bases: *spack.environment.NameValueModifier*

execute()

class spack.environment.**RemovePath**(*name*, *value*, ***kwargs*)
Bases: *spack.environment.NameValueModifier*

execute()

class spack.environment.**SetEnv**(*name*, *value*, ***kwargs*)
Bases: *spack.environment.NameValueModifier*

execute()

class spack.environment.**SetPath**(*name*, *value*, ***kwargs*)
Bases: *spack.environment.NameValueModifier*

execute()

class spack.environment.**UnsetEnv**(*name*, ***kwargs*)
Bases: *spack.environment.NameModifier*

execute()

spack.environment.**concatenate_paths**(*paths*, *separator=':'*)
Concatenates an iterable of paths into a string of paths separated by separator, defaulting to colon

Args: paths: iterable of paths separator: the separator to use, default ':'

Returns: string

spack.environment.**filter_environment_blacklist**(*env*, *variables*)
Generator that filters out any change to environment variables present in the input list

Args: env: list of environment modifications variables: list of variable names to be filtered

Yields: items in env if they are not in variables

spack.environment.**set_or_unset_not_first**(*variable*, *changes*, *errstream*)
Check if we are going to set or unset something after other modifications have already been requested

spack.environment.**validate**(*env*, *errstream*)
Validates the environment modifications to check for the presence of suspicious patterns. Prompts a warning for everything that was found

Current checks: - set or unset variables after other changes on the same variable

Args: env: list of environment modifications

12.13 spack.error module

exception spack.error.**NoNetworkConnectionError**(*message*, *url*)
Bases: *spack.error.SpackError*

Raised when an operation needs an internet connection.

exception spack.error.**SpackError**(*message*, *long_message=None*)
Bases: exceptions.Exception

This is the superclass for all Spack errors. Subclasses can be found in the modules they have to do with.

die()

long_message

exception spack.error.**UnsupportedPlatformError**(*message*)
Bases: *spack.error.SpackError*

Raised by packages when a platform is not supported

12.14 spack.fetch_strategy module

Fetch strategies are used to download source code into a staging area in order to build it. They need to define the following methods:

- **fetch()** This should attempt to download/check out source from somewhere.
- **check()** Apply a checksum to the downloaded source code, e.g. for an archive. May not do anything if the fetch method was safe to begin with.
- **expand()** Expand (e.g., an archive) downloaded file to source.
- **reset()** Restore original state of downloaded code. Used by clean commands. This may just remove the expanded source and re-expand an archive, or it may run something like git reset –hard.

- **archive()** Archive a source directory, e.g. for creating a mirror.

class spack.fetch_strategy.**CacheURLFetchStrategy**(**args*, ***kwargs*)
Bases: *spack.fetch_strategy.URLFetchStrategy*

The resource associated with a cache URL may be out of date.

fetch(**args*, ***kwargs*)

exception spack.fetch_strategy.**ChecksumError**(*message*, *long_msg=None*)
Bases: *spack.fetch_strategy.FetchError*

Raised when archive fails to checksum.

exception spack.fetch_strategy.**FailedDownloadError**(*url*, *msg=''*)
Bases: *spack.fetch_strategy.FetchError*

Raised wen a download fails.

exception spack.fetch_strategy.**FetchError**(*msg*, *long_msg=None*)
Bases: *spack.error.SpackError*

class spack.fetch_strategy.**FetchStrategy**
Bases: object

Superclass of all fetch strategies.

archive(*destination*)

check()

enabled = False

expand()

fetch()

classmethod matches(*args*)

required_attributes = None

reset()

set_stage(*stage*)
This is called by Stage before any of the fetching methods are called on the stage.

class spack.fetch_strategy.**FsCache**(*root*)
Bases: object

destroy()

fetcher(*targetPath*, *digest*, ***kwargs*)

store(*fetcher*, *relativeDst*)

class spack.fetch_strategy.**GitFetchStrategy**(***kwargs*)
Bases: *spack.fetch_strategy.VCSFetchStrategy*

Fetch strategy that gets source code from a git repository. Use like this in a package:

version('name', git='https://github.com/project/repo.git')

Optionally, you can provide a branch, or commit to check out, e.g.:

version('1.1', git='https://github.com/project/repo.git', tag='v1.1')

You can use these three optional attributes in addition to git:

•branch: Particular branch to build from (default is master)

- `tag`: Particular tag to check out
- `commit`: Particular commit hash in the repo

archive (*destination*)

enabled = True

fetch (**args*, ***kwargs*)

git

git_version

required_attributes = ('git',)

reset (**args*, ***kwargs*)

class `spack.fetch_strategy.`**GoFetchStrategy**(***kwargs*)
Bases: *spack.fetch_strategy.VCSFetchStrategy*

Fetch strategy that employs the *go get* infrastructure Use like this in a package:

version('name', go='github.com/monochromegane/the_platinum_searcher/...')

Go get does not natively support versions, they can be faked with git

archive (*destination*)

enabled = True

fetch (**args*, ***kwargs*)

go

go_version

required_attributes = ('go',)

reset (**args*, ***kwargs*)

class `spack.fetch_strategy.`**HgFetchStrategy**(***kwargs*)
Bases: *spack.fetch_strategy.VCSFetchStrategy*

Fetch strategy that gets source code from a Mercurial repository. Use like this in a package:

version('name', hg='https://jay.grs.rwth-aachen.de/hg/lwm2')

Optionally, you can provide a branch, or revision to check out, e.g.:

version('torus', hg='https://jay.grs.rwth-aachen.de/hg/lwm2', branch='torus')

You can use the optional 'revision' attribute to check out a branch, tag, or particular revision in hg. To prevent non-reproducible builds, using a moving target like a branch is discouraged.

- `revision`: Particular revision, branch, or tag.

archive (*destination*)

enabled = True

fetch (**args*, ***kwargs*)

hg

required_attributes = ['hg']

reset (**args*, ***kwargs*)

exception spack.fetch_strategy.**InvalidArgsError**(*pkg*, *version*)
Bases: *spack.fetch_strategy.FetchError*

exception spack.fetch_strategy.**NoArchiveFileError**(*msg*, *long_msg*)
Bases: *spack.fetch_strategy.FetchError*

exception spack.fetch_strategy.**NoDigestError**(*msg*, *long_msg=None*)
Bases: *spack.fetch_strategy.FetchError*

exception spack.fetch_strategy.**NoStageError**(*method*)
Bases: *spack.fetch_strategy.FetchError*

Raised when fetch operations are called before set_stage().

class spack.fetch_strategy.**SvnFetchStrategy**(***kwargs*)
Bases: *spack.fetch_strategy.VCSFetchStrategy*

Fetch strategy that gets source code from a subversion repository. Use like this in a package:

version('name', svn='http://www.example.com/svn/trunk')

Optionally, you can provide a revision for the URL:

version('name', svn='http://www.example.com/svn/trunk', revision='1641')

archive(*destination*)

enabled = True

fetch(**args*, ***kwargs*)

required_attributes = ['svn']

reset(**args*, ***kwargs*)

svn

class spack.fetch_strategy.**URLFetchStrategy**(*url=None*, *digest=None*, ***kwargs*)
Bases: *spack.fetch_strategy.FetchStrategy*

FetchStrategy that pulls source code from a URL for an archive, checks the archive against a checksum,and decompresses the archive.

archive(*destination*)
Just moves this archive to the destination.

archive_file
Path to the source archive within this stage directory.

check(**args*, ***kwargs*)
Check the downloaded archive against a checksum digest. No-op if this stage checks code out of a repository.

enabled = True

expand(**args*, ***kwargs*)

fetch(**args*, ***kwargs*)

required_attributes = ['url']

reset(**args*, ***kwargs*)
Removes the source path if it exists, then re-expands the archive.

class spack.fetch_strategy.**VCSFetchStrategy**(*name*, **rev_types*, ***kwargs*)
Bases: *spack.fetch_strategy.FetchStrategy*

archive(**args*, ***kwargs*)

check(**args*, ***kwargs*)

expand(**args*, ***kwargs*)

spack.fetch_strategy.**args_are_for**(*args*, *fetcher*)

spack.fetch_strategy.**for_package_version**(*pkg*, *version*)
Determine a fetch strategy based on the arguments supplied to version() in the package description.

spack.fetch_strategy.**from_kwargs**(***kwargs*)
Construct the appropriate FetchStrategy from the given keyword arguments.

Parameters **kwargs** – dictionary of keyword arguments

Returns fetcher or raise a FetchError exception

spack.fetch_strategy.**from_url**(*url*)
Given a URL, find an appropriate fetch strategy for it. Currently just gives you a URLFetchStrategy that uses curl.

TODO: make this return appropriate fetch strategies for other types of URLs.

12.15 spack.file_cache module

exception spack.file_cache.**CacheError**(*message*, *long_message=None*)
Bases: *spack.error.SpackError*

class spack.file_cache.**FileCache**(*root*)
Bases: object

This class manages cached data in the filesystem.

- Cache files are fetched and stored by unique keys. Keys can be relative paths, so that thre can be some hierarchy in the cache.
- The FileCache handles locking cache files for reading and writing, so client code need not manage locks for cache entries.

cache_path(*key*)
Path to the file in the cache for a particular key.

destroy()
Remove all files under the cache root.

init_entry(*key*)
Ensure we can access a cache file. Create a lock for it if needed.

Return whether the cache file exists yet or not.

mtime(*key*)
Return modification time of cache file, or 0 if it does not exist.

Time is in units returned by os.stat in the mtime field, which is platform-dependent.

read_transaction(*key*)
Get a read transaction on a file cache item.

Returns a ReadTransaction context manager and opens the cache file for reading. You can use it like this:

with spack.user_cache.read_transaction(key) as cache_file: cache_file.read()

remove(*key*)

write_transaction(*key*)
Get a write transaction on a file cache item.

Returns a WriteTransaction context manager that opens a temporary file for writing. Once the context manager finishes, if nothing went wrong, moves the file into place on top of the old file atomically.

12.16 spack.graph module

Functions for graphing DAGs of dependencies.

This file contains code for graphing DAGs of software packages (i.e. Spack specs). There are two main functions you probably care about:

graph_ascii() will output a colored graph of a spec in ascii format, kind of like the graph git shows with "git log –graph", e.g.:

```
o  mpileaks
|    | |    | o |  callpath
|/| |
| |\|
| |\     | | |\     | | | | o  adept-utils
| |_|_|/|
|/| | | |
o | | | |  mpi
 / / / /
| | o |  dyninst
| |/| |
|/|/| |
| | |/
| o |  libdwarf
|/ /
o |  libelf
 /
o  boost
```

graph_dot() will output a graph of a spec (or multiple specs) in dot format.

Note that `graph_ascii` assumes a single spec while `graph_dot` can take a number of specs as input.

`spack.graph.`**topological_sort**(*spec*, *reverse=False*, *deptype=None*)
Topological sort for specs.

Return a list of dependency specs sorted topologically. The spec argument is not modified in the process.

`spack.graph.`**graph_ascii**(*spec*, *node='o'*, *out=None*, *debug=False*, *indent=0*, *color=None*, *deptype=None*)

class `spack.graph.`**AsciiGraph**
Bases: `object`

write(*spec*, ***kwargs*)
Write out an ascii graph of the provided spec.

Arguments: spec – spec to graph. This only handles one spec at a time.

Optional arguments:

out – file object to write out to (default is sys.stdout)

color – whether to write in color. Default is to autodetect based on output file.

spack.graph.**graph_dot**(*specs*, *deptype=None*, *static=False*, *out=None*)
Generate a graph in dot format of all provided specs.

Print out a dot formatted graph of all the dependencies between package. Output can be passed to graphviz, e.g.:

spack graph –dot qt | dot -Tpdf > spack-graph.pdf

12.17 spack.mirror module

This file contains code for creating spack mirror directories. A mirror is an organized hierarchy containing specially named archive files. This enabled spack to know where to find files in a mirror if the main server for a particular package is down. Or, if the computer where spack is run is not connected to the internet, it allows spack to download packages directly from a mirror (e.g., on an intranet).

exception spack.mirror.**MirrorError**(*msg*, *long_msg=None*)
Bases: *spack.error.SpackError*

Superclass of all mirror-creation related errors.

spack.mirror.**add_single_spec**(*spec*, *mirror_root*, *categories*, ***kwargs*)

spack.mirror.**create**(*path*, *specs*, ***kwargs*)
Create a directory to be used as a spack mirror, and fill it with package archives.

Arguments: path: Path to create a mirror directory hierarchy in. specs: Any package versions matching these specs will be added to the mirror.

Keyword args: no_checksum: If True, do not checkpoint when fetching (default False) num_versions: Max number of versions to fetch per spec, if spec is ambiguous (default is 0 for all of them)

Return Value: Returns a tuple of lists: (present, mirrored, error)

- present: Package specs that were already present.
- mirrored: Package specs that were successfully mirrored.
- error: Package specs that failed to mirror due to some error.

This routine iterates through all known package versions, and it creates specs for those versions. If the version satisfies any spec in the specs list, it is downloaded and added to the mirror.

spack.mirror.**get_matching_versions**(*specs*, ***kwargs*)
Get a spec for EACH known version matching any spec in the list.

spack.mirror.**mirror_archive_filename**(*spec*, *fetcher*)
Get the name of the spec's archive in the mirror.

spack.mirror.**mirror_archive_path**(*spec*, *fetcher*)
Get the relative path to the spec's archive within a mirror.

spack.mirror.**suggest_archive_basename**(*resource*)
Return a tentative basename for an archive.

Raises an exception if the name is not an allowed archive type.

Parameters **fetcher** –

Returns

12.18 spack.modules module

This module contains code for creating environment modules, which can include dotkits, tcl modules, lmod, and others.

The various types of modules are installed by post-install hooks and removed after an uninstall by post-uninstall hooks. This class consolidates the logic for creating an abstract description of the information that module systems need.

This module also includes logic for coming up with unique names for the module files so that they can be found by the various shell-support files in $SPACK/share/spack/setup-env.*.

Each hook in hooks/ implements the logic for writing its specific type of module file.

class spack.modules.**EnvModule**(*spec=None*)
Bases: object

autoload(*spec*)

blacklisted

category

file_name
Subclasses should implement this to return the name of the file where this module lives.

formats = {}

header

module_specific_content(*configuration*)

name = 'env_module'

naming_scheme

prerequisite(*spec*)

process_environment_command(*env*)

remove()

tokens
Tokens that can be substituted in environment variable values and naming schemes

upper_tokens
Tokens that can be substituted in environment variable names

use_name
Subclasses should implement this to return the name the module command uses to refer to the package.

write(*overwrite=False*)
Writes out a module file for this object.

This method employs a template pattern and expects derived classes to: - override the header property - provide formats for autoload, prerequisites and environment changes

class spack.modules.**Dotkit**(*spec=None*)
Bases: *spack.modules.EnvModule*

autoload_format = 'dk_op {module_file}\n'

default_naming_format = '{name}-{version}-{compiler.name}-{compiler.version}'

environment_modifications_formats = {<class 'spack.environment.RemovePath'>: 'dk_unalter {name} {value}

file_name

header

name = 'dotkit'

path = '/home/docs/checkouts/readthedocs.org/user_builds/spack-citibeth-fork/checkouts/efischer-docs2/share/spack/dotl

prerequisite(*spec*)

class spack.modules.**TclModule**(*spec=None*)
Bases: *spack.modules.EnvModule*

autoload_format = 'if ![is-loaded {module_file}] {{\n puts stderr "Autoloading {module_file}"\n module load {module

default_naming_format = '{name}-{version}-{compiler.name}-{compiler.version}'

file_name

header

module_specific_content(*configuration*)

name = 'tcl'

path = '/home/docs/checkouts/readthedocs.org/user_builds/spack-citibeth-fork/checkouts/efischer-docs2/share/spack/mod

prerequisite_format = 'prereq {module_file}\n'

process_environment_command(*env*)

12.19 spack.multimethod module

This module contains utilities for using multi-methods in spack. You can think of multi-methods like overloaded methods – they're methods with the same name, and we need to select a version of the method based on some criteria. e.g., for overloaded methods, you would select a version of the method to call based on the types of its arguments.

In spack, multi-methods are used to ease the life of package authors. They allow methods like install() (or other methods called by install()) to declare multiple versions to be called when the package is instantiated with different specs. e.g., if the package is built with OpenMPI on x86_64,, you might want to call a different install method than if it was built for mpich2 on BlueGene/Q. Likewise, you might want to do a different type of install for different versions of the package.

Multi-methods provide a simple decorator-based syntax for this that avoids overly complicated rat nests of if statements. Obviously, depending on the scenario, regular old conditionals might be clearer, so package authors should use their judgement.

exception spack.multimethod.**MultiMethodError**(*message*)
Bases: *spack.error.SpackError*

Superclass for multimethod dispatch errors

exception spack.multimethod.**NoSuchMethodError**(*cls*, *method_name*, *spec*, *possible_specs*)
Bases: *spack.error.SpackError*

Raised when we can't find a version of a multi-method.

class spack.multimethod.**SpecMultiMethod**(*default=None*)
Bases: object

This implements a multi-method for Spack specs. Packages are instantiated with a particular spec, and you may want to execute different versions of methods based on what the spec looks like. For example, you might want to call a different version of install() for one platform than you call on another.

The SpecMultiMethod class implements a callable object that handles method dispatch. When it is called, it looks through registered methods and their associated specs, and it tries to find one that matches the package's spec. If it finds one (and only one), it will call that method.

The package author is responsible for ensuring that only one condition on multi-methods ever evaluates to true. If multiple methods evaluate to true, this will raise an exception.

This is intended for use with decorators (see below). The decorator (see docs below) creates SpecMultiMethods and registers method versions with them.

To register a method, you can do something like this: mm = SpecMultiMethod()
mm.register("^chaos_5_x86_64_ib", some_method)

The object registered needs to be a Spec or some string that will parse to be a valid spec.

When the mm is actually called, it selects a version of the method to call based on the sys_type of the object it is called on.

See the docs for decorators below for more details.

register(*spec*, *method*)
Register a version of a method for a particular sys_type.

class spack.multimethod.**when**(*spec*)
Bases: object

This annotation lets packages declare multiple versions of methods like install() that depend on the package's spec. For example:

```
class SomePackage(Package):
    ...

    def install(self, prefix):
        # Do default install

    @when('arch=chaos_5_x86_64_ib')
    def install(self, prefix):
        # This will be executed instead of the default install if
        # the package's platform() is chaos_5_x86_64_ib.

    @when('arch=bgqos_0")
    def install(self, prefix):
        # This will be executed if the package's sys_type is bgqos_0
```

This allows each package to have a default version of install() AND specialized versions for particular platforms. The version that is called depends on the architecutre of the instantiated package.

Note that this works for methods other than install, as well. So, if you only have part of the install that is platform specific, you could do this:

```
class SomePackage(Package):
    ...
    # virtual dependence on MPI.
    # could resolve to mpich, mpich2, OpenMPI
    depends_on('mpi')

    def setup(self):
        # do nothing in the default case
        pass

    @when('^openmpi')
    def setup(self):
```

```
        # do something special when this is built with OpenMPI for
        # its MPI implementations.


    def install(self, prefix):
        # Do common install stuff
        self.setup()
        # Do more common install stuff
```

There must be one (and only one) @when clause that matches the package's spec. If there is more than one, or if none match, then the method will raise an exception when it's called.

Note that the default version of decorated methods must *always* come first. Otherwise it will override all of the platform-specific versions. There's not much we can do to get around this because of the way decorators work.

12.20 spack.package module

This is where most of the action happens in Spack. See the Package docs for detailed instructions on how the class works and on how to write your own packages.

The spack package structure is based strongly on Homebrew (http://wiki.github.com/mxcl/homebrew/), mainly because Homebrew makes it very easy to create packages. For a complete rundown on spack and how it differs from homebrew, look at the README.

exception spack.package.**ActivationError**(*msg*, *long_msg=None*)
Bases: *spack.package.ExtensionError*

class spack.package.**CMakePackage**(*spec*)
Bases: *spack.package.StagedPackage*

configure_args()
Returns package-specific arguments to be provided to the configure command.

configure_env()
Returns package-specific environment under which the configure command should be run.

install_build()

install_configure()

install_install()

install_setup()

make_make()

transitive_inc_path()

exception spack.package.**DependencyConflictError**(*conflict*)
Bases: *spack.error.SpackError*

Raised when the dependencies cannot be flattened as asked for.

exception spack.package.**ExtensionConflictError**(*path*)
Bases: *spack.package.ExtensionError*

exception spack.package.**ExtensionError**(*message*, *long_msg=None*)
Bases: *spack.package.PackageError*

exception spack.package.**ExternalPackageError**(*message*, *long_msg=None*)
Bases: *spack.package.InstallError*

Raised by install() when a package is only for external use.

exception spack.package.**FetchError**(*message*, *long_msg=None*)
Bases: *spack.error.SpackError*

Raised when something goes wrong during fetch.

exception spack.package.**InstallError**(*message*, *long_msg=None*)
Bases: *spack.error.SpackError*

Raised when something goes wrong during install or uninstall.

exception spack.package.**NoURLError**(*cls*)
Bases: *spack.package.PackageError*

Raised when someone tries to build a URL for a package with no URLs.

class spack.package.**Package**(*spec*)
Bases: object

This is the superclass for all spack packages.

The Package class

Package is where the bulk of the work of installing packages is done.

A package defines how to fetch, verfiy (via, e.g., md5), build, and install a piece of software. A Package also defines what other packages it depends on, so that dependencies can be installed along with the package itself. Packages are written in pure python.

Packages are all submodules of spack.packages. If spack is installed in `$prefix`, all of its python files are in `$prefix/lib/spack`. Most of them are in the spack module, so all the packages live in `$prefix/lib/spack/spack/packages`.

All you have to do to create a package is make a new subclass of Package in this directory. Spack automatically scans the python files there and figures out which one to import when you invoke it.

An example package

Let's look at the cmake package to start with. This package lives in `$prefix/var/spack/repos/builtin/packages/cmake/package.py`:

```
from spack import *
class Cmake(Package):
    homepage  = 'https://www.cmake.org'
    url       = 'http://www.cmake.org/files/v2.8/cmake-2.8.10.2.tar.gz'
    md5       = '097278785da7182ec0aea8769d06860c'

    def install(self, spec, prefix):
        configure('--prefix=%s'   % prefix,
                  '--parallel=%s' % make_jobs)
        make()
        make('install')
```

Naming conventions

There are two names you should care about:

1. The module name, `cmake`.
   - User will refers to this name, e.g. 'spack install cmake'.
   - It can include _, −, and numbers (it can even start with a number).

2.The class name, "Cmake". This is formed by converting - or _ in the module name to camel case. If the name starts with a number, we prefix the class name with _. Examples:

Module Name	Class Name
foo_bar	FooBar
docbook-xml	DocbookXml
FooBar	Foobar
3proxy	_3proxy

The class name is what spack looks for when it loads a package module.

Required Attributes

Aside from proper naming, here is the bare minimum set of things you need when you make a package:

homepage: informational URL, so that users know what they're installing.

url or url_for_version(self, version): If url, then the URL of the source archive that spack will fetch. If url_for_version(), then a method returning the URL required to fetch a particular version.

install(): This function tells spack how to build and install the software it downloaded.

Optional Attributes

You can also optionally add these attributes, if needed:

list_url: Webpage to scrape for available version strings. Default is the directory containing the tarball; use this if the default isn't correct so that invoking 'spack versions' will work for this package.

url_version(self, version): When spack downloads packages at particular versions, it just converts version to string with str(version). Override this if your package needs special version formatting in its URL. boost is an example of a package that needs this.

Creating Packages

As a package creator, you can probably ignore most of the preceding information, because you can use the 'spack create' command to do it all automatically.

You as the package creator generally only have to worry about writing your install function and specifying dependencies.

spack create

Most software comes in nicely packaged tarballs, like this one

http://www.cmake.org/files/v2.8/cmake-2.8.10.2.tar.gz

Taking a page from homebrew, spack deduces pretty much everything it needs to know from the URL above. If you simply type this:

```
spack create http://www.cmake.org/files/v2.8/cmake-2.8.10.2.tar.gz
```

Spack will download the tarball, generate an md5 hash, figure out the version and the name of the package from the URL, and create a new package file for you with all the names and attributes set correctly.

Once this skeleton code is generated, spack pops up the new package in your $EDITOR so that you can modify the parts that need changes.

Dependencies

If your package requires another in order to build, you can specify that like this:

```
class Stackwalker(Package):
    ...
    depends_on("libdwarf")
    ...
```

This tells spack that before it builds stackwalker, it needs to build the libdwarf package as well. Note that this is the module name, not the class name (The class name is really only used by spack to find your package).

Spack will download an install each dependency before it installs your package. In addtion, it will add -L, -I, and rpath arguments to your compiler and linker for each dependency. In most cases, this allows you to avoid specifying any dependencies in your configure or cmake line; you can just run configure or cmake without any additional arguments and it will find the dependencies automatically.

The Install Function

The install function is designed so that someone not too terribly familiar with Python could write a package installer. For example, we put a number of commands in install scope that you can use almost like shell commands. These include make, configure, cmake, rm, rmtree, mkdir, mkdirp, and others.

You can see above in the cmake script that these commands are used to run configure and make almost like they're used on the command line. The only difference is that they are python function calls and not shell commands.

It may be puzzling to you where the commands and functions in install live. They are NOT instance variables on the class; this would require us to type 'self.' all the time and it makes the install code unnecessarily long. Rather, spack puts these commands and variables in *module* scope for your Package subclass. Since each package has its own module, this doesn't pollute other namespaces, and it allows you to more easily implement an install function.

For a full list of commands and variables available in module scope, see the add_commands_to_module() function in this class. This is where most of them are created and set on the module.

Parallel Builds

By default, Spack will run make in parallel when you run make() in your install function. Spack figures out how many cores are available on your system and runs make with -j<cores>. If you do not want this behavior, you can explicitly mark a package not to use parallel make:

```
class SomePackage(Package):
    ...
    parallel = False
    ...
```

This changes thd default behavior so that make is sequential. If you still want to build some parts in parallel, you can do this in your install function:

```
make(parallel=True)
```

Likewise, if you do not supply parallel = True in your Package, you can keep the default parallel behavior and run make like this when you want a sequential build:

```
make(parallel=False)
```

Package Lifecycle

This section is really only for developers of new spack commands.

A package's lifecycle over a run of Spack looks something like this:

```
p = Package()             # Done for you by spack

p.do_fetch()              # downloads tarball from a URL
```

```
p.do_stage()              # expands tarball in a temp directory
p.do_patch()              # applies patches to expanded source
p.do_install()            # calls package's install() function
p.do_uninstall()          # removes install directory
```

There are also some other commands that clean the build area:

```
p.do_clean()              # removes the stage directory entirely
p.do_restage()            # removes the build directory and
                          # re-expands the archive.
```

The convention used here is that a `do_*` function is intended to be called internally by Spack commands (in spack.cmd). These aren't for package writers to override, and doing so may break the functionality of the Package class.

Package creators override functions like install() (all of them do this), clean() (some of them do this), and others to provide custom behavior.

activate(*extension*, ***kwargs*)
Symlinks all files from the extension into extendee's install dir.

Package authors can override this method to support other extension mechanisms. Spack internals (commands, hooks, etc.) should call do_activate() method so that proper checks are always executed.

activated

all_urls

build_log_path

compiler
Get the spack.compiler.Compiler object used to build this package

deactivate(*extension*, ***kwargs*)
Unlinks all files from extension out of this package's install dir.

Package authors can override this method to support other extension mechanisms. Spack internals (commands, hooks, etc.) should call do_deactivate() method so that proper checks are always executed.

dependencies = {}

dependencies_of_type(**deptypes*)
Get subset of the dependencies with certain types.

do_activate(*force=False*)
Called on an extension to invoke the extendee's activate method.

Commands should call this routine, and should not call activate() directly.

do_clean()
Removes the package's build stage and source tarball.

do_deactivate(***kwargs*)
Called on the extension to invoke extendee's deactivate() method.

do_fake_install()
Make a fake install directory contaiing a 'fake' file in bin.

do_fetch(*mirror_only=False*)
Creates a stage directory and downloads the tarball for this package. Working directory will be set to the stage directory.

do_install(*keep_prefix=False*, *keep_stage=False*, *ignore_deps=False*, *skip_patch=False*, *verbose=False*, *make_jobs=None*, *run_tests=False*, *fake=False*, *explicit=False*, *dirty=False*, *install_phases=set(['provenance', 'configure', 'install', 'build'])*)
Called by commands to install a package and its dependencies.

Package implementations should override install() to describe their build process.

Parameters

- **keep_prefix** – Keep install prefix on failure. By default, destroys it.
- **keep_stage** – By default, stage is destroyed only if there are no exceptions during build. Set to True to keep the stage even with exceptions.
- **ignore_deps** – Don't install dependencies before installing this package
- **fake** – Don't really build; install fake stub files instead.
- **skip_patch** – Skip patch stage of build if True.
- **verbose** – Display verbose build output (by default, suppresses it)
- **dirty** – Don't clean the build environment before installing.
- **make_jobs** – Number of make jobs to use for install. Default is ncpus
- **run_tests** – Run tests within the package's install()

do_install_dependencies(***kwargs*)

do_patch()
Calls do_stage(), then applied patches to the expanded tarball if they haven't been applied already.

do_restage()
Reverts expanded/checked out source to a pristine state.

do_stage(*mirror_only=False*)
Unpacks the fetched tarball, then changes into the expanded tarball directory.

do_uninstall(*force=False*)

extendable = False
List of prefix-relative file paths (or a single path). If these do not exist after install, or if they exist but are not files, sanity checks fail.

extendee_args
Spec of the extendee of this package, or None if it is not an extension

extendee_spec
Spec of the extendee of this package, or None if it is not an extension

extendees = {}

extends(*spec*)

fetch_remote_versions()
Try to find remote versions of this package using the list_url and any other URLs described in the package file.

fetcher

format_doc(***kwargs*)
Wrap doc string at 72 characters and format nicely

global_license_dir
Returns the directory where global license files for all packages are stored.

global_license_file
Returns the path where a global license file for this particular package should be stored.

install(*spec*, *prefix*)
Package implementations override this with their own configuration

install_phases = set(['provenance', 'configure', 'install', 'build'])

installed

installed_dependents

Return a list of the specs of all installed packages that depend on this one.

TODO: move this method to database.py?

is_extension

make_jobs = None
By default do not run tests within package's install()

module
Use this to add variables to the class's module's scope. This lets us use custom syntax in the install method.

namespace

nearest_url(*version*)
Finds the URL for the next lowest version with a URL. If there is no lower version with a URL, uses the package url property. If that isn't there, uses a *higher* URL, and if that isn't there raises an error.

package_dir
Return the directory where the package.py file lives.

parallel = True
jobs to use for parallel make. If set, overrides default of ncpus.

patches = {}

possible_dependencies(*visited=None*)
Return set of possible transitive dependencies of this package.

prefix
Get the prefix into which this package should be installed.

provided = {}

provides(*vpkg_name*)
True if this package provides a virtual package with the specified name

remove_prefix()
Removes the prefix for a package along with any empty parent directories

resources = {}

rpath
Get the rpath this package links with, as a list of paths.

rpath_args
Get the rpath args as a string, with -Wl,-rpath, for each element

run_tests = False
Most packages are NOT extendable. Set to True if you want extensions.

sanity_check_is_dir = []

sanity_check_is_file = []
List of prefix-relative directory paths (or a single path). If these do not exist after install, or if they exist but are not directories, sanity checks will fail.

sanity_check_prefix()
This function checks whether install succeeded.

setup_dependent_environment(*spack_env*, *run_env*, *dependent_spec*)
Set up the environment of packages that depend on this one.

This is similar to `setup_environment`, but it is used to modify the compile and runtime environments of packages that *depend* on this one. This gives packages like Python and others that follow the extension model a way to implement common environment or compile-time settings for dependencies.

By default, this delegates to `self.setup_environment()`

Example:

1. Installing python modules generally requires *PYTHONPATH* to point to the lib/pythonX.Y/site-packages directory in the module's install prefix. This could set that variable.

Args:

spack_env (EnvironmentModifications): list of modifications to be applied when the dependent package is bulit within Spack.

run_env (EnvironmentModifications): list of environment changes to be applied when the dependent package is run outside of Spack.

dependent_spec (Spec): The spec of the dependent package about to be built. This allows the extendee (self) to query the dependent's state. Note that *this* package's spec is available as *self.spec*.

This is useful if there are some common steps to installing all extensions for a certain package.

setup_dependent_package(*module*, *dependent_spec*)
Set up Python module-scope variables for dependent packages.

Called before the install() method of dependents.

Default implementation does nothing, but this can be overridden by an extendable package to set up the module of its extensions. This is useful if there are some common steps to installing all extensions for a certain package.

Example :

1. Extensions often need to invoke the *python* interpreter from the Python installation being extended. This routine can put a 'python' Executable object in the module scope for the extension package to simplify extension installs.
2. MPI compilers could set some variables in the dependent's scope that point to *mpicc*, *mpicxx*, etc., allowing them to be called by common names regardless of which MPI is used.
3. BLAS/LAPACK implementations can set some variables indicating the path to their libraries, since these paths differ by BLAS/LAPACK implementation.

Args:

module (module): The Python *module* object of the dependent package. Packages can use this to set module-scope variables for the dependent to use.

dependent_spec (Spec): The spec of the dependent package about to be built. This allows the extendee (self) to query the dependent's state. Note that *this* package's spec is available as *self.spec*.

This is useful if there are some common steps to installing all extensions for a certain package.

setup_environment(*spack_env*, *run_env*)
Set up the compile and runtime environments for a package.

spack_env and *run_env* are *EnvironmentModifications* objects. Package authors can call methods on them to alter the environment within Spack and at runtime.

Both *spack_env* and *run_env* are applied within the build process, before this package's *install()* method is called.

Modifications in *run_env* will *also* be added to the generated environment modules for this package.

Default implementation does nothing, but this can be overridden if the package needs a particular environment.

Examples:

1. Qt extensions need *QTDIR* set.

Args:

spack_env (EnvironmentModifications): list of modifications to be applied when this package is built within Spack.

run_env (EnvironmentModifications): list of environment changes to be applied when this package is run outside of Spack.

stage

url_for_version(*version*)
Returns a URL from which the specified version of this package may be downloaded.

version: class Version The version for which a URL is sought.

See Class Version (version.py)

url_version(*version*)
Given a version, this returns a string that should be substituted into the package's URL to download that version.

By default, this just returns the version string. Subclasses may need to override this, e.g. for boost versions where you need to ensure that there are _'s in the download URL.

variants = {}

version

version_urls = <functools.partial object>

versions = {}

exception spack.package.**PackageError**(*message*, *long_msg=None*)
Bases: *spack.error.SpackError*

Raised when something is wrong with a package definition.

exception spack.package.**PackageStillNeededError**(*spec*, *dependents*)
Bases: *spack.package.InstallError*

Raised when package is still needed by another on uninstall.

exception spack.package.**PackageVersionError**(*version*)
Bases: *spack.package.PackageError*

Raised when a version URL cannot automatically be determined.

class spack.package.**StagedPackage**(*spec*)
Bases: *spack.package.Package*

A Package subclass where the install() is split up into stages.

install(*spec*, *prefix*)

install_build()
Runs the build process.

install_configure()
Runs the configure process.

install_install()
Runs the install process.

install_setup()
Creates a spack_setup.py script to configure the package later.

exception spack.package.**VersionFetchError**(*cls*)
Bases: *spack.package.PackageError*

Raised when a version URL cannot automatically be determined.

spack.package.**dump_packages**(*spec*, *path*)
Dump all package information for a spec and its dependencies.

This creates a package repository within path for every namespace in the spec DAG, and fills the repos wtih package files and patch files for every node in the DAG.

spack.package.**flatten_dependencies**(*spec*, *flat_dir*)
Make each dependency of spec present in dir via symlink.

spack.package.**install_dependency_symlinks**(*pkg*, *spec*, *prefix*)
Execute a dummy install and flatten dependencies

spack.package.**make_executable**(*path*)

spack.package.**print_pkg**(*message*)
Outputs a message with a package icon.

spack.package.**use_cray_compiler_names**()
Compiler names for builds that rely on cray compiler names.

spack.package.**validate_package_url**(*url_string*)
Determine whether spack can handle a particular URL or not.

12.21 spack.package_test module

spack.package_test.**compare_output**(*current_output*, *blessed_output*)
Compare blessed and current output of executables.

spack.package_test.**compare_output_file**(*current_output*, *blessed_output_file*)
Same as above, but when the blessed output is given as a file.

spack.package_test.**compile_c_and_execute**(*source_file*, *include_flags*, *link_flags*)
Compile C @p source_file with @p include_flags and @p link_flags, run and return the output.

12.22 spack.parse module

exception spack.parse.**LexError**(*message*, *string*, *pos*)
Bases: *spack.parse.ParseError*

Raised when we don't know how to lex something.

class spack.parse.**Lexer**(*lexicon*)
Bases: object

Base class for Lexers that keep track of line numbers.

lex(*text*)

token(*type*, *value=''*)

exception spack.parse.**ParseError**(*message*, *string*, *pos*)
Bases: *spack.error.SpackError*

Raised when we don't hit an error while parsing.

class spack.parse.**Parser**(*lexer*)
Bases: object

Base class for simple recursive descent parsers.

accept(*id*)
Put the next symbol in self.token if accepted, then call gettok()

expect(*id*)
Like accept(), but fails if we don't like the next token.

gettok()
Puts the next token in the input stream into self.next.

last_token_error(*message*)
Raise an error about the previous token in the stream.

next_token_error(*message*)
Raise an error about the next token in the stream.

parse(*text*)

push_tokens(*iterable*)
Adds all tokens in some iterable to the token stream.

setup(*text*)

unexpected_token()

class spack.parse.**Token**(*type*, *value=''*, *start=0*, *end=0*)
Represents tokens; generated from input by lexer and fed to parse().

is_a(*type*)

12.23 spack.patch module

exception spack.patch.**NoSuchPatchFileError**(*package*, *path*)
Bases: *spack.error.SpackError*

Raised when user specifies a patch file that doesn't exist.

class spack.patch.**Patch**(*pkg*, *path_or_url*, *level*)
Bases: object

This class describes a patch to be applied to some expanded source code.

apply(*stage*)
Fetch this patch, if necessary, and apply it to the source code in the supplied stage.

12.24 spack.preferred_packages module

class spack.preferred_packages.**PreferredPackages**
Bases: object

architecture_compare(*pkgname*, *a*, *b*)
Return less-than-0, 0, or greater than 0 if architecture a of pkgname is respectively less-than, equal-to, or greater-than architecture b of pkgname. One architecture is less-than another if it is preferred over the other.

compiler_compare(*pkgname*, *a*, *b*)
Return less-than-0, 0, or greater than 0 if compiler a of pkgname is respecively less-than, equal-to, or greater-than compiler b of pkgname. One compiler is less-than another if it is preferred over the other.

provider_compare(*pkgname*, *provider_str*, *a*, *b*)
Return less-than-0, 0, or greater than 0 if a is respecively less-than, equal-to, or greater-than b. A and b are possible implementations of provider_str. One provider is less-than another if it is preferred over the other. For example, provider_compare('scorep', 'mpi', 'mvapich', 'openmpi') would return -1 if mvapich should be preferred over openmpi for scorep.

spec_has_preferred_provider(*pkgname*, *provider_str*)
Return True iff the named package has a list of preferred providers

spec_preferred_variants(*pkgname*)
Return a VariantMap of preferred variants and their values

variant_compare(*pkgname*, *a*, *b*)
Return less-than-0, 0, or greater than 0 if variant a of pkgname is respectively less-than, equal-to, or greater-than variant b of pkgname. One variant is less-than another if it is preferred over the other.

version_compare(*pkgname*, *a*, *b*)
Return less-than-0, 0, or greater than 0 if version a of pkgname is respectively less-than, equal-to, or greater-than version b of pkgname. One version is less-than another if it is preferred over the other.

12.25 spack.provider_index module

The virtual module contains utility classes for virtual dependencies.

class spack.provider_index.**ProviderIndex**(*specs=None*, *restrict=False*)
Bases: object

This is a dict of dicts used for finding providers of particular virtual dependencies. The dict of dicts looks like:

{ vpkg name : { full vpkg spec : set(packages providing spec) } }

Callers can use this to first find which packages provide a vpkg, then find a matching full spec. e.g., in this scenario:

{ 'mpi' :

{ **mpi@:1.1** [set([mpich]),] mpi@:2.3 : set([mpich2@1.9:]) } }

Calling providers_for(spec) will find specs that provide a matching implementation of MPI.

copy()
Deep copy of this ProviderIndex.

static from_yaml(*stream*)

merge(*other*)
Merge *other* ProviderIndex into this one.

providers_for(**vpkg_specs*)
Gives specs of all packages that provide virtual packages with the supplied specs.

remove_provider(*pkg_name*)
Remove a provider from the ProviderIndex.

satisfies(*other*)
Check that providers of virtual specs are compatible.

to_yaml(*stream=None*)

update(*spec*)

exception spack.provider_index.**ProviderIndexError**(*message*, *long_message=None*)
Bases: *spack.error.SpackError*

Raised when there is a problem with a ProviderIndex.

12.26 spack.repository module

exception spack.repository.**BadRepoError**(*message*, *long_message=None*)
Bases: *spack.repository.RepoError*

Raised when repo layout is invalid.

exception spack.repository.**DuplicateRepoError**(*message*, *long_message=None*)
Bases: *spack.repository.RepoError*

Raised when duplicate repos are added to a RepoPath.

exception spack.repository.**FailedConstructorError**(*name*, *exc_type*, *exc_obj*, *exc_tb*)
Bases: *spack.repository.PackageLoadError*

Raised when a package's class constructor fails.

exception spack.repository.**InvalidNamespaceError**(*message*, *long_message=None*)
Bases: *spack.repository.RepoError*

Raised when an invalid namespace is encountered.

exception spack.repository.**NoRepoConfiguredError**(*message*, *long_message=None*)
Bases: *spack.repository.RepoError*

Raised when there are no repositories configured.

exception spack.repository.**PackageLoadError**(*message*, *long_message=None*)
Bases: *spack.error.SpackError*

Superclass for errors related to loading packages.

class spack.repository.**Repo**(*root*, *namespace='spack.pkg'*)
Bases: object

Class representing a package repository in the filesystem.

Each package repository must have a top-level configuration file called *repo.yaml*.

Currently, *repo.yaml* this must define:

***namespace*:** A Python namespace where the repository's packages should live.

all_package_names()
Returns a sorted list of all package names in the Repo.

all_packages()
Iterator over all packages in the repository.

Use this with care, because loading packages is slow.

dirname_for_package_name(*spec_like*, **args*, ***kwargs*)

dump_provenance(*spec_like*, **args*, ***kwargs*)

exists(*pkg_name*)
Whether a package with the supplied name exists.

extensions_for(*spec_like*, **args*, ***kwargs*)

filename_for_package_name(*spec_like*, **args*, ***kwargs*)

find_module(*fullname*, *path=None*)
Python find_module import hook.

Returns this Repo if it can load the module; None if not.

get(*spec_like*, **args*, ***kwargs*)

get_pkg_class(*pkg_name*)
Get the class for the package out of its module.

First loads (or fetches from cache) a module for the package. Then extracts the package class from the module according to Spack's naming convention.

is_prefix(*fullname*)
True if fullname is a prefix of this Repo's namespace.

load_module(*fullname*)
Python importer load hook.

Tries to load the module; raises an ImportError if it can't.

provider_index
A provider index with names *specific* to this repo.

providers_for(*spec_like*, **args*, ***kwargs*)

purge()
Clear entire package instance cache.

real_name(*import_name*)
Allow users to import Spack packages using Python identifiers.

A python identifier might map to many different Spack package names due to hyphen/underscore ambiguity.

Easy example: num3proxy -> 3proxy

Ambiguous: foo_bar -> foo_bar, foo-bar

More ambiguous: foo_bar_baz -> foo_bar_baz, foo-bar-baz, foo_bar-baz, foo-bar_baz

exception spack.repository.**RepoError**(*message*, *long_message=None*)
Bases: *spack.error.SpackError*

Superclass for repository-related errors.

class spack.repository.**RepoPath**(**repo_dirs*, ***kwargs*)
Bases: object

A RepoPath is a list of repos that function as one.

It functions exactly like a Repo, but it operates on the combined results of the Repos in its list instead of on a single package repository.

all_package_names()
Return all unique package names in all repositories.

all_packages()

dirname_for_package_name(*pkg_name*)

dump_provenance(*spec_like*, **args*, ***kwargs*)

exists(*pkg_name*)

extensions_for(*spec_like*, **args*, ***kwargs*)

filename_for_package_name(*pkg_name*)

find_module(*fullname*, *path=None*)
Implements precedence for overlaid namespaces.

Loop checks each namespace in self.repos for packages, and also handles loading empty containing namespaces.

first_repo()
Get the first repo in precedence order.

get(*spec_like*, **args*, ***kwargs*)

get_pkg_class(*pkg_name*)
Find a class for the spec's package and return the class object.

get_repo(*namespace*, *default=<object object>*)
Get a repository by namespace.

Arguments:

namespace:

Look up this namespace in the RepoPath, and return it if found.

Optional Arguments:

default:

If default is provided, return it when the namespace isn't found. If not, raise an UnknownNamespaceError.

load_module(*fullname*)
Handles loading container namespaces when necessary.

See Repo for how actual package modules are loaded.

provider_index
Merged ProviderIndex from all Repos in the RepoPath.

providers_for(*spec_like*, **args*, ***kwargs*)

put_first(*repo*)
Add repo first in the search path.

put_last(*repo*)
Add repo last in the search path.

remove(*repo*)
Remove a repo from the search path.

repo_for_pkg(*spec_like*, **args*, ***kwargs*)

swap(*other*)
Convenience function to make swapping repostiories easier.

This is currently used by mock tests. TODO: Maybe there is a cleaner way.

class spack.repository.**SpackNamespace**(*namespace*)
Bases: module

Allow lazy loading of modules.

exception spack.repository.**UnknownNamespaceError**(*namespace*)
Bases: *spack.repository.PackageLoadError*

Raised when we encounter an unknown namespace

exception spack.repository.**UnknownPackageError**(*name*, *repo=None*)
Bases: *spack.repository.PackageLoadError*

Raised when we encounter a package spack doesn't have.

spack.repository.**canonicalize_path**(*path*)
Substitute $spack, expand user home, take abspath.

spack.repository.**create_repo**(*root*, *namespace=None*)
Create a new repository in root with the specified namespace.

If the namespace is not provided, use basename of root. Return the canonicalized path and namespace of the created repository.

spack.repository.**substitute_spack_prefix**(*path*)
Replaces instances of $spack with Spack's prefix.

12.27 spack.resource module

Describes an optional resource needed for a build.

Typically a bunch of sources that can be built in-tree within another package to enable optional features.

class spack.resource.**Resource**(*name*, *fetcher*, *destination*, *placement*)
Bases: object

Represents an optional resource to be fetched by a package.

Aggregates a name, a fetcher, a destination and a placement.

12.28 spack.spec module

Spack allows very fine-grained control over how packages are installed and over how they are built and configured. To make this easy, it has its own syntax for declaring a dependence. We call a descriptor of a particular package configuration a "spec".

The syntax looks like this:

```
$ spack install mpileaks ^openmpi @1.2:1.4 +debug %intel @12.1 =bgqos_0
                 0        1        2         3      4      5     6
```

The first part of this is the command, 'spack install'. The rest of the line is a spec for a particular installation of the mpileaks package.

0. The package to install
1. A dependency of the package, prefixed by ^
2. A version descriptor for the package. This can either be a specific version, like "1.2", or it can be a range of versions, e.g. "1.2:1.4". If multiple specific versions or multiple ranges are acceptable, they can be separated by commas, e.g. if a package will only build with versions 1.0, 1.2-1.4, and 1.6-1.8 of mavpich, you could say:

   depends_on("mvapich@1.0,1.2:1.4,1.6:1.8")

3. A compile-time variant of the package. If you need openmpi to be built in debug mode for your package to work, you can require it by adding +debug to the openmpi spec when you depend on it. If you do NOT want the debug option to be enabled, then replace this with -debug.
4. The name of the compiler to build with.
5. The versions of the compiler to build with. Note that the identifier for a compiler version is the same '@' that is used for a package version. A version list denoted by '@' is associated with the compiler only if if it comes immediately after the compiler name. Otherwise it will be associated with the current package spec.
6. The architecture to build with. This is needed on machines where cross-compilation is required

Here is the EBNF grammar for a spec:

```
spec-list    = { spec [ dep-list ] }
dep_list     = { ^ spec }
spec         = id [ options ]
options      = { @version-list | +variant | -variant | ~variant |
                 %compiler | arch=architecture | [ flag ]=value}
flag         = { cflags | cxxflags | fcflags | fflags | cppflags |
                 ldflags | ldlibs }
variant      = id
architecture = id
compiler     = id [ version-list ]
version-list = version [ { , version } ]
version      = id | id: | :id | id:id
id           = [A-Za-z0-9_][A-Za-z0-9_.-]*
```

Identifiers using the <name>=<value> command, such as architectures and compiler flags, require a space before the name.

There is one context-sensitive part: ids in versions may contain '.', while other ids may not.

There is one ambiguity: since '-' is allowed in an id, you need to put whitespace space before -variant for it to be tokenized properly. You can either use whitespace, or you can just use ~variant since it means the same thing. Spack uses ~variant in directory names and in the canonical form of specs to avoid ambiguity. Both are provided because ~ can cause shell expansion when it is the first character in an id typed on the command line.

class spack.spec.**Spec**(*spec_like*, **dep_like*, ***kwargs*)
Bases: object

colorized()

common_dependencies(*other*)
Return names of dependencies that self an other have in common.

concrete
A spec is concrete if it can describe only ONE build of a package. If any of the name, version, architecture, compiler, variants, or depdenencies are ambiguous,then it is not concrete.

concretize()
A spec is concrete if it describes one build of a package uniquely. This will ensure that this spec is concrete.

If this spec could describe more than one version, variant, or build of a package, this will add constraints to make it concrete.

Some rigorous validation and checks are also performed on the spec. Concretizing ensures that it is self-consistent and that it's consistent with requirements of its pacakges. See flatten() and normalize() for more details on this.

concretized()
This is a non-destructive version of concretize(). First clones, then returns a concrete version of this package without modifying this package.

constrain(*other*, *deps=True*)
Merge the constraints of other with self.

Returns True if the spec changed as a result, False if not.

constrained(*other*, *deps=True*)
Return a constrained copy without modifying this spec.

copy(*deps=True*)
Return a copy of this spec.

By default, returns a deep copy. To control how dependencies are copied, supply:

deps=True: deep copy

deps=False: shallow copy (no dependencies)

deps=('link', 'build'): only build and link dependencies. Similar for other deptypes.

cshort_spec
Returns a version of the spec with the dependencies hashed instead of completely enumerated.

dag_hash(*length=None*)
Return a hash of the entire spec DAG, including connectivity.

dep_difference(*other*)
Returns dependencies in self that are not in other.

dep_string()

dependencies(*deptype=None*)

dependencies_dict(*deptype=None*)

dependents(*deptype=None*)

dependents_dict(*deptype=None*)

eq_dag(*other*)
True if the full dependency DAGs of specs are equal

eq_node (*other*)
Equality with another spec, not including dependencies.

flat_dependencies (***kwargs*)

flat_dependencies_with_deptype (***kwargs*)
Return a DependencyMap containing all of this spec's dependencies with their constraints merged.

If copy is True, returns merged copies of its dependencies without modifying the spec it's called on.

If copy is False, clears this spec's dependencies and returns them.

format (*format_string='$_$@$%@+$+$=', **kwargs*)
Prints out particular pieces of a spec, depending on what is in the format string. The format strings you can provide are:

```
$_   Package name
$.   Full package name (with namespace)
$@   Version with '@' prefix
$%   Compiler with '%' prefix
$%@  Compiler with '%' prefix & compiler version with '@' prefix
$%+  Compiler with '%' prefix & compiler flags prefixed by name
$%@+ Compiler, compiler version, and compiler flags with same
     prefixes as above
$+   Options
$=   Architecture prefixed by 'arch='
$#   7-char prefix of DAG hash with '-' prefix
$$   $
```

You can also use full-string versions, which elide the prefixes:

```
${PACKAGE}       Package name
${VERSION}       Version
${COMPILER}      Full compiler string
${COMPILERNAME}  Compiler name
${COMPILERVER}   Compiler version
${COMPILERFLAGS} Compiler flags
${OPTIONS}       Options
${ARCHITECTURE}  Architecture
${SHA1}          Dependencies 8-char sha1 prefix

${SPACK_ROOT}    The spack root directory
${SPACK_INSTALL} The default spack install directory,
                 ${SPACK_PREFIX}/opt
```

Optionally you can provide a width, e.g. `$20_` for a 20-wide name. Like printf, you can provide '-' for left justification, e.g. `$-20_` for a left-justified name.

Anything else is copied verbatim into the output stream.

Example: `$_$@$+` translates to the name, version, and options of the package, but no dependencies, arch, or compiler.

TODO: allow, e.g., `$6#` to customize short hash length TODO: allow, e.g., `$##` for full hash.

static from_node_dict (*node*)

static from_yaml (*stream*)
Construct a spec from YAML.

Parameters: stream – string or file object to read from.

TODO: currently discards hashes. Include hashes when they represent more than the DAG does.

fullname

get_dependency(*name*)

index(*deptype=None*)
Return DependencyMap that points to all the dependencies in this spec.

static **is_virtual**(*name*)
Test if a name is virtual without requiring a Spec.

ne_dag(*other*)
True if the full dependency DAGs of specs are not equal

ne_node(*other*)
Inequality with another spec, not including dependencies.

normalize(*force=False*)
When specs are parsed, any dependencies specified are hanging off the root, and ONLY the ones that were explicitly provided are there. Normalization turns a partial flat spec into a DAG, where:

1. Known dependencies of the root package are in the DAG.
2. Each node's dependencies dict only contains its known direct deps.
3. There is only ONE unique spec for each package in the DAG.
   - This includes virtual packages. If there a non-virtual package that provides a virtual package that is in the spec, then we replace the virtual package with the non-virtual one.

TODO: normalize should probably implement some form of cycle detection, to ensure that the spec is actually a DAG.

normalized()
Return a normalized copy of this spec without modifying this spec.

package

package_class
Internal package call gets only the class object for a package. Use this to just get package metadata.

prefix

static **read_yaml_dep_specs**(*dependency_dict*)
Read the DependencySpec portion of a YAML-formatted Spec.

This needs to be backward-compatible with older spack spec formats so that reindex will work on old specs/databases.

root
Follow dependent links and find the root of this spec's DAG. In spack specs, there should be a single root (the package being installed). This will throw an assertion error if that is not the case.

satisfies(*other*, *deps=True*, *strict=False*)
Determine if this spec satisfies all constraints of another.

There are two senses for satisfies:

- *loose* (default): the absence of a constraint in self implies that it *could* be satisfied by other, so we only check that there are no conflicts with other for constraints that this spec actually has.
- *strict*: strict means that we *must* meet all the constraints specified on other.

satisfies_dependencies(*other*, *strict=False*)
This checks constraints on common dependencies against each other.

short_spec
Returns a version of the spec with the dependencies hashed instead of completely enumerated.

sorted_deps()
Return a list of all dependencies sorted by name.

to_node_dict()

to_yaml(*stream=None*)

traverse(*visited=None*, *deptype=None*, ***kwargs*)

traverse_with_deptype(*visited=None*, *d=0*, *deptype=None*, *deptype_query=None*, *_self_deptype=None*, ***kwargs*)
Generic traversal of the DAG represented by this spec. This will yield each node in the spec. Options:

order [=pre|post] Order to traverse spec nodes. Defaults to preorder traversal. Options are:

'pre': Pre-order traversal; each node is yielded before its children in the dependency DAG.

'post': Post-order traversal; each node is yielded after its children in the dependency DAG.

cover [=nodes|edges|paths] Determines how extensively to cover the dag. Possible vlaues:

'nodes': Visit each node in the dag only once. Every node yielded by this function will be unique.

'edges': If a node has been visited once but is reached along a new path from the root, yield it but do not descend into it. This traverses each 'edge' in the DAG once.

'paths': Explore every unique path reachable from the root. This descends into visited subtrees and will yield nodes twice if they're reachable by multiple paths.

depth [=False] Defaults to False. When True, yields not just nodes in the spec, but also their depth from the root in a (depth, node) tuple.

key [=id] Allow a custom key function to track the identity of nodes in the traversal.

root [=True] If False, this won't yield the root node, just its descendents.

direction [=children|parents] If 'children', does a traversal of this spec's children. If 'parents', traverses upwards in the DAG towards the root.

tree(***kwargs*)
Prints out this spec and its dependencies, tree-formatted with indentation.

validate_names()
This checks that names of packages and compilers in this spec are real. If they're not, it will raise either UnknownPackageError or UnsupportedCompilerError.

version

virtual
Right now, a spec is virtual if no package exists with its name.

TODO: revisit this – might need to use a separate namespace and be more explicit about this. Possible idea: just use conventin and make virtual deps all caps, e.g., MPI vs mpi.

virtual_dependencies()
Return list of any virtual deps in this spec.

spack.spec.**canonical_deptype**(*deptype*)

spack.spec.**validate_deptype**(*deptype*)

spack.spec.**parse**(*string*)
Returns a list of specs from an input string. For creating one spec, see Spec() constructor.

spack.spec.**parse_anonymous_spec**(*spec_like*, *pkg_name*)
Allow the user to omit the package name part of a spec if they know what it has to be already.

e.g., provides('mpi@2', when='@1.9:') says that this package provides MPI-3 when its version is higher than 1.9.

exception spack.spec.**SpecError**(*message*, *long_message=None*)
Bases: *spack.error.SpackError*

Superclass for all errors that occur while constructing specs.

exception spack.spec.**SpecParseError**(*parse_error*)
Bases: *spack.spec.SpecError*

Wrapper for ParseError for when we're parsing specs.

exception spack.spec.**DuplicateDependencyError**(*message*, *long_message=None*)
Bases: *spack.spec.SpecError*

Raised when the same dependency occurs in a spec twice.

exception spack.spec.**DuplicateVariantError**(*message*, *long_message=None*)
Bases: *spack.spec.SpecError*

Raised when the same variant occurs in a spec twice.

exception spack.spec.**DuplicateCompilerSpecError**(*message*, *long_message=None*)
Bases: *spack.spec.SpecError*

Raised when the same compiler occurs in a spec twice.

exception spack.spec.**UnsupportedCompilerError**(*compiler_name*)
Bases: *spack.spec.SpecError*

Raised when the user asks for a compiler spack doesn't know about.

exception spack.spec.**UnknownVariantError**(*pkg*, *variant*)
Bases: *spack.spec.SpecError*

Raised when the same variant occurs in a spec twice.

exception spack.spec.**DuplicateArchitectureError**(*message*, *long_message=None*)
Bases: *spack.spec.SpecError*

Raised when the same architecture occurs in a spec twice.

exception spack.spec.**InconsistentSpecError**(*message*, *long_message=None*)
Bases: *spack.spec.SpecError*

Raised when two nodes in the same spec DAG have inconsistent constraints.

exception spack.spec.**InvalidDependencyError**(*message*, *long_message=None*)
Bases: *spack.spec.SpecError*

Raised when a dependency in a spec is not actually a dependency of the package.

exception spack.spec.**InvalidDependencyTypeError**(*message*, *long_message=None*)
Bases: *spack.spec.SpecError*

Raised when a dependency type is not a legal Spack dep type.

exception spack.spec.**NoProviderError**(*vpkg*)
Bases: *spack.spec.SpecError*

Raised when there is no package that provides a particular virtual dependency.

exception spack.spec.**MultipleProviderError**(*vpkg*, *providers*)
Bases: *spack.spec.SpecError*

Raised when there is no package that provides a particular virtual dependency.

exception spack.spec.**UnsatisfiableSpecError**(*provided*, *required*, *constraint_type*)
Bases: *spack.spec.SpecError*

Raised when a spec conflicts with package constraints. Provide the requirement that was violated when raising.

exception spack.spec.**UnsatisfiableSpecNameError**(*provided*, *required*)
Bases: *spack.spec.UnsatisfiableSpecError*

Raised when two specs aren't even for the same package.

exception spack.spec.**UnsatisfiableVersionSpecError**(*provided*, *required*)
Bases: *spack.spec.UnsatisfiableSpecError*

Raised when a spec version conflicts with package constraints.

exception spack.spec.**UnsatisfiableCompilerSpecError**(*provided*, *required*)
Bases: *spack.spec.UnsatisfiableSpecError*

Raised when a spec comiler conflicts with package constraints.

exception spack.spec.**UnsatisfiableVariantSpecError**(*provided*, *required*)
Bases: *spack.spec.UnsatisfiableSpecError*

Raised when a spec variant conflicts with package constraints.

exception spack.spec.**UnsatisfiableCompilerFlagSpecError**(*provided*, *required*)
Bases: *spack.spec.UnsatisfiableSpecError*

Raised when a spec variant conflicts with package constraints.

exception spack.spec.**UnsatisfiableArchitectureSpecError**(*provided*, *required*)
Bases: *spack.spec.UnsatisfiableSpecError*

Raised when a spec architecture conflicts with package constraints.

exception spack.spec.**UnsatisfiableProviderSpecError**(*provided*, *required*)
Bases: *spack.spec.UnsatisfiableSpecError*

Raised when a provider is supplied but constraints don't match a vpkg requirement

exception spack.spec.**UnsatisfiableDependencySpecError**(*provided*, *required*)
Bases: *spack.spec.UnsatisfiableSpecError*

Raised when some dependency of constrained specs are incompatible

exception spack.spec.**SpackYAMLError**(*msg*, *yaml_error*)
Bases: *spack.error.SpackError*

exception spack.spec.**AmbiguousHashError**(*msg*, **specs*)
Bases: *spack.spec.SpecError*

12.29 spack.stage module

exception spack.stage.**ChdirError**(*message*, *long_message=None*)
Bases: *spack.stage.StageError*

Raised when Spack can't change directories.

class spack.stage.**DIYStage**(*path*)
Bases: object

Simple class that allows any directory to be a spack stage.

cache_local()

chdir()

chdir_to_source()

check()

destroy()

expand_archive()

fetch(*mirror_only*)

restage()

class spack.stage.**ResourceStage**(*url_or_fetch_strategy*, *root*, *resource*, ***kwargs*)
Bases: *spack.stage.Stage*

expand_archive()

exception spack.stage.**RestageError**(*message*, *long_message=None*)
Bases: *spack.stage.StageError*

"Error encountered during restaging.

class spack.stage.**Stage**(*url_or_fetch_strategy*, *name=None*, *mirror_path=None*, *keep=False*, *path=None*)
Bases: object

Manages a temporary stage directory for building.

A Stage object is a context manager that handles a directory where some source code is downloaded and built before being installed. It handles fetching the source code, either as an archive to be expanded or by checking it out of a repository. A stage's lifecycle looks like this:

```
with Stage() as stage:      # Context manager creates and destroys the
                            # stage directory
    stage.fetch()           # Fetch a source archive into the stage.
    stage.expand_archive()  # Expand the source archive.
    <install>               # Build and install the archive.
                            # (handled by user of Stage)
```

When used as a context manager, the stage is automatically destroyed if no exception is raised by the context. If an excpetion is raised, the stage is left in the filesystem and NOT destroyed, for potential reuse later.

You can also use the stage's create/destroy functions manually, like this:

```
stage = Stage()
try:
    stage.create()          # Explicitly create the stage directory.
    stage.fetch()           # Fetch a source archive into the stage.
    stage.expand_archive()  # Expand the source archive.
    <install>               # Build and install the archive.
                            # (handled by user of Stage)
finally:
    stage.destroy()         # Explicitly destroy the stage directory.
```

If spack.use_tmp_stage is True, spack will attempt to create stages in a tmp directory. Otherwise, stages are created directly in spack.stage_path.

There are two kinds of stages: named and unnamed. Named stages can persist between runs of spack, e.g. if you fetched a tarball but didn't finish building it, you won't have to fetch it again.

Unnamed stages are created using standard mkdtemp mechanisms or similar, and are intended to persist for only one run of spack.

archive_file
Path to the source archive within this stage directory.

cache_local()

chdir()
Changes directory to the stage path. Or dies if it is not set up.

chdir_to_source()
Changes directory to the expanded archive directory. Dies with an error if there was no expanded archive.

check()
Check the downloaded archive against a checksum digest. No-op if this stage checks code out of a repository.

create()
Creates the stage directory

If self.tmp_root evaluates to False, the stage directory is created directly under spack.stage_path, otherwise this will attempt to create a stage in a temporary directory and link it into spack.stage_path.

Spack will use the first writable location in spack.tmp_dirs to create a stage. If there is no valid location in tmp_dirs, fall back to making the stage inside spack.stage_path.

destroy()
Removes this stage directory.

expand_archive()
Changes to the stage directory and attempt to expand the downloaded archive. Fail if the stage is not set up or if the archive is not yet downloaded.

expected_archive_files
Possible archive file paths.

fetch(*mirror_only=False*)
Downloads an archive or checks out code from a repository.

restage()
Removes the expanded archive path if it exists, then re-expands the archive.

save_filename

source_path
Returns the path to the expanded/checked out source code.

To find the source code, this method searches for the first subdirectory of the stage that it can find, and returns it. This assumes nothing besides the archive file will be in the stage path, but it has the advantage that we don't need to know the name of the archive or its contents.

If the fetch strategy is not supposed to expand the downloaded file, it will just return the stage path. If the archive needs to be expanded, it will return None when no archive is found.

exception spack.stage.**StageError**(*message*, *long_message=None*)
Bases: *spack.error.SpackError*

"Superclass for all errors encountered during staging.

spack.stage.**ensure_access**(*file='/home/docs/checkouts/readthedocs.org/user_builds/spack-citibeth-fork/checkouts/efischer-docs2/var/spack/stage'*)
Ensure we can access a directory and die with an error if we can't.

spack.stage.**find_tmp_root**()

spack.stage.**purge**()
Remove all build directories in the top-level stage path.

12.30 spack.url module

This module has methods for parsing names and versions of packages from URLs. The idea is to allow package creators to supply nothing more than the download location of the package, and figure out version and name information from there.

Example: when spack is given the following URL:

ftp://ftp.ruby-lang.org/pub/ruby/1.9/ruby-1.9.1-p243.tar.gz

It can figure out that the package name is ruby, and that it is at version 1.9.1-p243. This is useful for making the creation of packages simple: a user just supplies a URL and skeleton code is generated automatically.

Spack can also figure out that it can most likely download 1.8.1 at this URL:

ftp://ftp.ruby-lang.org/pub/ruby/1.9/ruby-1.8.1.tar.gz

This is useful if a user asks for a package at a particular version number; spack doesn't need anyone to tell it where to get the tarball even though it's never been told about that version before.

exception spack.url.**UndetectableNameError**(*path*)
Bases: *spack.url.UrlParseError*

Raised when we can't parse a package name from a string.

exception spack.url.**UndetectableVersionError**(*path*)
Bases: *spack.url.UrlParseError*

Raised when we can't parse a version from a string.

exception spack.url.**UrlParseError**(*msg*, *path*)
Bases: *spack.error.SpackError*

Raised when the URL module can't parse something correctly.

spack.url.**color_url**(*path*, ***kwargs*)
Color the parts of the url according to Spack's parsing.

Colors are: Cyan: The version found by parse_version_offset(). Red: The name found by parse_name_offset().

Green: Instances of version string from substitute_version(). Magenta: Instances of the name (protected from substitution).

Optional args: errors=True Append parse errors at end of string. subs=True Color substitutions as well as parsed name/version.

spack.url.**cumsum**(*elts*, *init=0*, *fn=<function <lambda>>*)
Return cumulative sum of result of fn on each element in elts.

spack.url.**downloaded_file_extension**(*path*)
This returns the type of archive a URL refers to. This is sometimes confusing because of URLs like:

1. https://github.com/petdance/ack/tarball/1.93_02

Where the URL doesn't actually contain the filename. We need to know what type it is so that we can appropriately name files in mirrors.

spack.url.**find_list_url**(*url*)
Finds a good list URL for the supplied URL. This depends on the site. By default, just assumes that a good list URL is the dirname of an archive path. For github URLs, this returns the URL of the project's releases page.

spack.url.**insensitize**(*string*)
Change upper and lowercase letters to be case insensitive in the provided string. e.g., 'a' because '[Aa]', 'B' becomes '[bB]', etc. Use for building regexes.

spack.url.**parse_name**(*path*, *ver=None*)

spack.url.**parse_name_and_version**(*path*)

spack.url.**parse_name_offset**(*path*, *v=None*)

spack.url.**parse_version**(*path*)
Given a URL or archive name, extract a version from it and return a version object.

spack.url.**parse_version_offset**(*path*)
Try to extract a version string from a filename or URL. This is taken largely from Homebrew's Version class.

spack.url.**split_url_extension**(*path*)
Some URLs have a query string, e.g.:

1. https://github.com/losalamos/CLAMR/blob/packages/PowerParser_v2.0.7.tgz?raw=true
2. http://www.apache.org/dyn/closer.cgi?path=/cassandra/1.2.0/apache-cassandra-1.2.0-rc2-bin.tar.gz

In (1), the query string needs to be stripped to get at the extension, but in (2), the filename is IN a single final query argument.

This strips the URL into three pieces: prefix, ext, and suffix. The suffix contains anything that was stripped off the URL to get at the file extension. In (1), it will be '?raw=true', but in (2), it will be empty. e.g.:

1. ('https://github.com/losalamos/CLAMR/blob/packages/PowerParser_v2.0.7', '.tgz', '?raw=true')
2. **('http://www.apache.org/dyn/closer.cgi?path=/cassandra/1.2.0/apache-cassandra-1.2.0-rc2-bin',** '.tar.gz', None)

spack.url.**strip_query_and_fragment**(*path*)

spack.url.**substitute_version**(*path*, *new_version*)
Given a URL or archive name, find the version in the path and substitute the new version for it. Replace all occurrences of the version *if* they don't overlap with the package name.

Simple example:: substitute_version('http://www.mr511.de/software/libelf-0.8.13.tar.gz', '2.9.3') ->'http://www.mr511.de/software/libelf-2.9.3.tar.gz'

Complex examples:: substitute_version('http://mvapich.cse.ohio-state.edu/download/mvapich/mv2/mvapich2-2.0.tar.gz', 2.1) -> 'http://mvapich.cse.ohio-state.edu/download/mvapich/mv2/mvapich2-2.1.tar.gz'

In this string, the "2" in mvapich2 is NOT replaced. substitute_version('http://mvapich.cse.ohio-state.edu/download/mvapich/mv2/mvapich2-2.tar.gz', 2.1) -> 'http://mvapich.cse.ohio-state.edu/download/mvapich/mv2/mvapich2-2.1.tar.gz'

spack.url.**substitution_offsets**(*path*)
This returns offsets for substituting versions and names in the provided path. It is a helper for substitute_version().

spack.url.**wildcard_version**(*path*)
Find the version in the supplied path, and return a regular expression that will match this path with any version in its place.

12.31 spack.variant module

Variant is a class describing flags on builds, or "variants".

Could be generalized later to describe aribitrary parameters, but currently variants are just flags.

class spack.variant.**Variant**(*default*, *description*)
Bases: object

Represents a variant on a build. Can be either on or off.

12.32 spack.version module

This module implements Version and version-ish objects. These are:

Version A single version of a package.

VersionRange A range of versions of a package.

VersionList A list of Versions and VersionRanges.

All of these types support the following operations, which can be called on any of the types:

```
__eq__, __ne__, __lt__, __gt__, __ge__, __le__, __hash__
__contains__
satisfies
overlaps
union
intersection
concrete
```

class spack.version.**Version**(*string*)
Bases: object

Class to represent versions

concrete

dashed

dotted

highest()

intersection(*a*, *b*, **args*, ***kwargs*)

is_predecessor(*other*)
True if the other version is the immediate predecessor of this one. That is, NO versions v exist such that: (self < v < other and v not in self).

is_successor(*other*)

lowest()

overlaps(*a*, *b*, **args*, ***kwargs*)

satisfies(*a*, *b*, **args*, ***kwargs*)
A Version 'satisfies' another if it is at least as specific and has a common prefix. e.g., we want gcc@4.7.3 to satisfy a request for gcc@4.7 so that when a user asks to build with gcc@4.7, we can find a suitable compiler.

underscored

union(*a*, *b*, **args*, ***kwargs*)

up_to(*index*)
Return a version string up to the specified component, exclusive. e.g., if this is 10.8.2, self.up_to(2) will return '10.8'.

wildcard()
Create a regex that will match variants of this version string.

class spack.version.**VersionRange**(*start*, *end*)
Bases: object

concrete

highest()

intersection(*a*, *b*, **args*, ***kwargs*)

lowest()

overlaps(*a*, *b*, **args*, ***kwargs*)

satisfies(*a*, *b*, **args*, ***kwargs*)
A VersionRange satisfies another if some version in this range would satisfy some version in the other range. To do this it must either:

1.Overlap with the other range

2.The start of this range satisfies the end of the other range.

This is essentially the same as overlaps(), but overlaps assumes that its arguments are specific. That is, 4.7 is interpreted as 4.7.0.0.0.0... . This funciton assumes that 4.7 woudl be satisfied by 4.7.3.5, etc.

Rationale:

If a user asks for gcc@4.5:4.7, and a package is only compatible with gcc@4.7.3:4.8, then that package should be able to build under the constraints. Just using overlaps() would not work here.

Note that we don't need to check whether the end of this range would satisfy the start of the other range, because overlaps() already covers that case.

Note further that overlaps() is a symmetric operation, while satisfies() is not.

union(*a*, *b*, **args*, ***kwargs*)

class spack.version.**VersionList**(*vlist=None*)
Bases: object

Sorted, non-redundant list of Versions and VersionRanges.

add(*version*)

concrete

copy()

static from_dict(*dictionary*)
Parse dict from to_dict.

highest()
Get the highest version in the list.

intersect(*a*, *b*, **args*, ***kwargs*)
Intersect this spec's list with other.

Return True if the spec changed as a result; False otherwise

intersection(*a*, *b*, **args*, ***kwargs*)

lowest()
Get the lowest version in the list.

overlaps(*a*, *b*, **args*, ***kwargs*)

satisfies(*a*, *b*, **args*, ***kwargs*)
A VersionList satisfies another if some version in the list would satisfy some version in the other list. This uses essentially the same algorithm as overlaps() does for VersionList, but it calls satisfies() on member Versions and VersionRanges.

If strict is specified, this version list must lie entirely *within* the other in order to satisfy it.

to_dict()
Generate human-readable dict for YAML.

union(*a*, *b*, **args*, ***kwargs*)

update(*a*, *b*, **args*, ***kwargs*)

spack.version.**ver**(*obj*)
Parses a Version, VersionRange, or VersionList from a string or list of strings.

12.33 spack.yaml_version_check module

Yaml Version Check is a module for ensuring that config file formats are compatible with the current version of Spack.

spack.yaml_version_check.**check_compiler_yaml_version**()

spack.yaml_version_check.**check_yaml_versions**()

12.34 Module contents

class spack.**Package**(*spec*)
Bases: object

This is the superclass for all spack packages.

The Package class

Package is where the bulk of the work of installing packages is done.

A package defines how to fetch, verfiy (via, e.g., md5), build, and install a piece of software. A Package also defines what other packages it depends on, so that dependencies can be installed along with the package itself. Packages are written in pure python.

Packages are all submodules of spack.packages. If spack is installed in `$prefix`, all of its python files are in `$prefix/lib/spack`. Most of them are in the spack module, so all the packages live in `$prefix/lib/spack/spack/packages`.

All you have to do to create a package is make a new subclass of Package in this directory. Spack automatically scans the python files there and figures out which one to import when you invoke it.

An example package

Let's look at the cmake package to start with. This package lives in `$prefix/var/spack/repos/builtin/packages/cmake/package.py`:

```
from spack import *
class Cmake(Package):
    homepage  = 'https://www.cmake.org'
    url       = 'http://www.cmake.org/files/v2.8/cmake-2.8.10.2.tar.gz'
    md5       = '097278785da7182ec0aea8769d06860c'

    def install(self, spec, prefix):
        configure('--prefix=%s'   % prefix,
                  '--parallel=%s' % make_jobs)
        make()
        make('install')
```

Naming conventions

There are two names you should care about:

1. The module name, `cmake`.
   - User will refers to this name, e.g. 'spack install cmake'.
   - It can include _, −, and numbers (it can even start with a number).
2. The class name, "Cmake". This is formed by converting - or _ in the module name to camel case. If the name starts with a number, we prefix the class name with _. Examples:

Module Name	Class Name
foo_bar	FooBar
docbook-xml	DocbookXml
FooBar	Foobar
3proxy	_3proxy

The class name is what spack looks for when it loads a package module.

Required Attributes

Aside from proper naming, here is the bare minimum set of things you need when you make a package:

homepage: informational URL, so that users know what they're installing.

url or url_for_version(self, version): If url, then the URL of the source archive that spack will fetch. If url_for_version(), then a method returning the URL required to fetch a particular version.

install(): This function tells spack how to build and install the software it downloaded.

Optional Attributes

You can also optionally add these attributes, if needed:

list_url: Webpage to scrape for available version strings. Default is the directory containing the tarball; use this if the default isn't correct so that invoking 'spack versions' will work for this package.

url_version(self, version): When spack downloads packages at particular versions, it just converts version to string with str(version). Override this if your package needs special version formatting in its URL. boost is an example of a package that needs this.

Creating Packages

As a package creator, you can probably ignore most of the preceding information, because you can use the 'spack create' command to do it all automatically.

You as the package creator generally only have to worry about writing your install function and specifying dependencies.

spack create

Most software comes in nicely packaged tarballs, like this one

http://www.cmake.org/files/v2.8/cmake-2.8.10.2.tar.gz

Taking a page from homebrew, spack deduces pretty much everything it needs to know from the URL above. If you simply type this:

```
spack create http://www.cmake.org/files/v2.8/cmake-2.8.10.2.tar.gz
```

Spack will download the tarball, generate an md5 hash, figure out the version and the name of the package from the URL, and create a new package file for you with all the names and attributes set correctly.

Once this skeleton code is generated, spack pops up the new package in your $EDITOR so that you can modify the parts that need changes.

Dependencies

If your package requires another in order to build, you can specify that like this:

```
class Stackwalker(Package):
    ...
    depends_on("libdwarf")
    ...
```

This tells spack that before it builds stackwalker, it needs to build the libdwarf package as well. Note that this is the module name, not the class name (The class name is really only used by spack to find your package).

Spack will download an install each dependency before it installs your package. In addtion, it will add -L, -I, and rpath arguments to your compiler and linker for each dependency. In most cases, this allows you to avoid specifying any dependencies in your configure or cmake line; you can just run configure or cmake without any additional arguments and it will find the dependencies automatically.

The Install Function

The install function is designed so that someone not too terribly familiar with Python could write a package installer. For example, we put a number of commands in install scope that you can use almost like shell commands. These include make, configure, cmake, rm, rmtree, mkdir, mkdirp, and others.

You can see above in the cmake script that these commands are used to run configure and make almost like they're used on the command line. The only difference is that they are python function calls and not shell commands.

It may be puzzling to you where the commands and functions in install live. They are NOT instance variables on the class; this would require us to type 'self.' all the time and it makes the install code unnecessarily long. Rather, spack puts these commands and variables in *module* scope for your Package subclass. Since each package has its own module, this doesn't pollute other namespaces, and it allows you to more easily implement an install function.

For a full list of commands and variables available in module scope, see the add_commands_to_module() function in this class. This is where most of them are created and set on the module.

Parallel Builds

By default, Spack will run make in parallel when you run make() in your install function. Spack figures out how many cores are available on your system and runs make with -j<cores>. If you do not want this behavior, you can explicitly mark a package not to use parallel make:

```
class SomePackage(Package):
    ...
    parallel = False
    ...
```

This changes thd default behavior so that make is sequential. If you still want to build some parts in parallel, you can do this in your install function:

```
make(parallel=True)
```

Likewise, if you do not supply parallel = True in your Package, you can keep the default parallel behavior and run make like this when you want a sequential build:

```
make(parallel=False)
```

Package Lifecycle

This section is really only for developers of new spack commands.

A package's lifecycle over a run of Spack looks something like this:

```
p = Package()             # Done for you by spack

p.do_fetch()              # downloads tarball from a URL
p.do_stage()              # expands tarball in a temp directory
p.do_patch()              # applies patches to expanded source
p.do_install()            # calls package's install() function
p.do_uninstall()          # removes install directory
```

There are also some other commands that clean the build area:

```
p.do_clean()              # removes the stage directory entirely
p.do_restage()            # removes the build directory and
                          # re-expands the archive.
```

The convention used here is that a `do_*` function is intended to be called internally by Spack commands (in spack.cmd). These aren't for package writers to override, and doing so may break the functionality of the Package class.

Package creators override functions like install() (all of them do this), clean() (some of them do this), and others to provide custom behavior.

activate(*extension*, ***kwargs*)
Symlinks all files from the extension into extendee's install dir.

Package authors can override this method to support other extension mechanisms. Spack internals (commands, hooks, etc.) should call do_activate() method so that proper checks are always executed.

activated

all_urls

build_log_path

compiler
Get the spack.compiler.Compiler object used to build this package

deactivate(*extension*, ***kwargs*)
Unlinks all files from extension out of this package's install dir.

Package authors can override this method to support other extension mechanisms. Spack internals (commands, hooks, etc.) should call do_deactivate() method so that proper checks are always executed.

dependencies = {}

dependencies_of_type(**deptypes*)
Get subset of the dependencies with certain types.

do_activate(*force=False*)
Called on an extension to invoke the extendee's activate method.

Commands should call this routine, and should not call activate() directly.

do_clean()
Removes the package's build stage and source tarball.

do_deactivate(***kwargs*)
Called on the extension to invoke extendee's deactivate() method.

do_fake_install()
Make a fake install directory contaiing a 'fake' file in bin.

do_fetch(*mirror_only=False*)
Creates a stage directory and downloads the tarball for this package. Working directory will be set to the stage directory.

do_install(*keep_prefix=False*, *keep_stage=False*, *ignore_deps=False*, *skip_patch=False*, *verbose=False*, *make_jobs=None*, *run_tests=False*, *fake=False*, *explicit=False*, *dirty=False*, *install_phases=set(['provenance', 'configure', 'install', 'build'])*)
Called by commands to install a package and its dependencies.

Package implementations should override install() to describe their build process.

Parameters

- **keep_prefix** – Keep install prefix on failure. By default, destroys it.
- **keep_stage** – By default, stage is destroyed only if there are no exceptions during build. Set to True to keep the stage even with exceptions.
- **ignore_deps** – Don't install dependencies before installing this package
- **fake** – Don't really build; install fake stub files instead.
- **skip_patch** – Skip patch stage of build if True.
- **verbose** – Display verbose build output (by default, suppresses it)
- **dirty** – Don't clean the build environment before installing.
- **make_jobs** – Number of make jobs to use for install. Default is ncpus
- **run_tests** – Run tests within the package's install()

do_install_dependencies(***kwargs*)

do_patch()
Calls do_stage(), then applied patches to the expanded tarball if they haven't been applied already.

do_restage()
Reverts expanded/checked out source to a pristine state.

do_stage(*mirror_only=False*)
Unpacks the fetched tarball, then changes into the expanded tarball directory.

do_uninstall(*force=False*)

extendable = False

extendee_args
Spec of the extendee of this package, or None if it is not an extension

extendee_spec
Spec of the extendee of this package, or None if it is not an extension

extendees = {}

extends(*spec*)

fetch_remote_versions()
Try to find remote versions of this package using the list_url and any other URLs described in the package file.

fetcher

format_doc(***kwargs*)
Wrap doc string at 72 characters and format nicely

global_license_dir
Returns the directory where global license files for all packages are stored.

global_license_file
Returns the path where a global license file for this particular package should be stored.

install(*spec*, *prefix*)
Package implementations override this with their own configuration

install_phases = set(['provenance', 'configure', 'install', 'build'])

installed

installed_dependents

Return a list of the specs of all installed packages that depend on this one.

TODO: move this method to database.py?

is_extension

make_jobs = None

module
Use this to add variables to the class's module's scope. This lets us use custom syntax in the install method.

namespace

nearest_url(*version*)
Finds the URL for the next lowest version with a URL. If there is no lower version with a URL, uses the package url property. If that isn't there, uses a *higher* URL, and if that isn't there raises an error.

package_dir
Return the directory where the package.py file lives.

parallel = True

patches = {}

possible_dependencies(*visited=None*)
Return set of possible transitive dependencies of this package.

prefix
Get the prefix into which this package should be installed.

provided = {}

provides(*vpkg_name*)
True if this package provides a virtual package with the specified name

remove_prefix()
Removes the prefix for a package along with any empty parent directories

resources = {}

rpath
Get the rpath this package links with, as a list of paths.

rpath_args
Get the rpath args as a string, with -Wl,-rpath, for each element

run_tests = False

sanity_check_is_dir = []

sanity_check_is_file = []

sanity_check_prefix()
This function checks whether install succeeded.

setup_dependent_environment(*spack_env*, *run_env*, *dependent_spec*)
Set up the environment of packages that depend on this one.

This is similar to `setup_environment`, but it is used to modify the compile and runtime environments of packages that *depend* on this one. This gives packages like Python and others that follow the extension model a way to implement common environment or compile-time settings for dependencies.

By default, this delegates to `self.setup_environment()`

Example:

1. Installing python modules generally requires *PYTHONPATH* to point to the lib/pythonX.Y/site-packages directory in the module's install prefix. This could set that variable.

Args:

spack_env (EnvironmentModifications): list of modifications to be applied when the dependent package is bulit within Spack.

run_env (EnvironmentModifications): list of environment changes to be applied when the dependent package is run outside of Spack.

dependent_spec (Spec): The spec of the dependent package about to be built. This allows the extendee (self) to query the dependent's state. Note that *this* package's spec is available as *self.spec*.

This is useful if there are some common steps to installing all extensions for a certain package.

setup_dependent_package(*module*, *dependent_spec*)
Set up Python module-scope variables for dependent packages.

Called before the install() method of dependents.

Default implementation does nothing, but this can be overridden by an extendable package to set up the module of its extensions. This is useful if there are some common steps to installing all extensions for a certain package.

Example :

1. Extensions often need to invoke the *python* interpreter from the Python installation being extended. This routine can put a 'python' Executable object in the module scope for the extension package to simplify extension installs.

2. MPI compilers could set some variables in the dependent's scope that point to *mpicc*, *mpicxx*, etc., allowing them to be called by common names regardless of which MPI is used.

3. BLAS/LAPACK implementations can set some variables indicating the path to their libraries, since these paths differ by BLAS/LAPACK implementation.

Args:

module (module): The Python *module* object of the dependent package. Packages can use this to set module-scope variables for the dependent to use.

dependent_spec (Spec): The spec of the dependent package about to be built. This allows the extendee (self) to query the dependent's state. Note that *this* package's spec is available as *self.spec*.

This is useful if there are some common steps to installing all extensions for a certain package.

setup_environment(*spack_env*, *run_env*)

Set up the compile and runtime environments for a package.

spack_env and *run_env* are *EnvironmentModifications* objects. Package authors can call methods on them to alter the environment within Spack and at runtime.

Both *spack_env* and *run_env* are applied within the build process, before this package's *install()* method is called.

Modifications in *run_env* will *also* be added to the generated environment modules for this package.

Default implementation does nothing, but this can be overridden if the package needs a particular environment.

Examples:

1. Qt extensions need *QTDIR* set.

Args:

spack_env (EnvironmentModifications): list of modifications to be applied when this package is built within Spack.

run_env (EnvironmentModifications): list of environment changes to be applied when this package is run outside of Spack.

stage

url_for_version(*version*)

Returns a URL from which the specified version of this package may be downloaded.

version: class Version The version for which a URL is sought.

See Class Version (version.py)

url_version(*version*)

Given a version, this returns a string that should be substituted into the package's URL to download that version.

By default, this just returns the version string. Subclasses may need to override this, e.g. for boost versions where you need to ensure that there are _'s in the download URL.

variants = {}

version

version_urls = <functools.partial object>

versions = {}

class spack.**StagedPackage**(*spec*)

Bases: *spack.package.Package*

A Package subclass where the install() is split up into stages.

install(*spec*, *prefix*)

install_build()
Runs the build process.

install_configure()
Runs the configure process.

install_install()
Runs the install process.

install_setup()
Creates a spack_setup.py script to configure the package later.

class spack.**CMakePackage**(*spec*)
Bases: *spack.package.StagedPackage*

configure_args()
Returns package-specific arguments to be provided to the configure command.

configure_env()
Returns package-specific environment under which the configure command should be run.

install_build()

install_configure()

install_install()

install_setup()

make_make()

transitive_inc_path()

class spack.**Version**(*string*)
Bases: object

Class to represent versions

concrete

dashed

dotted

highest()

intersection(*a*, *b*, **args*, ***kwargs*)

is_predecessor(*other*)
True if the other version is the immediate predecessor of this one. That is, NO versions v exist such that: (self < v < other and v not in self).

is_successor(*other*)

lowest()

overlaps(*a*, *b*, **args*, ***kwargs*)

satisfies(*a*, *b*, **args*, ***kwargs*)
A Version 'satisfies' another if it is at least as specific and has a common prefix. e.g., we want gcc@4.7.3 to satisfy a request for gcc@4.7 so that when a user asks to build with gcc@4.7, we can find a suitable compiler.

underscored

union(*a*, *b*, **args*, ***kwargs*)

up_to(*index*)
Return a version string up to the specified component, exclusive. e.g., if this is 10.8.2, self.up_to(2) will return '10.8'.

wildcard()
Create a regex that will match variants of this version string.

class spack.**when**(*spec*)
Bases: object

This annotation lets packages declare multiple versions of methods like install() that depend on the package's spec. For example:

```
class SomePackage(Package):
    ...

    def install(self, prefix):
        # Do default install

    @when('arch=chaos_5_x86_64_ib')
    def install(self, prefix):
        # This will be executed instead of the default install if
        # the package's platform() is chaos_5_x86_64_ib.

    @when('arch=bgqos_0")
    def install(self, prefix):
        # This will be executed if the package's sys_type is bgqos_0
```

This allows each package to have a default version of install() AND specialized versions for particular platforms. The version that is called depends on the architecutre of the instantiated package.

Note that this works for methods other than install, as well. So, if you only have part of the install that is platform specific, you could do this:

```
class SomePackage(Package):
    ...
    # virtual dependence on MPI.
    # could resolve to mpich, mpich2, OpenMPI
    depends_on('mpi')

    def setup(self):
        # do nothing in the default case
        pass

    @when('^openmpi')
    def setup(self):
        # do something special when this is built with OpenMPI for
        # its MPI implementations.


    def install(self, prefix):
        # Do common install stuff
        self.setup()
        # Do more common install stuff
```

There must be one (and only one) @when clause that matches the package's spec. If there is more than one, or if none match, then the method will raise an exception when it's called.

Note that the default version of decorated methods must *always* come first. Otherwise it will override all of the platform-specific versions. There's not much we can do to get around this because of the way decorators work.

spack.**ver**(*obj*)
Parses a Version, VersionRange, or VersionList from a string or list of strings.

spack.**set_install_permissions**(*path*)
Set appropriate permissions on the installed file.

spack.**install**(*src*, *dest*)
Manually install a file to a particular location.

spack.**install_tree**(*src*, *dest*, ***kwargs*)
Manually install a directory tree to a particular location.

spack.**traverse_tree**(*source_root*, *dest_root*, *rel_path=''*, ***kwargs*)
Traverse two filesystem trees simultaneously.

Walks the LinkTree directory in pre or post order. Yields each file in the source directory with a matching path from the dest directory, along with whether the file is a directory. e.g., for this tree:

```
root/
  a/
    file1
    file2
  b/
    file3
```

When called on dest, this yields:

```
('root',         'dest')
('root/a',       'dest/a')
('root/a/file1', 'dest/a/file1')
('root/a/file2', 'dest/a/file2')
('root/b',       'dest/b')
('root/b/file3', 'dest/b/file3')
```

Optional args:

order=[pre|post] – Whether to do pre- or post-order traversal.

ignore=<predicate> – Predicate indicating which files to ignore.

follow_nonexisting – Whether to descend into directories in src that do not exit in dest. Default True.

follow_links – Whether to descend into symlinks in src.

spack.**expand_user**(*path*)
Find instances of '%u' in a path and replace with the current user's username.

spack.**working_dir**(**args*, ***kwds*)

spack.**touch**(*path*)
Creates an empty file at the specified path.

spack.**touchp**(*path*)
Like touch, but creates any parent directories needed for the file.

spack.**mkdirp**(**paths*)
Creates a directory, as well as parent directories if needed.

spack.**force_remove**(**paths*)
Remove files without printing errors. Like rm -f, does NOT remove directories.

spack.**join_path**(*prefix*, **args*)

spack.**ancestor**(*dir*, *n=1*)
Get the nth ancestor of a directory.

spack.**can_access**(*file_name*)
True if we have read/write access to the file.

spack.**filter_file**(*regex*, *repl*, **filenames*, ***kwargs*)
Like sed, but uses python regular expressions.

Filters every line of each file through regex and replaces the file with a filtered version. Preserves mode of filtered files.

As with re.sub, `repl` can be either a string or a callable. If it is a callable, it is passed the match object and should return a suitable replacement string. If it is a string, it can contain , , etc. to represent back-substitution as sed would allow.

Keyword Options: string[=False] If True, treat regex as a plain string. backup[=True] Make backup file(s) suffixed with ~ ignore_absent[=False] Ignore any files that don't exist.

class spack.**FileFilter**(**filenames*)
Bases: `object`

Convenience class for calling filter_file a lot.

filter(*regex*, *repl*, ***kwargs*)

spack.**change_sed_delimiter**(*old_delim*, *new_delim*, **filenames*)
Find all sed search/replace commands and change the delimiter. e.g., if the file contains seds that look like 's///', you can call change_sed_delimiter('/', '@', file) to change the delimiter to '@'.

NOTE that this routine will fail if the delimiter is ' or ". Handling those is left for future work.

spack.**is_exe**(*path*)
True if path is an executable file.

spack.**force_symlink**(*src*, *dest*)

spack.**set_executable**(*path*)

spack.**copy_mode**(*src*, *dest*)

spack.**unset_executable_mode**(*path*)

spack.**remove_dead_links**(*root*)
Removes any dead link that is present in root

Args: root: path where to search for dead links

spack.**remove_linked_tree**(*path*)
Removes a directory and its contents. If the directory is a symlink, follows the link and removes the real directory before removing the link.

Args: path: directory to be removed

spack.**fix_darwin_install_name**(*path*)
Fix install name of dynamic libraries on Darwin to have full path. There are two parts of this task: (i) use install_name('-id',...) to change install name of a single lib; (ii) use install_name('-change',...) to change the cross linking between libs. The function assumes that all libraries are in one folder and currently won't follow subfolders.

Args: path: directory in which .dylib files are located

spack.**find_libraries**(*args*, *root*, *shared=True*, *recurse=False*)
Returns an iterable object containing a list of full paths to libraries if found.

Args: args: iterable object containing a list of library names to search for (e.g. 'libhdf5') root: root folder where to start searching shared: if True searches for shared libraries, otherwise for static recurse: if False search only root folder, if True descends top-down from the root

Returns: list of full paths to the libraries that have been found

class spack.**LibraryList**(*libraries*)

Bases: _abcoll.Sequence

Sequence of absolute paths to libraries

Provides a few convenience methods to manipulate library paths and get commonly used compiler flags or names

basenames

Stable de-duplication of the base-names in the list

```
>>> l = LibraryList(['/dir1/liba.a', '/dir2/libb.a', '/dir3/liba.a'])
>>> assert l.basenames == ['liba.a', 'libb.a']
```

directories

Stable de-duplication of the directories where the libraries reside

```
>>> l = LibraryList(['/dir1/liba.a', '/dir2/libb.a', '/dir1/libc.a'])
>>> assert l.directories == ['/dir1', '/dir2']
```

joined(*separator=' '*)

ld_flags

Search flags + link flags

```
>>> l = LibraryList(['/dir1/liba.a', '/dir2/libb.a', '/dir1/liba.so'])
>>> assert l.search_flags == '-L/dir1 -L/dir2 -la -lb'
```

link_flags

Link flags for the libraries

```
>>> l = LibraryList(['/dir1/liba.a', '/dir2/libb.a', '/dir1/liba.so'])
>>> assert l.search_flags == '-la -lb'
```

names

Stable de-duplication of library names in the list

```
>>> l = LibraryList(['/dir1/liba.a', '/dir2/libb.a', '/dir3/liba.so'])
>>> assert l.names == ['a', 'b']
```

search_flags

Search flags for the libraries

```
>>> l = LibraryList(['/dir1/liba.a', '/dir2/libb.a', '/dir1/liba.so'])
>>> assert l.search_flags == '-L/dir1 -L/dir2'
```

spack.**depends_on**(**args*, ***kwargs*)

Creates a dict of deps with specs defining when they apply. This directive is to be used inside a Package definition to declare that the package requires other packages to be built first. @see The section "Dependency specs" in the Spack Packaging Guide.

spack.**extends**(**args*, ***kwargs*)

Same as depends_on, but dependency is symlinked into parent prefix.

This is for Python and other language modules where the module needs to be installed into the prefix of the Python installation. Spack handles this by installing modules into their own prefix, but allowing ONE module version to be symlinked into a parent Python install at a time.

keyword arguments can be passed to extends() so that extension packages can pass parameters to the extendee's extension mechanism.

spack.**provides**(**args*, ***kwargs*)
Allows packages to provide a virtual dependency. If a package provides 'mpi', other packages can declare that they depend on "mpi", and spack can use the providing package to satisfy the dependency.

spack.**patch**(**args*, ***kwargs*)
Packages can declare patches to apply to source. You can optionally provide a when spec to indicate that a particular patch should only be applied when the package's spec meets certain conditions (e.g. a particular version).

spack.**version**(**args*, ***kwargs*)
Adds a version and metadata describing how to fetch it. Metadata is just stored as a dict in the package's versions dictionary. Package must turn it into a valid fetch strategy later.

spack.**variant**(**args*, ***kwargs*)
Define a variant for the package. Packager can specify a default value (on or off) as well as a text description.

spack.**resource**(**args*, ***kwargs*)
Define an external resource to be fetched and staged when building the package. Based on the keywords present in the dictionary the appropriate FetchStrategy will be used for the resource. Resources are fetched and staged in their own folder inside spack stage area, and then moved into the stage area of the package that needs them.

List of recognized keywords:

- 'when' : (optional) represents the condition upon which the resource is needed
- 'destination' : (optional) path where to move the resource. This path must be relative to the main package stage area.
- 'placement' : (optional) gives the possibility to fine tune how the resource is moved into the main package stage area.

class spack.**Executable**(*name*)
Bases: object

Class representing a program that can be run on the command line.

add_default_arg(*arg*)

command

spack.**which**(*name*, ***kwargs*)
Finds an executable in the path like command-line which.

exception spack.**ProcessError**(*msg*, *long_message=None*)
Bases: *spack.error.SpackError*

long_message

spack.**install_dependency_symlinks**(*pkg*, *spec*, *prefix*)
Execute a dummy install and flatten dependencies

spack.**flatten_dependencies**(*spec*, *flat_dir*)
Make each dependency of spec present in dir via symlink.

exception spack.**DependencyConflictError**(*conflict*)
Bases: *spack.error.SpackError*

Raised when the dependencies cannot be flattened as asked for.

exception spack.**InstallError**(*message*, *long_msg=None*)
Bases: *spack.error.SpackError*

Raised when something goes wrong during install or uninstall.

exception spack.**ExternalPackageError**(*message*, *long_msg=None*)
Bases: *spack.package.InstallError*

Raised by install() when a package is only for external use.

CHAPTER 13

Indices and tables

- genindex
- modindex
- search

S

spack.abi, 183
spack.architecture, 183
spack.build_environment, 185
spack.cmd, 144
spack.cmd.activate, 133
spack.cmd.arch, 133
spack.cmd.bootstrap, 133
spack.cmd.cd, 134
spack.cmd.checksum, 134
spack.cmd.clean, 134
spack.cmd.common, 133
spack.cmd.common.arguments, 133
spack.cmd.compiler, 134
spack.cmd.compilers, 134
spack.cmd.config, 134
spack.cmd.create, 135
spack.cmd.deactivate, 135
spack.cmd.debug, 135
spack.cmd.dependents, 135
spack.cmd.diy, 135
spack.cmd.doc, 135
spack.cmd.edit, 136
spack.cmd.env, 136
spack.cmd.extensions, 136
spack.cmd.fetch, 136
spack.cmd.find, 136
spack.cmd.graph, 136
spack.cmd.help, 136
spack.cmd.info, 136
spack.cmd.install, 137
spack.cmd.list, 137
spack.cmd.load, 137
spack.cmd.location, 137
spack.cmd.md5, 137
spack.cmd.mirror, 137
spack.cmd.module, 138
spack.cmd.package_list, 138
spack.cmd.patch, 138
spack.cmd.pkg, 138
spack.cmd.providers, 139
spack.cmd.purge, 139
spack.cmd.python, 139
spack.cmd.reindex, 139
spack.cmd.repo, 139
spack.cmd.restage, 140
spack.cmd.setup, 140
spack.cmd.spec, 140
spack.cmd.stage, 140
spack.cmd.test, 140
spack.cmd.test_install, 140
spack.cmd.uninstall, 141
spack.cmd.unload, 141
spack.cmd.unuse, 142
spack.cmd.url_parse, 142
spack.cmd.urls, 142
spack.cmd.use, 142
spack.cmd.versions, 142
spack.cmd.view, 142
spack.compiler, 187
spack.compilers, 148
spack.compilers.cce, 144
spack.compilers.clang, 145
spack.compilers.gcc, 145
spack.compilers.intel, 146
spack.compilers.nag, 146
spack.compilers.pgi, 147
spack.compilers.xl, 147
spack.concretize, 188
spack.config, 189
spack.database, 192
spack.directives, 193
spack.directory_layout, 194
spack.environment, 197
spack.error, 199
spack.fetch_strategy, 199
spack.file_cache, 203
spack.graph, 204
spack.hooks, 150
spack.hooks.dotkit, 149
spack.hooks.extensions, 149

spack.hooks.licensing, 149
spack.hooks.lmodmodule, 150
spack.hooks.sbang, 150
spack.hooks.tclmodule, 150
spack.mirror, 205
spack.modules, 206
spack.multimethod, 207
spack.operating_systems, 151
spack.operating_systems.cnl, 150
spack.operating_systems.linux_distro, 151
spack.operating_systems.mac_os, 151
spack.package, 209
spack.package_test, 218
spack.parse, 219
spack.patch, 219
spack.platforms, 152
spack.platforms.bgq, 151
spack.platforms.cray, 151
spack.platforms.darwin, 152
spack.platforms.linux, 152
spack.platforms.test, 152
spack.preferred_packages, 220
spack.provider_index, 220
spack.repository, 221
spack.resource, 224
spack.schema, 153
spack.schema.compilers, 152
spack.schema.mirrors, 153
spack.schema.modules, 153
spack.schema.packages, 153
spack.schema.repos, 153
spack.schema.targets, 153
spack.spec, 225
spack.stage, 231
spack.test, 178
spack.test.architecture, 155
spack.test.build_system_guess, 155
spack.test.cc, 156
spack.test.cmd, 155
spack.test.cmd.find, 153
spack.test.cmd.module, 153
spack.test.cmd.test_compiler_cmd, 154
spack.test.cmd.test_install, 154
spack.test.cmd.uninstall, 154
spack.test.concretize, 156
spack.test.concretize_preferences, 157
spack.test.config, 158
spack.test.database, 158
spack.test.directory_layout, 159
spack.test.environment, 159
spack.test.file_cache, 160
spack.test.git_fetch, 160
spack.test.hg_fetch, 161
spack.test.install, 161
spack.test.library_list, 161
spack.test.link_tree, 162
spack.test.lock, 162
spack.test.make_executable, 163
spack.test.mirror, 163
spack.test.mock_database, 164
spack.test.mock_packages_test, 164
spack.test.mock_repo, 164
spack.test.modules, 165
spack.test.multimethod, 166
spack.test.namespace_trie, 167
spack.test.optional_deps, 167
spack.test.package_sanity, 167
spack.test.packages, 167
spack.test.pattern, 168
spack.test.provider_index, 168
spack.test.python_version, 169
spack.test.sbang, 169
spack.test.spec_dag, 169
spack.test.spec_semantics, 170
spack.test.spec_syntax, 171
spack.test.spec_yaml, 172
spack.test.stage, 172
spack.test.svn_fetch, 173
spack.test.tally_plugin, 174
spack.test.url_extrapolate, 174
spack.test.url_parse, 175
spack.test.url_substitution, 176
spack.test.versions, 177
spack.test.yaml, 178
spack.url, 234
spack.util, 183
spack.util.compression, 178
spack.util.crypto, 179
spack.util.debug, 179
spack.util.environment, 179
spack.util.executable, 180
spack.util.multiproc, 180
spack.util.naming, 180
spack.util.pattern, 181
spack.util.prefix, 182
spack.util.spack_yaml, 182
spack.util.string, 182
spack.util.web, 182
spack.variant, 236
spack.version, 236
spack.yaml_version_check, 238

A

ABI (class in spack.abi), 183
accept() (spack.parse.Parser method), 219
acquire_read() (spack.test.lock.LockTest method), 162
acquire_write() (spack.test.lock.LockTest method), 162
activate() (in module spack.cmd.activate), 133
activate() (spack.package.Package method), 213
activated (spack.package.Package attribute), 213
ActivationError, 209
add() (spack.database.Database method), 192
add() (spack.version.VersionList method), 237
add_common_arguments() (in module spack.cmd.common.arguments), 133
add_compilers_to_config() (in module spack.compilers), 148
add_default_arg() (spack.util.executable.Executable method), 180
add_extension() (spack.directory_layout.DirectoryLayout method), 194
add_extension() (spack.directory_layout.YamlDirectoryLayout method), 196
add_operating_system() (spack.architecture.Platform method), 185
add_single_spec() (in module spack.mirror), 205
add_target() (spack.architecture.Platform method), 185
addError() (spack.test.tally_plugin.Tally method), 174
addFailure() (spack.test.tally_plugin.Tally method), 174
addSuccess() (spack.test.tally_plugin.Tally method), 174
all_compiler_types() (in module spack.compilers), 148
all_compilers() (in module spack.compilers), 148
all_compilers_config() (in module spack.compilers), 148
all_os_classes() (in module spack.compilers), 148
all_package_names() (spack.repository.Repo method), 222
all_package_names() (spack.repository.RepoPath method), 223
all_packages() (spack.repository.Repo method), 222
all_packages() (spack.repository.RepoPath method), 223
all_specs() (spack.directory_layout.DirectoryLayout method), 195
all_specs() (spack.directory_layout.YamlDirectoryLayout method), 196
all_urls (spack.package.Package attribute), 213
allowed_archive() (in module spack.util.compression), 178
AmbiguousHashError, 231
append() (spack.cmd.test_install.TestSuite method), 141
append_path() (spack.environment.EnvironmentModifications method), 197
AppendPath (class in spack.environment), 197
apply() (spack.patch.Patch method), 220
apply_modifications() (spack.environment.EnvironmentModifications method), 197
Arch (class in spack.architecture), 184
arch() (in module spack.cmd.arch), 133
arch_from_dict() (in module spack.architecture), 185
architecture_compare() (spack.preferred_packages.PreferredPackages method), 220
architecture_compatible() (spack.abi.ABI method), 183
ArchitectureTest (class in spack.test.architecture), 155
archive() (spack.fetch_strategy.FetchStrategy method), 200
archive() (spack.fetch_strategy.GitFetchStrategy method), 201
archive() (spack.fetch_strategy.GoFetchStrategy method), 201
archive() (spack.fetch_strategy.HgFetchStrategy method), 201
archive() (spack.fetch_strategy.SvnFetchStrategy method), 202
archive() (spack.fetch_strategy.URLFetchStrategy method), 202
archive() (spack.fetch_strategy.VCSFetchStrategy method), 202
archive_file (spack.fetch_strategy.URLFetchStrategy attribute), 202
archive_file (spack.stage.Stage attribute), 233
args_are_for() (in module spack.fetch_strategy), 203
AsciiGraph (class in spack.graph), 204
ask_for_confirmation() (in module spack.cmd), 144

assert_canonical() (spack.test.versions.VersionsTest method), 177
assert_does_not_satisfy() (spack.test.versions.VersionsTest method), 177
assert_in() (spack.test.versions.VersionsTest method), 177
assert_no_overlap() (spack.test.versions.VersionsTest method), 177
assert_not_detected() (spack.test.url_parse.UrlParseTest method), 175
assert_not_in() (spack.test.versions.VersionsTest method), 177
assert_overlaps() (spack.test.versions.VersionsTest method), 177
assert_rev() (spack.test.git_fetch.GitFetchTest method), 160
assert_rev() (spack.test.svn_fetch.SvnFetchTest method), 173
assert_satisfies() (spack.test.versions.VersionsTest method), 177
assert_variant_values() (spack.test.concretize_preferences.ConcretizePreferencesTest method), 157
assert_ver_eq() (spack.test.versions.VersionsTest method), 177
assert_ver_gt() (spack.test.versions.VersionsTest method), 177
assert_ver_lt() (spack.test.versions.VersionsTest method), 177
assuredir() (in module spack.cmd.view), 142
autoload() (spack.modules.EnvModule method), 206
autoload_format (spack.modules.Dotkit attribute), 206
autoload_format (spack.modules.TclModule attribute), 207

B

back_end (spack.architecture.Platform attribute), 185
back_end (spack.platforms.bgq.Bgq attribute), 151
back_end (spack.platforms.darwin.Darwin attribute), 152
back_end (spack.platforms.test.Test attribute), 152
back_os (spack.architecture.Platform attribute), 185
back_os (spack.platforms.test.Test attribute), 152
BadRepoError, 221
Barrier (class in spack.util.multiproc), 180
Bgq (class in spack.platforms.bgq), 151
blacklisted (spack.modules.EnvModule attribute), 206
bootstrap() (in module spack.cmd.bootstrap), 133
build_env_path() (spack.directory_layout.YamlDirectoryLayout method), 196
build_log_path (spack.package.Package attribute), 213
build_log_path() (spack.directory_layout.YamlDirectoryLayout method), 196
build_packages_path() (spack.directory_layout.YamlDirectoryLayout method), 196
BuildSystemGuesser (class in spack.cmd.create), 135
Bunch (class in spack.util.pattern), 181

C

cache_local() (spack.stage.DIYStage method), 232
cache_local() (spack.stage.Stage method), 233
cache_path() (spack.file_cache.FileCache method), 203
CacheError, 203
CacheURLFetchStrategy (class in spack.fetch_strategy), 200
canonical_deptype() (in module spack.spec), 229
canonicalize_path() (in module spack.repository), 224
category (spack.modules.EnvModule attribute), 206
cc_names (spack.compiler.Compiler attribute), 187
cc_names (spack.compilers.cce.Cce attribute), 144
cc_names (spack.compilers.clang.Clang attribute), 145
cc_names (spack.compilers.gcc.Gcc attribute), 145
cc_names (spack.compilers.intel.Intel attribute), 146
cc_names (spack.compilers.nag.Nag attribute), 146
cc_names (spack.compilers.pgi.Pgi attribute), 147
cc_names (spack.compilers.xl.Xl attribute), 147
cc_rpath_arg (spack.compiler.Compiler attribute), 187
cc_version() (spack.compiler.Compiler class method), 187
Cce (class in spack.compilers.cce), 144
cd() (in module spack.cmd.cd), 134
chdir() (spack.stage.DIYStage method), 232
chdir() (spack.stage.Stage method), 233
chdir_to_source() (spack.stage.DIYStage method), 232
chdir_to_source() (spack.stage.Stage method), 233
ChdirError, 231
check() (spack.fetch_strategy.FetchStrategy method), 200
check() (spack.fetch_strategy.URLFetchStrategy method), 202
check() (spack.fetch_strategy.VCSFetchStrategy method), 203
check() (spack.stage.DIYStage method), 232
check() (spack.stage.Stage method), 233
check() (spack.test.url_parse.UrlParseTest method), 175
check() (spack.util.crypto.Checker method), 179
check_activated() (spack.directory_layout.DirectoryLayout method), 195
check_activated() (spack.directory_layout.YamlDirectoryLayout method), 196
check_archive() (spack.test.build_system_guess.InstallTest method), 155
check_cc() (spack.test.cc.CompilerTest method), 156
check_chdir() (spack.test.stage.StageTest method), 172
check_chdir_to_source() (spack.test.stage.StageTest method), 173
check_compiler_yaml_version() (in module spack.yaml_version_check), 238
check_concretize() (spack.test.concretize.ConcretizeTest method), 156

check_config() (spack.test.config.ConfigTest method), 158
check_constrain() (spack.test.spec_semantics.SpecSematicsTest method), 170
check_constrain_changed() (spack.test.spec_semantics.SpecSematicsTest method), 170
check_constrain_not_changed() (spack.test.spec_semantics.SpecSematicsTest method), 170
check_cpp() (spack.test.cc.CompilerTest method), 156
check_cxx() (spack.test.cc.CompilerTest method), 156
check_db() (spack.test.package_sanity.PackageSanityTest method), 167
check_destroy() (spack.test.stage.StageTest method), 173
check_dir() (spack.test.link_tree.LinkTreeTest method), 162
check_expand_archive() (spack.test.stage.StageTest method), 173
check_extension_conflict() (spack.directory_layout.DirectoryLayout method), 195
check_extension_conflict() (spack.directory_layout.YamlDirectoryLayout method), 196
check_fc() (spack.test.cc.CompilerTest method), 156
check_fetch() (spack.test.stage.StageTest method), 173
check_file_link() (spack.test.link_tree.LinkTreeTest method), 162
check_installed() (spack.directory_layout.DirectoryLayout method), 195
check_installed() (spack.directory_layout.YamlDirectoryLayout method), 196
check_intersection() (spack.test.versions.VersionsTest method), 177
check_invalid_constraint() (spack.test.spec_semantics.SpecSematicsTest method), 170
check_ld() (spack.test.cc.CompilerTest method), 156
check_lex() (spack.test.spec_syntax.SpecSyntaxTest method), 171
check_links() (spack.test.spec_dag.SpecDagTest method), 169
check_mirror() (spack.test.mirror.MirrorTest method), 163
check_normalize() (spack.test.optional_deps.ConcretizeTest method), 167
check_one() (in module spack.cmd.view), 143
check_parse() (spack.test.spec_syntax.SpecSyntaxTest method), 171
check_python_versions() (spack.test.python_version.PythonVersionTest method), 169
check_satisfies() (spack.test.spec_semantics.SpecSematicsTest method), 170
check_setup() (spack.test.stage.StageTest method), 173
check_spec() (spack.test.concretize.ConcretizeTest method), 156
check_union() (spack.test.versions.VersionsTest method), 177
check_unsatisfiable() (spack.test.spec_semantics.SpecSematicsTest method), 170
check_url() (spack.test.url_extrapolate.UrlExtrapolateTest method), 174
check_yaml_round_trip() (spack.test.spec_yaml.SpecYamlTest method), 172
check_yaml_versions() (in module spack.yaml_version_check), 238
Checker (class in spack.util.crypto), 179
checksum() (in module spack.cmd.checksum), 134
checksum() (in module spack.util.crypto), 179
ChecksumError, 200
choose_virtual_or_external() (spack.concretize.DefaultConcretizer method), 188
Clang (class in spack.compilers.clang), 145
class_for_compiler_name() (in module spack.compilers), 148
clean() (in module spack.cmd.clean), 134
cleanmock() (spack.test.mock_packages_test.MockPackagesTest method), 164
clear() (spack.config.ConfigScope method), 191
clear() (spack.environment.EnvironmentModifications method), 197
clear_config_caches() (in module spack.config), 191
CMakePackage (class in spack.package), 209
cmp_specs() (in module spack.concretize), 189
Cnl (class in spack.operating_systems.cnl), 150
color_url() (in module spack.url), 234
colorized() (spack.spec.Spec method), 226
comma_and() (in module spack.util.string), 182
comma_list() (in module spack.util.string), 182
comma_or() (in module spack.util.string), 182
command (spack.util.executable.Executable attribute), 180
common_dependencies() (spack.spec.Spec method), 226
compare_output() (in module spack.package_test), 218
compare_output_file() (in module spack.package_test), 218
compatible() (spack.abi.ABI method), 183
compile_c_and_execute() (in module spack.package_test), 218
Compiler (class in spack.compiler), 187
compiler (spack.package.Package attribute), 213
compiler() (in module spack.cmd.compiler), 134

compiler_compare() (spack.preferred_packages.PreferredPackages method), 220
compiler_compatible() (spack.abi.ABI method), 183
compiler_find() (in module spack.cmd.compiler), 134
compiler_for_spec() (in module spack.compilers), 148
compiler_info() (in module spack.cmd.compiler), 134
compiler_list() (in module spack.cmd.compiler), 134
compiler_remove() (in module spack.cmd.compiler), 134
CompilerCmdTest (class in spack.test.cmd.test_compiler_cmd), 154
compilers() (in module spack.cmd.compilers), 134
compilers_for_spec() (in module spack.compilers), 148
CompilerSpecInsufficientlySpecificError, 148
CompilerTest (class in spack.test.cc), 156
composite() (in module spack.util.pattern), 181
CompositeTest (class in spack.test.pattern), 168
compute_md5_checksum() (in module spack.cmd.md5), 137
concatenate_paths() (in module spack.environment), 198
concrete (spack.architecture.Arch attribute), 184
concrete (spack.spec.Spec attribute), 226
concrete (spack.version.Version attribute), 236
concrete (spack.version.VersionList attribute), 237
concrete (spack.version.VersionRange attribute), 237
concretize() (spack.spec.Spec method), 226
concretize() (spack.test.concretize_preferences.ConcretizePreferencesTest method), 157
concretize_architecture() (spack.concretize.DefaultConcretizer method), 188
concretize_compiler() (spack.concretize.DefaultConcretizer method), 188
concretize_compiler_flags() (spack.concretize.DefaultConcretizer method), 188
concretize_specs() (in module spack.cmd.uninstall), 141
concretize_variants() (spack.concretize.DefaultConcretizer method), 188
concretize_version() (spack.concretize.DefaultConcretizer method), 188
concretized() (spack.spec.Spec method), 226
ConcretizePreferencesTest (class in spack.test.concretize_preferences), 157
ConcretizeTest (class in spack.test.concretize), 156
ConcretizeTest (class in spack.test.optional_deps), 167
config() (in module spack.cmd.config), 134
config_edit() (in module spack.cmd.config), 134
config_get() (in module spack.cmd.config), 134
ConfigError, 190
ConfigFileError, 190
ConfigFormatError, 190
ConfigSanityError, 190
ConfigScope (class in spack.config), 190
ConfigTest (class in spack.test.config), 158
configuration_alter_environment (spack.test.modules.LmodTests attribute), 165
configuration_alter_environment (spack.test.modules.TclTests attribute), 166
configuration_autoload_all (spack.test.modules.LmodTests attribute), 165
configuration_autoload_all (spack.test.modules.TclTests attribute), 166
configuration_autoload_direct (spack.test.modules.LmodTests attribute), 165
configuration_autoload_direct (spack.test.modules.TclTests attribute), 166
configuration_blacklist (spack.test.modules.LmodTests attribute), 165
configuration_blacklist (spack.test.modules.TclTests attribute), 166
configuration_conflicts (spack.test.modules.TclTests attribute), 166
configuration_dotkit (spack.test.modules.DotkitTests attribute), 165
configuration_prerequisites_all (spack.test.modules.TclTests attribute), 166
configuration_prerequisites_direct (spack.test.modules.TclTests attribute), 166
configuration_suffix (spack.test.modules.TclTests attribute), 166
configuration_wrong_conflicts (spack.test.modules.TclTests attribute), 166
configure() (spack.test.tally_plugin.Tally method), 174
configure_args() (spack.package.CMakePackage method), 209
configure_env() (spack.package.CMakePackage method), 209
constrain() (spack.spec.Spec method), 226
constrained() (spack.spec.Spec method), 226
copy() (spack.provider_index.ProviderIndex method), 221
copy() (spack.spec.Spec method), 226
copy() (spack.version.VersionList method), 237
CorruptDatabaseError, 192
Cray (class in spack.platforms.cray), 151
create() (in module spack.cmd.create), 135
create() (in module spack.mirror), 205
create() (spack.stage.Stage method), 233
create_db_tarball() (in module spack.cmd.debug), 135
create_install_directory() (spack.directory_layout.DirectoryLayout method), 195
create_install_directory() (spack.directory_layout.YamlDirectoryLayout method), 197

create_repo() (in module spack.repository), 224
cshort_spec (spack.spec.Spec attribute), 226
cumsum() (in module spack.url), 234
cxx11_flag (spack.compiler.Compiler attribute), 187
cxx11_flag (spack.compilers.clang.Clang attribute), 145
cxx11_flag (spack.compilers.gcc.Gcc attribute), 145
cxx11_flag (spack.compilers.intel.Intel attribute), 146
cxx11_flag (spack.compilers.nag.Nag attribute), 146
cxx11_flag (spack.compilers.pgi.Pgi attribute), 147
cxx11_flag (spack.compilers.xl.Xl attribute), 147
cxx14_flag (spack.compiler.Compiler attribute), 187
cxx14_flag (spack.compilers.gcc.Gcc attribute), 145
cxx_names (spack.compiler.Compiler attribute), 187
cxx_names (spack.compilers.cce.Cce attribute), 144
cxx_names (spack.compilers.clang.Clang attribute), 145
cxx_names (spack.compilers.gcc.Gcc attribute), 145
cxx_names (spack.compilers.intel.Intel attribute), 146
cxx_names (spack.compilers.nag.Nag attribute), 146
cxx_names (spack.compilers.pgi.Pgi attribute), 147
cxx_names (spack.compilers.xl.Xl attribute), 147
cxx_rpath_arg (spack.compiler.Compiler attribute), 187
cxx_version() (spack.compiler.Compiler class method), 187

D

dag_hash() (spack.spec.Spec method), 226
dag_hash() (spack.test.cmd.test_install.MockSpec method), 154
Darwin (class in spack.platforms.darwin), 152
dashed (spack.version.Version attribute), 236
Database (class in spack.database), 192
DatabaseTest (class in spack.test.database), 158
deactivate() (in module spack.cmd.deactivate), 135
deactivate() (spack.package.Package method), 213
debug() (in module spack.cmd.debug), 135
debug_handler() (in module spack.util.debug), 179
decompressor_for() (in module spack.util.compression), 178
default (spack.architecture.Platform attribute), 185
default (spack.platforms.bgq.Bgq attribute), 151
default (spack.platforms.darwin.Darwin attribute), 152
default (spack.platforms.test.Test attribute), 152
default_compiler() (in module spack.compilers), 149
default_naming_format (spack.modules.Dotkit attribute), 206
default_naming_format (spack.modules.TclModule attribute), 207
default_os (spack.architecture.Platform attribute), 185
default_os (spack.platforms.test.Test attribute), 152
default_version() (spack.compiler.Compiler class method), 187
default_version() (spack.compilers.cce.Cce class method), 144
default_version() (spack.compilers.clang.Clang class method), 145
default_version() (spack.compilers.intel.Intel class method), 146
default_version() (spack.compilers.nag.Nag class method), 146
default_version() (spack.compilers.pgi.Pgi class method), 147
default_version() (spack.compilers.xl.Xl class method), 147
DefaultConcretizer (class in spack.concretize), 188
dep_difference() (spack.spec.Spec method), 226
dep_string() (spack.spec.Spec method), 226
dependencies (spack.package.Package attribute), 213
dependencies() (spack.spec.Spec method), 226
dependencies() (spack.test.cmd.test_install.MockSpec method), 154
dependencies_dict() (spack.spec.Spec method), 226
dependencies_of_type() (spack.package.Package method), 213
DependencyConflictError, 209
dependents() (in module spack.cmd.dependents), 135
dependents() (spack.spec.Spec method), 226
dependents() (spack.test.cmd.test_install.MockSpec method), 154
dependents_dict() (spack.spec.Spec method), 226
depends_on() (in module spack.directives), 194
destroy() (spack.fetch_strategy.FsCache method), 200
destroy() (spack.file_cache.FileCache method), 203
destroy() (spack.stage.DIYStage method), 232
destroy() (spack.stage.Stage method), 233
destroy() (spack.test.mock_repo.MockRepo method), 165
detect() (spack.architecture.Platform class method), 185
detect() (spack.platforms.bgq.Bgq class method), 151
detect() (spack.platforms.cray.Cray class method), 151
detect() (spack.platforms.darwin.Darwin class method), 152
detect() (spack.platforms.linux.Linux class method), 152
detect() (spack.platforms.test.Test class method), 152
die() (spack.error.SpackError method), 199
diff_packages() (in module spack.cmd.pkg), 138
DirectoryLayout (class in spack.directory_layout), 194
DirectoryLayoutError, 195
DirectoryLayoutTest (class in spack.test.directory_layout), 159
dirname_for_package_name() (spack.repository.Repo method), 222
dirname_for_package_name() (spack.repository.RepoPath method), 223
disambiguate_spec() (in module spack.cmd), 144
display_specs() (in module spack.cmd), 144
diy() (in module spack.cmd.diy), 135
DIYStage (class in spack.stage), 231
do_activate() (spack.package.Package method), 213

do_clean() (spack.package.Package method), 213
do_deactivate() (spack.package.Package method), 213
do_fake_install() (spack.package.Package method), 213
do_fetch() (spack.package.Package method), 213
do_install() (spack.package.Package method), 213
do_install() (spack.test.cmd.test_install.MockPackage method), 154
do_install_dependencies() (spack.package.Package method), 214
do_patch() (spack.package.Package method), 214
do_restage() (spack.package.Package method), 214
do_stage() (spack.package.Package method), 214
do_uninstall() (in module spack.cmd.uninstall), 141
do_uninstall() (spack.package.Package method), 214
doc() (in module spack.cmd.doc), 135
Dotkit (class in spack.modules), 206
DotkitTests (class in spack.test.modules), 165
dotted (spack.version.Version attribute), 236
downloaded_file_extension() (in module spack.url), 234
dump() (in module spack.util.spack_yaml), 182
dump_environment() (in module spack.util.environment), 179
dump_packages() (in module spack.package), 218
dump_provenance() (spack.repository.Repo method), 222
dump_provenance() (spack.repository.RepoPath method), 223
DuplicateArchitectureError, 230
DuplicateCompilerSpecError, 230
DuplicateDependencyError, 230
DuplicateRepoError, 221
DuplicateVariantError, 230

E

edit() (in module spack.cmd.edit), 136
edit_package() (in module spack.cmd.edit), 136
elide_list() (in module spack.cmd), 144
enabled (spack.fetch_strategy.FetchStrategy attribute), 200
enabled (spack.fetch_strategy.GitFetchStrategy attribute), 201
enabled (spack.fetch_strategy.GoFetchStrategy attribute), 201
enabled (spack.fetch_strategy.HgFetchStrategy attribute), 201
enabled (spack.fetch_strategy.SvnFetchStrategy attribute), 202
enabled (spack.fetch_strategy.URLFetchStrategy attribute), 202
ensure_access() (in module spack.stage), 233
env() (in module spack.cmd.env), 136
env_flag() (in module spack.util.environment), 179
environment_modifications_formats (spack.modules.Dotkit attribute), 206
EnvironmentModifications (class in spack.environment), 197
EnvironmentTest (class in spack.test.environment), 159
EnvModule (class in spack.modules), 206
eq_dag() (spack.spec.Spec method), 226
eq_node() (spack.spec.Spec method), 226
ERRORED (spack.cmd.test_install.TestResult attribute), 140
Executable (class in spack.util.executable), 180
execute() (spack.environment.AppendPath method), 197
execute() (spack.environment.PrependPath method), 198
execute() (spack.environment.RemovePath method), 198
execute() (spack.environment.SetEnv method), 198
execute() (spack.environment.SetPath method), 198
execute() (spack.environment.UnsetEnv method), 198
exists() (spack.repository.Repo method), 222
exists() (spack.repository.RepoPath method), 223
expand() (spack.fetch_strategy.FetchStrategy method), 200
expand() (spack.fetch_strategy.URLFetchStrategy method), 202
expand() (spack.fetch_strategy.VCSFetchStrategy method), 203
expand_archive() (spack.stage.DIYStage method), 232
expand_archive() (spack.stage.ResourceStage method), 232
expand_archive() (spack.stage.Stage method), 233
expect() (spack.parse.Parser method), 219
expected_archive_files (spack.stage.Stage attribute), 233
extend() (spack.environment.EnvironmentModifications method), 197
extend_with_default() (in module spack.config), 191
extendable (spack.package.Package attribute), 214
extendee_args (spack.package.Package attribute), 214
extendee_spec (spack.package.Package attribute), 214
extendees (spack.package.Package attribute), 214
extends() (in module spack.directives), 194
extends() (spack.package.Package method), 214
extension() (in module spack.util.compression), 179
extension_file_path() (spack.directory_layout.YamlDirectoryLayout method), 197
extension_map() (spack.directory_layout.DirectoryLayout method), 195
extension_map() (spack.directory_layout.YamlDirectoryLayout method), 197
ExtensionAlreadyInstalledError, 195
ExtensionConflictError, 195, 209
ExtensionError, 209
extensions() (in module spack.cmd.extensions), 136
extensions_for() (spack.repository.Repo method), 222
extensions_for() (spack.repository.RepoPath method), 223
ExternalPackageError, 209

F

f77_names (spack.compiler.Compiler attribute), 187
f77_names (spack.compilers.cce.Cce attribute), 144
f77_names (spack.compilers.clang.Clang attribute), 145
f77_names (spack.compilers.gcc.Gcc attribute), 145
f77_names (spack.compilers.intel.Intel attribute), 146
f77_names (spack.compilers.nag.Nag attribute), 146
f77_names (spack.compilers.pgi.Pgi attribute), 147
f77_names (spack.compilers.xl.Xl attribute), 148
f77_rpath_arg (spack.compiler.Compiler attribute), 187
f77_rpath_arg (spack.compilers.nag.Nag attribute), 146
f77_version() (spack.compiler.Compiler class method), 187
f77_version() (spack.compilers.gcc.Gcc class method), 145
f77_version() (spack.compilers.xl.Xl class method), 148
factory (spack.test.modules.LmodTests attribute), 165
factory (spack.test.modules.TclTests attribute), 166
FAILED (spack.cmd.test_install.TestResult attribute), 140
failed_dependencies() (in module spack.cmd.test_install), 141
FailedConstructorError, 221
FailedDownloadError, 200
fake_fetchify() (spack.test.install.InstallTest method), 161
fc_names (spack.compiler.Compiler attribute), 187
fc_names (spack.compilers.cce.Cce attribute), 144
fc_names (spack.compilers.clang.Clang attribute), 145
fc_names (spack.compilers.gcc.Gcc attribute), 145
fc_names (spack.compilers.intel.Intel attribute), 146
fc_names (spack.compilers.nag.Nag attribute), 146
fc_names (spack.compilers.pgi.Pgi attribute), 147
fc_names (spack.compilers.xl.Xl attribute), 148
fc_rpath_arg (spack.compiler.Compiler attribute), 187
fc_rpath_arg (spack.compilers.nag.Nag attribute), 146
fc_version() (spack.compiler.Compiler class method), 187
fc_version() (spack.compilers.gcc.Gcc class method), 145
fc_version() (spack.compilers.xl.Xl class method), 148
fetch() (in module spack.cmd.fetch), 136
fetch() (spack.cmd.test.MockCacheFetcher method), 140
fetch() (spack.fetch_strategy.CacheURLFetchStrategy method), 200
fetch() (spack.fetch_strategy.FetchStrategy method), 200
fetch() (spack.fetch_strategy.GitFetchStrategy method), 201
fetch() (spack.fetch_strategy.GoFetchStrategy method), 201
fetch() (spack.fetch_strategy.HgFetchStrategy method), 201
fetch() (spack.fetch_strategy.SvnFetchStrategy method), 202
fetch() (spack.fetch_strategy.URLFetchStrategy method), 202
fetch() (spack.stage.DIYStage method), 232
fetch() (spack.stage.Stage method), 233
fetch_log() (in module spack.cmd.test_install), 141
fetch_remote_versions() (spack.package.Package method), 214
fetch_tarballs() (in module spack.cmd.create), 135
fetcher (spack.package.Package attribute), 214
fetcher() (spack.cmd.test.MockCache method), 140
fetcher() (spack.fetch_strategy.FsCache method), 200
FetchError, 200, 210
FetchStrategy (class in spack.fetch_strategy), 200
file_name (spack.modules.Dotkit attribute), 206
file_name (spack.modules.EnvModule attribute), 206
file_name (spack.modules.TclModule attribute), 207
FileCache (class in spack.file_cache), 203
FileCacheTest (class in spack.test.file_cache), 160
filename_for_package_name() (spack.repository.Repo method), 222
filename_for_package_name() (spack.repository.RepoPath method), 223
filter_environment_blacklist() (in module spack.environment), 199
filter_exclude() (in module spack.cmd.view), 143
filter_shebang() (in module spack.hooks.sbang), 150
filter_shebangs_in_directory() (in module spack.hooks.sbang), 150
finalize() (spack.test.tally_plugin.Tally method), 174
find() (in module spack.cmd.find), 136
find() (in module spack.cmd.module), 138
find() (in module spack.compilers), 149
find_compiler() (spack.architecture.OperatingSystem method), 184
find_compiler() (spack.operating_systems.cnl.Cnl method), 151
find_compilers() (in module spack.compilers), 149
find_compilers() (spack.architecture.OperatingSystem method), 184
find_compilers() (spack.operating_systems.cnl.Cnl method), 151
find_list_url() (in module spack.url), 235
find_module() (spack.repository.Repo method), 222
find_module() (spack.repository.RepoPath method), 223
find_repository() (in module spack.cmd.create), 135
find_spec() (in module spack.concretize), 189
find_tmp_root() (in module spack.stage), 234
find_versions_of_archive() (in module spack.util.web), 182
FindTest (class in spack.test.cmd.find), 153
first_repo() (spack.repository.RepoPath method), 223
flat_dependencies() (spack.spec.Spec method), 227
flat_dependencies_with_deptype() (spack.spec.Spec method), 227
flatten() (in module spack.cmd.view), 143
flatten_dependencies() (in module spack.package), 218

for_package_version() (in module spack.fetch_strategy), 203
fork() (in module spack.build_environment), 186
format() (spack.spec.Spec method), 227
format_doc() (spack.package.Package method), 214
formats (spack.modules.EnvModule attribute), 206
from_dict() (spack.database.InstallRecord class method), 193
from_dict() (spack.version.VersionList static method), 237
from_kwargs() (in module spack.fetch_strategy), 203
from_node_dict() (spack.spec.Spec static method), 227
from_sourcing_files() (spack.environment.EnvironmentModifications static method), 197
from_url() (in module spack.fetch_strategy), 203
from_yaml() (spack.provider_index.ProviderIndex static method), 221
from_yaml() (spack.spec.Spec static method), 227
front_end (spack.architecture.Platform attribute), 185
front_end (spack.platforms.bgq.Bgq attribute), 151
front_end (spack.platforms.darwin.Darwin attribute), 152
front_end (spack.platforms.test.Test attribute), 152
front_os (spack.architecture.Platform attribute), 185
FsCache (class in spack.fetch_strategy), 200
fullname (spack.spec.Spec attribute), 227

G

Gcc (class in spack.compilers.gcc), 145
get() (spack.repository.Repo method), 222
get() (spack.repository.RepoPath method), 223
get() (spack.test.cmd.test_install.MockPackageDb method), 154
get_checksums() (in module spack.cmd.checksum), 134
get_cmd_function_name() (in module spack.cmd), 144
get_command() (in module spack.cmd), 144
get_compiler_config() (in module spack.compilers), 149
get_compiler_version() (in module spack.compiler), 188
get_config() (in module spack.config), 191
get_config_filename() (in module spack.config), 191
get_dependency() (spack.spec.Spec method), 228
get_filename() (in module spack.cmd.test_install), 141
get_git() (in module spack.cmd.pkg), 138
get_matching_versions() (in module spack.mirror), 205
get_module() (in module spack.cmd), 144
get_modulefile_content() (spack.test.modules.DotkitTests method), 165
get_modulefile_content() (spack.test.modules.ModuleFileGeneratorTests method), 165
get_origin_info() (in module spack.cmd.bootstrap), 133
get_path() (in module spack.config), 191
get_path() (in module spack.util.environment), 180
get_path_from_module() (in module spack.build_environment), 186
get_pkg_class() (spack.repository.Repo method), 222
get_pkg_class() (spack.repository.RepoPath method), 223
get_record() (spack.database.Database method), 192
get_repo() (spack.repository.RepoPath method), 223
get_rev() (spack.test.mock_repo.MockHgRepo method), 164
get_rpaths() (in module spack.build_environment), 186
get_section() (spack.config.ConfigScope method), 191
get_section_filename() (spack.config.ConfigScope method), 191
get_stage_path() (spack.test.stage.StageTest method), 173
get_top_spec_or_die() (in module spack.cmd.test_install), 141
gettok() (spack.parse.Parser method), 219
git (spack.fetch_strategy.GitFetchStrategy attribute), 201
git_version (spack.fetch_strategy.GitFetchStrategy attribute), 201
GitFetchStrategy (class in spack.fetch_strategy), 200
GitFetchTest (class in spack.test.git_fetch), 160
github_url() (in module spack.cmd.package_list), 138
global_license_dir (spack.package.Package attribute), 214
global_license_file (spack.package.Package attribute), 214
go (spack.fetch_strategy.GoFetchStrategy attribute), 201
go_version (spack.fetch_strategy.GoFetchStrategy attribute), 201
GoFetchStrategy (class in spack.fetch_strategy), 201
graph() (in module spack.cmd.graph), 136
graph_ascii() (in module spack.graph), 204
graph_dot() (in module spack.graph), 204
gray_hash() (in module spack.cmd), 144
group_by_name() (spack.environment.EnvironmentModifications method), 198
guess_name_and_version() (in module spack.cmd.create), 135

H

handle_starttag() (spack.util.web.LinkParser method), 182
has_value() (spack.util.naming.NamespaceTrie method), 181
hash_name (spack.util.crypto.Checker attribute), 179
header (spack.modules.Dotkit attribute), 206
header (spack.modules.EnvModule attribute), 206
header (spack.modules.TclModule attribute), 207
help() (in module spack.cmd.help), 136
HelperFunctionsTests (class in spack.test.modules), 165
hg (spack.fetch_strategy.HgFetchStrategy attribute), 201
HgFetchStrategy (class in spack.fetch_strategy), 201
HgFetchTest (class in spack.test.hg_fetch), 161

hidden_file_paths (spack.directory_layout.DirectoryLayout attribute), 195
hidden_file_paths (spack.directory_layout.YamlDirectoryLayout attribute), 197
highest() (spack.version.Version method), 236
highest() (spack.version.VersionList method), 237
highest() (spack.version.VersionRange method), 237
highest_precedence_scope() (in module spack.config), 191
HookRunner (class in spack.hooks), 150

I

InconsistentInstallDirectoryError, 196
InconsistentSpecError, 230
index() (spack.spec.Spec method), 228
info() (in module spack.cmd.info), 136
init_entry() (spack.file_cache.FileCache method), 203
initmock() (spack.test.mock_packages_test.MockPackagesTest method), 164
insensitize() (in module spack.url), 235
install() (in module spack.cmd.install), 137
install() (spack.package.Package method), 215
install() (spack.package.StagedPackage method), 218
install_build() (spack.package.CMakePackage method), 209
install_build() (spack.package.StagedPackage method), 218
install_configure() (spack.package.CMakePackage method), 209
install_configure() (spack.package.StagedPackage method), 218
install_dependency_symlinks() (in module spack.package), 218
install_install() (spack.package.CMakePackage method), 209
install_install() (spack.package.StagedPackage method), 218
install_phases (spack.package.Package attribute), 215
install_setup() (spack.package.CMakePackage method), 209
install_setup() (spack.package.StagedPackage method), 218
install_single_spec() (in module spack.cmd.test_install), 141
InstallDirectoryAlreadyExistsError, 196
installed (spack.package.Package attribute), 215
installed_dependents (spack.package.Package attribute), 215
installed_dependents() (in module spack.cmd.uninstall), 141
installed_extensions_for() (spack.database.Database method), 192
InstallError, 186, 210
InstallRecord (class in spack.database), 193
InstallTest (class in spack.test.build_system_guess), 155
InstallTest (class in spack.test.install), 161
Intel (class in spack.compilers.intel), 146
intersect() (spack.version.VersionList method), 237
intersection() (spack.version.Version method), 236
intersection() (spack.version.VersionList method), 237
intersection() (spack.version.VersionRange method), 237
InvalidArgsError, 201
InvalidCompilerConfigurationError, 148
InvalidDatabaseVersionError, 193
InvalidDependencyError, 230
InvalidDependencyTypeError, 230
InvalidExtensionSpecError, 196
InvalidNamespaceError, 221
is_a() (spack.parse.Token method), 219
is_apple (spack.compilers.clang.Clang attribute), 145
is_extension (spack.package.Package attribute), 215
is_leaf() (spack.util.naming.NamespaceTrie method), 181
is_predecessor() (spack.version.Version method), 236
is_prefix() (spack.repository.Repo method), 222
is_prefix() (spack.util.naming.NamespaceTrie method), 181
is_spec_buildable() (in module spack.config), 191
is_successor() (spack.version.Version method), 236
is_virtual() (spack.spec.Spec static method), 228

L

last_token_error() (spack.parse.Parser method), 219
lex() (spack.parse.Lexer method), 219
Lexer (class in spack.parse), 219
LexError, 219
LibraryListTest (class in spack.test.library_list), 161
link_one() (in module spack.cmd.view), 143
link_paths (spack.compilers.cce.Cce attribute), 144
link_paths (spack.compilers.clang.Clang attribute), 145
link_paths (spack.compilers.gcc.Gcc attribute), 145
link_paths (spack.compilers.intel.Intel attribute), 146
link_paths (spack.compilers.nag.Nag attribute), 146
link_paths (spack.compilers.pgi.Pgi attribute), 147
link_paths (spack.compilers.xl.Xl attribute), 148
LinkParser (class in spack.util.web), 182
LinkTreeTest (class in spack.test.link_tree), 162
Linux (class in spack.platforms.linux), 152
LinuxDistro (class in spack.operating_systems.linux_distro), 151
list() (in module spack.cmd.list), 137
list_packages() (in module spack.cmd.pkg), 138
list_tests() (in module spack.test), 178
LmodTests (class in spack.test.modules), 165
load() (in module spack.cmd.load), 137
load() (in module spack.util.spack_yaml), 182
load_external_modules() (in module spack.build_environment), 186
load_module() (in module spack.build_environment), 186

load_module() (spack.repository.Repo method), 222
load_module() (spack.repository.RepoPath method), 223
loads() (in module spack.cmd.module), 138
location() (in module spack.cmd.location), 137
LockTest (class in spack.test.lock), 162
long_message (spack.error.SpackError attribute), 199
long_message (spack.util.executable.ProcessError attribute), 180
lowest() (spack.version.Version method), 236
lowest() (spack.version.VersionList method), 238
lowest() (spack.version.VersionRange method), 237

M

MacOs (class in spack.operating_systems.mac_os), 151
make_executable() (in module spack.package), 218
make_jobs (spack.package.Package attribute), 215
make_make() (spack.package.CMakePackage method), 209
make_mock_compiler() (in module spack.test.cmd.test_compiler_cmd), 154
make_version_calls() (in module spack.cmd.create), 135
MakeExecutable (class in spack.build_environment), 186
MakeExecutableTest (class in spack.test.make_executable), 163
matches() (spack.fetch_strategy.FetchStrategy class method), 200
md5() (in module spack.cmd.md5), 137
merge() (spack.provider_index.ProviderIndex method), 221
metadata_path() (spack.directory_layout.YamlDirectoryLayout method), 197
mirror() (in module spack.cmd.mirror), 137
mirror_add() (in module spack.cmd.mirror), 137
mirror_archive_filename() (in module spack.mirror), 205
mirror_archive_path() (in module spack.mirror), 205
mirror_create() (in module spack.cmd.mirror), 137
mirror_list() (in module spack.cmd.mirror), 137
mirror_remove() (in module spack.cmd.mirror), 137
MirrorError, 205
MirrorTest (class in spack.test.mirror), 163
missing() (spack.database.Database method), 192
mock_fetch_log() (in module spack.test.cmd.test_install), 154
mock_open() (in module spack.test.cmd.test_install), 154
mock_open() (in module spack.test.modules), 166
MockArchive (class in spack.test.mock_repo), 164
MockArgs (class in spack.test.cmd.test_compiler_cmd), 154
MockArgs (class in spack.test.cmd.test_install), 154
MockArgs (class in spack.test.cmd.uninstall), 154
MockCache (class in spack.cmd.test), 140
MockCacheFetcher (class in spack.cmd.test), 140
MockDatabase (class in spack.test.mock_database), 164
MockGitRepo (class in spack.test.mock_repo), 164
MockHgRepo (class in spack.test.mock_repo), 164
MockPackage (class in spack.test.cmd.test_install), 154
MockPackageDb (class in spack.test.cmd.test_install), 154
MockPackagesTest (class in spack.test.mock_packages_test), 164
MockRepo (class in spack.test.mock_repo), 164
MockSpec (class in spack.test.cmd.test_install), 154
MockSvnRepo (class in spack.test.mock_repo), 165
MockVCSRepo (class in spack.test.mock_repo), 165
mod_to_class() (in module spack.util.naming), 180
module (spack.package.Package attribute), 215
module() (in module spack.cmd.module), 138
module_specific_content() (spack.modules.EnvModule method), 206
module_specific_content() (spack.modules.TclModule method), 207
ModuleFileGeneratorTests (class in spack.test.modules), 165
mtime() (spack.file_cache.FileCache method), 203
MultiMethodError, 207
MultiMethodTest (class in spack.test.multimethod), 166
MultipleMatches, 138
MultipleProviderError, 230
multiproc_test() (spack.test.lock.LockTest method), 162

N

Nag (class in spack.compilers.nag), 146
name (spack.modules.Dotkit attribute), 207
name (spack.modules.EnvModule attribute), 206
name (spack.modules.TclModule attribute), 207
name (spack.test.tally_plugin.Tally attribute), 174
NameModifier (class in spack.environment), 198
namespace (spack.package.Package attribute), 215
NamespaceTrie (class in spack.util.naming), 181
NamespaceTrie.Element (class in spack.util.naming), 181
NamespaceTrieTest (class in spack.test.namespace_trie), 167
NameValueModifier (class in spack.environment), 198
naming_scheme (spack.modules.EnvModule attribute), 206
ne_dag() (spack.spec.Spec method), 228
ne_node() (spack.spec.Spec method), 228
nearest_url() (spack.package.Package method), 215
next_token_error() (spack.parse.Parser method), 219
NoArchiveFileError, 202
NoBuildError, 188
NoCompilerForSpecError, 148
NoCompilersError, 148
NoDigestError, 202
NoMatch, 138
NonConcreteSpecAddError, 193
NoNetworkConnectionError, 199
NoPlatformError, 184

NoProviderError, 230
NoRepoConfiguredError, 221
normalize() (spack.spec.Spec method), 228
normalized() (spack.spec.Spec method), 228
NoStageError, 202
NoSuchExtensionError, 196
NoSuchMethodError, 207
NoSuchPatchFileError, 219
NoURLError, 210
NoValidVersionError, 189
numberOfTestsRun (spack.test.tally_plugin.Tally attribute), 174

O

openmp_flag (spack.compiler.Compiler attribute), 187
openmp_flag (spack.compilers.clang.Clang attribute), 145
openmp_flag (spack.compilers.gcc.Gcc attribute), 145
openmp_flag (spack.compilers.intel.Intel attribute), 146
openmp_flag (spack.compilers.nag.Nag attribute), 147
openmp_flag (spack.compilers.pgi.Pgi attribute), 147
openmp_flag (spack.compilers.xl.Xl attribute), 148
operating_system() (spack.architecture.Platform method), 185
OperatingSystem (class in spack.architecture), 184
options() (spack.test.tally_plugin.Tally method), 174
overlaps() (spack.version.Version method), 236
overlaps() (spack.version.VersionList method), 238
overlaps() (spack.version.VersionRange method), 237

P

Package (class in spack.package), 210
package (spack.spec.Spec attribute), 228
package_class (spack.spec.Spec attribute), 228
package_dir (spack.package.Package attribute), 215
package_list() (in module spack.cmd.package_list), 138
package_py_files() (spack.test.python_version.PythonVersionTest method), 169
PackageError, 217
PackageLoadError, 221
PackageSanityTest (class in spack.test.package_sanity), 167
PackageSanityTest (class in spack.test.url_substitution), 176
PackagesTest (class in spack.test.packages), 167
PackageStillNeededError, 217
PackageVersionError, 217
padder() (in module spack.cmd.info), 136
parallel (spack.package.Package attribute), 215
parent_class_modules() (in module spack.build_environment), 187
parmap() (in module spack.util.multiproc), 180
parse() (in module spack.spec), 229
parse() (spack.parse.Parser method), 219
parse_anonymous_spec() (in module spack.spec), 229
parse_name() (in module spack.url), 235
parse_name_and_version() (in module spack.url), 235
parse_name_offset() (in module spack.url), 235
parse_specs() (in module spack.cmd), 144
parse_version() (in module spack.url), 235
parse_version_offset() (in module spack.url), 235
ParseError, 219
Parser (class in spack.parse), 219
PASSED (spack.cmd.test_install.TestResult attribute), 141
Patch (class in spack.patch), 219
patch() (in module spack.cmd.patch), 138
patch() (in module spack.directives), 194
patches (spack.package.Package attribute), 215
path (spack.modules.Dotkit attribute), 207
path (spack.modules.TclModule attribute), 207
path_for_spec() (spack.directory_layout.DirectoryLayout method), 195
path_put_first() (in module spack.util.environment), 180
path_set() (in module spack.util.environment), 180
Pgi (class in spack.compilers.pgi), 147
pkg() (in module spack.cmd.pkg), 138
pkg_add() (in module spack.cmd.pkg), 138
pkg_added() (in module spack.cmd.pkg), 138
pkg_diff() (in module spack.cmd.pkg), 139
pkg_list() (in module spack.cmd.pkg), 139
pkg_removed() (in module spack.cmd.pkg), 139
Platform (class in spack.architecture), 184
possible_dependencies() (spack.package.Package method), 215
possible_spack_module_names() (in module spack.util.naming), 181
post_install() (in module spack.hooks.dotkit), 149
post_install() (in module spack.hooks.licensing), 149
post_install() (in module spack.hooks.lmodmodule), 150
post_install() (in module spack.hooks.sbang), 150
post_install() (in module spack.hooks.tclmodule), 150
post_uninstall() (in module spack.hooks.dotkit), 149
post_uninstall() (in module spack.hooks.lmodmodule), 150
post_uninstall() (in module spack.hooks.tclmodule), 150
pre_install() (in module spack.hooks.licensing), 149
pre_uninstall() (in module spack.hooks.extensions), 149
PreferredPackages (class in spack.preferred_packages), 220
Prefix (class in spack.util.prefix), 182
prefix (spack.package.Package attribute), 215
prefix (spack.spec.Spec attribute), 228
prefixes (spack.compiler.Compiler attribute), 187
prepend_path() (spack.environment.EnvironmentModifications method), 198
PrependPath (class in spack.environment), 198
prerequisite() (spack.modules.Dotkit method), 207

prerequisite() (spack.modules.EnvModule method), 206
prerequisite_format (spack.modules.TclModule attribute), 207
PrgEnv (spack.compiler.Compiler attribute), 187
PrgEnv (spack.compilers.cce.Cce attribute), 144
PrgEnv (spack.compilers.gcc.Gcc attribute), 145
PrgEnv (spack.compilers.intel.Intel attribute), 146
PrgEnv (spack.compilers.pgi.Pgi attribute), 147
PrgEnv_compiler (spack.compiler.Compiler attribute), 187
PrgEnv_compiler (spack.compilers.cce.Cce attribute), 144
PrgEnv_compiler (spack.compilers.gcc.Gcc attribute), 145
PrgEnv_compiler (spack.compilers.intel.Intel attribute), 146
PrgEnv_compiler (spack.compilers.pgi.Pgi attribute), 147
print_name_and_version() (in module spack.cmd.url_parse), 142
print_pkg() (in module spack.package), 218
print_rst_package_list() (in module spack.cmd.package_list), 138
print_section() (in module spack.config), 191
print_text_info() (in module spack.cmd.info), 136
priority (spack.architecture.Platform attribute), 185
priority (spack.platforms.bgq.Bgq attribute), 151
priority (spack.platforms.cray.Cray attribute), 151
priority (spack.platforms.darwin.Darwin attribute), 152
priority (spack.platforms.linux.Linux attribute), 152
priority (spack.platforms.test.Test attribute), 152
process_environment_command() (spack.modules.EnvModule method), 206
process_environment_command() (spack.modules.TclModule method), 207
ProcessError, 180
provided (spack.package.Package attribute), 215
provider_compare() (spack.preferred_packages.PreferredPackages method), 220
provider_index (spack.repository.Repo attribute), 222
provider_index (spack.repository.RepoPath attribute), 223
ProviderIndex (class in spack.provider_index), 220
ProviderIndexError, 221
ProviderIndexTest (class in spack.test.provider_index), 168
providers() (in module spack.cmd.providers), 139
providers_for() (spack.provider_index.ProviderIndex method), 221
providers_for() (spack.repository.Repo method), 222
providers_for() (spack.repository.RepoPath method), 224
provides() (in module spack.directives), 194
provides() (spack.package.Package method), 215
purge() (in module spack.cmd.purge), 139
purge() (in module spack.stage), 234
purge() (spack.repository.Repo method), 222
purge_empty_directories() (in module spack.cmd.view), 143
push_tokens() (spack.parse.Parser method), 219
put_first() (spack.repository.RepoPath method), 224
put_last() (spack.repository.RepoPath method), 224
pyfiles() (spack.test.python_version.PythonVersionTest method), 169
python() (in module spack.cmd.python), 139
PythonVersionTest (class in spack.test.python_version), 169

Q

query() (spack.database.Database method), 192
query_arguments() (in module spack.cmd.find), 136
query_one() (spack.database.Database method), 192

R

read_spec() (spack.directory_layout.YamlDirectoryLayout method), 197
read_transaction() (spack.database.Database method), 193
read_transaction() (spack.file_cache.FileCache method), 203
read_yaml_dep_specs() (spack.spec.Spec static method), 228
real_name() (spack.repository.Repo method), 222
refresh() (in module spack.cmd.module), 138
register() (spack.multimethod.SpecMultiMethod method), 208
register_interrupt_handler() (in module spack.util.debug), 179
reindex() (in module spack.cmd.reindex), 139
reindex() (spack.database.Database method), 193
relative_path_for_spec() (spack.directory_layout.DirectoryLayout method), 195
relative_path_for_spec() (spack.directory_layout.YamlDirectoryLayout method), 197
relative_to() (in module spack.cmd.view), 143
remove() (spack.database.Database method), 193
remove() (spack.file_cache.FileCache method), 203
remove() (spack.modules.EnvModule method), 206
remove() (spack.repository.RepoPath method), 224
remove_compiler_from_config() (in module spack.compilers), 149
remove_extension() (spack.directory_layout.DirectoryLayout method), 195
remove_extension() (spack.directory_layout.YamlDirectoryLayout method), 197
remove_install_directory() (spack.directory_layout.DirectoryLayout method), 195
remove_one() (in module spack.cmd.view), 143

remove_path() (spack.environment.EnvironmentModifications method), 198
remove_prefix() (spack.package.Package method), 215
remove_provider() (spack.provider_index.ProviderIndex method), 221
RemoveFailedError, 196
RemovePath (class in spack.environment), 198
Repo (class in spack.repository), 221
repo() (in module spack.cmd.repo), 139
repo_add() (in module spack.cmd.repo), 139
repo_create() (in module spack.cmd.repo), 139
repo_for_pkg() (spack.repository.RepoPath method), 224
repo_list() (in module spack.cmd.repo), 139
repo_remove() (in module spack.cmd.repo), 139
RepoError, 223
RepoPath (class in spack.repository), 223
required_attributes (spack.fetch_strategy.FetchStrategy attribute), 200
required_attributes (spack.fetch_strategy.GitFetchStrategy attribute), 201
required_attributes (spack.fetch_strategy.GoFetchStrategy attribute), 201
required_attributes (spack.fetch_strategy.HgFetchStrategy attribute), 201
required_attributes (spack.fetch_strategy.SvnFetchStrategy attribute), 202
required_attributes (spack.fetch_strategy.URLFetchStrategy attribute), 202
reset() (spack.fetch_strategy.FetchStrategy method), 200
reset() (spack.fetch_strategy.GitFetchStrategy method), 201
reset() (spack.fetch_strategy.GoFetchStrategy method), 201
reset() (spack.fetch_strategy.HgFetchStrategy method), 201
reset() (spack.fetch_strategy.SvnFetchStrategy method), 202
reset() (spack.fetch_strategy.URLFetchStrategy method), 202
Resource (class in spack.resource), 224
resource() (in module spack.directives), 194
resources (spack.package.Package attribute), 215
ResourceStage (class in spack.stage), 232
restage() (in module spack.cmd.restage), 140
restage() (spack.stage.DIYStage method), 232
restage() (spack.stage.Stage method), 233
RestageError, 232
results (spack.cmd.test_install.TestCase attribute), 140
rev_hash() (spack.test.mock_repo.MockGitRepo method), 164
rm() (in module spack.cmd.module), 138
root (spack.spec.Spec attribute), 228
rpath (spack.package.Package attribute), 215
rpath_args (spack.package.Package attribute), 215
rst_table() (in module spack.cmd.package_list), 138
run() (in module spack.test), 178
run_tests (spack.package.Package attribute), 215

S

sanity_check_is_dir (spack.package.Package attribute), 215
sanity_check_is_file (spack.package.Package attribute), 215
sanity_check_prefix() (spack.package.Package method), 216
satisfies() (spack.provider_index.ProviderIndex method), 221
satisfies() (spack.spec.Spec method), 228
satisfies() (spack.version.Version method), 236
satisfies() (spack.version.VersionList method), 238
satisfies() (spack.version.VersionRange method), 237
satisfies_dependencies() (spack.spec.Spec method), 228
save_filename (spack.stage.Stage attribute), 233
SbangTest (class in spack.test.sbang), 169
section_schemas (in module spack.config), 191
set() (spack.environment.EnvironmentModifications method), 198
set_build_environment_variables() (in module spack.build_environment), 187
set_compiler_environment_variables() (in module spack.build_environment), 187
set_module_variables_for_package() (in module spack.build_environment), 187
set_or_unset_not_first() (in module spack.environment), 199
set_path() (spack.environment.EnvironmentModifications method), 198
set_pkg_dep() (spack.test.mock_packages_test.MockPackagesTest method), 164
set_result() (spack.cmd.test_install.TestCase method), 140
set_stage() (spack.cmd.test.MockCacheFetcher method), 140
set_stage() (spack.fetch_strategy.FetchStrategy method), 200
set_up_license() (in module spack.hooks.licensing), 149
set_up_package() (spack.test.mirror.MirrorTest method), 163
SetEnv (class in spack.environment), 198
SetPath (class in spack.environment), 198
setup() (in module spack.cmd.setup), 140
setup() (spack.parse.Parser method), 219
setUp() (spack.test.architecture.ArchitectureTest method), 155
setUp() (spack.test.build_system_guess.InstallTest method), 155
setUp() (spack.test.cc.CompilerTest method), 156

setUp() (spack.test.cmd.test_install.TestInstallTest method), 154
setUp() (spack.test.concretize_preferences.ConcretizePreferencesTest method), 157
setUp() (spack.test.config.ConfigTest method), 158
setUp() (spack.test.directory_layout.DirectoryLayoutTest method), 159
setUp() (spack.test.environment.EnvironmentTest method), 159
setUp() (spack.test.file_cache.FileCacheTest method), 160
setUp() (spack.test.git_fetch.GitFetchTest method), 160
setUp() (spack.test.hg_fetch.HgFetchTest method), 161
setUp() (spack.test.install.InstallTest method), 161
setUp() (spack.test.library_list.LibraryListTest method), 161
setUp() (spack.test.link_tree.LinkTreeTest method), 162
setUp() (spack.test.lock.LockTest method), 162
setUp() (spack.test.make_executable.MakeExecutableTest method), 163
setUp() (spack.test.mirror.MirrorTest method), 163
setUp() (spack.test.mock_database.MockDatabase method), 164
setUp() (spack.test.mock_packages_test.MockPackagesTest method), 164
setUp() (spack.test.modules.DotkitTests method), 165
setUp() (spack.test.modules.ModuleFileGeneratorTests method), 165
setUp() (spack.test.namespace_trie.NamespaceTrieTest method), 167
setUp() (spack.test.pattern.CompositeTest method), 168
setUp() (spack.test.sbang.SbangTest method), 169
setUp() (spack.test.stage.StageTest method), 173
setUp() (spack.test.svn_fetch.SvnFetchTest method), 173
setUp() (spack.test.yaml.YamlTest method), 178
setup_dependent_environment() (spack.package.Package method), 216
setup_dependent_package() (spack.package.Package method), 216
setup_environment() (spack.package.Package method), 217
setup_package() (in module spack.build_environment), 187
setup_parser() (in module spack.cmd.activate), 133
setup_parser() (in module spack.cmd.bootstrap), 133
setup_parser() (in module spack.cmd.cd), 134
setup_parser() (in module spack.cmd.checksum), 134
setup_parser() (in module spack.cmd.clean), 134
setup_parser() (in module spack.cmd.compiler), 134
setup_parser() (in module spack.cmd.compilers), 134
setup_parser() (in module spack.cmd.config), 134
setup_parser() (in module spack.cmd.create), 135
setup_parser() (in module spack.cmd.deactivate), 135
setup_parser() (in module spack.cmd.debug), 135
setup_parser() (in module spack.cmd.dependents), 135
setup_parser() (in module spack.cmd.diy), 135
setup_parser() (in module spack.cmd.doc), 135
setup_parser() (in module spack.cmd.edit), 136
setup_parser() (in module spack.cmd.env), 136
setup_parser() (in module spack.cmd.extensions), 136
setup_parser() (in module spack.cmd.fetch), 136
setup_parser() (in module spack.cmd.find), 136
setup_parser() (in module spack.cmd.graph), 136
setup_parser() (in module spack.cmd.help), 136
setup_parser() (in module spack.cmd.info), 137
setup_parser() (in module spack.cmd.install), 137
setup_parser() (in module spack.cmd.list), 137
setup_parser() (in module spack.cmd.load), 137
setup_parser() (in module spack.cmd.location), 137
setup_parser() (in module spack.cmd.md5), 137
setup_parser() (in module spack.cmd.mirror), 137
setup_parser() (in module spack.cmd.module), 138
setup_parser() (in module spack.cmd.patch), 138
setup_parser() (in module spack.cmd.pkg), 139
setup_parser() (in module spack.cmd.providers), 139
setup_parser() (in module spack.cmd.purge), 139
setup_parser() (in module spack.cmd.python), 139
setup_parser() (in module spack.cmd.repo), 139
setup_parser() (in module spack.cmd.restage), 140
setup_parser() (in module spack.cmd.setup), 140
setup_parser() (in module spack.cmd.spec), 140
setup_parser() (in module spack.cmd.stage), 140
setup_parser() (in module spack.cmd.test), 140
setup_parser() (in module spack.cmd.test_install), 141
setup_parser() (in module spack.cmd.uninstall), 141
setup_parser() (in module spack.cmd.unload), 141
setup_parser() (in module spack.cmd.unuse), 142
setup_parser() (in module spack.cmd.url_parse), 142
setup_parser() (in module spack.cmd.urls), 142
setup_parser() (in module spack.cmd.use), 142
setup_parser() (in module spack.cmd.versions), 142
setup_parser() (in module spack.cmd.view), 143
setup_platform_environment() (spack.architecture.Platform class method), 185
setup_platform_environment() (spack.platforms.cray.Cray class method), 151
shebang_too_long() (in module spack.hooks.sbang), 150
short_spec (spack.spec.Spec attribute), 228
short_spec (spack.test.cmd.test_install.MockSpec attribute), 154
SKIPPED (spack.cmd.test_install.TestResult attribute), 141
sorted_deps() (spack.spec.Spec method), 229
source_path (spack.stage.Stage attribute), 233
spack.abi (module), 183
spack.architecture (module), 183

spack.build_environment (module), 185
spack.cmd (module), 144
spack.cmd.activate (module), 133
spack.cmd.arch (module), 133
spack.cmd.bootstrap (module), 133
spack.cmd.cd (module), 134
spack.cmd.checksum (module), 134
spack.cmd.clean (module), 134
spack.cmd.common (module), 133
spack.cmd.common.arguments (module), 133
spack.cmd.compiler (module), 134
spack.cmd.compilers (module), 134
spack.cmd.config (module), 134
spack.cmd.create (module), 135
spack.cmd.deactivate (module), 135
spack.cmd.debug (module), 135
spack.cmd.dependents (module), 135
spack.cmd.diy (module), 135
spack.cmd.doc (module), 135
spack.cmd.edit (module), 136
spack.cmd.env (module), 136
spack.cmd.extensions (module), 136
spack.cmd.fetch (module), 136
spack.cmd.find (module), 136
spack.cmd.graph (module), 136
spack.cmd.help (module), 136
spack.cmd.info (module), 136
spack.cmd.install (module), 137
spack.cmd.list (module), 137
spack.cmd.load (module), 137
spack.cmd.location (module), 137
spack.cmd.md5 (module), 137
spack.cmd.mirror (module), 137
spack.cmd.module (module), 138
spack.cmd.package_list (module), 138
spack.cmd.patch (module), 138
spack.cmd.pkg (module), 138
spack.cmd.providers (module), 139
spack.cmd.purge (module), 139
spack.cmd.python (module), 139
spack.cmd.reindex (module), 139
spack.cmd.repo (module), 139
spack.cmd.restage (module), 140
spack.cmd.setup (module), 140
spack.cmd.spec (module), 140
spack.cmd.stage (module), 140
spack.cmd.test (module), 140
spack.cmd.test_install (module), 140
spack.cmd.uninstall (module), 141
spack.cmd.unload (module), 141
spack.cmd.unuse (module), 142
spack.cmd.url_parse (module), 142
spack.cmd.urls (module), 142
spack.cmd.use (module), 142
spack.cmd.versions (module), 142
spack.cmd.view (module), 142
spack.compiler (module), 187
spack.compilers (module), 148
spack.compilers.cce (module), 144
spack.compilers.clang (module), 145
spack.compilers.gcc (module), 145
spack.compilers.intel (module), 146
spack.compilers.nag (module), 146
spack.compilers.pgi (module), 147
spack.compilers.xl (module), 147
spack.concretize (module), 188
spack.config (module), 189
spack.database (module), 192
spack.directives (module), 193
spack.directory_layout (module), 194
spack.environment (module), 197
spack.error (module), 199
spack.fetch_strategy (module), 199
spack.file_cache (module), 203
spack.graph (module), 204
spack.hooks (module), 150
spack.hooks.dotkit (module), 149
spack.hooks.extensions (module), 149
spack.hooks.licensing (module), 149
spack.hooks.lmodmodule (module), 150
spack.hooks.sbang (module), 150
spack.hooks.tclmodule (module), 150
spack.mirror (module), 205
spack.modules (module), 206
spack.multimethod (module), 207
spack.operating_systems (module), 151
spack.operating_systems.cnl (module), 150
spack.operating_systems.linux_distro (module), 151
spack.operating_systems.mac_os (module), 151
spack.package (module), 209
spack.package_test (module), 218
spack.parse (module), 219
spack.patch (module), 219
spack.platforms (module), 152
spack.platforms.bgq (module), 151
spack.platforms.cray (module), 151
spack.platforms.darwin (module), 152
spack.platforms.linux (module), 152
spack.platforms.test (module), 152
spack.preferred_packages (module), 220
spack.provider_index (module), 220
spack.repository (module), 221
spack.resource (module), 224
spack.schema (module), 153
spack.schema.compilers (module), 152
spack.schema.mirrors (module), 153
spack.schema.modules (module), 153
spack.schema.packages (module), 153

spack.schema.repos (module), 153
spack.schema.targets (module), 153
spack.spec (module), 225
spack.stage (module), 231
spack.test (module), 178
spack.test.architecture (module), 155
spack.test.build_system_guess (module), 155
spack.test.cc (module), 156
spack.test.cmd (module), 155
spack.test.cmd.find (module), 153
spack.test.cmd.module (module), 153
spack.test.cmd.test_compiler_cmd (module), 154
spack.test.cmd.test_install (module), 154
spack.test.cmd.uninstall (module), 154
spack.test.concretize (module), 156
spack.test.concretize_preferences (module), 157
spack.test.config (module), 158
spack.test.database (module), 158
spack.test.directory_layout (module), 159
spack.test.environment (module), 159
spack.test.file_cache (module), 160
spack.test.git_fetch (module), 160
spack.test.hg_fetch (module), 161
spack.test.install (module), 161
spack.test.library_list (module), 161
spack.test.link_tree (module), 162
spack.test.lock (module), 162
spack.test.make_executable (module), 163
spack.test.mirror (module), 163
spack.test.mock_database (module), 164
spack.test.mock_packages_test (module), 164
spack.test.mock_repo (module), 164
spack.test.modules (module), 165
spack.test.multimethod (module), 166
spack.test.namespace_trie (module), 167
spack.test.optional_deps (module), 167
spack.test.package_sanity (module), 167
spack.test.packages (module), 167
spack.test.pattern (module), 168
spack.test.provider_index (module), 168
spack.test.python_version (module), 169
spack.test.sbang (module), 169
spack.test.spec_dag (module), 169
spack.test.spec_semantics (module), 170
spack.test.spec_syntax (module), 171
spack.test.spec_yaml (module), 172
spack.test.stage (module), 172
spack.test.svn_fetch (module), 173
spack.test.tally_plugin (module), 174
spack.test.url_extrapolate (module), 174
spack.test.url_parse (module), 175
spack.test.url_substitution (module), 176
spack.test.versions (module), 177
spack.test.yaml (module), 178
spack.url (module), 234
spack.util (module), 183
spack.util.compression (module), 178
spack.util.crypto (module), 179
spack.util.debug (module), 179
spack.util.environment (module), 179
spack.util.executable (module), 180
spack.util.multiproc (module), 180
spack.util.naming (module), 180
spack.util.pattern (module), 181
spack.util.prefix (module), 182
spack.util.spack_yaml (module), 182
spack.util.string (module), 182
spack.util.web (module), 182
spack.variant (module), 236
spack.version (module), 236
spack.yaml_version_check (module), 238
spack_module_to_python_module() (in module spack.util.naming), 181
SpackError, 199
SpackNamespace (class in spack.repository), 224
SpackYAMLError, 231
spawn() (in module spack.util.multiproc), 180
Spec (class in spack.spec), 225
spec() (in module spack.cmd.spec), 140
spec_externals() (in module spack.config), 191
spec_file_path() (spack.directory_layout.YamlDirectoryLayout method), 197
spec_has_preferred_provider() (spack.preferred_packages.PreferredPackages method), 220
spec_preferred_variants() (spack.preferred_packages.PreferredPackages method), 220
SpecDagTest (class in spack.test.spec_dag), 169
SpecError, 230
SpecHashCollisionError, 196
SpecMultiMethod (class in spack.multimethod), 207
SpecParseError, 230
SpecReadError, 196
specs_by_hash() (spack.directory_layout.YamlDirectoryLayout method), 197
SpecSematicsTest (class in spack.test.spec_semantics), 170
SpecSyntaxTest (class in spack.test.spec_syntax), 171
SpecYamlTest (class in spack.test.spec_yaml), 172
spider() (in module spack.util.web), 183
split_url_extension() (in module spack.url), 235
Stage (class in spack.stage), 232
stage (spack.package.Package attribute), 217
stage() (in module spack.cmd.stage), 140
StagedPackage (class in spack.package), 217
StageError, 233
StageTest (class in spack.test.stage), 172

stdcxx_libs (spack.compilers.gcc.Gcc attribute), 145
stdcxx_libs (spack.compilers.intel.Intel attribute), 146
store() (spack.cmd.test.MockCache method), 140
store() (spack.fetch_strategy.FsCache method), 200
strip_extension() (in module spack.util.compression), 179
strip_query_and_fragment() (in module spack.url), 235
subcommand() (in module spack.cmd.module), 138
substitute_spack_prefix() (in module spack.repository), 224
substitute_version() (in module spack.url), 235
substitution_offsets() (in module spack.url), 235
suffixes (spack.compiler.Compiler attribute), 188
suffixes (spack.compilers.cce.Cce attribute), 144
suffixes (spack.compilers.gcc.Gcc attribute), 145
suggest_archive_basename() (in module spack.mirror), 205
supported() (in module spack.compilers), 149
supported_compilers() (in module spack.compilers), 149
svn (spack.fetch_strategy.SvnFetchStrategy attribute), 202
SvnFetchStrategy (class in spack.fetch_strategy), 202
SvnFetchTest (class in spack.test.svn_fetch), 173
swap() (spack.repository.RepoPath method), 224
symlink_license() (in module spack.hooks.licensing), 149

T

Tally (class in spack.test.tally_plugin), 174
Target (class in spack.architecture), 185
target() (spack.architecture.Platform method), 185
TclModule (class in spack.modules), 207
TclTests (class in spack.test.modules), 166
tearDown() (spack.test.architecture.ArchitectureTest method), 155
tearDown() (spack.test.build_system_guess.InstallTest method), 155
tearDown() (spack.test.cc.CompilerTest method), 156
tearDown() (spack.test.cmd.test_install.TestInstallTest method), 154
tearDown() (spack.test.concretize_preferences.ConcretizePreferencesTest method), 158
tearDown() (spack.test.config.ConfigTest method), 158
tearDown() (spack.test.directory_layout.DirectoryLayoutTest method), 159
tearDown() (spack.test.environment.EnvironmentTest method), 159
tearDown() (spack.test.file_cache.FileCacheTest method), 160
tearDown() (spack.test.git_fetch.GitFetchTest method), 160
tearDown() (spack.test.hg_fetch.HgFetchTest method), 161
tearDown() (spack.test.install.InstallTest method), 161
tearDown() (spack.test.link_tree.LinkTreeTest method), 162
tearDown() (spack.test.lock.LockTest method), 162
tearDown() (spack.test.make_executable.MakeExecutableTest method), 163
tearDown() (spack.test.mirror.MirrorTest method), 163
tearDown() (spack.test.mock_database.MockDatabase method), 164
tearDown() (spack.test.mock_packages_test.MockPackagesTest method), 164
tearDown() (spack.test.modules.DotkitTests method), 165
tearDown() (spack.test.modules.ModuleFileGeneratorTests method), 165
tearDown() (spack.test.sbang.SbangTest method), 169
tearDown() (spack.test.stage.StageTest method), 173
tearDown() (spack.test.svn_fetch.SvnFetchTest method), 173
Test (class in spack.platforms.test), 152
test() (in module spack.cmd.test), 140
test_005_db_exists() (spack.test.database.DatabaseTest method), 158
test_010_all_install_sanity() (spack.test.database.DatabaseTest method), 158
test_015_write_and_read() (spack.test.database.DatabaseTest method), 158
test_020_db_sanity() (spack.test.database.DatabaseTest method), 158
test_025_reindex() (spack.test.database.DatabaseTest method), 158
test_030_db_sanity_from_another_process() (spack.test.database.DatabaseTest method), 158
test_040_ref_counts() (spack.test.database.DatabaseTest method), 158
test_050_basic_query() (spack.test.database.DatabaseTest method), 159
test_060_remove_and_add_root_package() (spack.test.database.DatabaseTest method), 159
test_070_remove_and_add_dependency_package() (spack.test.database.DatabaseTest method), 159
test_080_root_ref_counts() (spack.test.database.DatabaseTest method), 159
test_090_non_root_ref_counts() (spack.test.database.DatabaseTest method), 159
test_100_no_write_with_exception_on_remove() (spack.test.database.DatabaseTest method), 159
test_110_no_write_with_exception_on_install() (spack.test.database.DatabaseTest method), 159

test_add() (spack.test.library_list.LibraryListTest method), 161
test_add_multiple() (spack.test.namespace_trie.NamespaceTrieTest method), 167
test_add_none_multiple() (spack.test.namespace_trie.NamespaceTrieTest method), 167
test_add_none_single() (spack.test.namespace_trie.NamespaceTrieTest method), 167
test_add_single() (spack.test.namespace_trie.NamespaceTrieTest method), 167
test_add_three() (spack.test.namespace_trie.NamespaceTrieTest method), 167
test_all_deps() (spack.test.cc.CompilerTest method), 156
test_all_mirror() (spack.test.mirror.MirrorTest method), 164
test_alpha() (spack.test.versions.VersionsTest method), 177
test_alpha_beta() (spack.test.versions.VersionsTest method), 177
test_alpha_with_dots() (spack.test.versions.VersionsTest method), 177
test_alter_environment() (spack.test.modules.LmodTests method), 165
test_alter_environment() (spack.test.modules.TclTests method), 166
test_ambiguous() (spack.test.spec_syntax.SpecSyntaxTest method), 172
test_ambiguous_version_spec() (spack.test.spec_yaml.SpecYamlTest method), 172
test_angband_version_style() (spack.test.url_parse.UrlParseTest method), 175
test_another_erlang_version_style() (spack.test.url_parse.UrlParseTest method), 175
test_apache_version_style() (spack.test.url_parse.UrlParseTest method), 175
test_as_mode() (spack.test.cc.CompilerTest method), 156
test_astyle_verson_style() (spack.test.url_parse.UrlParseTest method), 175
test_autoload() (spack.test.modules.LmodTests method), 165
test_autoload() (spack.test.modules.TclTests method), 166
test_autotools() (spack.test.build_system_guess.InstallTest method), 155
test_basic_version_satisfaction() (spack.test.versions.VersionsTest method), 177
test_basic_version_satisfaction_in_lists() (spack.test.versions.VersionsTest method), 177
test_blacklist() (spack.test.modules.LmodTests method), 165
test_blacklist() (spack.test.modules.TclTests method), 166
test_bootstrap() (spack.test.architecture.ArchitectureTest method), 155
test_boost_version_style() (spack.test.url_parse.UrlParseTest method), 175
test_canonicalize() (spack.test.spec_syntax.SpecSyntaxTest method), 172
test_canonicalize_list() (spack.test.versions.VersionsTest method), 177
test_ccld_mode() (spack.test.cc.CompilerTest method), 156
test_chained_mpi() (spack.test.optional_deps.ConcretizeTest method), 167
test_chdir() (spack.test.stage.StageTest method), 173
test_close_numbers() (spack.test.versions.VersionsTest method), 177
test_cmake() (spack.test.build_system_guess.InstallTest method), 155
test_compiler_add() (spack.test.cmd.test_compiler_cmd.CompilerCmdTest method), 154
test_compiler_child() (spack.test.concretize.ConcretizeTest method), 156
test_compiler_inheritance() (spack.test.concretize.ConcretizeTest method), 156
test_compiler_remove() (spack.test.cmd.test_compiler_cmd.CompilerCmdT method), 154
test_complex_acquire_and_release_chain() (spack.test.lock.LockTest method), 162
test_composite_from_interface() (spack.test.pattern.CompositeTest method), 168
test_composite_from_method_list() (spack.test.pattern.CompositeTest method), 168
test_concrete_spec() (spack.test.spec_yaml.SpecYamlTest method), 172
test_concretize_dag() (spack.test.concretize.ConcretizeTest method), 156
test_concretize_no_deps() (spack.test.concretize.ConcretizeTest method), 156
test_concretize_preferred_version() (spack.test.concretize.ConcretizeTest method), 157
test_concretize_two_virtuals() (spack.test.concretize.ConcretizeTest method),

157
test_concretize_two_virtuals_with_dual_provider() (spack.test.concretize.ConcretizeTest method), 157
test_concretize_two_virtuals_with_dual_provider_and_a_conflict() (spack.test.concretize.ConcretizeTest method), 157
test_concretize_two_virtuals_with_one_bound() (spack.test.concretize.ConcretizeTest method), 157
test_concretize_two_virtuals_with_two_bound() (spack.test.concretize.ConcretizeTest method), 157
test_concretize_variant() (spack.test.concretize.ConcretizeTest method), 157
test_concretize_with_provides_when() (spack.test.concretize.ConcretizeTest method), 157
test_concretize_with_restricted_virtual() (spack.test.concretize.ConcretizeTest method), 157
test_concretize_with_virtual() (spack.test.concretize.ConcretizeTest method), 157
test_conflicting_package_constraints() (spack.test.spec_dag.SpecDagTest method), 169
test_conflicting_spec_constraints() (spack.test.spec_dag.SpecDagTest method), 169
test_conflicts() (spack.test.modules.TclTests method), 166
test_conretize_compiler_flags() (spack.test.concretize.ConcretizeTest method), 157
test_constrain_architecture() (spack.test.spec_semantics.SpecSematicsTest method), 170
test_constrain_changed() (spack.test.spec_semantics.SpecSematicsTest method), 170
test_constrain_compiler() (spack.test.spec_semantics.SpecSematicsTest method), 170
test_constrain_compiler_flags() (spack.test.spec_semantics.SpecSematicsTest method), 170
test_constrain_dependency_changed() (spack.test.spec_semantics.SpecSematicsTest method), 170
test_constrain_dependency_not_changed() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_constrain_not_changed() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_constrain_variants() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_contains() (spack.test.spec_dag.SpecDagTest method), 169
test_contains() (spack.test.versions.VersionsTest method), 177
test_copy() (spack.test.provider_index.ProviderIndexTest method), 168
test_copy_concretized() (spack.test.spec_dag.SpecDagTest method), 169
test_copy_normalized() (spack.test.spec_dag.SpecDagTest method), 169
test_copy_simple() (spack.test.spec_dag.SpecDagTest method), 169
test_core_module_compatibility() (spack.test.python_version.PythonVersionTest method), 169
test_cpp_mode() (spack.test.cc.CompilerTest method), 156
test_dash_rc_style() (spack.test.url_parse.UrlParseTest method), 175
test_dash_version_dash_style() (spack.test.url_parse.UrlParseTest method), 175
test_date_stamps() (spack.test.versions.VersionsTest method), 177
test_debian_style_1() (spack.test.url_parse.UrlParseTest method), 175
test_debian_style_2() (spack.test.url_parse.UrlParseTest method), 175
test_default_variant() (spack.test.optional_deps.ConcretizeTest method), 167
test_default_works() (spack.test.multimethod.MultiMethodTest method), 166
test_dep_include() (spack.test.cc.CompilerTest method), 156
test_dep_index() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_dep_lib() (spack.test.cc.CompilerTest method), 156
test_dep_rpath() (spack.test.cc.CompilerTest method), 156
test_dependencies_with_versions() (spack.test.spec_syntax.SpecSyntaxTest method), 172
test_dependency_already_installed() (spack.test.cmd.test_install.TestInstallTest method), 154
test_dependency_match() (spack.test.multimethod.MultiMethodTest method), 166

test_dependents_and_dependencies_are_correct() (spack.test.spec_dag.SpecDagTest method), 169
test_deptype_traversal() (spack.test.spec_dag.SpecDagTest method), 169
test_deptype_traversal_full() (spack.test.spec_dag.SpecDagTest method), 169
test_deptype_traversal_run() (spack.test.spec_dag.SpecDagTest method), 169
test_deptype_traversal_with_builddeps() (spack.test.spec_dag.SpecDagTest method), 170
test_dict_functions_for_architecture() (spack.test.architecture.ArchitectureTest method), 155
test_dict_order() (spack.test.yaml.YamlTest method), 178
test_dotkit() (spack.test.modules.DotkitTests method), 165
test_double_alpha() (spack.test.versions.VersionsTest method), 177
test_duplicate_compiler() (spack.test.spec_syntax.SpecSyntaxTest method), 172
test_duplicate_depdendence() (spack.test.spec_syntax.SpecSyntaxTest method), 172
test_duplicate_variant() (spack.test.spec_syntax.SpecSyntaxTest method), 172
test_dyninst_version() (spack.test.url_extrapolate.UrlExtrapolateTest method), 174
test_equal() (spack.test.provider_index.ProviderIndexTest method), 168
test_equal() (spack.test.spec_dag.SpecDagTest method), 170
test_erlang_version_style() (spack.test.url_parse.UrlParseTest method), 175
test_error_conditions() (spack.test.pattern.CompositeTest method), 168
test_expand_archive() (spack.test.stage.StageTest method), 173
test_expand_archive_with_chdir() (spack.test.stage.StageTest method), 173
test_extend() (spack.test.environment.EnvironmentTest method), 159
test_external_and_virtual() (spack.test.concretize.ConcretizeTest method), 157
test_external_package() (spack.test.concretize.ConcretizeTest method), 157
test_external_package_module() (spack.test.concretize.ConcretizeTest method), 157
test_extra_arguments() (spack.test.environment.EnvironmentTest method), 159
test_fann_version() (spack.test.url_parse.UrlParseTest method), 175
test_fetch() (spack.test.stage.StageTest method), 173
test_fetch_branch() (spack.test.git_fetch.GitFetchTest method), 160
test_fetch_commit() (spack.test.git_fetch.GitFetchTest method), 160
test_fetch_default() (spack.test.hg_fetch.HgFetchTest method), 161
test_fetch_default() (spack.test.svn_fetch.SvnFetchTest method), 174
test_fetch_master() (spack.test.git_fetch.GitFetchTest method), 160
test_fetch_r1() (spack.test.svn_fetch.SvnFetchTest method), 174
test_fetch_rev0() (spack.test.hg_fetch.HgFetchTest method), 161
test_fetch_tag() (spack.test.git_fetch.GitFetchTest method), 160
test_find() (spack.test.directory_layout.DirectoryLayoutTest method), 159
test_find_spec_children() (spack.test.concretize.ConcretizeTest method), 157
test_find_spec_none() (spack.test.concretize.ConcretizeTest method), 157
test_find_spec_parents() (spack.test.concretize.ConcretizeTest method), 157
test_find_spec_self() (spack.test.concretize.ConcretizeTest method), 157
test_find_spec_sibling() (spack.test.concretize.ConcretizeTest method), 157
test_flags() (spack.test.cc.CompilerTest method), 156
test_flags() (spack.test.library_list.LibraryListTest method), 161
test_formatted_strings() (spack.test.versions.VersionsTest method), 177
test_full_specs() (spack.test.spec_syntax.SpecSyntaxTest method), 172
test_gcc() (spack.test.url_extrapolate.UrlExtrapolateTest method), 175
test_gcc_version() (spack.test.url_parse.UrlParseTest method), 175
test_gcc_version_precedence() (spack.test.url_parse.UrlParseTest method), 175
test_get_all_mock_packages() (spack.test.package_sanity.PackageSanityTest method), 167
test_get_all_packages() (spack.test.package_sanity.PackageSanityTest method), 167

test_get_item() (spack.test.library_list.LibraryListTest method), 161
test_get_item() (spack.test.versions.VersionsTest method), 177
test_git_mirror() (spack.test.mirror.MirrorTest method), 164
test_github_raw() (spack.test.url_extrapolate.UrlExtrapolateTest method), 175
test_github_raw_url() (spack.test.url_parse.UrlParseTest method), 175
test_gloox_beta_style() (spack.test.url_parse.UrlParseTest method), 175
test_handle_unknown_package() (spack.test.directory_layout.DirectoryLayoutTest method), 159
test_haxe_version() (spack.test.url_parse.UrlParseTest method), 175
test_hdf5_version() (spack.test.url_parse.UrlParseTest method), 175
test_hg_mirror() (spack.test.mirror.MirrorTest method), 164
test_hypre_url_substitution() (spack.test.url_substitution.PackageSanityTest method), 176
test_iges_version() (spack.test.url_parse.UrlParseTest method), 175
test_ignore() (spack.test.link_tree.LinkTreeTest method), 162
test_imagemagick_style() (spack.test.url_parse.UrlParseTest method), 175
test_imap_version() (spack.test.url_parse.UrlParseTest method), 175
test_import_class_from_package() (spack.test.packages.PackagesTest method), 167
test_import_module_from_package() (spack.test.packages.PackagesTest method), 167
test_import_namespace_container_modules() (spack.test.packages.PackagesTest method), 167
test_import_package() (spack.test.packages.PackagesTest method), 168
test_import_package_as() (spack.test.packages.PackagesTest method), 168
test_in_list() (spack.test.versions.VersionsTest method), 177
test_inspect_path() (spack.test.modules.HelperFunctionsTests method), 165
test_install() (in module spack.cmd.test_install), 141
test_install_and_uninstall() (spack.test.install.InstallTest method), 161
test_install_environment() (spack.test.install.InstallTest method), 161
test_installing_both() (spack.test.cmd.test_install.TestInstallTest method), 154
test_intersect_with_containment() (spack.test.versions.VersionsTest method), 177
test_intersection() (spack.test.versions.VersionsTest method), 177
test_invalid_constraint() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_invalid_dep() (spack.test.spec_dag.SpecDagTest method), 170
test_joined_and_str() (spack.test.library_list.LibraryListTest method), 162
test_jpeg_style() (spack.test.url_parse.UrlParseTest method), 175
test_keep_exceptions() (spack.test.stage.StageTest method), 173
test_keep_without_exceptions() (spack.test.stage.StageTest method), 173
test_lame_version_style() (spack.test.url_parse.UrlParseTest method), 175
test_ld_deps() (spack.test.cc.CompilerTest method), 156
test_ld_deps_reentrant() (spack.test.cc.CompilerTest method), 156
test_ld_mode() (spack.test.cc.CompilerTest method), 156
test_libdwarf_version() (spack.test.url_extrapolate.UrlExtrapolateTest method), 175
test_libelf_version() (spack.test.url_extrapolate.UrlExtrapolateTest method), 175
test_line_numbers() (spack.test.yaml.YamlTest method), 178
test_lists_overlap() (spack.test.versions.VersionsTest method), 177
test_load_package() (spack.test.packages.PackagesTest method), 168
test_make_explicit() (spack.test.make_executable.MakeExecutableTest method), 163
test_make_normal() (spack.test.make_executable.MakeExecutableTest method), 163
test_make_one_job() (spack.test.make_executable.MakeExecutableTest method), 163
test_make_parallel_disabled() (spack.test.make_executable.MakeExecutableTest method), 163
test_make_parallel_false() (spack.test.make_executable.MakeExecutableTest method), 163
test_make_parallel_precedence() (spack.test.make_executable.MakeExecutableTest method), 163

test_merge_to_existing_directory() (spack.test.link_tree.LinkTreeTest method), 162
test_merge_to_new_directory() (spack.test.link_tree.LinkTreeTest method), 162
test_merge_with_empty_directories() (spack.test.link_tree.LinkTreeTest method), 162
test_minimal_spaces() (spack.test.spec_syntax.SpecSyntaxTest method), 172
test_module_common_operations() (spack.test.cmd.module.TestModule method), 153
test_mpi_providers() (spack.test.provider_index.ProviderIndexTest method), 168
test_mpi_version() (spack.test.multimethod.MultiMethodTest method), 166
test_mpileaks_version() (spack.test.url_extrapolate.UrlExtrapolateTest method), 175
test_mpileaks_version() (spack.test.url_parse.UrlParseTest method), 175
test_multiple_conditionals() (spack.test.optional_deps.ConcretizeTest method), 167
test_mvapich2_19_version() (spack.test.url_parse.UrlParseTest method), 176
test_mvapich2_20_version() (spack.test.url_parse.UrlParseTest method), 176
test_my_dep_depends_on_provider_of_my_virtual_dep() (spack.test.concretize.ConcretizeTest method), 157
test_new_github_style() (spack.test.url_parse.UrlParseTest method), 176
test_no_keep_with_exceptions() (spack.test.stage.StageTest method), 173
test_no_keep_without_exceptions() (spack.test.stage.StageTest method), 173
test_no_version() (spack.test.url_parse.UrlParseTest method), 176
test_no_version_match() (spack.test.multimethod.MultiMethodTest method), 166
test_nobuild_package() (spack.test.concretize.ConcretizeTest method), 157
test_nonexisting_package_filename() (spack.test.packages.PackagesTest method), 168
test_normal_spec() (spack.test.spec_yaml.SpecYamlTest method), 172
test_normalize_a_lot() (spack.test.spec_dag.SpecDagTest method), 170
test_normalize_mpileaks() (spack.test.spec_dag.SpecDagTest method), 170
test_normalize_simple_conditionals() (spack.test.optional_deps.ConcretizeTest method), 167
test_normalize_twice() (spack.test.spec_dag.SpecDagTest method), 170
test_normalize_with_virtual_package() (spack.test.spec_dag.SpecDagTest method), 170
test_normalize_with_virtual_spec() (spack.test.spec_dag.SpecDagTest method), 170
test_num_alpha_no_separator_single_digit() (spack.test.url_parse.UrlParseTest method), 176
test_num_alpha_with_no_separator() (spack.test.versions.VersionsTest method), 177
test_nums_and_patch() (spack.test.versions.VersionsTest method), 177
test_omega_version_style() (spack.test.url_parse.UrlParseTest method), 176
test_one_version_match() (spack.test.multimethod.MultiMethodTest method), 166
test_openssl_version() (spack.test.url_parse.UrlParseTest method), 176
test_otf2_url_substitution() (spack.test.url_substitution.PackageSanityTest method), 177
test_overlap_with_containment() (spack.test.versions.VersionsTest method), 178
test_p7zip_version_style() (spack.test.url_parse.UrlParseTest method), 176
test_package_class_names() (spack.test.packages.PackagesTest method), 168
test_package_filename() (spack.test.packages.PackagesTest method), 168
test_package_module_compatibility() (spack.test.python_version.PythonVersionTest method), 169
test_package_name() (spack.test.packages.PackagesTest method), 168
test_package_names() (spack.test.spec_syntax.SpecSyntaxTest method), 172
test_padded_numbers() (spack.test.versions.VersionsTest method), 178
test_parse() (spack.test.yaml.YamlTest method), 178

test_parse_errors() (spack.test.spec_syntax.SpecSyntaxTest method), 172
test_partial_version_prefix() (spack.test.url_extrapolate.UrlExtrapolateTest method), 175
test_patch() (spack.test.versions.VersionsTest method), 178
test_path_manipulation() (spack.test.environment.EnvironmentTest method), 159
test_paths_manipulation() (spack.test.library_list.LibraryListTest method), 162
test_platform() (spack.test.architecture.ArchitectureTest method), 155
test_postorder_edge_traversal() (spack.test.spec_dag.SpecDagTest method), 170
test_postorder_node_traversal() (spack.test.spec_dag.SpecDagTest method), 170
test_postorder_path_traversal() (spack.test.spec_dag.SpecDagTest method), 170
test_preferred_compilers() (spack.test.concretize_preferences.ConcretizePreferencesTest method), 158
test_preferred_providers() (spack.test.concretize_preferences.ConcretizePreferencesTest method), 158
test_preferred_variants() (spack.test.concretize_preferences.ConcretizePreferencesTest method), 158
test_preferred_versions() (spack.test.concretize_preferences.ConcretizePreferencesTest method), 158
test_preorder_edge_traversal() (spack.test.spec_dag.SpecDagTest method), 170
test_preorder_node_traversal() (spack.test.spec_dag.SpecDagTest method), 170
test_preorder_path_traversal() (spack.test.spec_dag.SpecDagTest method), 170
test_prerequisites() (spack.test.modules.TclTests method), 166
test_providers_for_simple() (spack.test.provider_index.ProviderIndexTest method), 168
test_pypy_version() (spack.test.url_parse.UrlParseTest method), 176
test_python() (spack.test.build_system_guess.InstallTest method), 155
test_query_arguments() (spack.test.cmd.find.FindTest method), 153
test_R() (spack.test.build_system_guess.InstallTest method), 155
test_ranges_overlap() (spack.test.versions.VersionsTest method), 178
test_rc_style() (spack.test.url_parse.UrlParseTest method), 176
test_rc_versions() (spack.test.versions.VersionsTest method), 178
test_read_and_write_spec() (spack.test.directory_layout.DirectoryLayoutTest method), 159
test_read_lock_timeout_on_write() (spack.test.lock.LockTest method), 162
test_read_lock_timeout_on_write_2() (spack.test.lock.LockTest method), 162
test_read_lock_timeout_on_write_3() (spack.test.lock.LockTest method), 162
test_remove() (spack.test.file_cache.FileCacheTest method), 160
test_repr() (spack.test.library_list.LibraryListTest method), 162
test_repr_and_str() (spack.test.versions.VersionsTest method), 178
test_restage() (spack.test.stage.StageTest method), 173
test_rpm_oddities() (spack.test.versions.VersionsTest method), 178
test_scheme_version_style() (spack.test.url_parse.UrlParseTest method), 176
test_satisfaction_with_lists() (spack.test.versions.VersionsTest method), 178
test_satisfies() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_satisfies_architecture() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_satisfies_compiler() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_satisfies_compiler_version() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_satisfies_dependencies() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_satisfies_dependency_versions() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_satisfies_matching_compiler_flag() (spack.test.spec_semantics.SpecSematicsTest method), 171

test_satisfies_matching_variant() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_satisfies_namespace() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_satisfies_namespaced_dep() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_satisfies_unconstrained_compiler_flag() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_satisfies_unconstrained_variant() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_satisfies_virtual() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_satisfies_virtual_dep_with_virtual_constraint() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_satisfies_virtual_dependencies() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_satisfies_virtual_dependency_versions() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_scalasca_partial_version() (spack.test.url_extrapolate.UrlExtrapolateTest method), 175
test_scalasca_version() (spack.test.url_parse.UrlParseTest method), 176
test_scons() (spack.test.build_system_guess.InstallTest method), 155
test_self_index() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_set() (spack.test.environment.EnvironmentTest method), 159
test_set_path() (spack.test.environment.EnvironmentTest method), 159
test_setup_and_destroy_name_with_tmp() (spack.test.stage.StageTest method), 173
test_setup_and_destroy_name_without_tmp() (spack.test.stage.StageTest method), 173
test_setup_and_destroy_no_name_with_tmp() (spack.test.stage.StageTest method), 173
test_setup_and_destroy_no_name_without_tmp() (spack.test.stage.StageTest method), 173
test_shebang_handles_non_writable_files() (spack.test.sbang.SbangTest method), 169
test_shebang_handling() (spack.test.sbang.SbangTest method), 169
test_simple_case() (spack.test.modules.LmodTests method), 165
test_simple_case() (spack.test.modules.TclTests method), 166
test_simple_dependence() (spack.test.spec_syntax.SpecSyntaxTest method), 172
test_simple_spec() (spack.test.spec_yaml.SpecYamlTest method), 172
test_source_files() (spack.test.environment.EnvironmentTest method), 160
test_spaces_between_dependences() (spack.test.spec_syntax.SpecSyntaxTest method), 172
test_spaces_between_options() (spack.test.spec_syntax.SpecSyntaxTest method), 172
test_spec_contains_deps() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_sphinx_beta_style() (spack.test.url_parse.UrlParseTest method), 176
test_stable_suffix() (spack.test.url_parse.UrlParseTest method), 176
test_suffixes() (spack.test.modules.TclTests method), 166
test_suite3270_version() (spack.test.url_parse.UrlParseTest method), 176
test_svn_mirror() (spack.test.mirror.MirrorTest method), 164
test_synergy_version() (spack.test.url_parse.UrlParseTest method), 176
test_target_match() (spack.test.multimethod.MultiMethodTest method), 166
test_three_segments() (spack.test.versions.VersionsTest method), 178
test_transaction() (spack.test.lock.LockTest method), 162
test_transaction_with_context_manager() (spack.test.lock.LockTest method), 162
test_transaction_with_context_manager_and_exception() (spack.test.lock.LockTest method), 162
test_transaction_with_exception() (spack.test.lock.LockTest method), 162
test_transitive_chain() (spack.test.optional_deps.ConcretizeTest method), 167
test_two_segments() (spack.test.versions.VersionsTest method), 178
test_undefined_mpi_version() (spack.test.multimethod.MultiMethodTest method), 166
test_underscores() (spack.test.versions.VersionsTest method), 178
test_uninstall() (spack.test.cmd.uninstall.TestUninstall method), 155
test_union_with_containment() (spack.test.versions.VersionsTest method), 178

test_unknown() (spack.test.build_system_guess.InstallTest method), 155
test_unsatisfiable_architecture() (spack.test.spec_dag.SpecDagTest method), 170
test_unsatisfiable_compiler() (spack.test.spec_dag.SpecDagTest method), 170
test_unsatisfiable_compiler_flag() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_unsatisfiable_compiler_flag_mismatch() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_unsatisfiable_compiler_version() (spack.test.spec_dag.SpecDagTest method), 170
test_unsatisfiable_variant_mismatch() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_unsatisfiable_variants() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_unsatisfiable_version() (spack.test.spec_dag.SpecDagTest method), 170
test_unset() (spack.test.environment.EnvironmentTest method), 160
test_update_dictionary_extending_list() (spack.test.modules.HelperFunctionsTests method), 165
test_url_mirror() (spack.test.mirror.MirrorTest method), 164
test_url_versions() (spack.test.package_sanity.PackageSanityTest method), 167
test_user_back_end_input() (spack.test.architecture.ArchitectureTest method), 155
test_user_defaults() (spack.test.architecture.ArchitectureTest method), 155
test_user_front_end_input() (spack.test.architecture.ArchitectureTest method), 155
test_user_input_combination() (spack.test.architecture.ArchitectureTest method), 155
test_using_ordered_dict() (spack.test.spec_dag.SpecDagTest method), 170
test_vcheck_mode() (spack.test.cc.CompilerTest method), 156
test_version_all_dots() (spack.test.url_parse.UrlParseTest method), 176
test_version_developer_that_hates_us_format() (spack.test.url_parse.UrlParseTest method), 176
test_version_dos2unix() (spack.test.url_parse.UrlParseTest method), 176
test_version_github() (spack.test.url_parse.UrlParseTest method), 176
test_version_github_with_high_patch_number() (spack.test.url_parse.UrlParseTest method), 176
test_version_internal_dash() (spack.test.url_parse.UrlParseTest method), 176
test_version_overlap() (spack.test.multimethod.MultiMethodTest method), 166
test_version_range_satisfaction() (spack.test.versions.VersionsTest method), 178
test_version_range_satisfaction_in_lists() (spack.test.versions.VersionsTest method), 178
test_version_ranges() (spack.test.versions.VersionsTest method), 178
test_version_regular() (spack.test.url_parse.UrlParseTest method), 176
test_version_single_digit() (spack.test.url_parse.UrlParseTest method), 176
test_version_sourceforge_download() (spack.test.url_parse.UrlParseTest method), 176
test_version_underscore_separator() (spack.test.url_parse.UrlParseTest method), 176
test_virtual_dep_match() (spack.test.multimethod.MultiMethodTest method), 166
test_virtual_index() (spack.test.spec_semantics.SpecSematicsTest method), 171
test_virtual_is_fully_expanded_for_callpath() (spack.test.concretize.ConcretizeTest method), 157
test_virtual_is_fully_expanded_for_mpileaks() (spack.test.concretize.ConcretizeTest method), 157
test_way_too_many_spaces() (spack.test.spec_syntax.SpecSyntaxTest method), 172
test_write_and_read_cache_file() (spack.test.file_cache.FileCacheTest method), 160
test_write_and_write_cache_file() (spack.test.file_cache.FileCacheTest method), 160

test_write_key_in_memory() (spack.test.config.ConfigTest method), 158
test_write_key_to_disk() (spack.test.config.ConfigTest method), 158
test_write_list_in_memory() (spack.test.config.ConfigTest method), 158
test_write_lock_timeout_on_read() (spack.test.lock.LockTest method), 162
test_write_lock_timeout_on_read_2() (spack.test.lock.LockTest method), 162
test_write_lock_timeout_on_read_3() (spack.test.lock.LockTest method), 163
test_write_lock_timeout_on_write() (spack.test.lock.LockTest method), 163
test_write_lock_timeout_on_write_2() (spack.test.lock.LockTest method), 163
test_write_lock_timeout_on_write_3() (spack.test.lock.LockTest method), 163
test_write_lock_timeout_with_multiple_readers_2_1() (spack.test.lock.LockTest method), 163
test_write_lock_timeout_with_multiple_readers_2_2() (spack.test.lock.LockTest method), 163
test_write_lock_timeout_with_multiple_readers_3_1() (spack.test.lock.LockTest method), 163
test_write_lock_timeout_with_multiple_readers_3_2() (spack.test.lock.LockTest method), 163
test_write_to_same_priority_file() (spack.test.config.ConfigTest method), 158
test_wwwoffle_version() (spack.test.url_parse.UrlParseTest method), 176
test_xaw3d_version() (spack.test.url_parse.UrlParseTest method), 176
test_yaml_round_trip() (spack.test.provider_index.ProviderIndexTest method), 168
test_yaml_subdag() (spack.test.spec_yaml.SpecYamlTest method), 172
test_yet_another_erlang_version_style() (spack.test.url_parse.UrlParseTest method), 176
test_yet_another_version() (spack.test.url_parse.UrlParseTest method), 176
TestCase (class in spack.cmd.test_install), 140
TestInstallTest (class in spack.test.cmd.test_install), 154
TestModule (class in spack.test.cmd.module), 153
TestResult (class in spack.cmd.test_install), 140
TestSuite (class in spack.cmd.test_install), 141
TestUninstall (class in spack.test.cmd.uninstall), 155
timeout_read() (spack.test.lock.LockTest method), 163
timeout_write() (spack.test.lock.LockTest method), 163
to_dict() (spack.architecture.Arch method), 184
to_dict() (spack.architecture.OperatingSystem method), 184
to_dict() (spack.database.InstallRecord method), 193
to_dict() (spack.version.VersionList method), 238
to_node_dict() (spack.spec.Spec method), 229
to_yaml() (spack.provider_index.ProviderIndex method), 221
to_yaml() (spack.spec.Spec method), 229
Token (class in spack.parse), 219
token() (spack.parse.Lexer method), 219
tokens (spack.modules.EnvModule attribute), 206
topological_sort() (in module spack.graph), 204
transform_path() (in module spack.cmd.view), 143
transitive_inc_path() (spack.package.CMakePackage method), 209
traverse() (spack.spec.Spec method), 229
traverse() (spack.test.cmd.test_install.MockSpec method), 154
traverse_with_deptype() (spack.spec.Spec method), 229
tree() (spack.spec.Spec method), 229
try_fetch() (spack.test.git_fetch.GitFetchTest method), 160
try_fetch() (spack.test.hg_fetch.HgFetchTest method), 161
try_fetch() (spack.test.svn_fetch.SvnFetchTest method), 174

U

UnavailableCompilerVersionError, 189
underscored (spack.version.Version attribute), 236
UndetectableNameError, 234
UndetectableVersionError, 234
unexpected_token() (spack.parse.Parser method), 219
uninstall() (in module spack.cmd.uninstall), 141
union() (spack.version.Version method), 236
union() (spack.version.VersionList method), 238
union() (spack.version.VersionRange method), 237
UnknownNamespaceError, 224
UnknownPackageError, 224
UnknownVariantError, 230
unload() (in module spack.cmd.unload), 141
UnsatisfiableArchitectureSpecError, 231
UnsatisfiableCompilerFlagSpecError, 231
UnsatisfiableCompilerSpecError, 231
UnsatisfiableDependencySpecError, 231
UnsatisfiableProviderSpecError, 231
UnsatisfiableSpecError, 231
UnsatisfiableSpecNameError, 231
UnsatisfiableVariantSpecError, 231
UnsatisfiableVersionSpecError, 231
unset() (spack.environment.EnvironmentModifications method), 198
UnsetEnv (class in spack.environment), 198
UnsupportedCompilerError, 230
UnsupportedPlatformError, 199
unuse() (in module spack.cmd.unuse), 142
up_to() (spack.version.Version method), 237

update() (spack.provider_index.ProviderIndex method), 221
update() (spack.version.VersionList method), 238
update_args() (spack.environment.NameModifier method), 198
update_args() (spack.environment.NameValueModifier method), 198
update_config() (in module spack.config), 191
update_packages() (spack.test.concretize_preferences.ConcretizePreferences method), 158
upper_tokens (spack.modules.EnvModule attribute), 206
url_for_version() (spack.package.Package method), 217
url_parse() (in module spack.cmd.url_parse), 142
url_version() (spack.package.Package method), 217
UrlExtrapolateTest (class in spack.test.url_extrapolate), 174
URLFetchStrategy (class in spack.fetch_strategy), 202
UrlParseError, 234
UrlParseTest (class in spack.test.url_parse), 175
urls() (in module spack.cmd.urls), 142
use() (in module spack.cmd.use), 142
use_cray_compiler_names() (in module spack.package), 218
use_name (spack.modules.EnvModule attribute), 206
use_tmp() (in module spack.test.stage), 173

V

valid_fully_qualified_module_name() (in module spack.util.naming), 181
valid_module_name() (in module spack.util.naming), 181
validate() (in module spack.environment), 199
validate_deptype() (in module spack.spec), 229
validate_fully_qualified_module_name() (in module spack.util.naming), 181
validate_module_name() (in module spack.util.naming), 181
validate_names() (spack.spec.Spec method), 229
validate_package_url() (in module spack.package), 218
validate_scope() (in module spack.config), 191
validate_section() (in module spack.config), 191
validate_section_name() (in module spack.config), 192
Variant (class in spack.variant), 236
variant() (in module spack.directives), 194
variant_compare() (spack.preferred_packages.PreferredPackages method), 220
variants (spack.package.Package attribute), 217
VCSFetchStrategy (class in spack.fetch_strategy), 202
ver() (in module spack.version), 238
Version (class in spack.version), 236
version (spack.compiler.Compiler attribute), 188
version (spack.package.Package attribute), 217
version (spack.spec.Spec attribute), 229
version() (in module spack.directives), 194
version_compare() (spack.preferred_packages.PreferredPackages method), 220
version_urls (spack.package.Package attribute), 217
VersionFetchError, 218
VersionList (class in spack.version), 237
VersionRange (class in spack.version), 237
versions (spack.package.Package attribute), 217
versions() (in module spack.cmd.versions), 142
VersionsTest (class in spack.test.versions), 177
view() (in module spack.cmd.view), 143
virtual (spack.spec.Spec attribute), 229
virtual_dependencies() (spack.spec.Spec method), 229
visitor_add() (in module spack.cmd.view), 143
visitor_check() (in module spack.cmd.view), 143
visitor_hard() (in module spack.cmd.view), 143
visitor_hardlink() (in module spack.cmd.view), 143
visitor_remove() (in module spack.cmd.view), 143
visitor_rm() (in module spack.cmd.view), 143
visitor_soft() (in module spack.cmd.view), 143
visitor_statlink() (in module spack.cmd.view), 143
visitor_status() (in module spack.cmd.view), 143
visitor_symlink() (in module spack.cmd.view), 143

W

wait() (spack.util.multiproc.Barrier method), 180
when (class in spack.multimethod), 208
which() (in module spack.util.executable), 180
wildcard() (spack.version.Version method), 237
wildcard_version() (in module spack.url), 235
write() (spack.graph.AsciiGraph method), 204
write() (spack.modules.EnvModule method), 206
write_license_file() (in module spack.hooks.licensing), 149
write_section() (spack.config.ConfigScope method), 191
write_spec() (spack.directory_layout.YamlDirectoryLayout method), 197
write_transaction() (spack.database.Database method), 193
write_transaction() (spack.file_cache.FileCache method), 203

X

Xl (class in spack.compilers.xl), 147

Y

YamlDirectoryLayout (class in spack.directory_layout), 196
YamlTest (class in spack.test.yaml), 178